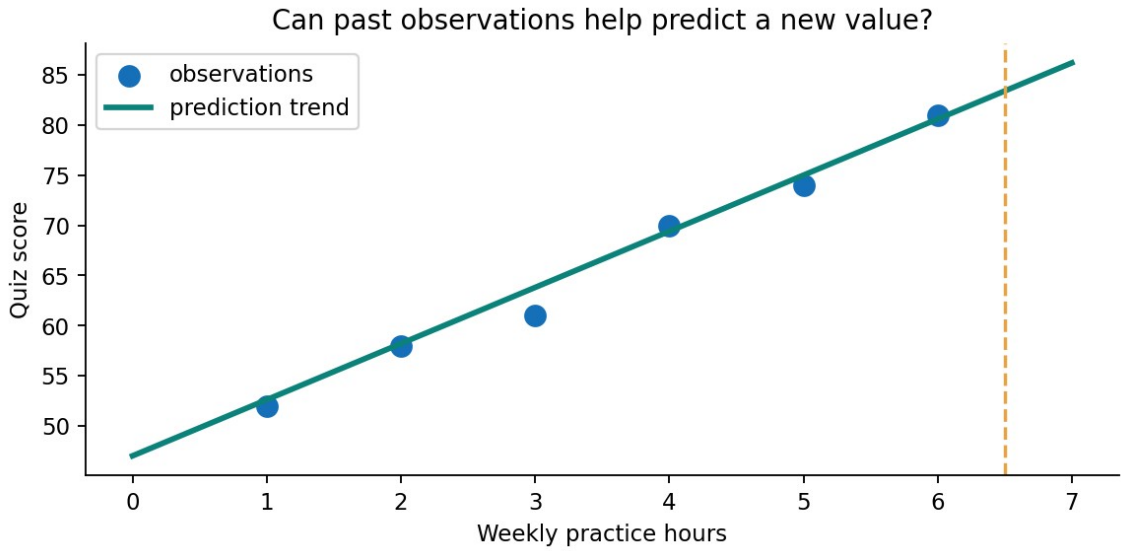
ספר מדעי הבינה המלאכותית

מסלול הלמידה העמוקה | פרק 5 בתכנית ההתמחות

רגרסיה ליניארית — מהחיזוי הראשון ועד לאימון מודל

מהדורת תלמיד מורחבת · מתמטיקה מוסברת · חישוב ידני · תרשימים · Python · תרגול לבגרות ולפרויקט גמר



איור: לומדים לזהות קשר בין מאפיין לתוצאה ולבדוק אם אפשר לחזות דוגמה חדשה.

מיקום בתכנית: פרק 5 — רגרסיה ליניארית (15 שעות) במסלול ההתמחות בלמידת מכונה ולמידה עמוקה, על פי מפת התכנית השמורה במאגר. הפרק נשען על מצגות ההשתלמות 4.1 ו־4.2, על תרגיל Iris ועל מחברת KC House. חלקים כגון גזירה של נוסחת הריבועים הפחותים ו־Gradient Descent מסומנים כהעמקה דידקטית ולא כדרישת בחינה מאומתת.

מטרות הפרק

בסיום הפרק תוכלו להבחין בין סיווג לרגרסיה, לקרוא את המשוואה $\hat{y} = wx + b$ ולהסביר כל סימן, לבצע תחזיות ידניות, לחשב שאריות ושגיאות, להבין כיצד נקבע הישר המתאים לנתונים, לממש LinearRegression ב־scikit-learn, לפרש MAE, MSE, RMSE ו־R^2, להרחיב למספר מאפיינים, ולהציע ניסוי הוגן על נתונים שלא שימשו לאימון.

ידע קודם והנחיות קריאה

לפני הקריאה נדרשת היכרות עם צירים, משוואת ישר, ממוצע, תרשים פיזור, וקטורים, טבלת DataFrame ופעולות בסיסיות ב־Python. אם פעולת המכפלה הפנימית אינה מוכרת, חזרו לפרק 1.1; אם מטריצת מאפיינים אינה ברורה, חזרו לפרק 4.1. תחילה נסביר כל פעולה בעברית, ורק אחר כך נשתמש בספרייה מקצועית.

מפת הדרך

בעיה אמיתית → משתנה מטרה → משוואת חיזוי → שיפוע וחיתוך → שארית → מדדי שגיאה → מציאת ישר מיטבי → Python → אימון/בדיקה → מספר מאפיינים → גשר לאופטימיזציה → תרגול עצמאי.

1. סיפור הפתיחה — האם ניתן לנבא תוצאה שטרם נמדדה?

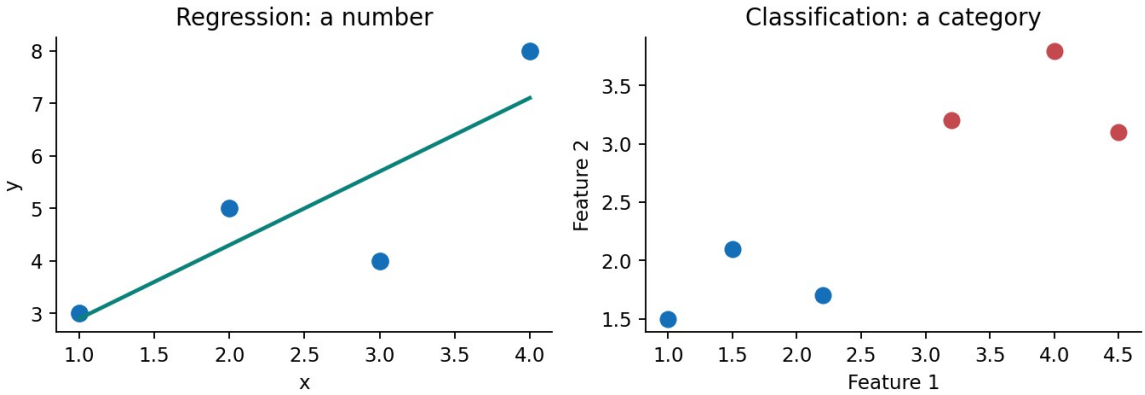
דודו מדריך תלמידים במעבדת תכנות. הוא מבחין שתלמידים המקדישים יותר שעות לתרגול מקבלים בדרך כלל ציונים גבוהים יותר בבוחן קצר. הוא אינו רוצה לנחש את הציון של כל תלמיד מחדש: הוא מחפש כלל מספרי שיוכל להשתמש בו לתלמידים נוספים. בידיו טבלה עם שעות תרגול והציונים שהתקבלו בעבר. משימתנו היא לבחון האם קיים קשר מועיל, כיצד לבטא אותו במספרים, וכיצד לדעת אם הכלל מצליח גם על תלמיד שלא השתתף באימון המודל.

שימו לב להבדל בין שאלה תיאורית לשאלת חיזוי: ״מהו ממוצע הציונים?״ היא שאלה סטטיסטית תיאורית. ״איזה ציון צפוי לתלמיד שתרגל שלוש שעות?״ היא שאלת חיזוי. גם אם התשובה המספרית נראית מדויקת, היא תחזית ולא הבטחה.

בדיקת הבנה: אם שני תלמידים למדו אותו מספר שעות וקיבלו ציונים שונים — האם פירוש הדבר שרגרסיה אינה אפשרית? נמקו.

2. מתי משתמשים ברגרסיה ומתי בסיווג?

למידה מונחית מתחילה בדוגמאות שבהן גם המאפיינים וגם התוצאה ידועים. ברגרסיה (Regression) התוצאה היא קטגוריה: ״תקין/תקול״, ״עיר/יער״, או סוג הפרח. אין לבחור אלגוריתם רק משום שהוא מוכר לנו; הבחירה מתחילה מהגדרת התוצאה.



איור: ברגרסיה מנבאים מספר על ציר רציף; בסיווג מנבאים תווית של מחלקה.

משפחה	הפלט	תכונות הקלט	שאלת חיזוי
רגרסיה	מספר דקות	גיל המחשב, סוג תקלה	כמה דקות יידרש לתיקון?
סיווג	כן / לא	גיל, תקלות עבר	האם המחשב יתקלקל?
רגרסיה	סכום כסף	שטח, מספר חדרים	מהו מחיר דירה?

חשוב: העובדה שמספר קטגוריות מסומנות בקוד כמספרים 0 ו־1 אינה הופכת אותן אוטומטית למשתנה רציף.

3. השפה של הנתונים: מאפיינים, מטרה, תחזית

לכל דוגמה יש מאפיינים (Features), שהם נתונים הידועים בזמן החיזוי. את התוצאה המבוקשת נקרא משתנה המטרה (Target), ומסמנים לרוב y. את כלל המאפיינים של הדוגמאות מסדרים במטריצה X.

כאשר יש רק מאפיין אחד, אפשר לצייר את הדוגמאות כנקודות במישור. כאשר משתמשים בשני מאפיינים, כל דוגמה היא וקטור בעל שני רכיבים, גם אם התוצאה המבוקשת נשארת מספר אחד.

Features (X) versus target (y)

Example	Practice hours	Breaks	Result
A	1	2	3
B	2	1	5
C	3	2	4
D	4	1	8

איור: ב־X שומרים את נתוני הקלט; בעמודת התוצאה y שומרים את הערך שאותו רוצים לחזות.

X: shape = (number of examples, number of features)

אם יש ארבע דוגמאות ושני מאפיינים לכל אחת, המטריצה X מכילה ארבע שורות ושתי עמודות. לעומת זאת, וקטור y מכיל ארבעה ערכים — ערך מטרה אחד לכל דוגמה. חשוב להתאים תמיד בין מספר השורות ב־X לבין אורך y.

```
import numpy as np
X = np.array([[1, 2], [2, 1], [3, 2], [4, 1]])
y = np.array([3, 5, 4, 8])
print(X.shape)     # (4, 2)
print(y.shape)     # (4,)
```

בדיקת הבנה: כאשר תוסיפו מאפיין שלישי לארבע הדוגמאות, מה ישתנה בממדי X, ומה לא ישתנה ב־y?

4. משוואת הישר: להפוך דפוס לכלל חיזוי

נפתח במודל הפשוט ביותר: ישר שקושר בין מאפיין אחד x לבין תחזית מספרית. הרעיון אינו שהנתונים עצמם חייבים להימצא על קו ישר, אלא שאנו משתמשים בקו כקירוב מסודר למגמה הכללית. סימן הגג מעל y נועד להבחין בין התוצאה שנמדדה בפועל לבין ערך שהמודל חישב.

$$\hat{y} = w\,x + b$$

סימן	השם	מה הוא אומר?
x	מאפיין	הנתון הידוע בעת החיזוי
w	מקדם / שיפוע	כמה תשתנה התחזית כאשר x גדל ביחידה אחת
b	הטיה / חיתוך	התחזית כאשר x שווה אפס
$\hat{y}$	תחזית	הערך שהמודל מחזיר; לא בהכרח הערך האמיתי

דוגמה: נניח שהמודל הוא $\hat{y} = 5x + 48$, כאשר x הוא מספר שעות תרגול והתחזית היא ציון. אם תלמיד מתרגל שלוש שעות, התחזית היא 63 נקודות: מכפילים שלוש בחמש ומוסיפים 48. אם השעות גדלות מארבע לחמש, התחזית עולה בחמש נקודות. זוהי משמעות השיפוע. את $b=48$ אין לפרש בהכרח כציון אמיתי ללא תרגול; ייתכן שערך $x=0$ כלל אינו בטווח הנתונים שעליו למדנו.

$$\hat{y}(3) = 5 \cdot 3 + 48 = 63$$

בדיקת הבנה: אם מחליפים את $w=5$ ב־$w=-5$, כיצד משתנה כיוון הקשר? מהי המשמעות של b שלילי?

5. חיזוי ידני: שלוש דוגמאות ללא Python

נבחר מודל לדוגמה: $\hat{y} = 1.4x + 1.5$. בשלב זה עדיין איננו טוענים שזהו הישר הטוב ביותר. מטרתנו להבין את פעולת החיזוי. נציב ערכי x שונים ונראה שהחישוב תמיד מתבצע באותו סדר: מכפילים את המאפיין בשיפוע, מוסיפים את החיתוך ומקבלים תחזית.

x	כפל בשיפוע	הוספת חיתוך	תחזית $\hat{y}$
1	$1.4 \times 1 = 1.4$	$1.4 + 1.5$	2.9
2	$1.4 \times 2 = 2.8$	$2.8 + 1.5$	4.3
3	$1.4 \times 3 = 4.2$	$4.2 + 1.5$	5.7
4	$1.4 \times 4 = 5.6$	$5.6 + 1.5$	7.1

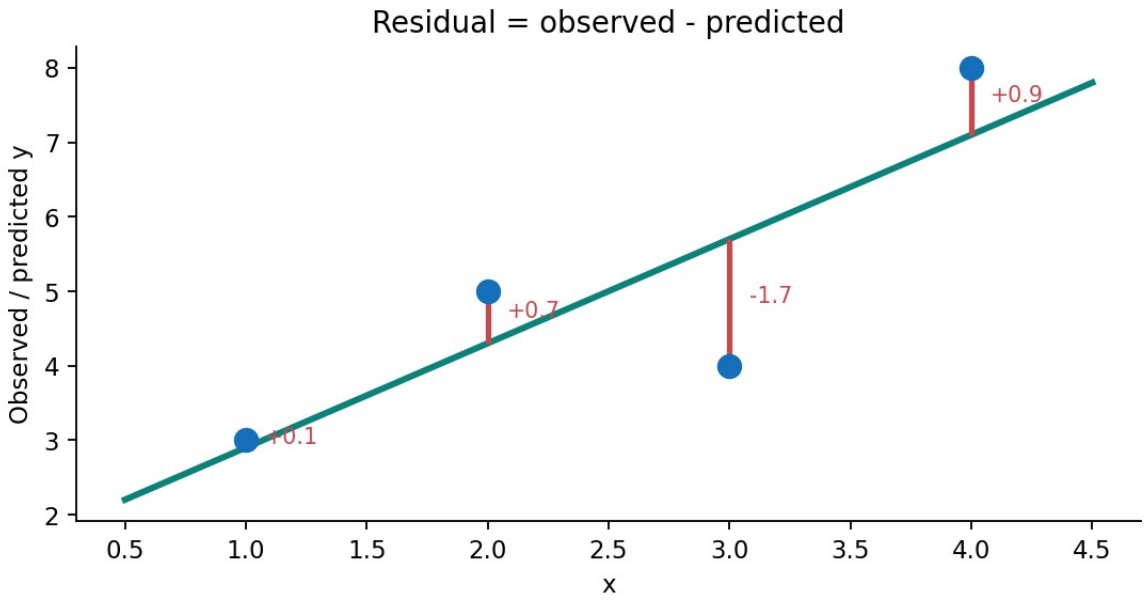
המודל מחזיר גם ערכים עשרוניים. אם אנו חוזים מחיר במאות שקלים זו התנהגות סבירה; אם אנו חוזים מספר תלמידים שאינם יכולים להתחלק לשבר, ייתכן שמודל אחר יתאים יותר. בחירת המודל אינה מסתיימת בפעולת ההצבה.

6. מהי שארית ומדוע היא חשובה?

נשתמש בארבע דוגמאות פשוטות. הערכים שנמדדו בפועל הם 3, 5, 4, 8 עבור $x=1,2,3,4$. התחזיות מהישר הקודם הן 2.9, 4.3, 5.7, 7.1. שארית (Residual) היא ההפרש בין הערך האמיתי לבין התחזית. שארית חיובית פירושה שהתחזית נמוכה מהערך האמיתי; שארית שלילית פירושה שהמודל העריך יותר מדי.

$$e_i = y_i - \hat{y}_i$$

x	ערך נמדד y	תחזית $\hat{y}$	שארית $y-\hat{y}$
1	3	2.9	0.1
2	5	4.3	0.7
3	4	5.7	−1.7
4	8	7.1	0.9



איור: הקווים האנכיים מייצגים את הפער בין הנקודות הנמדדות לקו החיזוי.

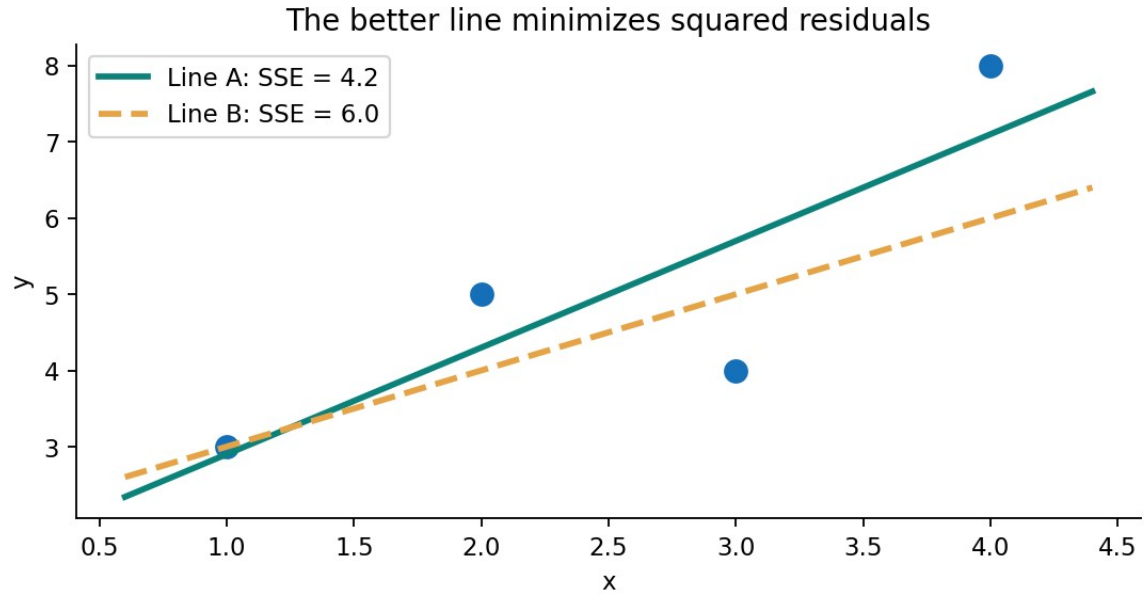
אם נחבר שאריות חיוביות ושליליות, הן עלולות לבטל זו את זו. לכן לא משתמשים בדרך כלל בסכום שאריות רגיל כמדד יחיד לטיב המודל. בהמשך נמדוד את גודל השגיאות באמצעות ערך מוחלט או באמצעות ריבוע.

7. מדוע אנו מחפשים את "הקו הטוב ביותר"?

אפשר להעביר אינסוף ישרים שונים דרך אותו אזור של נקודות. השאלה היא כיצד לבחור מתוכם באופן עקבי ולא רק לפי מה שנראה יפה בגרף. שיטת הריבועים הפחותים (Ordinary Least Squares, OLS) בוחרת שיפוע וחיתוך הממזערים את סכום ריבועי השאריות. ריבוע מבטיח ששגיאות חיוביות ושליליות יגדילו את הסכום, וששגיאות גדולות מקבלות משקל כבד יותר.

$$\mathbf{SSE = \Sigma\,(y_i - \hat{y}_i)^2}$$

לישר A מהדוגמה, סכום הריבועים הוא $0.7^2+0.1^2+(-1.7)^2+0.9^2 = 0.49+0.01+2.89+0.81 = 4.20$. נבדוק ישר חלופי $\hat{y} = x+2$:B. התחזיות הן 3,4,5,6 ולכן השאריות הן 1,2−,0,1 וסכום הריבועים $4+1+1+0=6$. בהשוואה בין שני הקווים על אותן ארבע דוגמאות, קו A מתאים טוב יותר לפי SSE.



איור: הישר בעל סכום ריבועי השאריות הקטן יותר נבחר לפי קריטריון *OLS*.

חשוב: SSE הוא מדד לשגיאה על הנתונים שנבדקו. העובדה שקו מסוים מנצח על נתוני האימון אינה מוכיחה שהוא ינבא טוב יותר נתונים חדשים.

8. כיצד מחשבים שיפוע וחיתוך מתוך הנתונים?

כעת נעבור מהצבה במודל קיים ללמידה של המודל עצמו. עבור רגרסיה ליניארית פשוטה, כאשר ערכי x אינם כולם זהים, ניתן לחשב את השיפוע שממזער את SSE ישירות מממוצעי x ו־y. אל תנסו לשנן את הנוסחה לפני שתבינו את מבנה המונה והמכנה: המונה בודק האם x ו־y משתנים יחד, והמכנה מודד כמה x משתנה סביב הממוצע שלו.

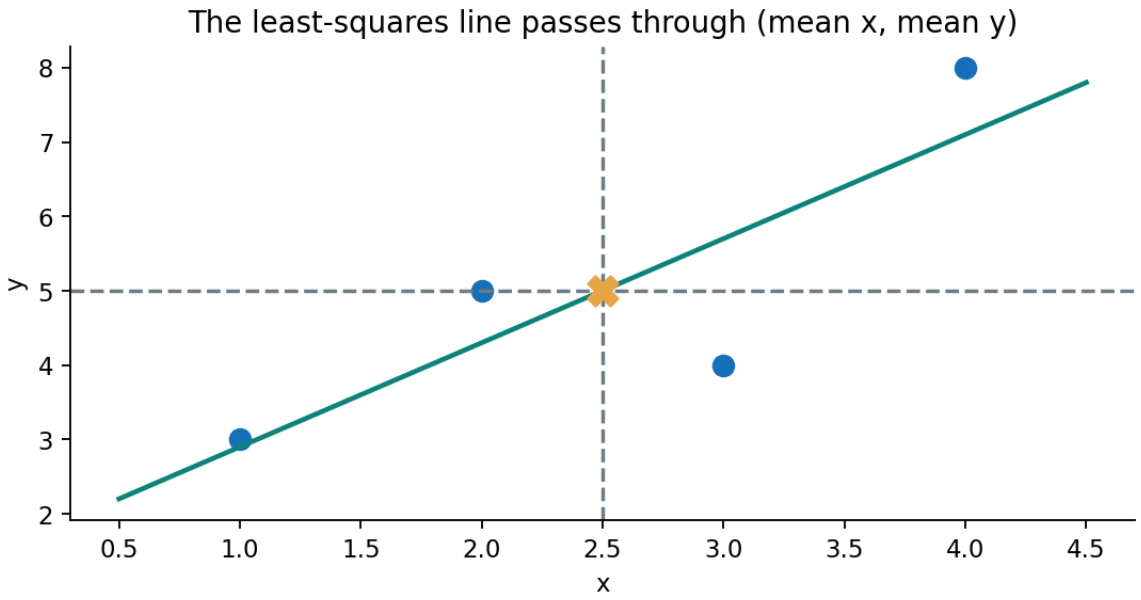
$$w = \Sigma[(x_i - \bar{x})(y_i - \bar{y})] \,/\, \Sigma[(x_i - \bar{x})^2]$$

$$b = \bar{y} - w\,\bar{x}$$

לנתונים $x=(1,2,3,4)$, $y=(3,5,4,8)$ נקבל $\bar{x}=2.5$, $\bar{y}=5$. נרכז את הסטיות בטבלה. כל ערך במונה נוצר ממכפלת שתי סטיות המתייחסות לאותה דוגמה. כל ערך במכנה הוא ריבוע הסטייה של x בלבד.

x	y	x−2.5	y−5	מכפלת סטיות	ריבוע סטיית x
1	3	−1.5	−2	3	2.25
2	5	−0.5	0	0	0.25
3	4	+0.5	−1	−0.5	0.25
4	8	+1.5	+3	4.5	2.25
סכום				7	5

$$w = 7/5 = 1.4 \qquad b = 5 - 1.4 \cdot 2.5 = 1.5$$



איור: ישר הריבועים הפחותים ברגרסיה פשוטה עם חיתוך עובר דרך נקודת הממוצעים $(\bar{x},\bar{y})$.

כעת אנו יודעים מדוע השתמשנו בישר $\hat{y}=1.4x+1.5$: הוא התקבל מחישוב המתאים לארבע הדוגמאות. אם נחליף תצפית או נוסיף נתונים, השיפוע והחיתוך עשויים להשתנות. זו משמעות ה״למידה״ ברגרסיה ליניארית: התאמת מקדמים מנתוני האימון.

בדיקת הבנה: מדוע הנוסחה אינה ניתנת לחישוב אם כל ערכי x זהים? מה מלמד אותנו המכנה במקרה זה?

9. מדדי שגיאה — לא כל מדד אומר אותו דבר

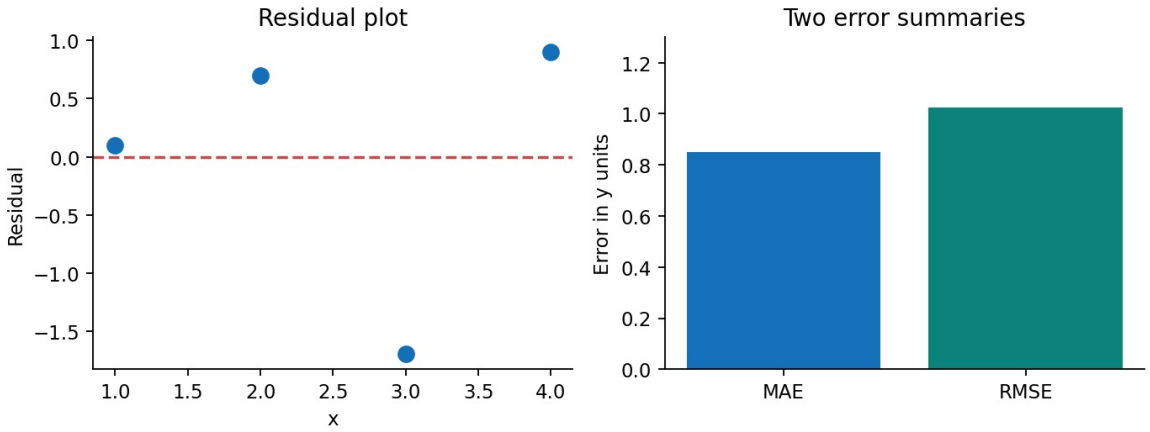
עד כה השתמשנו ב־SSE, אך לסכום שגיאות יש בעיה: ככל שמספר הדוגמאות גדל, גם הסכום נוטה לגדול. לכן משתמשים לעיתים בממוצע שגיאות. MAE מחשב את ממוצע הערכים המוחלטים של השאריות; MSE מחשב את ממוצע ריבועי השאריות; RMSE הוא שורש MSE ומחזיר אותנו ליחידות של משתנה המטרה.

$$\mathbf{MAE = (1/n)\ \Sigma\ |y_i - \hat{y}_i|}$$

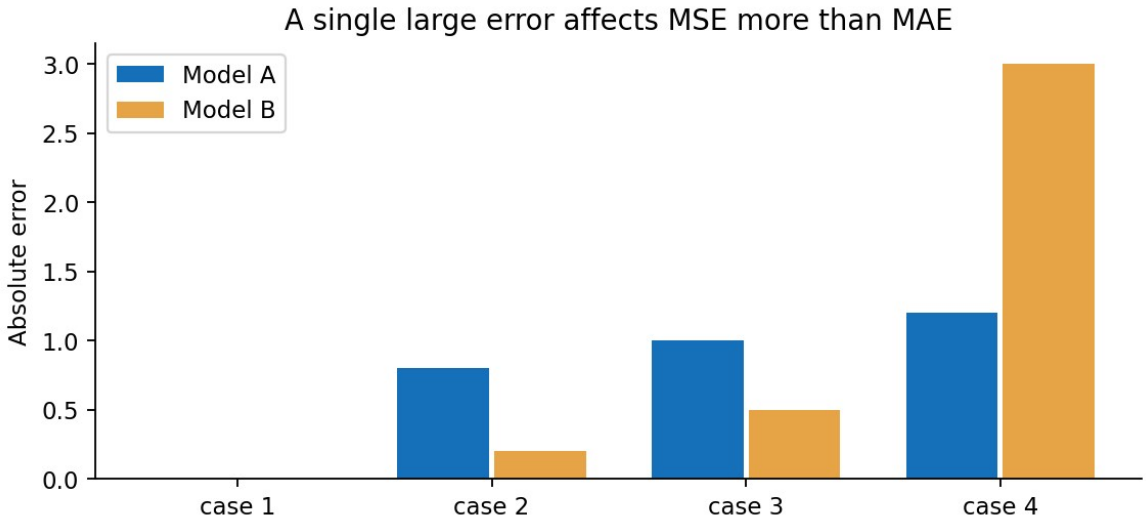
$$\mathbf{MSE = (1/n)\ \Sigma\ (y_i - \hat{y}_i)^2}$$

$$\mathbf{RMSE = \sqrt{MSE}}$$

בחישוב הידני שלנו: MAE=(0.1+0.7+1.7+0.9)/4=0.85. MSE=(0.01+0.49+2.89+0.81)/4=1.05. RMSE=1.051.0247. אם y נמדד בנקודות ציון, גם MAE ו־RMSE נמדדים בנקודות ציון, ואילו MSE נמדד בנקודות ציון בריבוע.



איור: תרשים שאריות ומדדי שגיאה: אין די בחישוב מספר אחד; חשוב להבין את יחידותיו.



איור: *MSE* מעניק משקל רב יותר לשגיאה גדולה בודדת, לעומת *MAE*.

בדיקת הבנה: אם שגיאה של 1 הוכפלה לשגיאה של 2, בכמה גדלה תרומתה ל־MAE ובכמה ל־MSE?

10. R^2 — כמה טוב יותר המודל מתחזית ממוצע?

מדד R^2 (מקדם ההסבר, Coefficient of Determination) בוחן את השיפור היחסי של המודל לעומת תחזית קבועה של הממוצע בנתונים הנבדקים. במונה נמצא סכום ריבועי השאריות של המודל; במכנה נמצא סכום ריבועי הסטיות של הערכים האמיתיים מממוצעם. זהו מדד יחסי ואינו מודד ישירות מספר שקלים או דקות של שגיאה.

$$R^2 = 1 - \Sigma(y_i - \hat{y}_i)^2 / \Sigma(y_i - \bar{y})^2$$

בדוגמה שלנו הממוצע הוא 5. כדי לחשב את המכנה, מחסרים 5 מכל אחד מארבעת הערכים האמיתיים, מעלים בריבוע ומחברים. מתקבלים ארבעה רכיבים: 4, 0, 1 ו־9. סכום 14. סכום שגיאות המודל הוא 4.2. לאחר הצבת הנתונים בנוסחה מקבלים R^2 השווה ל־0.70. פירוש זה מתייחס להתאמת המודל לנתונים הללו, לא להבטחה כי על כל תלמיד חדש יושג ציון דומה.

$$R^2 = 1 - 4.2/14 = 0.70$$

R^2 יכול להיות גם שלילי: נניח שבקבוצת בדיקה הערכים האמיתיים הם 3,5,7, אך המודל חוזה לכולם. סכום ריבועי השאריות הוא 83, וסכום ריבועי הסטיות ממוצע הבדיקה (5) הוא 8. לאחר הצבה בנוסחה נקבל R^2 שלילי שערכו מינוס 9.375. הוא אינו מוגבל לטווח 0 עד 1 כאשר בודקים תחזיות כלליות; המשמעות היא שבאותה קבוצת בדיקה המודל גרוע בהרבה מהתחזית הקבועה של ממוצע אותה קבוצה.

> חשוב: אם כל ערכי y זהים, המכנה בנוסחת R^2 הוא אפס והמדד דורש טיפול מיוחד. אל תציגו מקרה זה כמדד רגיל. הממוצע בביטוי R^2 של Test הוא אמת מידה מתמטית, ולא שימוש מותר ב־Test לשם אימון מודל.

11. מימוש ראשון ב־NumPy — לשחזר את המתמטיקה

לפני שמבקשים מספרייה לבחור את הישר, נוכל לשחזר את הנוסחה בעצמנו. חשוב להבחין בין חישוב מקדמים ובין חיזוי: בשלב הראשון משתמשים בנתוני האימון כדי למצוא w ו־b; בשלב השני מפעילים את אותו זוג מקדמים על x חדש. הנה קוד קצר שמדגים בדיוק את החישוב הידני.

```
import numpy as np
x = np.array([1., 2., 3., 4.])
y = np.array([3., 5., 4., 8.])
x_bar = x.mean()
y_bar = y.mean()
w = np.sum((x - x_bar) * (y - y_bar)) / np.sum((x - x_bar)**2)
b = y_bar - w * x_bar
predictions = w * x + b
print('w:', w, 'b:', b)
print('predictions:', predictions)
```

התוצאה הצפויה היא $w=1.4$ ו־$b=1.5$, והתחזיות 2.9, 4.3, 5.7, 7.1. זהו לא טריק של Python: כל פקודה מקבילה לשלב מסוים בטבלת החישוב הידני. אם תשנו את הנתונים ותבצעו את החישוב מחדש, תוכלו לבדוק כיצד הקו משתנה.

12. מימוש מקצועי ב־scikit-learn

בעבודה מקצועית משתמשים במחלקה LinearRegression. הפעולה fit מתאימה את מקדמי המודל לנתוני האימון; תכונות coef_ ו־intercept_ מאפשרות לקרוא את המקדמים שנלמדו; הפעולה predict מחשבת תחזיות. חשוב: גם כשיש רק מאפיין אחד, X צריך להיות מטריצה דו־ממדית עם שורה לכל דוגמה ועמודה אחת לפחות.

```
import numpy as np
from sklearn.linear_model import LinearRegression

X = np.array([[1.], [2.], [3.], [4.]])
y = np.array([3., 5., 4., 8.])
model = LinearRegression()
model.fit(X, y)
print(model.coef_, model.intercept_)
print(model.predict(np.array([[2.5], [3.5]])))
```

שימו לב לצורה של המערך החדש: [[2.5],[3.5]] פירושו שתי דוגמאות, שלכל אחת מהן מאפיין אחד. כתיבת [2.5,3.5] בלבד אינה מייצגת את אותו מבנה. אם יש שני מאפיינים לדוגמה חדשה אחת, צורת הקלט דווקא תהיה [[x1,x2]].

בדיקת הבנה: מה ההבדל בין x אחד עם שני מאפיינים לבין שתי דוגמאות שלכל אחת מאפיין אחד? תארו את ממדי X בשני המקרים.

13. איך מציגים את קו הרגרסיה בגרף?

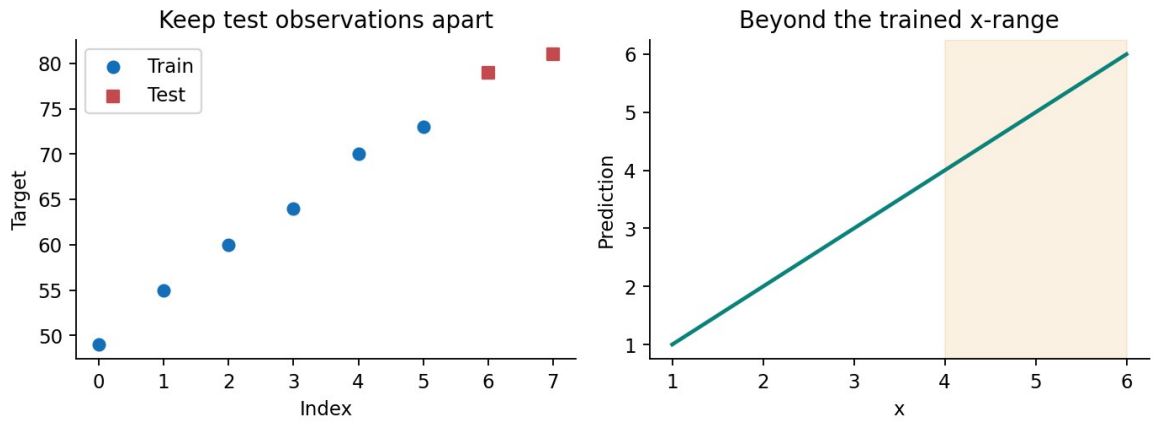
מטרתו של הגרף היא לאפשר לנו להשוות בצורה חזותית את הנקודות שנמדדו לקו התחזיות. לכן על אותו תרשים נציג את דוגמאות האימון כנקודות ואת התחזית של המודל עבור ערכי x רציפים כקו. כך ניתן לזהות נקודות שמרוחקות מהקו, מגמה שאינה ליניארית או אזור שבו אין לנו נתונים מספיקים.

```
import matplotlib.pyplot as plt
x_line = np.linspace(1, 4, 100).reshape(-1, 1)
y_line = model.predict(x_line)
plt.scatter(X[:, 0], y, label='Observed')
plt.plot(x_line[:, 0], y_line, label='Prediction')
plt.xlabel('Input feature')
plt.ylabel('Target')
plt.legend()
plt.show()
```

אל תציירו קו דרך הנקודות לפי סדר הופעתן בטבלה: כך עלול להתקבל ״זיגזג״ במקום ישר המודל. כדי לצייר קו תחזית, השתמשו ברצף מסודר של ערכי x והעבירו אותו דרך predict.

14. נתוני אימון ונתוני בדיקה — לא בודקים על מה שלמדנו בלבד

בדיקת ביצועים על נתוני האימון עשויה להסתיר בעיות. כדי לדעת כיצד המודל צפוי לעבוד על דוגמאות חדשות, נפריד מראש נתוני Train ו־Test. נלמד מקדמים מתוך Train בלבד, ונחשב מדדים על Test שלא השתתף באימון. אין להמשיך לשנות את המודל שוב ושוב לפי ציון Test ולהציג אותו אחר כך כהערכה בלתי תלויה.



איור: משמאל: הפרדה בין נתוני אימון ובדיקה. מימין: חיזוי מחוץ לטווח הנתונים הוא אקסטרפולציה.

```
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_absolute_error, r2_score

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42)
model = LinearRegression()
model.fit(X_train, y_train)
y_test_pred = model.predict(X_test)
print('Test MAE:', mean_absolute_error(y_test, y_test_pred))
```

חשוב: הדוגמה עם ארבע תצפיות בלבד ממחישה תחביר, אך היא קטנה מדי להערכת מודל אמינה. בפרויקט נעבוד עם מאגר משמעותי יותר, ונשווה לקו בסיס מתאים.

15. קו בסיס — האם צריך בכלל מודל מורכב?

לפני שמכריזים על הצלחה, כדאי להשוות למודל פשוט מאוד: תמיד לחזות את ממוצע ערכי y של נתוני האימון. זהו Baseline. אם הרגרסיה כמעט אינה משפרת את מדדי השגיאה ביחס לקו הבסיס, יש לשאול האם המאפיינים באמת מועילים, האם הנתונים מתאימים לבעיה והאם רמת הרעש גבוהה מדי. קו הבסיס אינו תחליף למודל אלא מבחן מינימלי לכך שהמודל החדש מביא ערך.

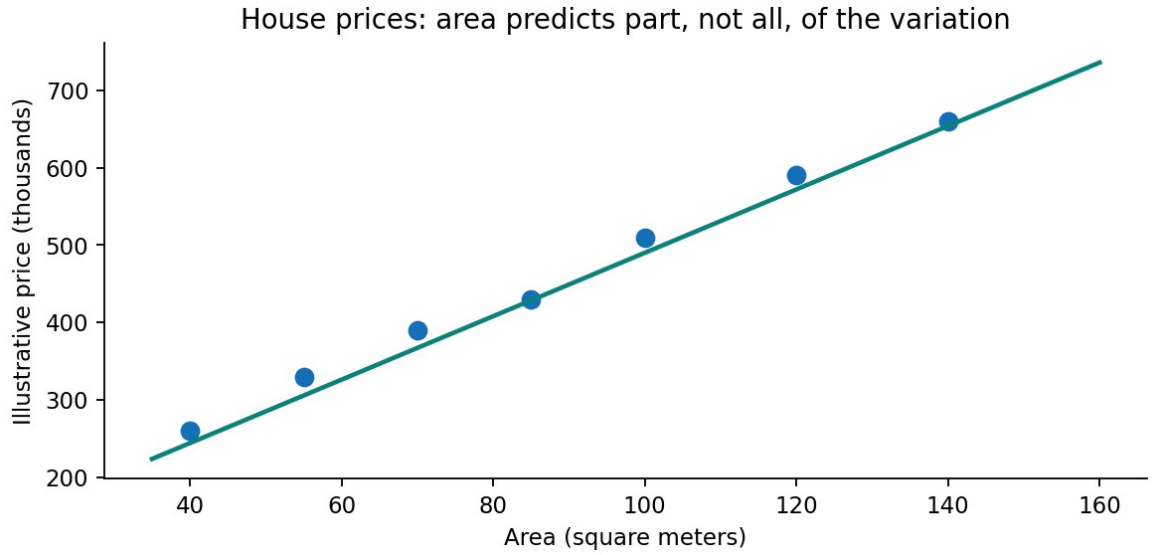
```
import numpy as np
baseline_value = np.mean(y_train)
baseline_predictions = np.full(len(y_test), baseline_value)
model_error = mean_absolute_error(y_test, y_test_pred)
base_error = mean_absolute_error(y_test, baseline_predictions)
print('Model MAE:', model_error)
print('Baseline MAE:', base_error)
```

16. רגרסיה עם יותר ממאפיין אחד

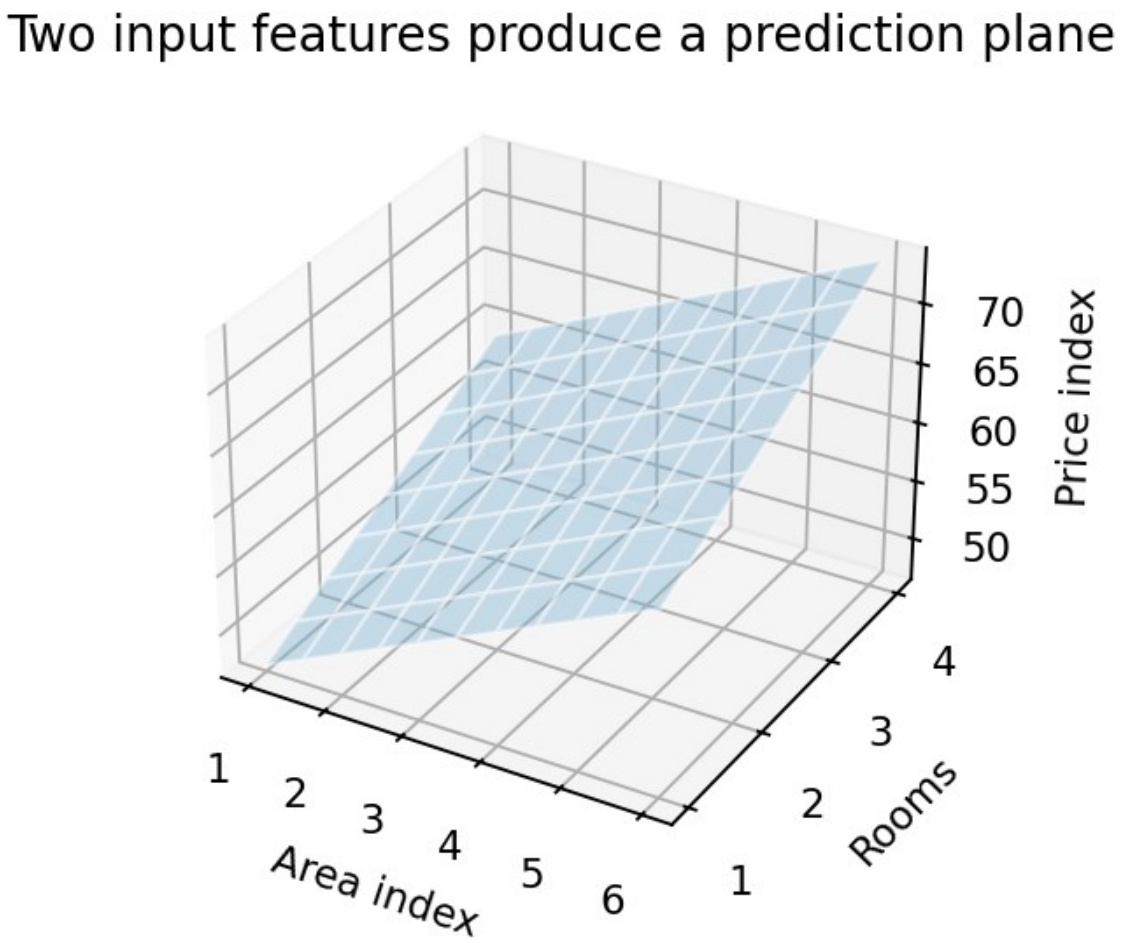
בחיזוי מחירי דירות, שטח לבדו אינו מתאר את כל ההבדלים בין דירות. נוכל להוסיף מספר חדרים, גיל בניין או מרחק ממרכז עיר. כאשר יש p מאפיינים, המודל מחשב מכפלה פנימית בין וקטור המאפיינים לווקטור המקדמים, ומוסיף b. זוהי אותה פעולה מתמטית שכבר למדנו בפרק 1.1 ושתופיע שוב במודל של נוירון.

$$\hat{y} = w_1x_1 + w_2x_2 + \ldots + w_px_p + b$$

נניח מודל המחזיר אומדן באלפי שקלים: $\hat{y}$=4×שטח+25×חדרים+80. לדירה בעלת שטח 70 מ״ר ו־3 חדרים מתקבלת תחזית 4×70+25×3+80=435 (אלפי שקלים). אלה מספרים מומצאים להמחשת פעולת המודל, ולא הערכת שוק אמיתית בישראל. השפעת המקדם 4 נמדדת באלפי שקלים למ״ר, בהינתן שמספר החדרים נשאר קבוע.



איור: קשר מומצא בין שטח לבין מחיר דירה — דוגמה להמחשה, לא נתוני שוק אמיתיים.



איור: כאשר יש שני מאפיינים, המודל מייצר מישור חיזוי במקום קו יחיד.

```
import pandas as pd
from sklearn.linear_model import LinearRegression

df = pd.DataFrame({
    'area_m2': [40, 55, 65, 80, 95, 110],
    'rooms':   [1, 2, 2, 3, 4, 4],
    'price_k': [250, 310, 360, 460, 550, 610]
})
X = df[['area_m2', 'rooms']]
y = df['price_k']
```

```
reg = LinearRegression().fit(X, y)
print(reg.coef_, reg.intercept_)
```

אם המקדמים משתנים לאחר הוספת מאפיין, אין בכך תקלה: כעת כל מקדם מתאר שינוי בתחזית כאשר יתר המאפיינים במודל מוחזקים קבועים. אין להסיק ישירות שקשר שנמצא בנתונים הוא קשר סיבתי. כמו כן, שני מאפיינים דומים מאוד עשויים להקשות על פירוש יציב של המקדמים.

17. תרגיל מבוסס Iris — ממדגם אמיתי לחיזוי רציף

במאגר Iris נמדדות ארבע תכונות של פרחים. אמנם רבים מכירים אותו מסיווג מינים, אך אפשר להשתמש באותן מדידות גם למשימת רגרסיה: לנבא אורך עלה כותרת מתוך אורך עלה הגביע. כך מתברר שאותו מאגר יכול לשמש שאלות חקר שונות, והמשתנה שאנו בוחרים לחזות קובע אם מדובר ברגרסיה או בסיווג.

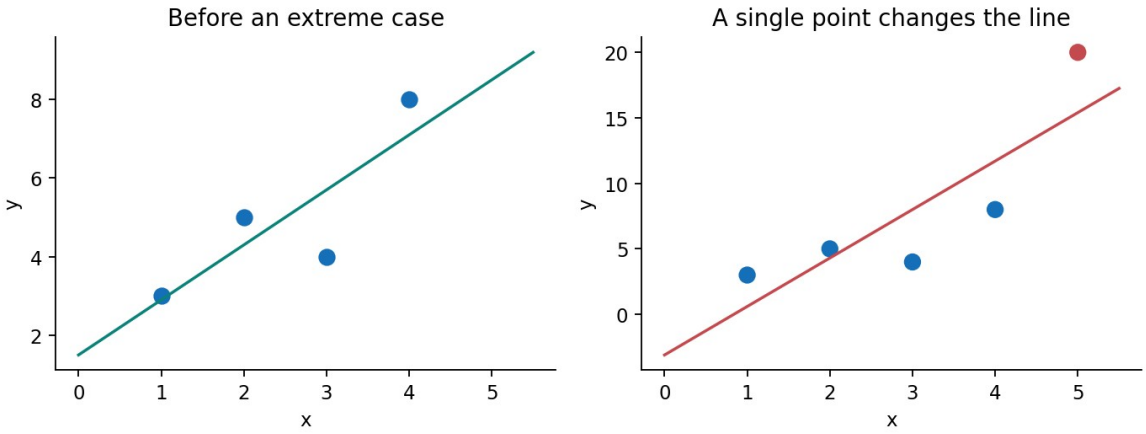
```
from sklearn.datasets import load_iris
from sklearn.linear_model import LinearRegression

iris = load_iris(as_frame=True).frame
X = iris[['sepal length (cm)']]
y = iris['petal length (cm)']
reg = LinearRegression().fit(X, y)
print('Slope:', reg.coef_[0])
print('Intercept:', reg.intercept_)
new_iris = X.iloc[[0]].copy()
new_iris.iloc[0, 0] = 6.0
print('Prediction:', reg.predict(new_iris))
```

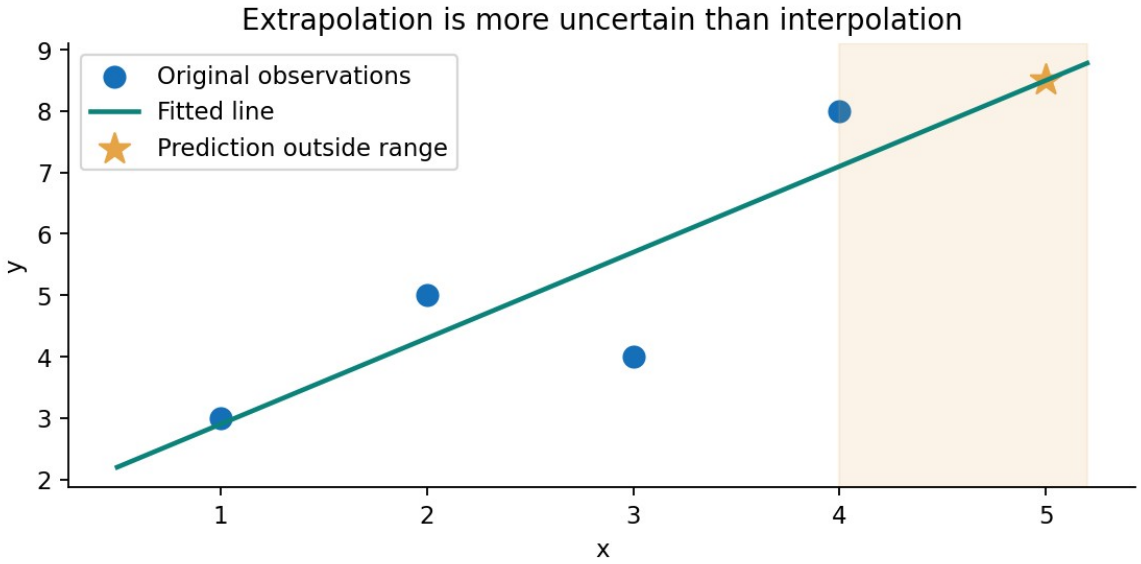
שאלת חקר: האם ייתכן שהקשר שונה בין מיני פרחים שונים? אם נבדוק את ההנחה הזו, חשוב להחליט מראש אם אנו מנתחים את המאגר כולו או תת־קבוצה מסוימת ולציין על אילו נתונים אומן המודל. מדידות פרחים אינן מודל לכל תופעה ביולוגית; זו דוגמה לימודית מוגדרת.

18. טעויות שיטתיות, חריגים והנחות

רגרסיה ליניארית היא מודל פשוט, אך אין פירושו שכל קשר אמיתי חייב להיות ישר. אם השאריות יוצרות צורת קשת בגרף, הדבר עשוי להעיד שהקשר אינו ליניארי. ערך חריג יכול למשוך את הישר לכיוונו. משתנה שנשמט מהמודל יכול ליצור הטיה, ומידע המופיע רק לאחר האירוע שאנו מנסים לחזות אינו מאפיין חוקי בזמן החיזוי.



איור: נקודת קיצון אחת יכולה להזיז משמעותית את הישר המותאם — אין למחוק חריג באופן אוטומטי.



איור: חיזוי ב־x שלא הופיע בתחום האימון הוא אקסטרפולציה; יש להתייחס אליו בזהירות.

בדקו גם אם המשתנה y מוגבל לטווח מסוים. ישר יכול להחזיר תחזית שלילית למחיר או תחזית מעל 100 לציון, אף שאין לתוצאות כאלה משמעות בבעיה. לעיתים נדרש מודל אחר, שינוי ייצוג או הגדרת טווח שימוש. התאמה סטטיסטית טובה אינה מבטלת צורך בביקורת מקצועית.

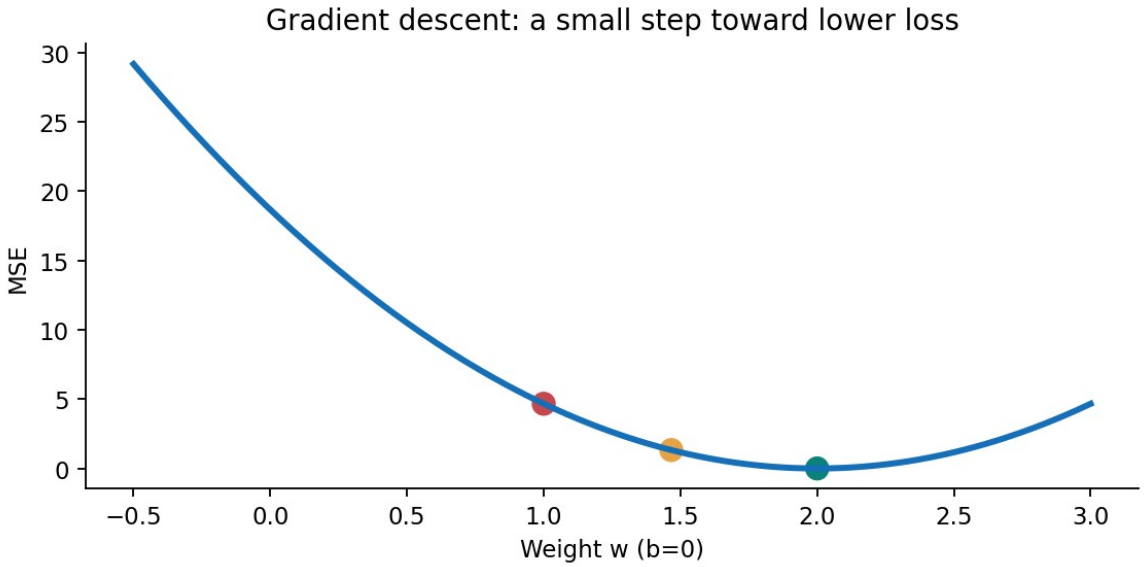
19. גשר ללמידה עמוקה — איך משקולות משתנות באימון?

עד כה מצאנו את השיפוע והחיתוך באמצעות נוסחה ישירה או ספריית LinearRegression. בהמשך הספר נלמד שיטה כללית יותר: להתחיל ממשקולות כלשהן, לחשב את השגיאה, להעריך כיצד השגיאה משתנה כשמשנים משקולות, ולעדכן אותן בכיוון שמקטין אותה. זהו הרעיון שמאחורי Gradient Descent, ומשם נמשיך לכלל השרשרת ולאימון רשתות עצביות.

$$w_{new} = w_{old} - \alpha \cdot (\partial L / \partial w)$$

להמחשה בלבד ניקח שלוש דוגמאות שבהן $x=(1,2,3)$ ו־$y=(2,4,6)$, ונגביל את המודל ל־$\hat{y}=wx$ (ללא b). אם מתחילים במשקל 1, סכום ריבועי השגיאות הוא 14, ולכן MSE הוא 14 חלקי 3. הנגזרת של MSE לפי המשקל היא מינוס 28 חלקי 3. בקצב למידה 0.05 מתקבל משקל מעודכן של כ־1.4667; הוא מקרב אותנו אל המשקל האופטימלי, שהוא 2.

$$w_{new} = 1 - 0.05 \cdot (-28/3) \approx 1.4667$$



איור: הנוסחה מתארת צעד קטן בכיוון ירידת השגיאה; זהו גשר לפרק האופטימיזציה הבא.

חשוב: גזירת הנגזרת וניתוח קצב הלמידה יילמדו בהרחבה בפרקי המתמטיקה ללמידה עמוקה.
המטרה כאן היא להבין מדוע המושג ״משקל״ ברגרסיה יחזור גם בפרספטרון וברשת עצבית.

20. טעויות נפוצות — כיצד מזהים ומתקנים אותן?

טעות שכיחה	למה היא טעות?	תיקון מומלץ
לקרוא לתחזית $\hat{y}$ ״התוצאה האמיתית״	המודל מחזיר אומדן	להבחין בכל טבלה בין y לבין $\hat{y}$
לחשוב שהישר עובר בכל הנקודות	רגרסיה מתאימה מגמה כללית	לשרטט שאריות
למסור X חד־ממדי ל־sklearn	הספרייה מצפה למטריצת מאפיינים	להשתמש ב־reshape(-1,1) או [[x]]
לדווח R^2 אימון כתוצאה סופית	אין הערכת הכללה	לשמור Test חיצוני
לפרש שיפוע כקשר סיבתי	מתאם אינו הוכחת סיבתיות	לדון במשתנים שלא נמדדו
לפרש R^2 שלילי כשגיאת קוד	ייתכן מודל גרוע מבסיס הממוצע	לחשב SSE ו־SST במפורש

21. מפת הקשרים לשלוש תכניות הלימוד

התוצר הנדרש	מה כבר ידענו / מה נוסף	מסלול
להסביר תחזית, שארית ומגבלה	גרף פיזור, X/y, מודל רגרסיה בסיסי ומדדים	יחידה 3 — כיתה י׳
להבין את התשתית המתמטית; רגרסיה היא גשר ולא פרק חובה מזוהה במפה	וקטורים, מכפלה פנימית, מדדי מרכז, מתאם והכללה	יחידה 5 — נושאי ליבה
להתאים מודל, לפרש מקדמים, להעריך ולתעד החלטות	רגרסיה עם מאפיין אחד ורבים, sklearn, ניסויים ו־R^2	למידה עמוקה — מסלול התמחות

22. תרגול מדורג — לפני המעבדה

רמה א׳ — הבנה וחישוב

1. עבור $\hat{y} = 4x + 12$ חשבו תחזיות ל־$x=2,5,8$. הסבירו את משמעות השיפוע ואת מגבלת פירוש החיתוך.

2. נתונות התוצאות $y=(10,13,14)$ והתחזיות $\hat{y}=(11,11,15)$. חשבו שארית לכל דוגמה וציינו מתי הייתה הערכת יתר.

3. הסבירו במילים מדוע MAE ומדד MSE אינם שווים בהכרח, אף ששניהם מבוססים על אותן שאריות.

4. תלמיד הזין למודל sklearn את $X=[1,2,3,4]$. מדוע עלולה להתקבל שגיאה ואיך תתקנו את צורת המערך?

רמה ב׳ — חקר ופרשנות

5. ניתנו $x=(1,2,3,4)$ ו־$y=(4,4,7,9)$. חשבו ממוצעים, שיפוע וחיתוך בשיטת OLS, ואז חשבו שתי תחזיות.

6. השוו שתי קבוצות תחזיות באמצעות MAE ו־MSE והציעו מתי חשוב במיוחד להעניש שגיאות גדולות.

7. קיבלתם $R^2=0.91$ ו־MAE=3.2 על Train, אך MAE=13 על Test. הציעו שני הסברים אפשריים ובדיקה נוספת.

8. התבוננו בגרף פיזור עם קשר מעוקל: האם רגרסיה ליניארית תהיה תמיד חסרת ערך? נמקו מתי היא קירוב סביר ומתי לא.

רמה ג׳ — שאלות מקוריות בסגנון בגרות ובהגנת פרויקט

9. נתונה טבלת ארבע דוגמאות, שני מאפיינים וערך מטרה. כתבו את ממדי X ואת אורך y, ואת צורת הקלט לחיזוי דוגמה חדשה אחת.

10. מודל א׳ מניב $R^2=0.72$ על Test ומודל ב׳ מניב $R^2=-0.4$. הסבירו מה מותר ומה אסור להסיק מההשוואה.

11. תלמיד טוען שהמודל שלו ״מדויק ב־95%״ משום ש־$R^2=0.95$. הסבירו מה שגוי בניסוח והציעו ניסוח מדעי מתאים.

12. בפיתוח מערכת מחירי דירות, תלמיד הוסיף כמאפיין את ״מחיר המכירה הסופי לאחר העסקה״ כדי לנבא את מחיר המכירה. זהו את הבעיה והציעו תיקון.

13. תכננו ניסוי המשווה מודל שטח בלבד למודל שטח+חדרים. פרטו את מדדי ההצלחה, חלוקת הנתונים והסיכון להסקת מסקנות מטעות.

14. הסבירו כיצד הנוסחה $\hat{y}=w\cdot x+b$ מופיעה הן ברגרסיה והן כשלב חישוב לפני פונקציית הפעלה של נוירון.

23. משימת יישום לקראת פרויקט הגמר

בחרו שאלה כמותית מעולם העניין שלכם: חיזוי משך טיפול בקריאת שירות, צריכת אנרגיה, זמן הגעה או מחיר מוצר. כתבו כרטיס פרויקט קצר הכולל: מהו y, אילו מאפיינים יהיו זמינים בזמן החיזוי, מקור הנתונים והזכויות להשתמש בהם, כיצד תחלקו Train/Test, מהו קו הבסיס, מהם שני המדדים שתשוו, ואילו בעיות פרטיות או הטיה עלולות לעלות. אל תתחילו מהקוד: קודם כתבו את שאלת המחקר ואת הקריטריונים להצלחה.

24. סיכום מושגים

מושג	הגדרה במשפט אחד
רגרסיה	ניבוי משתנה מטרה כמותי
שיפוע w	שינוי בתחזית ליחידת מאפיין אחת, בהינתן יתר המשתנים
חיתוך b	התחזית בנקודת המוצא של כל המאפיינים
שארית	הערך האמיתי פחות התחזית
OLS	בחירת מקדמים הממזערים סכום ריבועי שאריות
MAE	ממוצע גדלי השגיאות
MSE	ממוצע ריבועי השגיאות
RMSE	שורש MSE ביחידות משתנה המטרה
R^2	השוואת SSE לסכום סטיות מ־ממוצע ערכי הבדיקה
Baseline	תחזית פשוטה שמולה משווים את ערך המודל

25. מקורות והערות עריכה

עוגני משרד החינוך וההשתלמות: מפת התמחות למידת מכונה ולמידה עמוקה — פרק 5, 15 שעות; מצגות ההשתלמות 4.1 ו־4.2; מחברות Iris ו־KC House כפי שסוכמו במאגר GitHub של הקורס. תוכן הפרק הוא עיבוד דידקטי מקורי; אינו העתקה של מצגות או ספרים מסחריים. אין להסיק מהרחבות הנגזרות והבדיקות המתקדמות שהן דרישות חובה לבגרות ביחידה 5.

מקורות מקצועיים להעמקה: Aurélien Géron — Hands-On Machine Learning; James, Witten, Hastie, Tibshirani & Taylor — An Introduction to Statistical Learning with Applications in Python, פרק Linear Regression; Deisenroth, Faisal & Ong — Mathematics for Machine Learning; Luis Serrano — Grokking Machine Learning; התיעוד הרשמי של scikit-learn ול־LinearRegression ולמדדי רגרסיה.

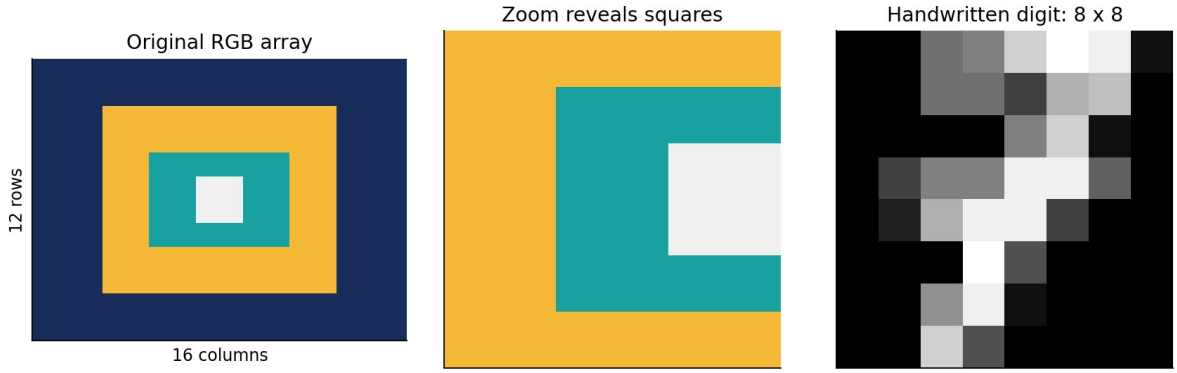
קישורים: https://www.statlearning.com/ ; https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LinearRegression.html ; https://scikit-learn.org/stable/modules/model_evaluation.html ; https://github.com/samiabo75/Data-Science-ML-Course/tree/main/03-Deep-Learning-5-Units/05-Linear-Regression

מדעי הבינה המלאכותית

התמחות למידת מכונה ולמידה עמוקה | פרק 6 בתכנית ההתמחות

עיבוד תמונה — מפיקסל בודד ועד לייצוג תמונה במודל AI

מהדורת תלמיד מורחבת · הסברים דידקטיים · נוסחאות וחישובים ידניים · 12 המחשות מקוריות · Python · תרגול עצמאי



איור: התמונה שאנחנו רואים כעצם שלם שמורה במחשב כמערך של ערכי פיקסלים. גם ספרות בכתב יד הן מערכים מספריים.

מיקום בסילבוס: פרק 6 — עיבוד תמונה, 15 שעות, בתכנית ההתמחות בלמידת מכונה ולמידה עמוקה. שיוך זה נבדק מול מפת הסילבוס השמורה במאגר הלימוד. רצף ההוראה והניסויים שלהלן הם עיבוד דידקטי מקורי; אין בידינו ברגע זה אישור שכל תת־נושא הוא דרישה פורמלית נפרדת בבחינה.

מה תדעו לעשות לאחר לימוד הפרק?

להסביר מהו פיקסל וכיצד מיוצגת תמונה כמערך; לזהות גובה, רוחב וערוצי צבע; לחשב ידנית פעולות על פיקסלים; לשנות בהירות וניגודיות בלי גלישה מטווח הערכים; לקרוא היסטוגרמת עוצמות; להבין חיתוך, שינוי גודל וסיכון לאיבוד מידע; לחשב מסנן ממוצע על חלון בגודל 3×3; לזהות את תפקיד סינון הקצוות; לכתוב קוד NumPy ו־Pillow; לבנות מעבדה משחזרת ולתאר כיצד קונבולוציה משמשת בסיס ל־CNN.

מפת דרכים לפרק

בעיה אמיתית → פיקסלים ומטריצות → צבע ותמונה אפורה → קואורדינטות ו־dtype → פעולות על תמונה → היסטוגרמה → חיתוך ושינוי גודל → רעש ומסננים → קונבולוציה → מבוא ל־CNN → הערכה, הטיות ופרויקט גמר.

1. סיפור הפתיחה — כיצד מכונה יכולה לקרוא כתב יד?

דודו בונה יישומון לתלמידים: מצלמים מספר שנכתב על דף, והיישומון מזהה את הספרה. לנו קל להבין שתמונה מציגה את הספרה 7, אך למחשב אין מושג טבעי של ״ספרה״ או ״קו״. הוא מקבל אוסף ערכים מספריים. כדי להגיע למערכת שמזהה כתב יד, עלינו להבין קודם מה נשמר בקובץ תמונה, כיצד משנים את הערכים וכיצד אפשר להפיק מהם דפוסים משמעותיים.

נניח שהמצלמה צילמה מספר בתאורה חלשה. התלמיד רואה ספרה כהה, אבל מכונה עלולה לקבל ערכים שההבדל בינם לבין הרקע קטן. לפני שנבנה מסווג, נצטרך לדעת לבדוק בהירות, להתאים גודל, להסיר רעש ולשמור על המידע החשוב. עיבוד תמונה אינו ״קישוט״: הוא עשוי לשנות את הקלט ואת איכות החיזוי של המודל.

שימו לב: שלב עיבוד התמונה יכול להתבצע בעזרת כללים קבועים שהמתכנת הגדיר, גם בלי למידת מכונה. רשת CNN תלמד בהמשך מסננים מתאימים מתוך נתוני האימון; ההבחנה הזאת היא אחת ממטרות הפרק.

בדיקת הבנה: אם נבהיר תמונה כהה, האם נוסיף מידע חדש לתמונה או נשנה את אופן הייצוג של המידע שכבר קיים?

2. מהו פיקסל ומהי רזולוציה?

פיקסל הוא יחידת מדידה במבנה של תמונה דיגיטלית. בתמונה אפורה, כל תא במטריצה מחזיק מספר המתאר עוצמת אור או בהירות. בתמונה צבעונית, לכל מיקום קיימים בדרך כלל כמה ערוצים — למשל אדום, ירוק וכחול. כאשר מציגים את הערכים יחד על מסך, העין שלנו רואה תמונה רציפה, אך במבט מוגדל אפשר להבחין בריבועים הקטנים.

רזולוציה מתארת את מספר הפיקסלים: גובה H כפול רוחב W. תמונה ברוחב 16 פיקסלים ובגובה 12 פיקסלים מכילה 192 מיקומי פיקסלים. אם זו תמונת RGB בת שלושה ערוצים, אפשר לייצג את הצבעים באמצעות 576 ערכי ערוץ. ככל שיש יותר פיקסלים אפשר לשמור פרטים מרחביים קטנים יותר, אך נדרש גם יותר זיכרון וחישוב.

$$N_{pixels} = H \times W \quad | \quad N_{RGB_values} = H \times W \times 3$$

נתון	גובה H	רוחב W	מספר פיקסלים	ערכי RGB
תמונה קטנה	12	16	192	576
תמונה רגילה	32	32	1,024	3,072
קלט גדול	224	224	50,176	150,528

הדוגמה האחרונה גם מבהירה למה קלט תמונה עשוי להיות יקר יותר מקלט טבלאי: במקום כמה מאפיינים, המודל מקבל מאות אלפי מספרים. המספרים אינם בלתי תלויים: פיקסלים סמוכים נוטים להשתייך לאותו קו, קצה או מרקם.

בדיקת הבנה: אם נכפיל את הגובה ואת הרוחב פי שניים, פי כמה יגדל מספר הפיקסלים?

3. תמונה אפורה היא מטריצה

נתחיל מתמונה בגווני אפור. נהוג להשתמש ב־uint8, סוג נתונים של מספר שלם בלתי שלילי שמיוצג בשמונה סיביות. לכן כל ערך יכול לנוע בין 0 ל־255: 0 הוא שחור, 255 הוא לבן, וערכי הביניים הם גוונים שונים של אפור. לא כל תמונה נשמרת כך: קיימות תמונות עם עומק צבע אחר, עם ערכים עשרוניים ועם קידוד אחר. אנו משתמשים ב־uint8 כדי להכיר את העיקרון בצורה ברורה.

עמודה 0	עמודה 1	עמודה 2	עמודה 3
0	0	0	0
0	80	160	0
0	160	255	0
0	0	0	0

כדי להציג את התמונה, הספרייה מתרגמת כל מספר לגוון מתאים. שימו לב שהערכים שבתוך המטריצה הם הנתונים עצמם, בעוד שצבעי התרשים הם דרך ההצגה שלהם. שינוי מפת הצבעים בתצוגה אינו בהכרח משנה את המטריצה.

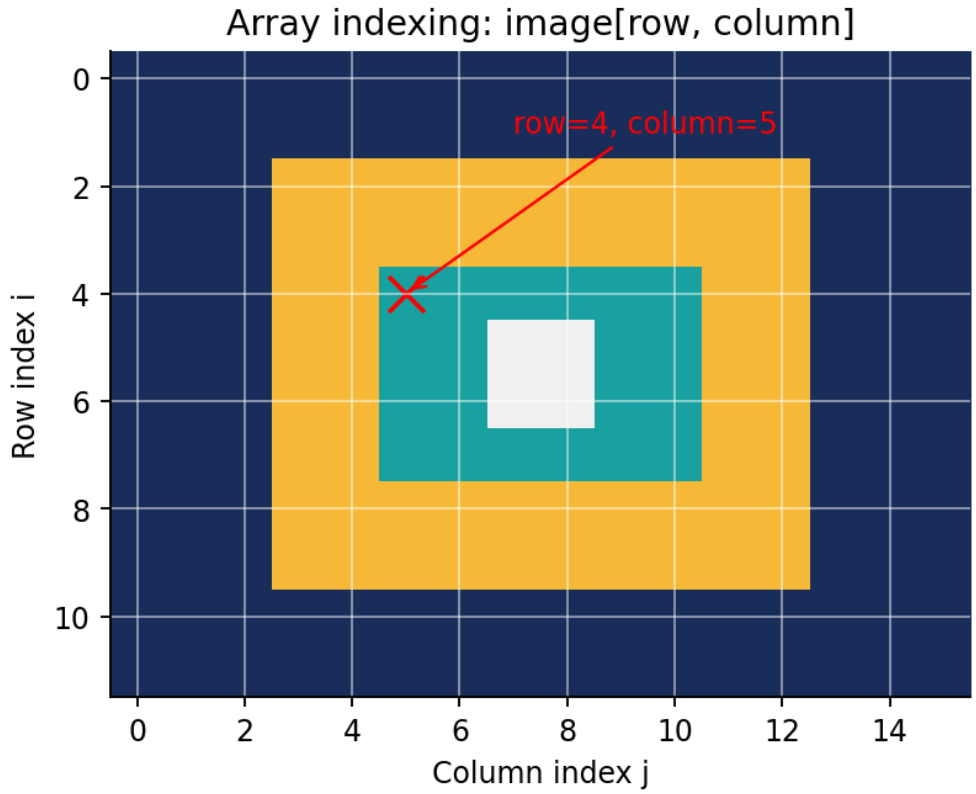
```
import numpy as np
import matplotlib.pyplot as plt
pixels = np.array([[0, 0, 0, 0],
                   [0, 80, 160, 0],
                   [0, 160, 255, 0],
```

```
                  [0, 0, 0, 0]], dtype=np.uint8)
print(pixels.shape)   # (4, 4)
plt.imshow(pixels, cmap="gray", vmin=0, vmax=255)
plt.show()
```

חשוב לזכור: מטריצה היא ייצוג של התמונה, אך אינה יודעת בעצמה מה מופיע בה. פירוש הסימנים שבתמונה הוא כבר משימה ברמה גבוהה יותר.

4. היכן נמצא פיקסל? שורות, עמודות ואינדקסים

בעבודה עם NumPy מציינים קודם שורה ואחר כך עמודה: image[row, column]. השורה הראשונה היא שורה 0; העמודה הראשונה היא עמודה 0. ציר השורות מתקדם בדרך כלל מלמעלה למטה בתמונה, וציר העמודות משמאל לימין. הדבר שונה במידת מה מהאינטואיציה של מערכות צירים במתמטיקה, שבהן ציר y נוטה לעלות כלפי מעלה.



איור: אינדקס של פיקסל: תחילה מספר שורה ואז מספר עמודה. נקודת (4,5) כאן מייצגת שורה 4 ועמודה 5.

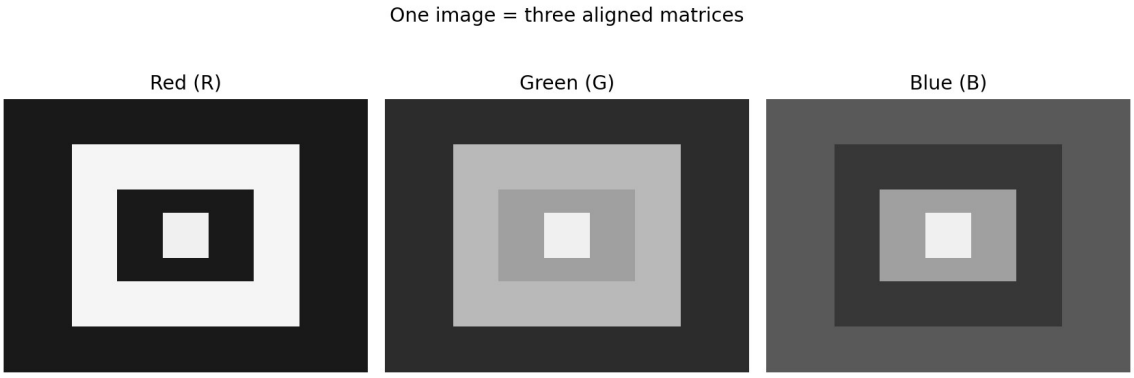
```
rgb = np.zeros((12, 16, 3), dtype=np.uint8)
rgb[:] = [25, 45, 90]
rgb[2:10, 3:13] = [245, 185, 55]
rgb[4:8, 5:11] = [25, 160, 160]
pixel = rgb[4, 5]       # three channel values
red_value = rgb[4, 5, 0]
print(rgb.shape)         # (12, 16, 3)
```

במרבית הפעולות נרצה לשנות אזור שלם ולא תא אחד. בעזרת חיתוך מערך אפשר לבחור טווח שורות וטווח עמודות. הביטוי image[2:8, 3:12] כולל שורות 2 עד 7 ועמודות 3 עד 11, מפני שהגבול העליון אינו נכלל בחיתוך.

בדיקת הבנה: בתמונה שממדיה 12×16, האם image[12,0] חוקי? ומהו אינדקס השורה האחרון?

5. תמונה צבעונית: שלושה ערוצי RGB

בייצוג RGB שומרים לכל פיקסל שלושה ערכים: עוצמת האדום (Red), הירוק (Green) והכחול (Blue). שלושת הערכים יוצרים צבע אחד על המסך. לדוגמה, RGB=(255,0,0) ייתן אדום מלא, RGB=(0,255,0) ירוק מלא, ו־RGB=(255,255,255) לבן. מערך RGB סטנדרטי ב־NumPy בנוי בגודל H×W×3.



איור: כל ערוץ צבע הוא מטריצה דו־ממדית נפרדת. שילוב שלושת הערוצים יוצר את התמונה הצבעונית.

חשוב להבחין בין גודל התמונה לבין מספר הערכים בכל פיקסל. תמונה בגודל 12×16 RGB עדיין מכילה 192 מיקומי פיקסלים, אך כל מיקום כולל שלושה מספרים. כאשר מנתחים משקל קובץ צריך גם לזכור שקובצי PNG/JPEG משתמשים בדחיסה, ולכן מספר הערכים בזיכרון אינו בהכרח שווה לגודל הקובץ על הדיסק.

צבע	R	G	B
אדום	255	0	0
ירוק	0	255	0
כחול	0	0	255
צהוב	255	255	0
אפור בינוני	128	128	128

```
from PIL import Image
color_image = Image.fromarray(rgb)
print(np.asarray(color_image).shape)
red_channel = np.asarray(color_image)[:, :, 0]
plt.imshow(red_channel, cmap="gray", vmin=0, vmax=255)
plt.show()
```

חשוב לזכור: ב־Pillow וב־Matplotlib הצבעים מטופלים לרוב כסדר RGB. פונקציות מסוימות ב־OpenCV משתמשות ב־BGR; בעת מעבר בין ספריות חובה לבדוק את סדר הערוצים.

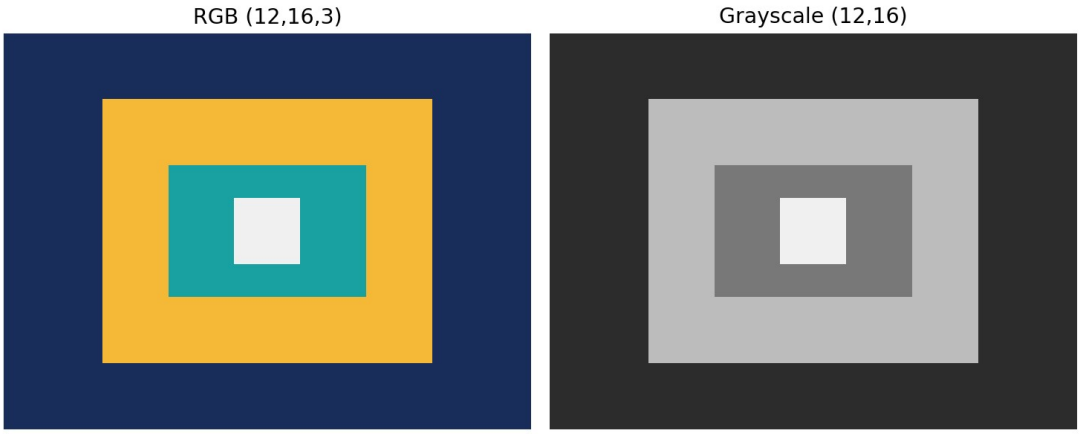
6. המרה לגווני אפור: מדוע לא עושים רק ממוצע פשוט?

לעיתים נרצה לצמצם תמונה צבעונית לערוץ אחד, למשל כדי להתמקד בקווי מתאר של ספרה. אפשר היה לחשב ממוצע רגיל של R,G,B, אך העין האנושית אינה רגישה לכל שלושת הצבעים באותה מידה. לכן מקובל לחשב קירוב לבהירות בעזרת משקולות שונות לערוצים. זו אחת משיטות ההמרה המקובלות, ואינה כל השיטות האפשריות.

$$\text{Gray} \approx 0.299R + 0.587G + 0.114B$$

נחשב ידנית פיקסל שערכיו R=200, G=100, B=50. התרומה של האדום היא 59.8, של הירוק 58.7 ושל הכחול 5.7. הסכום הוא 124.2, ולכן אם נדרוש ערך שלם נקבל בערך 124. לעומת זאת, ממוצע פשוט של שלושת הערכים הוא 116.67. ההבדל ממחיש שהמרה לגווני אפור היא כלל חישוב ולא היעלמות קסומה של הצבע.

0.299×200 + 0.587×100 + 0.114×50 = 124.2



איור: אותה תמונה ב־RGB ובגווני אפור: הממדים עוברים מ־$H \times W \times 3$ ל־$H \times W$.

```
rgb_float = rgb.astype(np.float32)
gray = 0.299 * rgb_float[:, :, 0]      + 0.587 * rgb_float[:, :, 1]      + 0.114 *
rgb_float[:, :, 2]
gray_u8 = np.clip(np.rint(gray), 0, 255).astype(np.uint8)
print(gray_u8.shape)
```

בדיקת הבנה: מדוע המרה לגווני אפור עלולה לפגוע במסווג שמבחין בין תפוחים אדומים לירוקים?

7. בהירות וניגודיות — לשנות פיקסלים בצורה מבוקרת

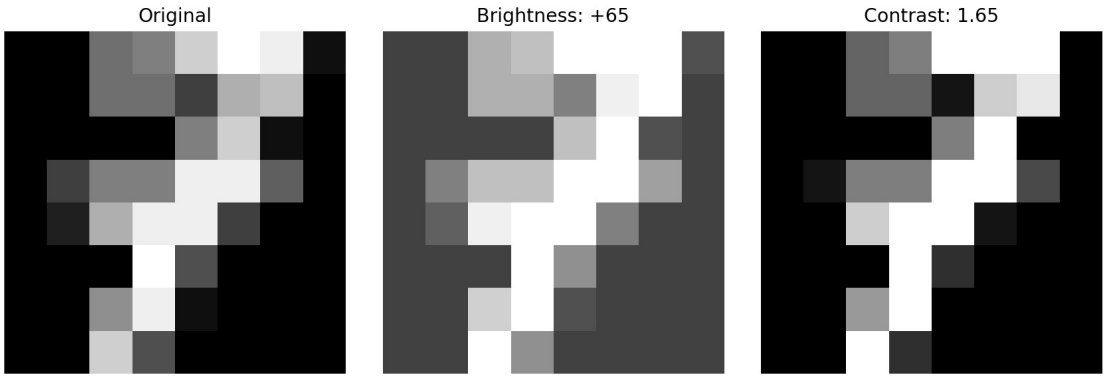
בהירות מתארת את הרמה הכללית של ערכי העוצמה; ניגודיות מתארת את ההבדל בין אזורים בהירים לכהים. אם תמונה חשוכה מדי אפשר לנסות להוסיף לכל ערך קבוע חיובי. אבל אם הפיקסל היה כבר 240 ונוסיף לו 40, התוצאה 280 חורגת מתחום 0–255. אין לשמור אותה ישירות כ־uint8, מפני שפעולות על מספרים בתחום מוגבל עלולות לגרום לגלישה או לליפוף ערכים במקום לרוויה לבנה. לכן מחשבים תחילה בסוג מספרי רחב וחותכים לטווח התקין.

brightness: I_new = clip(I_old + β, 0, 255)

פיקסל מקורי	תוספת β	תוצאה לפני clip	תוצאה סופית
60	40	100	100
220	40	260	255
0	40	40	40

ניגודיות אפשר לשנות באמצעות מתיחה של המרחק מהערך 128: ככל שמקדם α גדול מ־1, ערכים שמעל 128 מתרחקים כלפי מעלה וערכים שמתחתיו מתרחקים כלפי מטה. כאשר α בין 0 ל־1 ההבדלים נחלשים. נצטרך תמיד לחתוך את התוצאות לתחום החוקי. הגדלת ניגודיות עשויה להבליט קווים, אך עלולה גם לאבד פרטים אם אזורים רבים מגיעים ל־0 או ל־255.

contrast: I_new = clip(α(I_old − 128) + 128, 0, 255)



איור: אותה ספרה מוצגת עם בהירות וניגודיות שונים; התמונה נשמרת כמערך, אך הערכים משתנים.

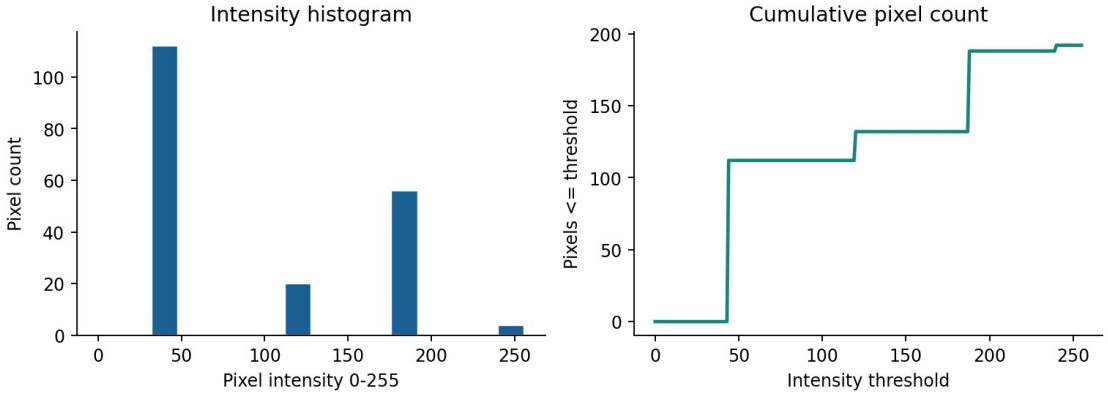
```
from sklearn.datasets import load_digits
digit8 = np.rint(load_digits().images[7] * 255 / 16).astype(np.uint8)
img_float = digit8.astype(np.float32)
brighter = np.clip(img_float + 65, 0, 255).astype(np.uint8)
stronger = np.clip(1.65 * (img_float - 128) + 128,
                   0, 255).astype(np.uint8)
```

חשוב לזכור: חשוב לשמור עותק של התמונה המקורית, כדי שאפשר יהיה לשחזר ולבדוק את השפעת כל שינוי בנפרד.

8. היסטוגרמת בהירות — לקרוא את התפלגות הפיקסלים

היסטוגרמה של תמונת אפור סופרת כמה פיקסלים נמצאים בכל טווח עוצמות. כאשר רוב הפיקסלים נמצאים בטווח נמוך, ייתכן שהתמונה כהה. כאשר הם מרוכזים בטווח גבוה, ייתכן שהיא בהירה. עם זאת, המשמעות תלויה בתוכן התמונה: תמונת לילה אמורה להכיל פיקסלים כהים רבים ואין בכך בהכרח תקלה.

נניח שלתמונה קטנה שישה פיקסלים: 0,0,50,50,50,255. ערך 0 מופיע פעמיים, 50 שלוש פעמים ו־255 פעם אחת. היסטוגרמה היא סיכום של עוצמות ולא של מיקומים. שתי תמונות שונות עשויות לקבל בדיוק אותה היסטוגרמה אם סידרנו מחדש את הפיקסלים, אף שהן מציגות צורות אחרות לחלוטין.



איור: היסטוגרמה סופרת עוצמות, אך אינה שומרת מידע על מיקום הפיקסלים.

```
pixel_values = gray_u8.ravel()
plt.hist(pixel_values, bins=np.arange(0, 257, 16),
         edgecolor="white")
plt.xlabel("Pixel intensity")
plt.ylabel("Count")
plt.show()
```

בדיקת הבנה: האם היסטוגרמה זהה של שתי תמונות מוכיחה שהתמונות זהות? הציגו דוגמה נגדית.

9. חיתוך תמונה ושינוי גודל — מדוע סדר הפעולות חשוב?

חיתוך (Crop) בוחר רק אזור מסוים מתוך התמונה. בתמונה עם מספר ועם רקע גדול מסביב, חיתוך יכול להסיר מידע שאינו מועיל ולהשאיר את הספרה במרכז. אבל חיתוך אגרסיבי עלול לחתוך גם חלק מהספרה. שינוי גודל (Resize) מתאים את ממדי התמונה לגודל הקלט הנדרש במודל, למשל 28×28. פעולה זו אינה מוסיפה מחדש פרטים שנעלמו בצילום המקורי.

שינוי הגודל מחייב כלל להערכת הערכים החדשים. באינטרפולציה הקרובה ביותר בוחרים ערך של פיקסל סמוך; בשיטה ביליניארית מחשבים שילוב משוקלל של כמה שכנים. לכן אותה תמונה יכולה להיראות משוננת או חלקה יותר לפי שיטת שינוי הגודל. חשוב לשמור על יחס רוחב־גובה, או להחליט במודע אם מוסיפים שוליים (Padding), כדי שלא לעוות את האובייקט הנלמד.



איור: אותו מערך קטן לאחר הגדלה בשתי שיטות: הפרטים החדשים הם אומדנים ולא מידע שנמדד במצלמה.

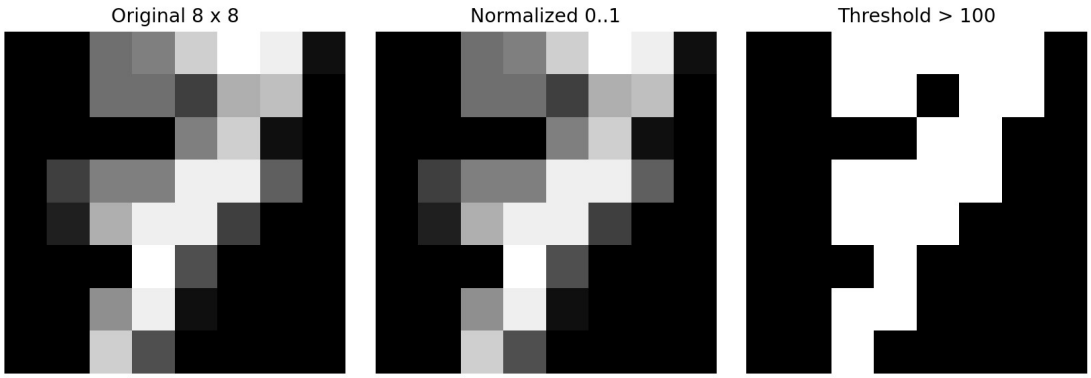
```
from PIL import Image
im = Image.fromarray(rgb)
crop = im.crop((3, 2, 13, 10))  # left, top, right, bottom
resized = im.resize((80, 60), Image.Resampling.BILINEAR)
print(np.array(crop).shape)
print(np.array(resized).shape)
```

חשוב לזכור: שימו לב להבדל: בחיתוך NumPy מסדרים [שורות, עמודות], ואילו Pillow.crop מקבל (left, top, right, bottom).

10. נרמול עוצמות — מ־0 עד 255 לטווח 0 עד 1

מודלים מסוימים נוחים יותר לאימון כאשר ערכי הקלט נמצאים בטווח מוגבל ומוכר. שינוי פשוט הוא חלוקת כל ערכי הפיקסלים ב־255. הפעולה אינה משנה את הסדר היחסי של הערכים ואף אינה משפרת אוטומטית את איכות התמונה; היא רק משנה את יחידות המדידה. פיקסל שהיה 128 יהפוך ל־0.50196 בקירוב, ופיקסל שהיה 255 יהפוך ל־1.

I_normalized = I_uint8 / 255.0



איור: הדגמה של אותם פיקסלים כערכי 0–255, כערכים עשרוניים בטווח 0–1 וכסף בינארי.

סף בינארי (Thresholding) הוא פעולה אחרת לחלוטין: למשל לקבוע שכל ערך מעל 100 יהפוך ל־1, וכל ערך אחר ל־0. הסף יכול לסייע בהפרדת ספרה מרקע, אבל הוא מוחק את רוב המידע על רמות הבהירות. לכן אין לבלבל בין נרמול לבין בינאריזציה.

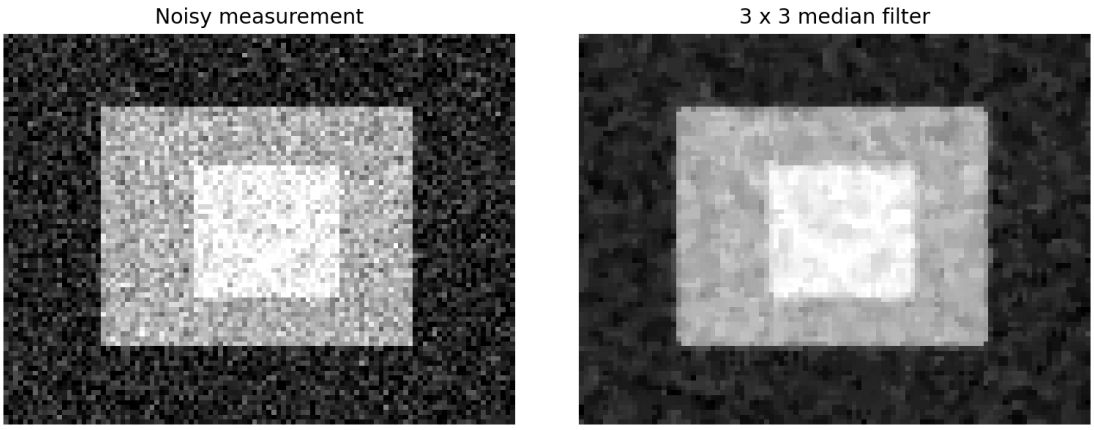
```
normalized = digit8.astype(np.float32) / 255.0
binary = (digit8 > 100).astype(np.uint8)
print(normalized.min(), normalized.max())
print(np.unique(binary))
```

חשוב לזכור: חלוקה ב־255 היא שינוי קבוע שאינו לומד פרמטר מנתוני האימון. לעומת זאת, תקנון לפי ממוצע וסטיית תקן מחייב התאמת הפרמטרים רק על Train, כדי למנוע דליפת מידע.

11. רעש — מה הוא עושה לתמונה?

בצילום אמיתי מופיעים לפעמים הפרעות: הבדלי תאורה, גרעיניות, נקודות בודדות או פיקסלים שאינם מייצגים היטב את המציאות. נוח לחשוב על רעש כהפרעה שמתווספת לאות הרצוי, אך לא כל שינוי קטן הוא רעש — לעיתים זהו פרט חשוב, כגון נקודה מעל אות או קצה של ספרה.

כדי להמחיש זאת נייצר תמונה פשוטה ונוסיף רעש אקראי. אפשר לנסות להקטין רעש באמצעות חציון מקומי: לוקחים את ערכי הסביבה, ממיינים ובוחרים את הערך האמצעי. מסנן חציון מועיל במיוחד לרעש של נקודות חריגות, אבל אינו מתאים בהכרח לכל סוג רעש. חשוב להשוות לתמונה המקורית ולוודא שלא נמחקו פרטים רלוונטיים.



איור: מסנן חציון מטפל ברעש מקומי, אך לעיתים גם מסיר פרטים קטנים.

בדיקת הבנה: אם מסנן מסיר נקודה מיותרת אך גם מסיר נקודה השייכת לאות ״ו״ — האם שיפרנו את התמונה? לפי איזו מטרה צריך להחליט?

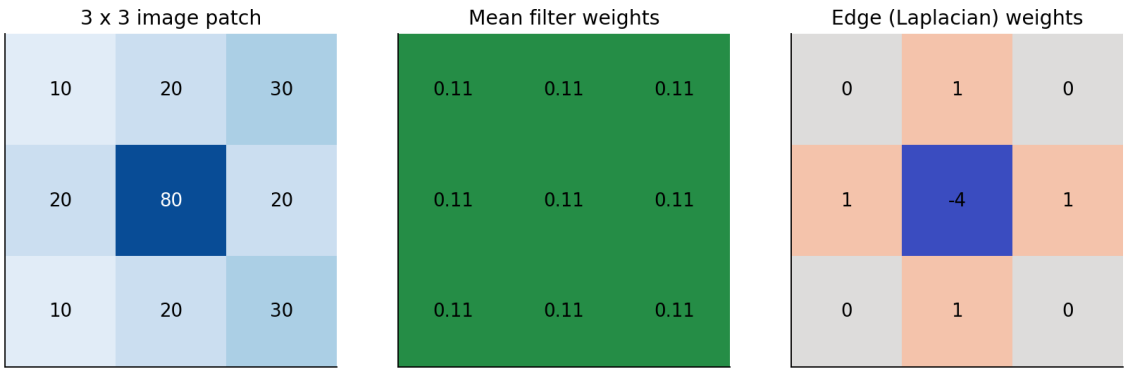
12. חלון שכנים ומסנן ממוצע — חישוב ידני

כעת נגיע לרעיון מרכזי בעיבוד תמונה: פיקסל אינו בהכרח מנותח לבדו. לעיתים נרצה לחקור את סביבתו הקרובה בעזרת חלון קטן, למשל שלוש שורות ושלוש עמודות. מסנן ממוצע מחליף את ערך הפיקסל המרכזי בממוצע של תשעת הערכים שבחלון. התוצאה נוטה להיות חלקה יותר, כי הבדלים חדים בין שכנים מצטמצמים.

10	20	30
20	80	20
10	20	30

בחלון שלפנינו סכום הערכים הוא 240. מחלקים בתשע מפני שיש בו תשעה תאים, ומקבלים 26.67 בקירוב. כלומר הפיקסל המרכזי שערכו 80 מקבל ערך חדש הקרוב ל־27. מה הרווחנו? שיטחנו שינוי מקומי חד. מה איבדנו? ייתכן שערך 80 היה סימן חשוב ולא רעש.

mean(3×3 patch) = 240 / 9 ≈ 26.67



איור: חלון 3×3, מסנן ממוצע ומשקולות מסנן להדגשת שינויים.

מסנן ממוצע הוא מקרה פשוט של פעולה משוקללת. אם לכל תא במשקל יש ערך 1/9, מכפילים כל פיקסל במשקל המתאים ומחברים. אפשר לבחור משקולות אחרות כדי להדגיש קצוות, לטשטש או לבצע פעולות נוספות.

```
patch = np.array([[10,20,30],
                  [20,80,20],
                  [10,20,30]], dtype=float)
weights = np.ones((3,3)) / 9
result = np.sum(patch * weights)
print(round(result, 2))   # 26.67
```

13. קונבולוציה ומסננים: הרעיון המתמטי

קונבולוציה היא פעולה שבה מעבירים מסנן קטן על פני מערך ומחשבים בכל מיקום סכום של מכפלות בין ערכי הסביבה למשקולות המסנן. כדי להבין את המנגנון, דמיינו חלון של 3×3 שמונח על תשעה פיקסלים. בכל צעד מכפילים תא מול תא, מחברים את תשע המכפלות ורושמים ערך בפלט; לאחר מכן מזיזים את החלון למיקום הבא.

$$S(i,j) = \Sigma_a \Sigma_b \; I(i+a, j+b) \cdot K(a,b)$$

הנוסחה שלפנינו היא למעשה פעולת Cross-Correlation, כפי שממומשת בדרך כלל בשכבות CNN. בקונבולוציה המתמטית הקלאסית יש גם היפוך של גרעין הסינון. להדגמת מסנן ממוצע סימטרי שתי הפעולות נותנות את אותה תוצאה; במסננים לא־סימטריים חשוב להבחין ביניהן. בפרק CNN נעסוק גם במשקולות שנלמדות מתוך נתונים.

מה נעשה בשוליים? חלון של 3×3 אינו יכול להיות ממורכז בפינה בלי לצאת מגבולות התמונה. אחת האפשרויות היא לחשב רק במיקומים שיש להם חלון מלא (Valid), ואז גובה ורוחב הפלט קטנים בשניים. אפשרות אחרת היא להוסיף שכבת שוליים מלאכותית (Padding), למשל אפסים, ולשמור על הגודל. הבחירה יכולה להשפיע על התוצאות ליד גבולות התמונה.

$$\text{valid 3×3 filter: } H_{out} = H - 2 \quad ; \quad W_{out} = W - 2$$

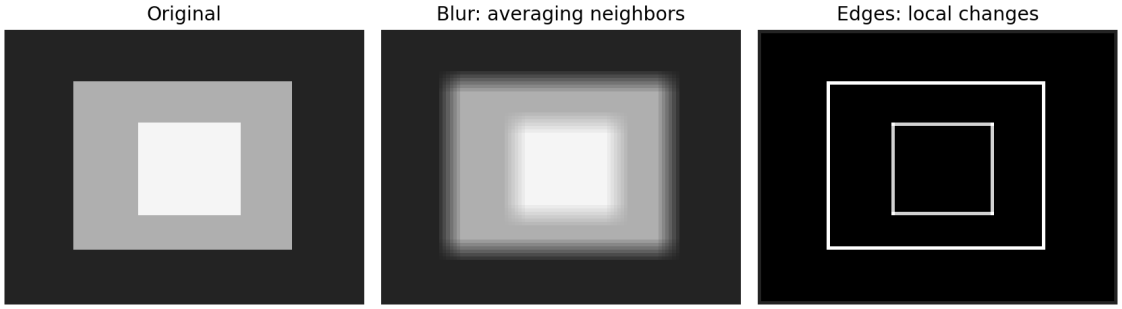
חשוב לזכור: כאשר stride = 1 וללא padding. בהמשך, עבור stride גדול יותר או padding אחר, נוסחת גודל הפלט משתנה.

14. זיהוי קצוות — מה אפשר ללמוד מהפרשים בין שכנים?

קצה בתמונה הוא אזור שבו ערכי הפיקסלים משתנים בחדות במרחב. למשל, במעבר מקיר כהה לחלון מואר נראה הבדל משמעותי בין פיקסלים סמוכים. מסננים מסוימים נועדו להבליט שינויים כאלה. קצה אינו ״שם של עצם״; הוא רק רמז מקומי שעשוי לסייע לזיהוי צורות.

נבחן מסנן סימטרי המדגיש הבדלים בין המרכז לארבעת שכניו הישירים. מקדמי השכנים הם 1, מקדם המרכז הוא מינוס 4, ובפינות מקדם 0. עבור חלון שבמרכזו 80 ובארבעת השכנים הישירים 20 נקבל 80 פחות 320, כלומר מינוס 240. לעיתים מציגים את גודל התגובה או ממירים אותה לטווח מותר לשם תצוגה. הסימן והערך המקורי עשויים להיות חשובים בחישוב.

response = 20+20+20+20 − 4×80 = −240



איור: טשטוש מפחית פרטים חדים; מסנן קצוות מדגיש שינויים מקומיים בעוצמה.

ב־CNN המסננים יכולים ללמוד להבליט דפוסים מתוך תמונות אימון, במקום שהמתכנת יבחר מראש את כל המשקולות. אולם תהליך הסכימה המקומית של ערכים כפול משקולות הוא אותו רעיון מרכזי.

15. מ־Pillow ו־NumPy אל ייצוג המתאים ללמידת מכונה

כאשר נטען תמונה ל־NumPy נקבל מערך מספרי. בשלב הבא יכול להיות שנרצה להפוך אותו לווקטור אחד ארוך, למשל 8×8 לאוסף של 64 תכונות. כך אפשר להפעיל מסווג רגיל כגון KNN. החיסרון הוא שהמיקום המרחבי פחות מפורש: פיקסלים צמודים בשורות שונות עלולים להתרחק במערך השטוח מבחינה אינדקסית, וקשרים מקומיים אינם נשמרים כשכבה נפרדת של תהליך הלמידה.

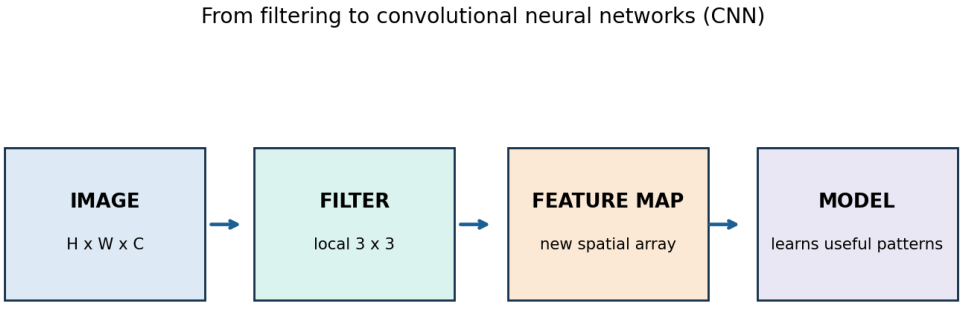
image shape (8,8) → flatten → feature vector length 64

```
from sklearn.datasets import load_digits
digits = load_digits()
X = digits.images.reshape(len(digits.images), -1)
y = digits.target
print(digits.images.shape)   # (n, 8, 8)
print(X.shape)               # (n, 64)
```

הספרייה load_digits מספקת דוגמאות אמיתיות של ספרות בכתב יד ברזולוציה של 8×8, עם ערכי עוצמה בין 0 ל־16. כדי להציגן בטווח 0–255 אפשר להכפיל ב־255/16, אבל זאת המרה לייצוג תצוגה; אין לבלבל בינה לבין העובדה שהערכים המקוריים של המאגר אינם שמונה־ביט מלאים.

בדיקת הבנה: מדוע תמונה של ספרה מתאימה גם ל־KNN וגם ל־CNN, אך לרשת CNN יש יתרון מבני בעבודה על שכונות פיקסלים?

16. גשר לפרק CNN: מסנן לומד לעומת מסנן קבוע



עד כה בחרנו ביד את משקולות המסננים: למשל כל משקל 1/9 לצורך מיצוע או מקדמים קבועים להדגשת קצוות. ברשת קונבולוציה, לעומת זאת, שכבות הקונבולוציה מכילות משקולות שנלמדות מן הנתונים בזמן האימון. בשכבות הראשונות עשויים להופיע מסננים המגיבים לקצוות או למרקמים; בשכבות עמוקות יותר יכולים להיווצר ייצוגים מורכבים. זהו תיאור רעיוני, לא הבטחה שכל רשת תלמד בדיוק אותו סדר של תכונות.

דימוי מועיל הוא ההבדל בין מסננת קבועה לבין כלי שמכוונים את המשקולות שלו לפי טעויות החיזוי. כעת אנו לומדים את הפעולה על חלון קטן; בפרק הרשתות נלמד כיצד השגיאה משנה את המשקולות בעזרת תהליך אימון.

למידה עמוקה אינה מחליפה חשיבה על איכות הנתונים. גם רשת עוצמתית עלולה להיכשל אם התמונות אינן מייצגות מצבי תאורה שונים, אם יש טעויות בתוויות, או אם נתוני Test הושפעו מן התהליך שבו בנינו את המודל.

17. הנדסת ניסוי נכון — מה בודקים לפני שמשווים מודלים?

נניח שמודל A אומן על תמונות בהירות ומודל B על תמונות כהות שנוצרו באמצעות עיבוד שונה. אם גם הנתונים, גם סוג העיבוד וגם המודל השתנו במקביל, קשה לדעת מה גרם לשינוי בביצועים. לכן ניסוי הוגן משנה גורם אחד בכל פעם, משאיר את נתוני הבדיקה מופרדים ומדווח לא רק על ציון אחד אלא גם על סוגי הטעויות.

לצורך פרויקט גמר רצוי לתעד תהליך קבוע: מקור התמונות והרשאות השימוש, גודל הקלט, סדר ערוצי הצבע, המרה לאפור או שמירת RGB, שיטת Resize, טווח נרמול והדרך שבה חולקו הדוגמאות ל־Train ול־Test. אם אותה תמונת מקור צולמה כמה פעמים בזוויות דומות, כדאי לבדוק שלא פיזרנו צילומים כמעט זהים בין אימון לבדיקה ויצרנו הערכה אופטימית מדי.

תיעוד מוצע	שאלת בדיקה	שלב
מקור, רישיון, סיכונים	האם התמונות דומות למצבים בעולם האמיתי?	נתונים
H×W×C, RGB/gray	מהם גודל התמונה וסדר הערוצים?	קלט
Crop, Resize, Normalize	האם שינינו את המידע?	עיבוד
חלוקה, מקור, מניעת כפילות	האם נשמר Test עצמאי?	בדיקה
מטריצת בלבול ודוגמאות	היכן המודל נכשל?	תוצאה

18. התנסות מודרכת מלאה — תמונה, מערך וגרף

הקוד הבא יוצר תמונה קטנה מקורית, מציג את צורת המערך ומחשב תמונה אפורה. הוא אינו דורש הורדת קובץ חיצוני, ולכן ניתן להפעילו במחברת Colab חדשה. לפני ההרצה נסו לנבא מה יהיו ממדי המערכים.

```
import numpy as np
import matplotlib.pyplot as plt
from PIL import Image

rgb = np.zeros((12, 16, 3), dtype=np.uint8)
rgb[:] = [25, 45, 90]
rgb[2:10, 3:13] = [245, 185, 55]
rgb[4:8, 5:11] = [25, 160, 160]

r, g, b = [rgb[:, :, i].astype(float) for i in range(3)]
gray = np.clip(0.299*r + 0.587*g + 0.114*b,
               0, 255).astype(np.uint8)
```

```
print('RGB shape:', rgb.shape)
print('Gray shape:', gray.shape)
fig, ax = plt.subplots(1, 2, figsize=(7, 3))
ax[0].imshow(rgb)
ax[1].imshow(gray, cmap='gray', vmin=0, vmax=255)
for a in ax: a.axis('off')
plt.show()
```

לאחר ההרצה הסבירו איזו פעולה שינתה את מספר הערוצים, ואיזו פעולה רק הציגה אותם על המסך. את התשובה נסחו בעברית ולא רק באמצעות הדפסת הצורות.

19. משימות תרגול מדורגות

רמה א׳ — הבנת ייצוג ופעולות

1. תמונה אפורה בגובה 20 ורוחב 30: כמה פיקסלים יש בה? מהי צורת המערך? מה יהיה ההבדל אם היא תמונת RGB?

2. הציגו שלושה פיקסלים שערכיהם 0, 128, 255 והסבירו כיצד כל אחד ייראה בתמונה אפורה.

3. עבור הפיקסל (200,100,50), בצעו חישוב ידני של ההמרה לגווני אפור. הסבירו מדוע הממוצע הפשוט נותן תוצאה אחרת.

4. כתבו את טווח השורות והעמודות בביטוי image[1:4, 2:6]. כמה תאים ייבחרו?

5. קבעו אם שינוי colormap בעת plt.imshow משנה את נתוני התמונה או רק את התצוגה.

רמה ב׳ — חשיבה מתמטית וניתוח קוד

6. לפיקסלים 30, 210, 250 מוסיפים β=40. חשבו את הערכים לפני clip ואחריו והסבירו מדוע דרושה פעולת clipping.

7. קבלו חלון 3×3 שבו במרכז 90 וכל שמונת השכנים הם 10. מצאו ממוצע וחציון, והשוו מה כל מסנן יעשה לערך המרכזי.

8. תמונה בגודל 10×12 עוברת מסנן 3×3, ללא Padding ובּ־Stride של 1. מה יהיו ממדי הפלט? נמקו.

9. השוו בין Resize בשיטת Nearest Neighbor לבין Bilinear. איזה מידע לא ניתן לשחזר מתוך תמונה קטנה במיוחד?

10. התלמיד טוען ש־Normalization ו־Thresholding הן אותה פעולה. כתבו דוגמה מספרית הסותרת את הטענה.

רמה ג׳ — העברה לבגרות ולפרויקט גמר

11. מערכת לזיהוי ספרות מצליחה באימון ונכשלת בצילומים בתאורה חלשה. הציעו ניסוי מבוקר שיבדוק את השפעת עיבוד הבהירות בלי לשנות את נתוני Test לצורך בחירת הפרמטרים.

12. שתי קבוצות תמונות הן בעלות היסטוגרמת בהירות זהה, אך שונות במיקום קווי המתאר. הסבירו מדוע לא כדאי לבסס סיווג ספרות על היסטוגרמה בלבד.

13. הציעו מדוע רשת CNN שמקבלת תמונה דו־ממדית עשויה להיות עדיפה על מסווג רגיל המקבל וקטור שטוח, גם כשמספר ערכי הקלט זהה.

14. תארו שלושה פרטים שמוכרחים להופיע בתיעוד של שלב עיבוד התמונה בפרויקט גמר, ונמקו את חשיבותם.

15. תכננו בדיקת שגיאות למסווג תמונות: אילו מקרים תציגו בפני הבוחן כדי להוכיח שאתם מבינים גם את מגבלות המערכת?

20. שאלת הערכה מקורית בסגנון בגרות

מערכת למיון מוצרים מצלמת כרטיס שעליו מופיע מספר סידורי. כל כרטיס נחתך לתמונה אפורה בגודל 6×6 פיקסלים, ומסווג באמצעות מודל שנלמד מנתוני אימון מסומנים. נתונה המטריצה הבאה המתארת חלון מקומי:

10	20	10
20	100	20
10	20	10

א. הסבירו מהו ערך הפיקסל המרכזי ומהו טווח הערכים האפשרי בתמונה מטיפוס uint8. ב. חשבו את ערך המסנן הממוצע על החלון והראו את שלבי הסכימה. ג. הסבירו כיצד סינון עשוי גם לפגוע בזיהוי הספרה. ד. אם נבצע חיתוך תמונה ו־Resize, באיזה סדר הייתם פועלים ומדוע? ה. הסבירו מדוע צריך לשמור קבוצת Test נפרדת בעת בחירת שיטת עיבוד תמונה.

דרישת ההערכה: נימוק מלא במילים, חישוב מנומק והבחנה בין פעולת עיבוד קבועה לבין מודל שלומד מנתונים. זו שאלה מקורית שנכתבה לצורכי ההוראה, ולא שאלה מקורית ממאגר בחינות משרד החינוך.

21. מעבדת Google Colab ומדדי הצלחה

המעבדה המצורפת כוללת 12 משימות המתקדמות מזיהוי ממדים וערכי פיקסלים אל יצירת מסננים וניתוח ניסוי. בתאי התרגול מופיעים מקומות להשלמת קוד, ולא פתרונות מלאים. לפני כל תא יש לכתוב תחזית לגבי התוצאה; אחרי ההרצה יש להסביר אותה. מעבדה תיחשב מוצלחת רק אם התלמיד מסוגל להסביר את צורות המערכים, את שינויי הערכים ואת הסיכון לאיבוד מידע.

מה התלמיד מציג	יכולת
פירוש צורה (H,W) או (H,W,3)	ייצוג
המרה לאפור ומסנן ממוצע באופן ידני	חישוב
פעולות NumPy/Pillow מוסברות	קוד
השוואת שיטות עיבוד תוך שמירה על אותם נתונים	חקירה
הסבר מגבלה והשפעתה על AI	הגנה

22. הקשר בשלוש תכניות הלימוד

תכנית	תפקיד התוכן	הסטטוס
יחידה 3 — מדעי הנתונים	חזרה על מערכים, מאפיינים, KNN והמחשה בסיסית	ידע קודם / גשר פדגוגי
יחידה 5 — נושאי ליבה	וקטורים, מטריצות, מרחקים, תקנון, חקר נתונים	קישור מושגי; לא פרק תמונה נפרד
התמחות למידה עמוקה	עיבוד תמונה שיטתי, הכנת קלט וגשר ל־CNN	פרק 6: 15 שעות לפי מפת התכנית

הספר נשען על מפת ההתמחות השמורה במאגר GitHub שלנו. הגדרות, דוגמאות וניסויים נוספו כדי להנגיש לתלמידים רעיונות המופיעים בספרי יסוד כגון Hands-On Machine Learning, Mathematics for Machine Learning, Dive into Deep Learning ו־Grokking Deep Learning. כל הדוגמאות והאיורים בפרק נוצרו עבור החוברת; לא הועתקו מהם פרקים או איורים מוגנים.

23. הצעת חלוקת 15 שעות הוראה

תוצר	נושאים	שעות
קריאת מערכים ותיאור תמונה	מבוא, פיקסלים, ממדים, RGB ואפור	1–3
חישוב ידני ותרשים	אינדקסים, בהירות, ניגודיות והיסטוגרמה	4–6
השוואת עיבודים מודרכת	רעש, Crop, Resize, Normalize	7–9
מימוש מסנן וניתוח פלט	מסננים, קונבולוציה וזיהוי קצוות	10–12
הסבר פרויקט ומגבלות	הכנה ל־CNN, מעבדה, משימת הערכה	13–15

חלוקת השעות היא הצעה פדגוגית ל־15 שעות הסילבוס, ולא טבלה רשמית של משרד החינוך. ניתן להאריך או לקצר פעילויות בהתאם לרמת הכיתה, תוך שמירת תוצרי החובה שהמורה קובע על פי המסמך התקף.

מקורות והמשך קריאה

תכנית ההתמחות במאגר הקורס:
https://github.com/samiabo75/Data-Science-ML-Course/tree/main/03-Deep-Learning-5-Units/06-Image-Processing

Aurélien Géron, Hands-On Machine Learning with Scikit-Learn and PyTorch — פרקי ייצוג והכנת נתונים ומודלי ראייה.

Zhang, Lipton, Li & Smola, Dive into Deep Learning — פרקי image processing ו־Convolutional Neural Networks; https://d2l.ai

Deisenroth, Faisal & Ong, Mathematics for Machine Learning — אלגברה ליניארית ופעולות מערכיות; https://mml-book.github.io

Pillow documentation: https://pillow.readthedocs.io

/NumPy documentation: https://numpy.org/doc/stable

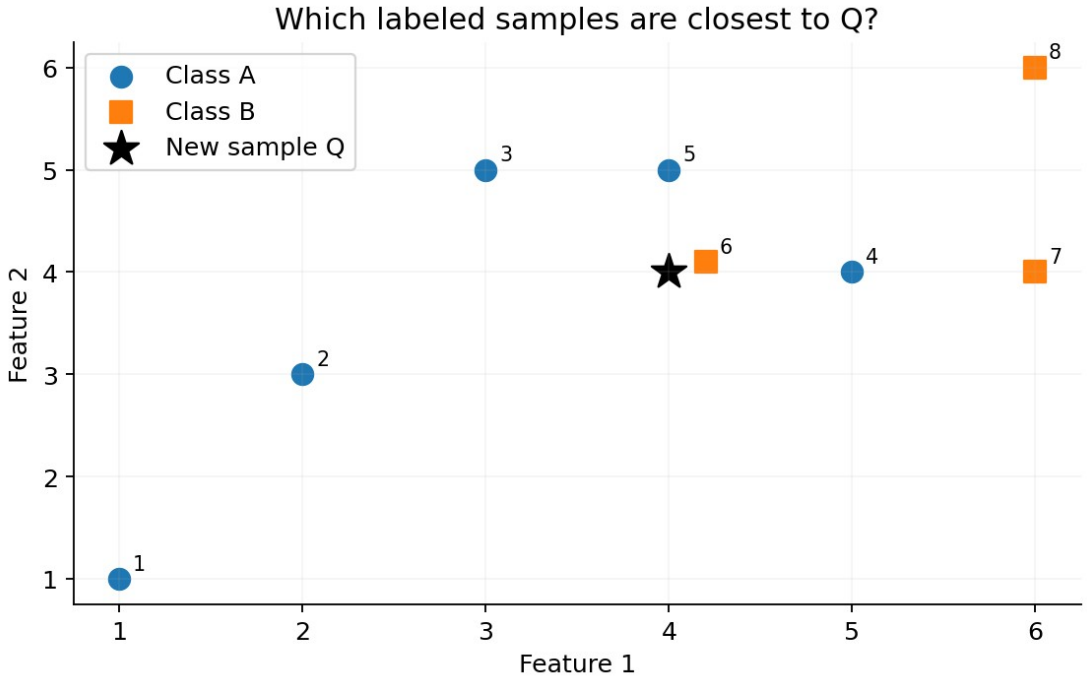
scikit-learn load_digits:
https://scikit-learn.org/stable/modules/generated/sklearn.datasets.load_digits.html

הערת מיפוי: הפרק מבוסס על כותרת פרק 6 והיקפו במאגר הסילבוס, אך ללא בדיקה חוזרת של כל שורות ה־PDF המקורי. ההרחבות בנושא קונבולוציה, הטיה וניהול ניסוי הוצגו כהכנה מקצועית ל־CNN ולפרויקט ולא כטענה אוטומטית לחובת בחינה.

למידה עמוקה | פרק 7 בסילבוס ההתמחות | 30 שעות מסגרת

פרק 7 — סיווג וKNN מתקדם

מהפעלת ספרייה למימוש אלגוריתם, חשיבה וקטורית ובדיקת הכללה



איור 1. אותו רעיון מוכר של שכנים קרובים — הפעם לומדים כיצד מממשים אותו בעצמנו.

פרק זה הוא המשך מעמיק לפרקים 4.1–4.4 של נושאי הליבה. במקום לשכפל את ההגדרות, נפרק כאן את השלבים שמבצע המסווג ונבנה אותם בהדרגה בקוד. תחילה נדע להסביר כל שורה, אחר כך נחשב ב־NumPy כמה מרחקים במקביל, ולבסוף נשווה את המימוש שלנו לכלי מקצועי. ההצלחה בפרק אינה נמדדת רק בקבלת תחזית נכונה, אלא ביכולת לזהות שגיאות, לבדוק נכונות ולהצדיק החלטות של מפתח AI.

עוגן: תכנית ההתמחות בלמידת מכונה ולמידה עמוקה, פרק 7 — בעיית הסיווג ו־KNN (30 שעות); חומרי השתלמות 5.1–5.3, כולל מימוש עצמאי ב־NumPy ותרגול Iris. חלוקת התרגול וההעמקה כאן היא הצעה דידקטית מקורית, לא נוסח רשמי של המחוון.

מפת הלמידה — מה נדע לעשות?

- להסביר כיצד אלגוריתם KNN מקבל את מטריצת המאפיינים X, את וקטור התוויות y ואת נקודת השאילתה q.
- לממש חישוב מרחקים, מיון אינדקסים, שמירת תוויות והצבעת רוב בעזרת לולאות, ואז בעזרת NumPy.
- להסביר Broadcasting, את המושג shape ואת משמעות axis בחישוב מרחקים לשאילתה אחת ולמספר שאילתות.
- לקבוע מראש מדיניות שוויון, להבחין בין תווית חזויה לציון הצבעה, ולהשוות הצבעה רגילה להצבעה משוקללת.

- לבדוק התאמת ממדים, תקנון ללא זליגת נתונים, חלוקת Train/Test, בחירת k באמצעות Cross-Validation והשוואה הוגנת.
- לנתח מורכבות חישובית, מגבלות ממדיות ולנסח החלטות מקצועיות בפרויקט גמר.

רצף מוצע ל־30 שעות ההתמחות

תוצר	מיקוד	שעות
טבלת מרחקים + הנמקת סיווג	ריענון סיווג וחישוב ידני	1–5
מימוש שקוף עם לולאות	פיתוח פונקציות Python	6–10
מימוש וקטורי ותיעוד shape	NumPy, Broadcasting וצורות	11–15
בדיקות נכונות והשוואה לספרייה	שוויון, משקולות ובדיקות	16–20
ניסוי Iris מתועד	Pipeline, CV ומדדים	21–25
דוח חקר קצר והגנה בעל־פה	ניתוח עלויות, העברה ותרגול	26–30

1. סיפור פתיחה — מדוע אי אפשר להסתפק ב־fit ו־predict?

דודו בונה מערכת המסייעת לסווג תקלות במעבדת מחשבים. לכל מחשב נשמרים שני מאפיינים מספריים: עומס במעבד ומדד התרעות. לכל מחשב שכבר נבדק קיימת גם תווית: A — תקלה קלה, B — דורש בדיקה נוספת. הדוגמה מומצאת להוראה, ואין להשתמש בה לקבלת החלטות תחזוקה אמיתיות ללא בדיקה מקצועית.

דודו הפעיל KNeighborsClassifier וקיבל תשובה — אבל המורה שאל: כיצד נדע שהמודל בחר את השכנים הנכונים? מה יקרה אם שני מחשבים יהיו באותו מרחק בדיוק? כיצד נבדוק שהנוסחה שהמרנו לקוד מחזירה את אותו המרחק שחישבנו על הנייר? אלה הן שאלות פיתוח תוכנה ומדע נתונים, ולא רק שאלות על פקודות.

$$q = (4,\ 4)$$

לצורך לימוד נשתמש בשמונה דוגמאות אימון ובשאילתה אחת q. כל שורה במטריצה היא מחשב; כל עמודה היא תכונה. התוויות נשמרות בנפרד, משום שאסור למודל להשתמש בתשובה הרצויה בתור מאפיין של הדוגמה החדשה.

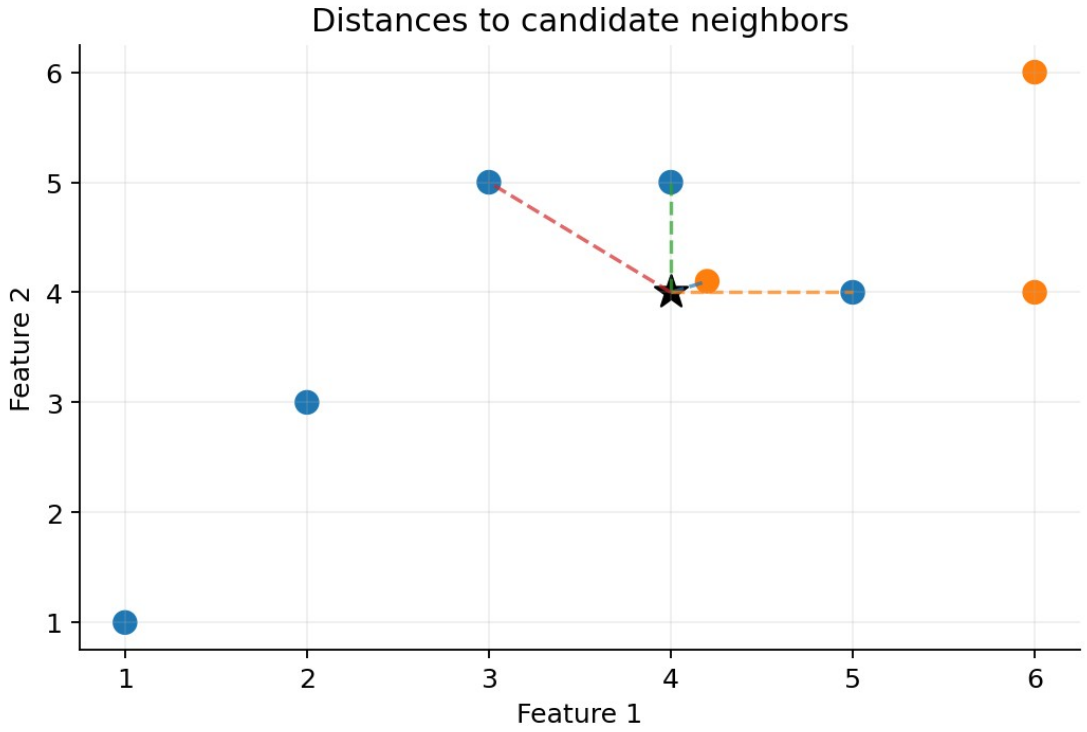
מספר	x_1	x_2	מחלקה
1	1	1	A
2	2	3	A
3	3	5	A
4	5	4	A
5	4	5	A
6	4.2	4.1	B
7	6	4	B
8	6	6	B

2. חזרה קצרה — האלגוריתם בחמש פעולות

תיאור האלגוריתם במילים חשוב לא פחות מקוד. התלמיד צריך להיות מסוגל לספר כיצד הופכת שורה חדשה לתווית חזויה: מודדים את המרחק לכל דוגמת אימון, מסדרים את הדוגמאות לפי מרחק, בוחרים k שכנים, סופרים את התוויות ומחזירים את המחלקה שנבחרה לפי כלל ההכרעה.

$$d(x, q) = \sqrt{\left[(x_1 - q_1)^2 + (x_2 - q_2)^2 \right]}$$

זהו מרחק אוקלידי. כבר למדנו אותו בפרק 1.2; כאן נשתמש בו שוב כדי לתכנן מבנה נתונים נכון. הדבר החשוב הוא שמרחק ותווית שייכים לאותה שורה: כאשר ממיינים מרחקים, אין למיין את התוויות בנפרד.



איור 2. הקווים המקווקווים מדגישים מרחקים של שכנים אפשריים אל השאילתה.

3. חישוב ידני מלא של סיווג אחד

נחשב קודם כמה מרחקים ללא מחשב. לתצפית מספר 6, הווקטור הוא (4.2,4.1). מרחקו מן השאילתה $q=(4,4)$ הוא השורש של סכום ריבועי ההפרשים. לתצפית מספר 4, הווקטור (5,4), ולכן המרחק שווה בדיוק ל־1.

$$d_6 = \sqrt{[(4.2 - 4)^2 + (4.1 - 4)^2]} = \sqrt{0.05} \approx 0.2236$$

$$d_4 = \sqrt{[(5 - 4)^2 + (4 - 4)^2]} = 1$$

$$d_5 = \sqrt{[(4 - 4)^2 + (5 - 4)^2]} = 1$$

לאחר חישוב כל המרחקים נרשום אותם לצד מספר התצפית והתווית. טבלה זו היא כלי בקרה חשוב: היא מאפשרת לאתר טעויות בקוד כגון שימוש בעמודה הלא נכונה או השמטת השורש.

סדר קרבה	תצפית	מרחק משוער	תווית
1	6	0.2236	B
2	4	1.0000	A

סדר קרבה	תצפית	מרחק משוער	תווית
3	5	1.0000	A
4	3	1.4142	A
5	7	2.0000	B
6	2	2.2361	A
7	8	2.8284	B
8	1	4.2426	A

התחזית כאשר k=1 היא B, מפני שהשכן הקרוב ביותר הוא תצפית 6. כאשר k=3, שלוש התוויות הראשונות הן B, A, A ולכן תחזית הרוב היא A. זוהי דוגמה לכך ששינוי k יכול להפוך החלטה גם כאשר כל הנתונים נשארו קבועים.



איור 3. סדר המרחקים לאחר המיון. יש לשמור את אינדקס השורה יחד עם המרחק שלה.

4. ממימוש רעיוני למימוש בפייתון עם לולאות

כדי להבין כיצד המחשב עובד, נתחיל בקוד שקוף ולא בפקודה אחת מוכנה. נשמור מערך של זוגות: המרחק ולידו אינדקס השורה. לאחר המיון נוכל למצוא את התווית מתוך y באותו אינדקס. זהו עיקרון תכנותי חשוב: אינדקס מחבר בין שתי טבלאות שמתארות את אותה תצפית.

```
import numpy as np
X = np.array([[1,1], [2,3], [3,5], [5,4],
              [4,5], [4.2,4.1], [6,4], [6,6]], dtype=float)
y = np.array(["A", "A", "A", "A", "A", "B", "B", "B"])
q = np.array([4.0, 4.0])
rows = []
for i in range(len(X)):
    total = 0.0
    for j in range(X.shape[1]):
        total += (X[i, j] - q[j]) ** 2
    rows.append((np.sqrt(total), i))
rows.sort(key=lambda pair: (pair[0], pair[1]))
print(rows[:3])
```

שאלו את עצמכם: מדוע הלולאה החיצונית עוברת על שורות והפנימית על מאפיינים? מדוע שומרים את i ולא רק את המרחק? מדוע במיון בחרנו צמד (מרחק, אינדקס)? הבחירה באינדקס כקריטריון משני הופכת את סדר השורות לשחזורי במקרה של מרחקים שווים, אבל היא עדיין אינה פתרון מלא לשוויון בהצבעה בין מחלקות.

5. פיתוח פונקציה עם חוזה ברור

לפני מימוש פונקציה נגדיר מה היא מקבלת ומה היא מחזירה. פונקציה לחיזוי אחת מקבלת X בגודל $n \times d$, תוויות y באורך n, נקודת שאילתה באורך d ו־k שלם בין 1 ל־n. תוצאתה היא תווית חזויה. פונקציה מקצועית גם צריכה לזהות קלט לא תקין, אבל לצורך ההיכרות נתחיל במקרה התקין ונסח בדיקות מתאימות בהמשך.

`X.shape = (n, d)` `y.shape = (n,)` `q.shape = (d,)`

```
def distances_one_query(X, q):
    result = np.zeros(X.shape[0], dtype=float)
    for i in range(X.shape[0]):
        total = 0.0
        for j in range(X.shape[1]):
            total += (X[i, j] - q[j]) ** 2
        result[i] = np.sqrt(total)
    return result

print(np.round(distances_one_query(X, q), 4))
```

הפרדת חישוב המרחקים מן ההצבעה מאפשרת לבדוק כל רכיב בנפרד. אם התקבלה תחזית שגויה, אפשר לשאול האם המרחקים שגויים, האם המיון שגוי, או האם ההצבעה שגויה. כך עובדים גם בפרויקט תוכנה גדול: מפרקים בעיה לבדיקות קטנות.

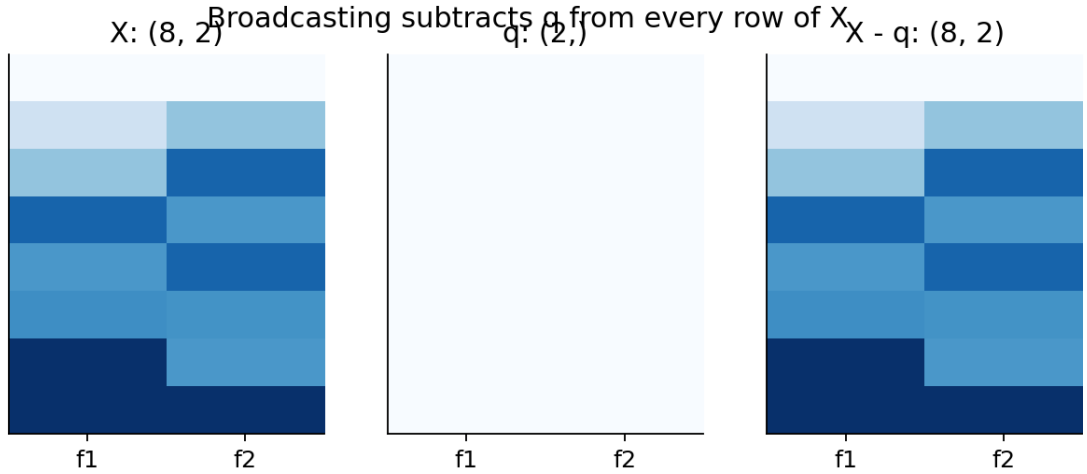
6. NumPy — מדוע לפעמים אפשר להחליף לולאות בפעולת מערכים?

למערך NumPy יש צורה (shape) שמתארת כמה שורות ועמודות יש בו. כאשר X היא מטריצה בעלת שתי עמודות ו־q הוא וקטור בן שני רכיבים, ניתן להחסיר את q מכל שורה ב־X. פעולה זו נקראת Broadcasting: NumPy מתאים מערכים בעלי צורות תואמות ומפעיל את אותה פעולה על קבוצת רכיבים. אין צורך לכתוב בעצמנו לולאה לכל שורה.

```
(8, 2) − (2,)  →  (8, 2)
```

```
differences = X - q
squared = differences ** 2
sums = np.sum(squared, axis=1)
distances = np.sqrt(sums)
print(differences.shape)
print(np.round(distances, 4))
```

ב־axis=1 אנחנו מסכמים כל שורה בנפרד, משום שכל שורה מתארת תצפית. אם בטעות נשתמש ב־axis=0, נקבל שני סכומים — אחד לכל מאפיין — במקום שמונה מרחקים. זו שגיאת תוכן ולא שגיאת תחביר, ולכן חשוב לנבא מראש מה אמורה להיות צורת התוצאה.



איור 4. הפחתת q מכל שורה ב־X ויצירת מטריצת הפרשים. שמירת shape נכונה קריטית למימוש.

7. מיון אינדקסים — מדוע argsort עדיף על sort?

הפקודה np.sort מחזירה את המרחקים כשהם מסודרים, אך בדרך זו נאבד את הקשר אל התווית של התצפית. לעומת זאת, np.argsort מחזירה את האינדקסים שמסדרים את המרחקים. לכן נוכל להשתמש באותם אינדקסים כדי לקרוא מתוך y את המחלקות של השכנים שנבחרו.

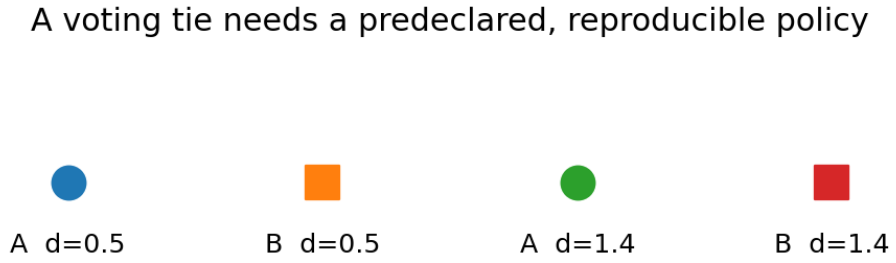
```
order = np.argsort(distances, kind="stable")
k = 3
neighbor_indices = order[:k]
neighbor_labels = y[neighbor_indices]
print(neighbor_indices)
print(neighbor_labels)
```

np.argsort היא אחת הפקודות החשובות בפרק: היא מחזירה מידע על סדר, לא את התוויות ולא את הערכים עצמם. בחרנו stable כדי לשמור על סדרם המקורי של פריטים בעלי מרחק שווה; עדיין צריך להגדיר כיצד מוכרעת הצבעה בין שתי מחלקות עם אותו מספר קולות.

8. הכרעת רוב ומדיניות שוויון — חלק מחוזה האלגוריתם

ב־k אי־זוגי ובבעיה שיש בה שתי מחלקות בלבד, הצבעה ברוב אינה יכולה להסתיים בשוויון במספר הקולות — בתנאי שנבחרו בדיוק k שכנים. אבל עם שלוש מחלקות, או כאשר k זוגי, ייתכן שוויון. נוסף לכך, ייתכן שוויון מרחק בגבול שבין השכן ה־k לבין השכן ה־k+1. אם לא נכתוב כלל מראש, שתי הרצות או שני מימושים עשויים לתת תשובות שונות.

לצורך המימוש הלימודי שלנו נקבע מדיניות מפורשת: במרחקים שווים נשמור תחילה על סדר האינדקסים המקורי; בהצבעת רוב שווה נבחר את המחלקה של השכן הקרוב ביותר מבין המחלקות הקשורות; ואם גם המרחק המינימלי שווה בדיוק, נבחר את שם המחלקה לפי סדר אלפביתי. זוהי מדיניות לימודית שבחרנו לצורך שחזוריות, ולא טענה שהמימוש של scikit-learn מפעיל בהכרח אותו כלל.



איור 5. שתי מחלקות עשויות לקבל מספר קולות זהה. יש לתעד כלל הכרעה לפני הפעלת המודל.

```
  def vote_with_tie_policy(labels, distances):
      classes, counts = np.unique(labels, return_counts=True)
      max_count = np.max(counts)
      tied = classes[counts == max_count]
      if len(tied) == 1:
          return tied[0]
      closest = []
      for label in tied:
          closest.append(np.min(distances[labels == label]))
      min_distance = np.min(closest)
      remaining = tied[np.isclose(closest, min_distance)]
      return sorted(remaining)[0]
```

דוגמה לבדיקה: עבור שכנים עם תוויות A,B,A,B ועם מרחקים 0.2,0.5,0.7,1.1, לכל מחלקה שני קולות. המדיניות שלנו בוחרת A, משום שהשכן הקרוב ביותר מבין המחלקות הקשורות שייך ל־A. אם השכן הקרוב ביותר היה B, הבחירה הייתה משתנה. תלמיד צריך להיות מסוגל להסביר את ההחלטה ללא הרצת הקוד.

9. בניית מסווג אחד קריא ומפורש

נשלב את רכיבי התהליך בפונקציה אחת לצורך הדגמה. נשתמש בפעולת NumPy לחישוב המרחקים, ב־argsort לשמירת אינדקסים ובפונקציית ההכרעה שכתבנו. השמות שנבחרו בקוד מבטאים תפקיד; הם אינם קיצור מסתורי שמחייב לזכור את גוף הפונקציה.

```
def predict_one(X, y, q, k=3):
    if X.ndim != 2 or len(y) != len(X):
        raise ValueError("Expected X: 2D and y: matching rows")
    if q.shape != (X.shape[1],):
        raise ValueError("Query feature count does not match")
    if not 1 <= k <= len(X):
        raise ValueError("k must be between 1 and n")
    distances = np.sqrt(np.sum((X - q) ** 2, axis=1))
    order = np.argsort(distances, kind="stable")[:k]
    return vote_with_tie_policy(y[order], distances[order])

print(predict_one(X, y, q, k=1))
print(predict_one(X, y, q, k=3))
```

שימו לב: עבור ערכי k הנבדקים בדוגמה זו אין שוויון בין מחלקות לאחר ההצבעה. כאשר משווים מימושים שונים על נתונים שיש בהם שוויון גבולי, אין לצפות בהכרח לאותה תוצאה ללא יישור של מדיניות השוויון.

10. מעבר לשאילתות רבות — שלושה ממדים במערך

עד כה חישבנו מרחקים בין שאילתה אחת לבין כל שורות האימון. אבל באפליקציה נחזה בדרך כלל כמה דוגמאות חדשות. נרצה מטריצת מרחקים שבה שורה אחת לכל שאילתה ועמודה לכל דוגמת אימון. כדי לאפשר זאת, נוסיף ממד בגודל 1 לכל אחד מהמערכים וניתן ל־Broadcasting ליצור את כל זוגות ההפרשים.

$$\texttt{Q.shape = (m, d) ; X.shape = (n, d)}$$

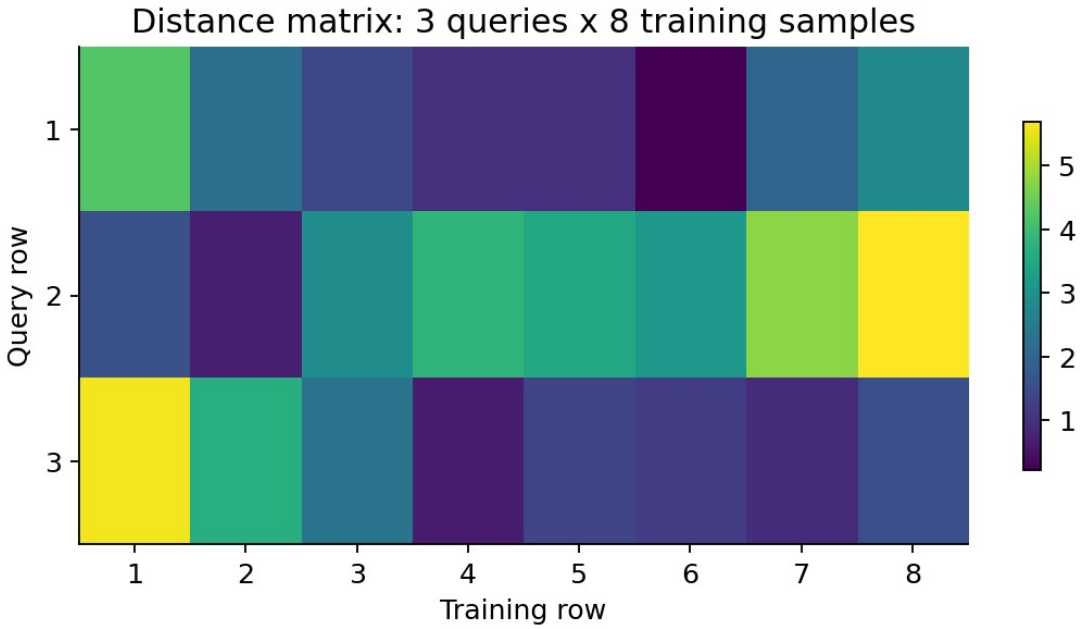
$$\texttt{Q[:, None, :].shape = (m, 1, d)}$$

$$\texttt{X[None, :, :].shape = (1, n, d)}$$

$$\texttt{differences.shape = (m, n, d)} \;\rightarrow\; \texttt{distances.shape = (m, n)}$$

```
Q = np.array([[4., 4.], [1.5, 2.5], [5.3, 4.6]])
all_diffs = Q[:, None, :] - X[None, :, :]
all_distances = np.linalg.norm(all_diffs, axis=2)
print(all_diffs.shape)
print(all_distances.shape)
```

המשמעות של axis=2 כאן שונה מ־axis=1 קודם: במערך החדש ציר 0 מייצג שאילתות, ציר 1 מייצג דוגמאות אימון, וציר 2 מייצג מאפיינים. סכימה על הציר האחרון מחשבת מרחק לכל זוג. אין לשנן axis ללא בדיקת shape.

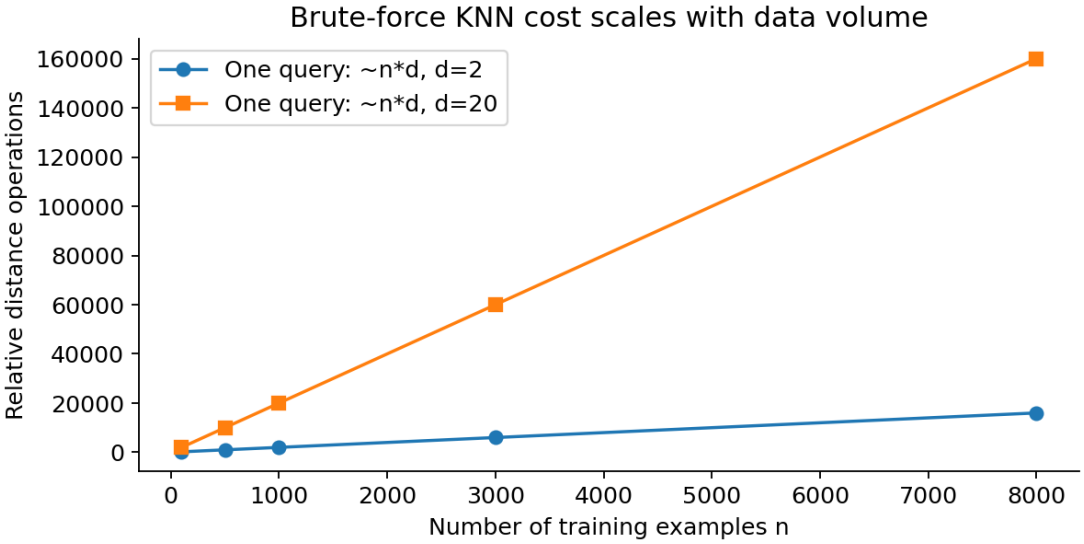


איור 6. מטריצת המרחקים מאפשרת להשוות כמה שאילתות מול כל דוגמאות האימון.

11. האם קוד קצר יותר בהכרח יעיל יותר?

שימוש ב־Broadcasting מקטין את הצורך בלולאות פייתון ויכול להיות מהיר מאוד. אולם עבור m שאילתות, n דוגמאות אימון ו־d מאפיינים, מערך הביניים של ההפרשים עשוי להכיל m×n×d מספרים. כאשר שלושת הגדלים גדולים, צריכת הזיכרון עלולה להפוך לבעיה ממשית. לכן לא נוכל להסיק ש״וקטוריזציה תמיד עדיפה״; נצטרך לשאול גם כמה זיכרון נדרש.

פתרונות אפשריים לפרויקט גדול הם חישוב בקבוצות קטנות (Batches), עבודה עם ספריות שמבצעות חישוב מותאם, או שימוש בשיטות חיפוש שכנים כגון KD-Tree ו־Ball-Tree כאשר תנאי הנתונים מתאימים. אין חובה לממש עצים בפרק זה; חשוב להבין שהמימוש הפשוט אינו הפתרון היחיד.



איור 7. במימוש brute-force, מספר חישובי המרחק גדל עם מספר דוגמאות האימון ומספר המאפיינים.

12. ניתוח מורכבות חישובית ברמת תיכון

נניח שיש n דוגמאות אימון ו־d מאפיינים. כדי לחשב את מרחקה של שאילתה אחת לכל הדוגמאות נבצע בסדר גודל של $n \times d$ פעולות חשבון. כאשר ממיינים את כל n המרחקים, המיון מוסיף עלות בקירוב $n \log n$. לכן עבור המימוש הלימודי עם argsort עלות החיזוי היא בקירוב $O(nd + n \log n)$ לשאילתה אחת, מעבר לעלות האחסון של הנתונים. קיימות שיטות לבחירת k איברים קטנים בלי למיין הכול, אך לא נכניס אותן לליבת המימוש הראשוני.

```
Training storage ≈ O(n d)    |    Distance calculation ≈ O(n d)
Full sort ≈ O(n log n)       |    m-query broadcast memory ≈ O(m n d)
```

יש להבדיל בין זמן חישוב לבין זיכרון. גם אם המחשב מצליח ליצור מערך עצום, אין פירוש הדבר שהפעולה יעילה או מתאימה להפעלה באפליקציה בזמן אמת. שאלה כזו נכנסת לפרק הגנת פרויקט: אילו מגבלות צפיתם, ואיזה שינוי עשיתם כשנפח הנתונים גדל?

13. הצבעה משוקללת לפי מרחק

במנגנון הצבעת רוב כל אחד מן השכנים הנבחרים מקבל קול אחד, גם אם אחד קרוב מאוד ואחר רחוק יותר. בשיטה משוקללת ניתן לתת לשכן קרוב משקל גבוה יותר, למשל באמצעות משקל הפוך למרחק. כך מקרה שבו יש שני קולות למרחקים גדולים וקול אחד קרוב מאוד עשוי להתהפך.

$$w_i = 1 / d_i \qquad (d_i > 0)$$

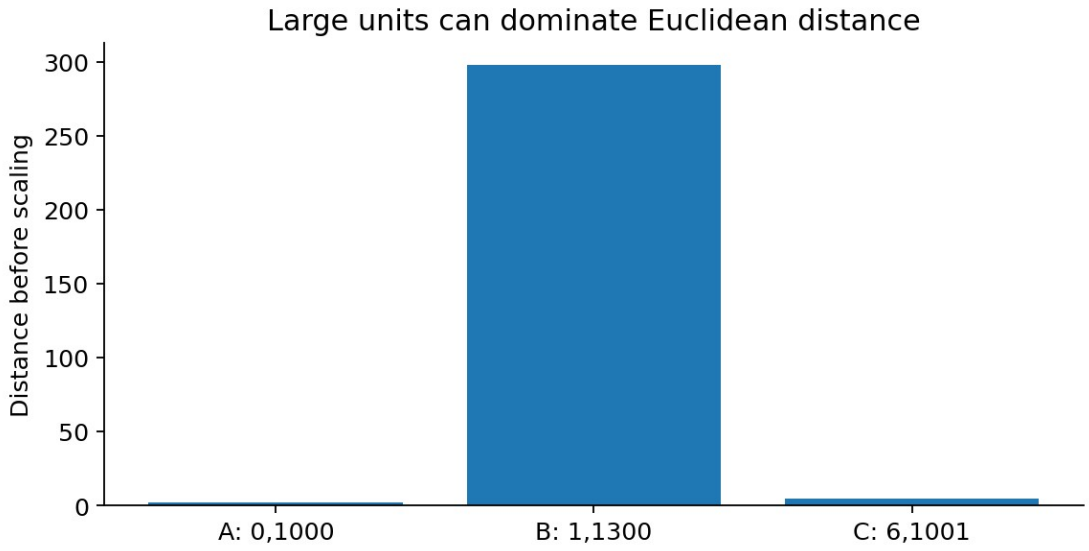
כאשר מרחק שווה לאפס, הביטוי $1/d$ אינו מוגדר. לכן יש צורך במדיניות נפרדת: אם יש שכנים זהים לשאילתה, ניתן להכריע רק ביניהם על פי התוויות שלהם, במקום לחלק באפס. בספריית scikit-learn קיימת אפשרות "weights="distance", אך היא מטפלת במקרים מיוחדים לפי המימוש שלה; אין לייחס לה באופן אוטומטי את כללי המימוש הלימודי שלנו.

```
from sklearn.neighbors import KNeighborsClassifier
model = KNeighborsClassifier(n_neighbors=3, weights="distance")
model.fit(X, y)
print(model.predict(q.reshape(1, -1)))
```

חשוב לא לבלבל משקל בהצבעת KNN עם משקל שנלמד ברגרסיה או ברשת עצבית: ב־KNN ההצבעה המשוקללת מבוססת על מרחקים בזמן החיזוי, ולא על עדכון הדרגתי של וקטור משקולות באמצעות Gradient Descent.

14. קנה מידה ותקנון — בדיקת שגיאה מקצועית

מרחק אוקלידי רגיש ליחידות המדידה. אם מאפיין אחד נע בין 0 ל־10 והשני בין 0 ל־10,000, ההפרש המספרי במאפיין השני עלול לשלוט במרחק. לפיכך, ייתכן שנקודה תיראה רחוקה בעיקר בגלל סולם המדידה ולא בגלל חשיבות אמיתית של המאפיין. כבר למדנו תקנון בפרק 2.3; כאן ניישם אותו באופן מקצועי.



איור 8. הבדל גדול ביחידות של מאפיין יכול להשתלט על המרחק הכולל.

את ממוצע האימון וסטיית התקן של האימון יש לחשב רק על נתוני האימון. נתוני Test עוברים transform באמצעות אותם פרמטרים; אין להשתמש בהם כדי להתאים את הממיר. בוולידציה צולבת יש להתאים מחדש את הממיר בתוך כל קיפול אימון, ולכן Pipeline הוא כלי העבודה המועדף.

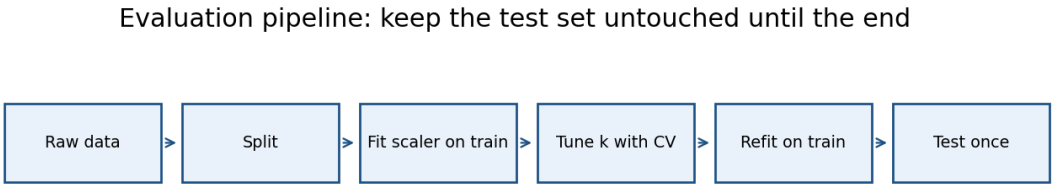
15. בניית ניסוי אמיתי על Iris בלי זליגת מידע

נעבור לדוגמה מקצועית מתוך Iris: קבוצת נתונים מוכרת בעלת ארבע תכונות מדידה ושלושה סוגי אירוסים. כאן נבחר לצורך גרפים שתי תכונות בלבד, ונשמור את יחס המחלקות בעת החלוקה. בנתוני פרויקט אמיתי חשוב לתאר גם מקור נתונים, איכותם, ערכים חסרים ומגבלות ייצוג האוכלוסייה.

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.neighbors import KNeighborsClassifier

iris = load_iris()
X = iris.data[:, [0, 2]]
y = iris.target
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.30, random_state=19, stratify=y
)
pipe = Pipeline([
    ("scale", StandardScaler()),
    ("knn", KNeighborsClassifier(n_neighbors=5))
])
pipe.fit(X_train, y_train)
print("Test accuracy:", pipe.score(X_test, y_test))
```

למה לא להפעיל scaler.fit_transform על כל X לפני החלוקה? משום שאז ממוצעי נתוני הבדיקה ישפיעו על אופן הכנת נתוני האימון. זוהי זליגת מידע. שימו לב גם ש־pipe.fit אינו ״מלמד״ את KNN משקולות כמו ברשת עצבית; הוא מתאים את הממיר ושומר את דוגמאות האימון שהמסווג צריך בזמן החיזוי.

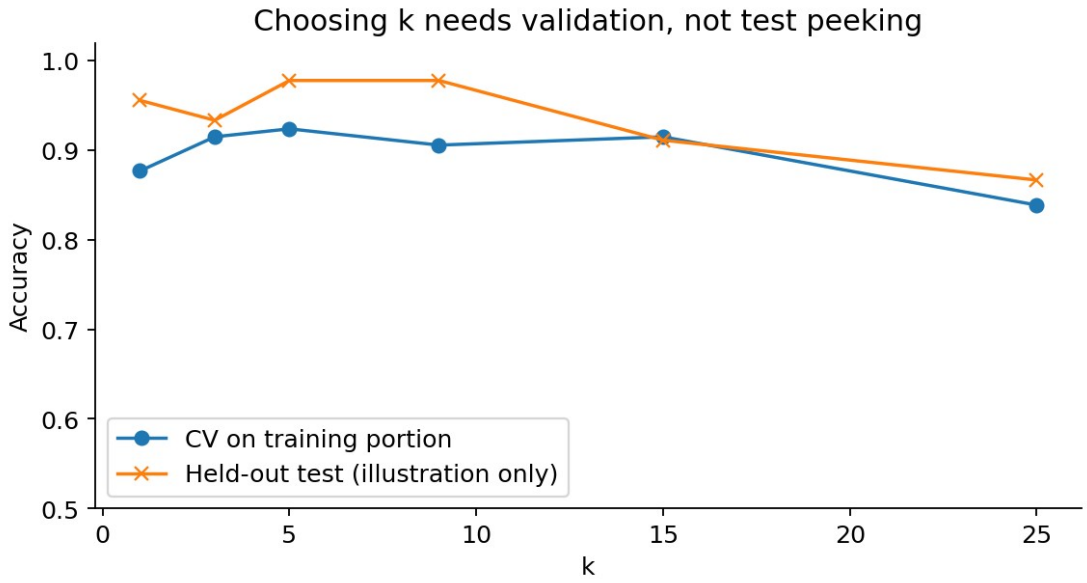


איור 9. סדר עבודה תקין: חלוקה, התאמת הכנה בתוך אימון, בחירת k בוולידציה ובדיקת Test בסוף.

16. בחירת k באמצעות Cross-Validation

היפר־פרמטר k משפיע על רגישות המודל. ערך קטן מאוד יכול לגרום להחלטה רגישה לרעש, וערך גדול מדי עשוי להחליק גבולות החלטה יתר על המידה. איננו בוחרים k לפי Test; בוחרים אותו באמצעות נתוני האימון, למשל באמצעות K-Fold או StratifiedKFold. קיפולים מספקים כמה אומדנים, אך אין לראות בממוצע שלהם הבטחה לתוצאה זהה בנתוני אמת.

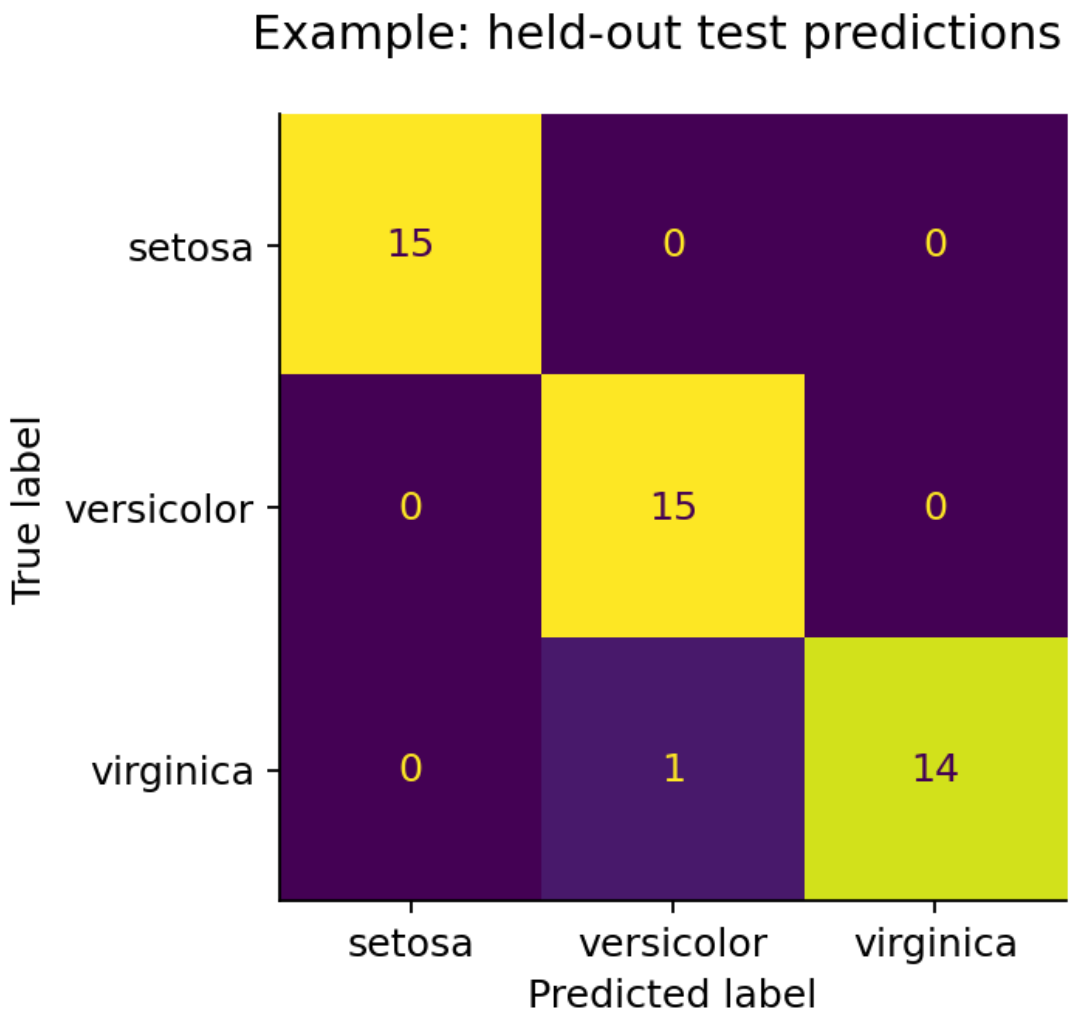
```
from sklearn.model_selection import StratifiedKFold, cross_val_score
cv = StratifiedKFold(n_splits=4, shuffle=True, random_state=19)
for k in [1, 3, 5, 9, 15]:
    candidate = Pipeline([
        ("scale", StandardScaler()),
        ("knn", KNeighborsClassifier(n_neighbors=k))
    ])
    scores = cross_val_score(candidate, X_train, y_train,
                             cv=cv, scoring="accuracy")
    print(k, round(scores.mean(), 3), round(scores.std(), 3))
```



איור 10. דוגמה מדגמית על Iris: עקומות CV ו־Test עשויות להשתנות עם החלוקה. בוחרים k מתוך CV בלבד; Test מוצג לצורך דיון לאחר הבחירה.

17. מדדי ביצוע — מעבר למספר Accuracy בודד

לפעמים מודל מסווג היטב את המחלקה הגדולה אך מחמיץ מחלקה נדירה וחשובה. כדי להבין מה קרה נבחן מטריצת בלבול, Precision, Recall ו־F1 לפי הצורך. במקרה של כמה מחלקות נדווח גם כיצד אוחדו המדדים בין מחלקות — למשל macro או weighted — ולא רק מספר ממוצע חסר הסבר.



איור 11. דוגמה למטריצת בלבול מתוך קבוצת בדיקה מופרדת; האלכסון מציין תחזיות נכונות.

בחינת הבגרות דורשת יכולת להסביר תוצאה, ולא להסתפק בשורת פלט. אם מודל קיבל Accuracy גבוה, התלמיד צריך לדעת לשאול: בכמה דוגמאות נבדק? מה יחס המחלקות? האם התבצעה התאמת תקנון על נתוני הבדיקה? מהו מחיר טעות מסוג מסוים?

18. בדיקות יחידה למימוש KNN שלנו

לפני שמציגים פרויקט גמר בודקים גם מקרים קטנים שביכולתנו לחשב ביד. בדיקת יחידה טובה אינה דורשת מאגר גדול: לעיתים דווקא שלוש נקודות מאפשרות לזהות שגיאה. רצוי לתכנן מראש בדיקה למרחק אפס, לשתי נקודות שוות מרחק, לאינדקסי מיון, לממדים לא תואמים ולמקרה שבו k גדול ממספר דוגמאות האימון.

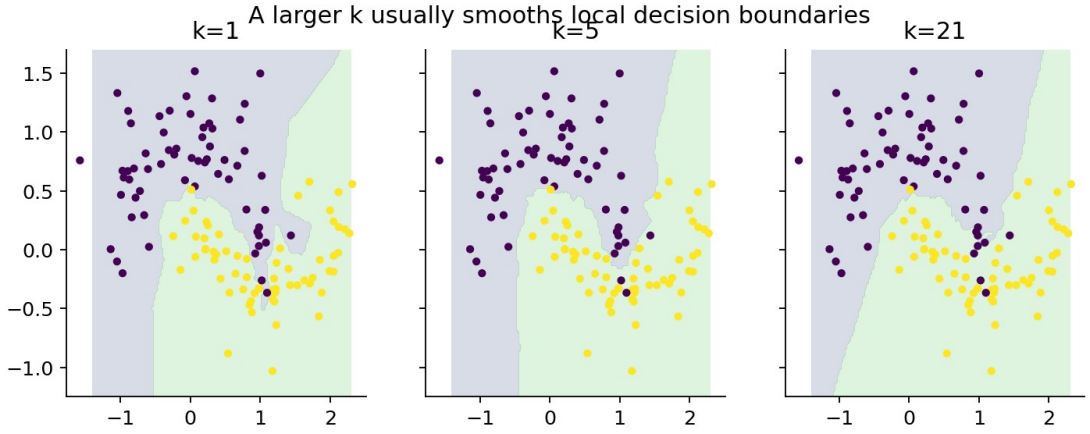
```
# Tests for our teaching implementation
assert predict_one(X=np.array([[0.,0.],[3.,0.]]),
                   y=np.array(["A", "B"]),
                   q=np.array([.2,0.]), k=1) == "A"

assert np.isclose(
    np.linalg.norm(np.array([3.,4.]) - np.array([0.,0.])), 5.0
)
print("Small deterministic tests passed")
```

חשוב להבדיל בין בדיקת נכונות לבין הערכת דיוק על מאגר: בדיקת יחידה אומרת שהמימוש פועל לפי הכללים שכתבנו במקרה ידוע; Accuracy על Test מודד כיצד המודל מכליל לדוגמאות חדשות. האחת אינה מחליפה את האחרת.

19. על אילו מקרים KNN עלול להיכשל?

KNN יכול להיכשל בשל בחירת מאפיינים לא מתאימים, קנה מידה, רעש, חוסר איזון במחלקות או מספר גדול של ממדים. במרחב עתיר ממדים, מרחקים בין נקודות עלולים להיות פחות מבחינים, תופעה המכונה לעיתים "קללת הממדיות". אין פירוש הדבר ש־KNN לעולם אינו עובד עם הרבה מאפיינים, אלא שיש לבדוק האם המרחק שנבחר משמעותי עבור המשימה והנתונים.



איור 12. שינוי k משפיע על צורת גבולות ההחלטה; הגבול החלק ביותר אינו בהכרח המתאים ביותר.

בנוסף, מאגר שיש בו שתי קבוצות של אותה מחלקה עשוי להיות מבלבל עבור Nearest Centroid, אך KNN יכול לעיתים לזהות את המבנה המקומי. מצד שני, כאשר יש מספר עצום של נקודות אימון ומערכת חייבת לענות במהירות רבה, העלויות של KNN יכולות להקשות על שימוש ישיר. בחירת המודל תמיד תלויה בבעיה.

20. כיצד מחברים KNN לעיבוד תמונה?

בפרק 6 למדנו שתמונה היא מערך מספרי. אפשר לשטח תמונת גווני אפור לווקטור ולהגדיר מרחק בין תמונות. אז לכאורה ניתן להפעיל KNN על פיקסלים: תמונה חדשה תסווג לפי התמונות הקרובות אליה. זהו קו בסיס לימודי שימושי, אך מרחק פיקסלים רגיש להזזה, תאורה וסיבוב, ולכן אינו מבטיח לזהות צורות סמנטיות דומות.

הקשר העמוק לפרקי CNN יהיה ברור בהמשך: במקום להשוות רק ערכי פיקסלים גולמיים, רשת עצבית יכולה ללמוד ייצוגים של מאפיינים חזותיים. לאחר לימוד רשתות נוכל לשאול אם KNN על ייצוגים נלמדים נותן תוצאות טובות יותר מ־KNN על פיקסלים. זו הרחבת גשר לפרויקט, ולא תחליף לפרק CNN.

21. יישום בפרויקט גמר — דף החלטות הנדסי

תלמיד שבוחר להשתמש ב־KNN בפרויקט צריך לתעד את שאלת המחקר, מקור הנתונים, שמות המאפיינים, יחידותיהם, דרך הטיפול בחסרים, אופן החלוקה, שיטת התקנון, רשימת ערכי k שנבדקו, מדדי ההערכה, מגבלות הביצועים והסיבה לבחירת המודל. יש להימנע מתיעוד טכני שנכתב לאחר מעשה ואינו משקף ניסוי שבוצע באמת.

מה צריך לנמק?	החלטה
האם הם זמינים בזמן חיזוי ובעלי משמעות?	מאפיינים
מדוע Euclidean או מדד אחר מתאים לנתונים?	מרחק
כיצד הותאם רק על Train?	תקנון
כיצד נבחר באמצעות Validation/CV?	k
מהו מחיר הטעות ומהי המחלקה החשובה?	מדד
מה עלול להשתבש בנתונים חדשים?	מגבלה

22. טעויות נפוצות שכדאי לעצור בזמן

- סידור distances במיון ושכחת לעדכן באותו סדר את התוויות המתאימות.
- שימוש ב־axis=0 במקום axis=1 בחישוב מרחקי שאילתה אחת, או במקום axis=2 בחישוב כמה שאילתות.
- הפעלת fit_transform על כל הדוגמאות לפני חלוקת Test או לפני CV.
- הנחה שהמימוש הידני ו־sklearn יכריעו כל שוויון באותה דרך.
- חשיבה ש־predict_proba בהכרח נותנת הסתברות מכוילת היטב לשימוש בהחלטה רגישה.
- שימוש ב־Accuracy אימון כהוכחה להכללה, או בחירת k לאחר הסתכלות ב־Test.
- בניית מטריצת Broadcasting ענקית בלי להבין את מחיר הזיכרון.
- תיאור k בתור פרמטר שנלמד מן הנתונים; k הוא היפר־פרמטר שהמפתח בוחר או מכוון.

23. סיכום מושגים, תפקידם והקשר ל־AI

מושג	מה צריך לדעת להסביר
shape	מה המשמעות של כל ציר במערך NumPy
Broadcasting	מתי ניתן לבצע פעולת מערך ללא לולאות Python מפורשות
argsort	כיצד לשמור קשר בין מרחק לאינדקס ולתווית
k	השפעת מספר השכנים על החלטת סיווג
Tie policy	מה קורה במקרה של שוויון מרחק או קולות

מה צריך לדעת להסביר	מושג
מדוע הכנת הנתונים חייבת להיכלל בתהליך האימון	Pipeline
כיצד בוחרים היפר־פרמטר בלי לגעת ב־Test	CV
מה מחיר מרחקים, מיון ואחסון בנתונים גדולים	Complexity

24. תרגול מדורג — ללא פתרונות

רמת יסוד: הבנת מבנה המימוש

- שאלה 1. נתונה מטריצה X בעלת 20 שורות ו־4 עמודות ושאילתה q בעלת 4 רכיבים. כתבו את shape של X, של q, של X-q ושל וקטור המרחקים.
- שאלה 2. לפניכם מרחקים [2.0,0.5,1.0] ותוויות [A,B,A]. קבעו אילו אינדקסים תחזיר argsort, ומה תהיה התחזית כאשר k=1 וכאשר k=3.
- שאלה 3. הסבירו במילים מדוע אי־אפשר פשוט למיין את מערך התוויות כאשר ממיינים את המרחקים.
- שאלה 4. תלמיד כתב np.sum((X-q)**2, axis=0). הסבירו מה הקוד מחשב ולמה זו אינה התוצאה הנדרשת.

רמת העמקה: מתמטיקה, ניסוי ותכנון

- שאלה 5. בנו מערכת של ארבע דוגמאות ושתי מחלקות שבה מספר הקולות ב־k=4 שווה. הגדירו כלל הכרעה ודונו בהשפעתו.
- שאלה 6. נתון מאגר עם 2,000 דוגמאות ו־30 מאפיינים. העריכו כמה פעולות הבדל־ריבוע נדרשות לשאילתה אחת במימוש brute-force.
- שאלה 7. הסבירו מדוע שימוש ב־StandardScaler לפני cross_val_score יכול לגרום לזליגת מידע, והציעו תיקון באמצעות Pipeline.
- שאלה 8. תכננו השוואה בין k=1, k=5 ו־k=15 על Iris שתשתמש ב־CV לבחירה וב־Test להערכה סופית בלבד.

רמת בגרות ופרויקט: נימוק מקצועי

- שאלה 9. תלמיד טוען כי ״כל מימוש NumPy מהיר יותר כי אין בו לולאות״. הסבירו מדוע הטענה גורפת מדי תוך התייחסות לזיכרון.
- שאלה 10. מערכת לזיהוי חריגות בנתוני תלמידים מקבלת Accuracy של 97%, אך כמעט אינה מזהה את המחלקה הנדירה. הציעו לפחות שני מדדים או ניסויים נוספים.
- שאלה 11. שני מימושים של KNN מחזירים תוצאה שונה רק כאשר השכן ה־k והשכן ה־k+1 נמצאים באותו מרחק. הציעו בדיקת יחידה והסבירו מקור אפשרי להבדל.
- שאלה 12. כתבו פסקת הגנה בעל־פה: מדוע בחרתם ב־KNN, מה יתרונו, מה חולשתו ומה תרצו לשנות אם המאגר יגדל פי מאה.

25. משימת סיכום — ממימוש למערכת קטנה

בנו מחברת Colab המשווה בין המימוש העצמאי שלכם לבין KNeighborsClassifier על אותה חלוקת נתונים. צרו טבלת השוואה של חיזויים, נתחו מקרים שבהם התחזיות שונות, בדקו תחילה את מדיניות השוויון, והשוו מדדי בדיקה. לאחר מכן כתבו דוח קצר שבו מוסברים מקורות הנתונים, השיטה, החלטת k, שיטת התקנון, מגבלה אחת והמלצה להמשך. הפעילו את כל התאים מהתחלה לפי סדרם כדי לוודא שחזוריות.

26. מפת הקשרים בין שלוש תכניות הלימודים

מעמד	מה נלמד או נדרש	מסלול
ידע קודם	הפעלה והבנה ראשונית של KNN ומדדי ביצוע	יחידה 3 — בסיס
עוגן בליבה	מרחקים, גבולות החלטה, k, ניתוח וסיווג	יחידה 5 — נושאי ליבה
עוגן התמחות	מימוש עצמי, NumPy, Broadcasting וניסויים על Iris	התמחות DL — פרק 7
העמקה דידקטית	בדיקות יחידה, זיכרון, מגבלות ומהירות	הרחבת פרויקט

27. קריאה משלימה ומקורות

עוגן תכנית: משרד החינוך, התמחות למידת מכונה ולמידה עמוקה, פרק 7 — בעיית הסיווג ו־KNN, 30 שעות (לפי מפת המאגר). מצגות ההשתלמות KNN 5.1, KNN with sklearn 5.2 ו־KNN with code 5.3; מחברות Iris ותרגולי המימוש העצמאי.

ספרי יסוד ששיטות ההסבר שלהם משמשות השראה לכתיבה מקורית: Luis Serrano, Grokking Machine Learning; Aurélien Géron, Hands-On Machine Learning; Deisenroth, Faisal & Ong, Mathematics for Machine Learning; James et al., An Introduction to Statistical Learning with Applications in Python; Zhang et al., Dive into Deep Learning.

תיעוד טכני: https://numpy.org/doc/stable/user/basics.broadcasting ; https://scikit-learn.org/stable/modules/generated/sklearn.neighbors.KNeighborsClassifier.html ; https://scikit-learn.org/stable/modules/cross_validation.html

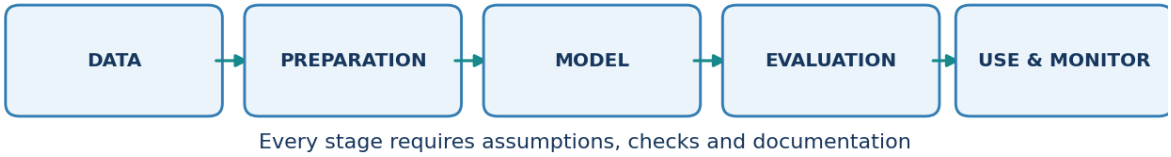
הערה: ההסברים, הדוגמאות, הטבלאות והאיורים נכתבו ויוצרו לפרק זה באופן מקורי. התאמה לנוסח המחייב של כל סעיף בבגרות טעונה בדיקה מול מסמכי משרד החינוך התקפים לשנת הלימודים.

פרק 8 — מושגי יסוד בלמידת מכונה

ממודל שמחשב תחזית למערכת שלומדת, נבדקת ומשתפרת

ספר תלמיד | התמחות למידת מכונה ולמידה עמוקה — 5 יחידות לימוד | 10 שעות על פי מפת התכנית שבמאגר.

מעמד הפרק: פרק 8 בתכנית ההתמחות. דוגמאות על Gradient Descent ורגולריזציה מובאות כהכנה לפרקי הפרספטרון והמתמטיקה של למידה עמוקה; אין לייחס להן מעמד של תת־סעיף חובה בפרק זה ללא הצלבת נוסח הסילבוס הרשמי המלא.



איור: מסלול פיתוח מודל: עבודה עם נתונים, הכנה, התאמת מודל, הערכה ושימוש אחראי לאורך זמן.

איך לומדים בעזרת הספר?

בכל מושג נתחיל משאלה מחיי היום־יום, נציג דוגמה שניתן לחשב ביד, ננסח את העיקרון המתמטי, נבדוק אותו ב־Python ולבסוף ננתח מקרה שבו התוצאה עלולה להטעות. אין צורך לזכור את שמות הפונקציות בעל פה: החשוב הוא להבין מה כל פעולה מקבלת, מה היא מחזירה ומדוע בחרנו לבצע אותה.

מטרות ביצועיות

- להבחין בין נתונים, מאפיינים, משתנה מטרה, תחזית, פרמטר והיפר־פרמטר.
- להסביר כיצד משתמשים בפונקציית שגיאה כדי להשוות תחזיות, ולחשב MAE ו־MSE ביד.
- להבין את רעיון שינוי הפרמטרים בעת אימון, כולל דוגמת צעד אחת של Gradient Descent כהכנה.
- לזהות Overfitting ו־Underfitting בעזרת גרפים והבדלים בין אימון לוולידציה.
- לתכנן ניסוי הוגן עם חלוקת נתונים, קו בסיס ומניעת זליגה.
- לנסח החלטה מקצועית מנומקת מתוך מדדים, מגבלות ויומן ניסויים.

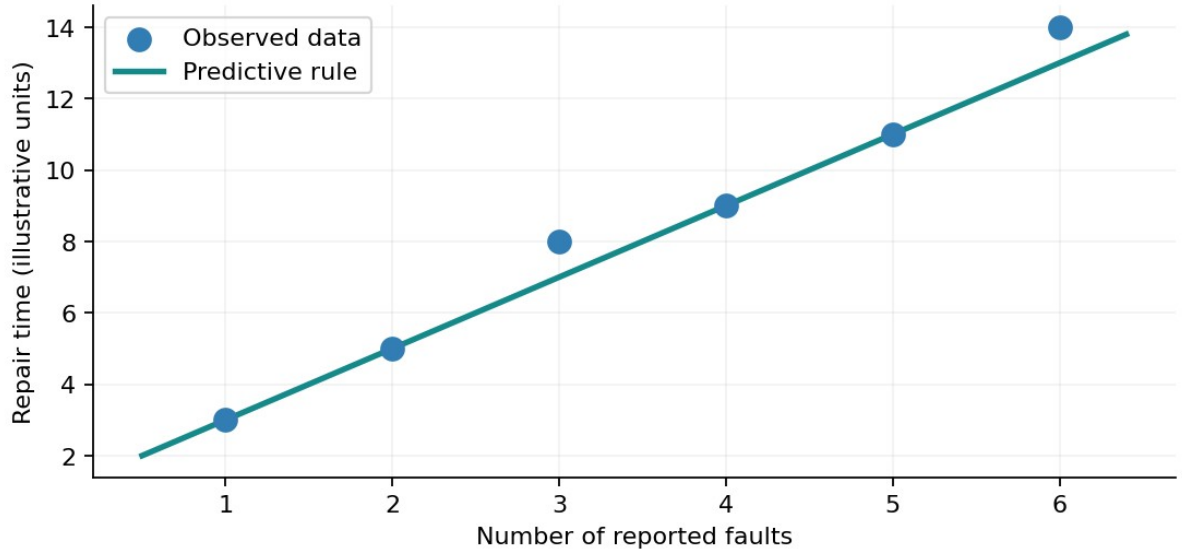
ידע קודם

הקורא כבר למד פונקציות בסיסיות, וקטורים, טבלאות וגרפים, רגרסיה ליניארית, KNN, מדדי הערכה ו־Train/Validation/Test. אין צורך ללמוד אותם מחדש. במקום זאת נשתמש בהם כדי להבין את תהליך האימון והבחירה במודל ברמה גבוהה יותר.

1. סיפור פתיחה: מחשב שחוזה זמן תיקון — האם הוא באמת לומד?

דודו אחראי על מעבדת המחשבים בבית הספר. לאחר כל תיקון הוא רושם כמה תקלות אותרו וכמה זמן ארך התיקון. הוא רוצה לבנות מערכת שתעריך מראש את משך הטיפול במחשב חדש, כדי לתכנן את עבודת הטכנאים. בתחילה הוא כותב כלל קבוע: ״כל תקלה מוסיפה עשר דקות״. אבל בבדיקת נתוני עבר הוא מגלה שהכלל מתאים לחלק מהמקרים ולאחרים פחות.

המעבר האמיתי ללמידת מכונה אינו עצם כתיבת נוסחה. בלמידת מכונה אנחנו בוחרים משפחה של כללים אפשריים, משתמשים בנתונים כדי להתאים את הפרמטרים של הכלל, ובודקים האם הכלל שנבחר עוזר לנו לחזות מקרים שלא שימשו להתאמתו.



איור: דוגמה לימודית של מספר תקלות מול זמן תיקון. הנקודות הן מדידות; הקו הוא כלל חיזוי. הוא אינו חייב לעבור בכל נקודה.

1.1 מה אנחנו יודעים ומה המודל צריך ללמוד?

תפקיד	דוגמה במעבדה	מרכיב
דוגמה אחת ללמידה	מחשב שטופל בעבר	תצפית
מידע ידוע בזמן החיזוי	מספר התקלות	מאפיין x
מה שרוצים לחזות	משך התיקון בפועל	מטרה y
תוצאת המודל	משך תיקון משוער	תחזית ŷ
ערכים המותאמים לנתונים	שיפוע וחיתוך	פרמטרים

שאלת הבנה: האם "משך התיקון בפועל" הוא מאפיין שמותר להזין למודל כאשר הוא מנבא משך תיקון שטרם התרחש? לא. זהו בדיוק המשתנה שאנחנו מנסים לחזות. הכנסת התוצאה האמיתית לתכונות בזמן האימון תייצר זליגת מידע חמורה.

2. מהו מודל וכיצד הוא שונה מאלגוריתם האימון?

מודל הוא הפונקציה שמקבלת נתונים ומפיקה תחזית. אלגוריתם האימון הוא התהליך שמחפש ערכי פרמטרים מתאימים. בסיום האימון שומרים את המודל המותאם, ולא את כל סדר השאלות והטעויות שביצענו בדרך. לדוגמה, כאשר עובדים עם רגרסיה ליניארית, המודל הוא ישר; שיטת התאמת המקדמים עשויה להיות חישוב מתמטי ישיר או תהליך איטרטיבי.

$$\hat{y} = wx + b$$

במשוואה זו x הוא מאפיין הקלט, w הוא המקדם שמכפיל אותו, b הוא החיתוך עם ציר y, ו־ŷ היא התחזית. שים לב להבדל בין y, הערך האמיתי, לבין ŷ, הניחוש של המודל. ההבדל ביניהם עומד בבסיס פונקציית השגיאה.

2.1 פרמטר לעומת היפר־פרמטר

פרמטר הוא ערך שנקבע מתוך נתוני האימון בהתאם לאלגוריתם. היפר־פרמטר הוא בחירה שמגדירה כיצד ייבנה או יופעל תהליך הלמידה. הוא עשוי להיבחר בידי המפתח או באמצעות חיפוש על נתוני ולידציה. חשוב שלא לבלבל היפר־פרמטר עם תכונה של דוגמה אחת.



איור: הבחנה בין ערכים שהמודל מתאים מנתונים לבין הגדרות תהליך שהמפתח בוחר או מכוון.

מודל	פרמטר שנלמד	דוגמה להיפר־פרמטר
רגרסיה ליניארית	משקולות w והטיה b	מידת רגולריזציה ב־Ridge
KNN	אין משקולות גלובליות מהסוג הזה; דוגמאות האימון נשמרות	מספר השכנים k
רשת עצבית	משקולות והטיות של הנוירונים	קצב למידה, מספר שכבות

שאלת חשיבה: נניח שבחרנו ב־KNN עם k=5. האם המודל "למד" את המספר 5? לא. זהו ערך שנקבע בתהליך תכנון או כוונון. לעומת זאת, ברשת עצבית המשקולות בדרך כלל משתנות תוך כדי האימון.

3. איך מודדים עד כמה התחזית טובה? פונקציית השגיאה

כדי לשפר מודל דרושה דרך כמותית לתאר עד כמה הוא טועה. לא מספיק לומר שהתחזית "קרובה" לערך האמיתי, משום שקשה להשוות בין מאות תחזיות ללא כלל חישוב משותף. פונקציית שגיאה (Loss) הופכת את הפער בין התחזית לערך האמיתי למספר. כשמקבלים מספר כזה אפשר להשוות בין מודלים או בין שלבי אימון שונים.

3.1 שארית — הצעד הקטן הראשון

$$e_i = y_i - \hat{y}_i$$

אם זמן התיקון בפועל הוא 8 יחידות זמן והמודל חזה 7, השארית היא 1. אם הוא חזה 9, השארית היא 1−. הסימן מספר לנו לאיזה כיוון טעה המודל, אבל כאשר מסכמים שאריות חיוביות ושליליות הן עלולות לבטל זו את זו. לכן משתמשים לעיתים בערך מוחלט או בריבוע של הטעות.

3.2 MAE — ממוצע השגיאות המוחלטות

$$\mathrm{MAE} = \frac{1}{n}\sum |y_i - \hat{y}_i|$$

MAE מודד בכמה יחידות, בממוצע, התחזית מתרחקת מן התוצאה האמיתית. היתרון הדידקטי שלו הוא שהיחידות נשמרות: אם אנחנו חוזים דקות, גם MAE נמדד בדקות. מדד זה נוח להסבר לאנשים שאינם מפתחים, אך הוא אינו מעניש שגיאות גדולות באותה עוצמה כמו ריבוע השגיאה.

3.3 MSE — ממוצע ריבועי השגיאות

$$\mathbf{MSE = (1/n)\ \Sigma\ (y_i - \hat{y}_i)^2}$$

MSE נותן משקל גדול יותר לשגיאות גדולות כי הוא מעלה כל שגיאה בריבוע. הוא שכיח מאוד בפונקציות אובדן לאימון מודלי רגרסיה, משום שהוא נוח לניתוח מתמטי. אולם היחידות שלו הן יחידות המדידה בריבוע, ולכן פירוש גודל התוצאה פחות מיידי. שורש ה־MSE, כלומר RMSE, מחזיר אותנו ליחידות המקור.

4. חישוב ידני מלא: איזה מודל עדיף?

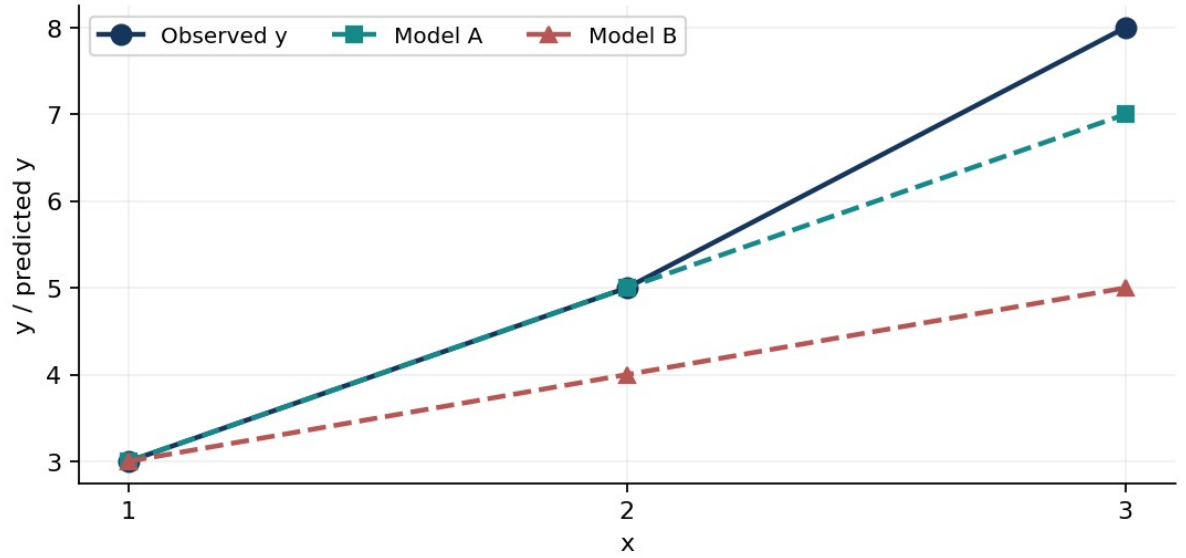
נשתמש בשלוש דוגמאות פשוטות. ערכי הקלט הם 1, 2, 3; ערכי המטרה הם 3, 5, 8. ננסה שני כללים: מודל A עם שיפוע 2 וחיתוך 1, ומודל B עם שיפוע 1 וחיתוך 2. נחשב תחזית לכל תצפית ולאחר מכן את השארית ואת ריבועה.

x	ערך אמיתי y	תחזית A: 2x+1	$(y-\hat{y}A)^2$	תחזית B: x+2	$(y-\hat{y}B)^2$
1	3	3	0	3	0
2	5	5	0	4	1
3	8	7	1	5	9
סכום			1		10

$$\mathbf{MSE(A) = (0 + 0 + 1) / 3 = 1/3 \approx 0.3333}$$

$$\mathbf{MSE(B) = (0 + 1 + 9) / 3 = 10/3 \approx 3.3333}$$

לכן על שלוש הדוגמאות האלה מודל A השיג MSE קטן יותר. אבל איננו יודעים מכאן שהוא יהיה טוב יותר לכל מחשב חדש. ייתכן שהנתונים קטנים מדי, וייתכן שהדוגמאות נבחרו באופן שאינו מייצג את האוכלוסייה. הבחנה זו תקפה לכל מדד: המדד מתאר את הביצועים על הנתונים שעליהם חושב.



איור: השוואת התחזיות של שני מודלים לאותן שלוש תצפיות. הפער בין קו לנקודת אמת הוא השגיאה.

```
import numpy as np
x = np.array([1, 2, 3])
y = np.array([3, 5, 8])
pred_a = 2 * x + 1
pred_b = x + 2
mse_a = np.mean((y - pred_a) ** 2)
mse_b = np.mean((y - pred_b) ** 2)
print("A:", mse_a)
print("B:", mse_b)
```

לפני ההרצה, נסו לחזות את המספרים שיתקבלו. לאחר מכן השוו בין הפלט לחישוב הידני. אם התוצאות אינן זהות, אל תחליפו מיד את החישוב: בדקו אילו ערכים נכנסו למערך ובאיזה שלב.

5. מה פירוש "אימון"? התאמת הפרמטרים

לאחר שבחרנו דרך למדוד שגיאה, אפשר לנסות לשנות את פרמטרי המודל כך שהשגיאה תקטן. יש אלגוריתמים שמוצאים את הפתרון באמצעות נוסחה; אחרים מתחילים מערכים כלשהם ומעדכנים אותם בהדרגה. לכן המילה "אימון" אינה מציינת בהכרח אותה שיטת חישוב בכל המודלים.

ברגרסיה ליניארית אפשר למצוא מקדמים באמצעות שיטת ריבועים פחותים. בהכנה ללמידה עמוקה חשוב להבין גם את הרעיון האיטרטיבי: מתחילים מניחוש, מחשבים שגיאה, בודקים באיזה כיוון שינוי הפרמטרים צפוי להקטין את השגיאה ומבצעים צעד קטן. ברשתות עצביות התהליך יהיה עשיר יותר ונלמד אותו בהמשך.

5.1 הדגמת צעד אחד של Gradient Descent — הרחבת גשר

נתחיל בדוגמה עם נתון יחיד כדי להתמקד במשמעות הפעולה. נניח שמודל פשוט מנבא $\hat{y}=wx$, כאשר $x=2$ והערך האמיתי $y=5$. נבחר תחילה $w=1$ ונחשב את השגיאה הריבועית.

$$L(w) = (2w - 5)^2$$

$$L(1) = (2 - 5)^2 = 9$$

כדי לדעת כיצד לשנות את המשקל נבחן את נגזרת פונקציית השגיאה. אין צורך בשלב זה לגזור פונקציות רבות; נשתמש בתוצאה ונפרש אותה. בפרק המתמטיקה של הלמידה העמוקה נלמד את הגזירה בצורה שיטתית.

$$dL/dw = 4(2w - 5)$$

$$dL/dw \mid (w=1) = -12$$

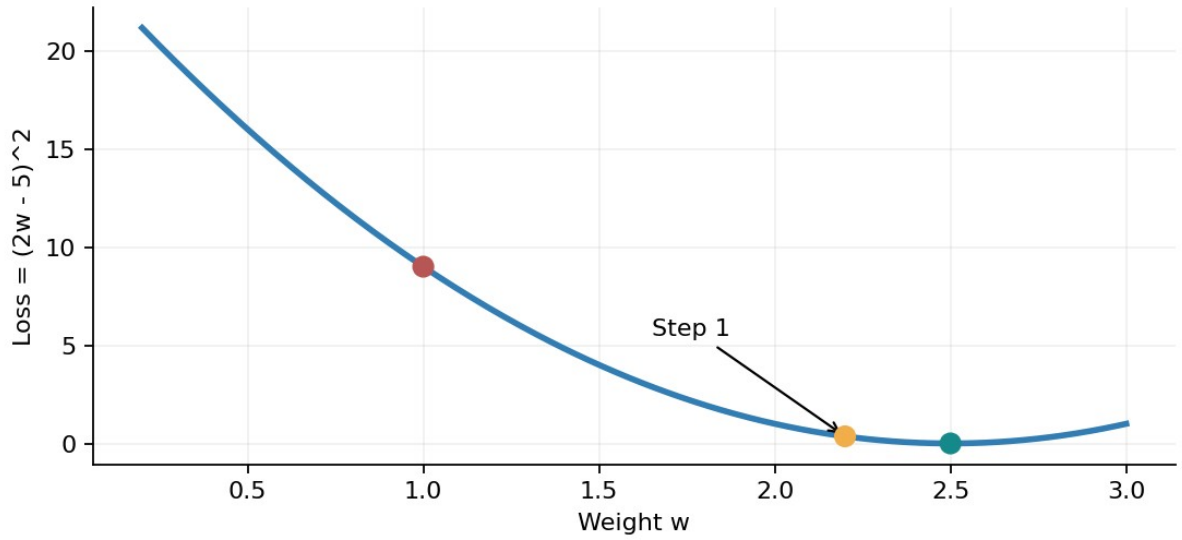
הנגזרת שלילית בנקודה זו. פירוש הדבר: אם נגדיל מעט את w, השגיאה אמורה לקטון. כלל העדכון מחסר מן המשקל את קצב הלמידה כפול הנגזרת; במקרה של נגזרת שלילית, החיסור גורם להגדלת המשקל.

$$w_{new} = w_{old} - \eta \, (dL/dw)$$

$$w_{new} = 1 - 0.1 \times (-12) = 2.2$$

$$L(2.2) = (4.4 - 5)^2 = 0.36$$

השגיאה ירדה מ־9 ל־0.36. זו דוגמה אחת בלבד, ולא הבטחה שכל גודל צעד ישפר תמיד את התוצאה. אם קצב הלמידה גדול מדי, נוכל לדלג מעל המינימום ואף להגדיל את השגיאה. זהו בדיוק אחד הנושאים שנחקור בהמשך הלמידה העמוקה.



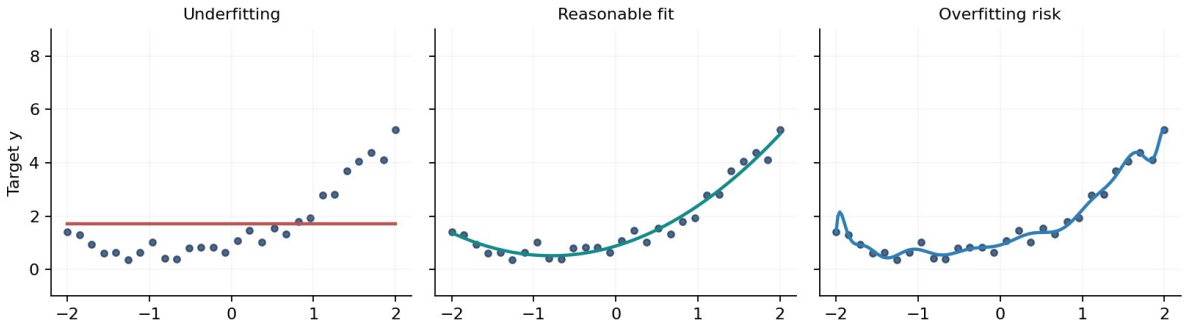
איור: פונקציית שגיאה כפונקציה של משקל יחיד. צעד לעבר המינימום עשוי להקטין את השגיאה.

```
w = 1.0
x = 2.0
y = 5.0
learning_rate = 0.1
loss_before = (w * x - y) ** 2
gradient = 2 * (w * x - y) * x
w = w - learning_rate * gradient
loss_after = (w * x - y) ** 2
print(w, loss_before, loss_after)
```

בדיקת הבנה: מה ישתנה אם קצב הלמידה יהיה 1 במקום 0.1? חשבו את הצעד ביד לפני שתריצו את הקוד, ונסו להסביר את התוצאה בעזרת גרף הפונקציה.

6. הצלחה באימון אינה הצלחה בעולם האמיתי

מודל עשוי לשנן דפוסים שמופיעים בנתוני האימון, כולל רעש, מקרים חריגים או פרטים ייחודיים שאינם חוזרים באוכלוסייה. אם הוא מצליח מאוד על נתוני האימון אבל נכשל על דוגמאות חדשות, יש חשד להתאמת יתר — Overfitting. מצב הפוך, שבו המודל פשוט מדי ואינו מזהה אפילו את הדפוס הבסיסי בנתונים, נקרא Underfitting.

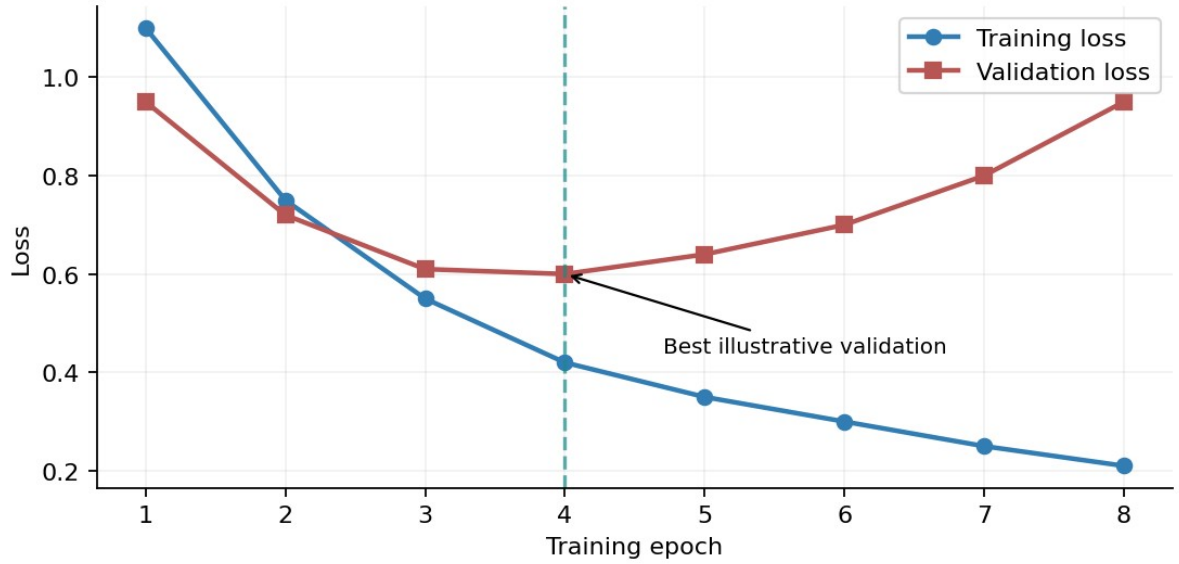


איור: שלוש דרגות התאמה: מודל פשוט מדי, מודל סביר ומודל מורכב מאוד שיכול לעקוב אחרי רעש המדגם.

חשוב להיזהר מהכללה: מודל מורכב אינו בהכרח מודל גרוע, ומודל פשוט אינו בהכרח טוב. איכות ההתאמה תלויה במורכבות הבעיה, בכמות הנתונים, ברמת הרעש ובאופן ההערכה. ההחלטה צריכה להסתמך על נתונים שאינם משמשים לבחירת הפרמטרים.

6.1 כיצד קוראים עקומות אימון?

במודלים מאומנים בתהליך איטרטיבי אפשר לצייר את השגיאה לאחר כל תקופת אימון (Epoch). כאשר שגיאת האימון יורדת ושגיאת הוולידציה יורדת יחד איתה, מתקבלת עדות לכך שהתהליך משפר את הכללת המודל. כאשר שגיאת האימון ממשיכה לרדת אך שגיאת הוולידציה מתחילה לעלות באופן עקבי, ייתכן שהמודל מתאים את עצמו לפרטי האימון מעבר לרצוי.



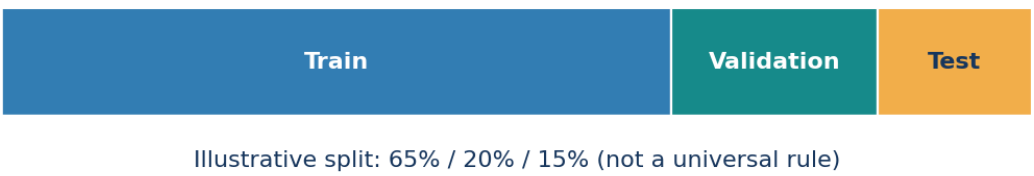
איור: עקומות אימון לדוגמה בלבד: לאחר תקופת אימון מסוימת, שגיאת הוולידציה עולה אף ששגיאת האימון יורדת.

הערה חשובה: עקומת אימון חריגה אחת אינה הוכחה מתמטית ל־Overfitting. יש לבחון כיצד נוצרו הקבוצות, האם חל שינוי בהתפלגות, האם ההבדלים יציבים בניסויים חוזרים והאם המדד מתאים לבעיה.

איזו בדיקה נבצע?	מה אפשר לחשוד?	דפוס
בדיקת מאפיינים, איכות נתונים ומורכבות	Underfitting או בעיית נתונים	שגיאת אימון גבוהה ושגיאת ולידציה גבוהה
בדיקה עצמאית, רגולריזציה או יותר נתונים	Overfitting אפשרי	אימון נמוך מאוד, ולידציה גבוהה
בדיקת Test חיצונית ובחינת מגבלות	הכללה טובה על הדגימה שנבדקה	אימון וולידציה דומים ונמוכים

7. מדוע מחלקים נתונים? אימון, בחירה ובדיקה

בפרק 5.2 של נושאי הליבה הכרנו Train, Validation ו־Test ואת K-Fold. כאן נשתמש בידע כדי לתכנן מערכת שלמה. קבוצת Train מאפשרת להתאים פרמטרים; Validation מסייעת לבחור מודל והיפר־פרמטרים; Test נועדה לתת הערכה סופית לאחר שהבחירות כבר התקבלו. אם נשתמש ב־Test שוב ושוב כדי לבחור מודלים, היא כבר לא תהיה מבחן עצמאי אמיתי.

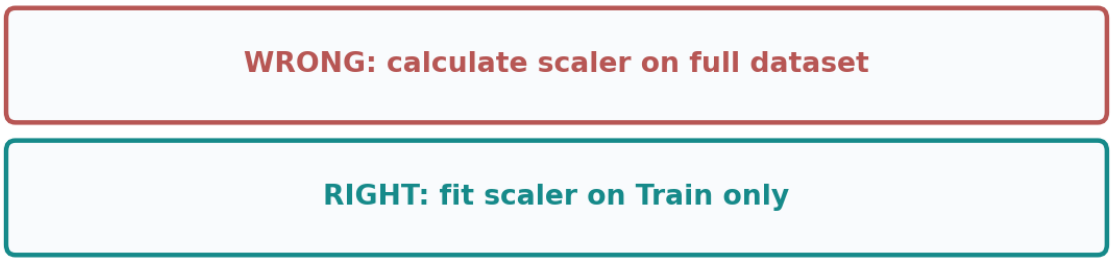


איור: חלוקת נתונים לדוגמה בלבד. אחוז החלוקה תלוי בכמות הנתונים, באיזון ובמבנה הזמן או הקבוצות.

אם יש כמה רשומות מאותו מחשב, צריך לשקול חלוקה לפי מזהה מחשב כדי שלא יהיו רשומות כמעט זהות גם באימון וגם בבדיקה. כאשר המטרה היא לחזות אירועים בעתיד, לעיתים נדרשת חלוקה לפי זמן: עבר לאימון ותקופה מאוחרת לבדיקה. חלוקה אקראית עלולה להיות בלתי מתאימה במקרה זה.

7.1 זליגת מידע — טעות שמציגה מודל מוצלח מדי

זליגת מידע מתרחשת כאשר מידע שלא היה זמין במועד החיזוי משפיע על בניית המודל או על הערכתו. לדוגמה, אם נשתמש במשך התיקון בפועל כאחד המאפיינים של המחשב, המודל יקבל במישרין או בעקיפין את התשובה. גם חישוב ממוצע וסטיית תקן על כל המאגר לפני חלוקה ל־Train/Test עלול לשלב מידע מן הבדיקה בתהליך האימון.



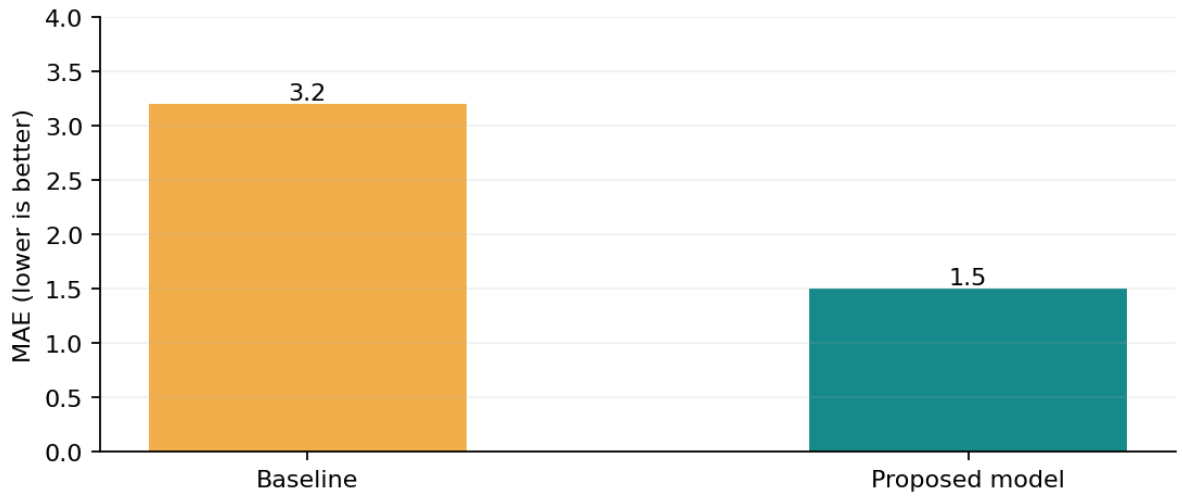
איור: התאמת פעולות עיבוד צריכה להישען על נתוני האימון בלבד; הנתונים האחרים עוברים באמצעות *transform* ההתאמה הקיימת.

הפתרון המקצועי הוא להפריד תחילה קבוצות ולהכניס פעולות עיבוד נלמדות ל־Pipeline. בתוך Cross-Validation הפעולות נלמדות מחדש על חלק האימון של כל קיפול. כך תוצאות ההערכה קרובות יותר למצב שבו המודל יראה דוגמה חדשה באמת.

8. מהו קו בסיס (Baseline) ומדוע הוא הכרחי?

מודל שמקבל ציון מסוים יכול להיראות מרשים, אך ללא השוואה אין לנו נקודת ייחוס. בבעיית רגרסיה אפשר לבנות קו בסיס שמנבא תמיד את ממוצע ערכי המטרה באימון. בבעיית סיווג אפשר לנבא תמיד את המחלקה הנפוצה ביותר, או להשתמש בכלל עסקי פשוט. קו הבסיס אינו אמור להיות חכם; הוא אמור להגדיר את הרף שהמודל החדש צריך להתעלות עליו.

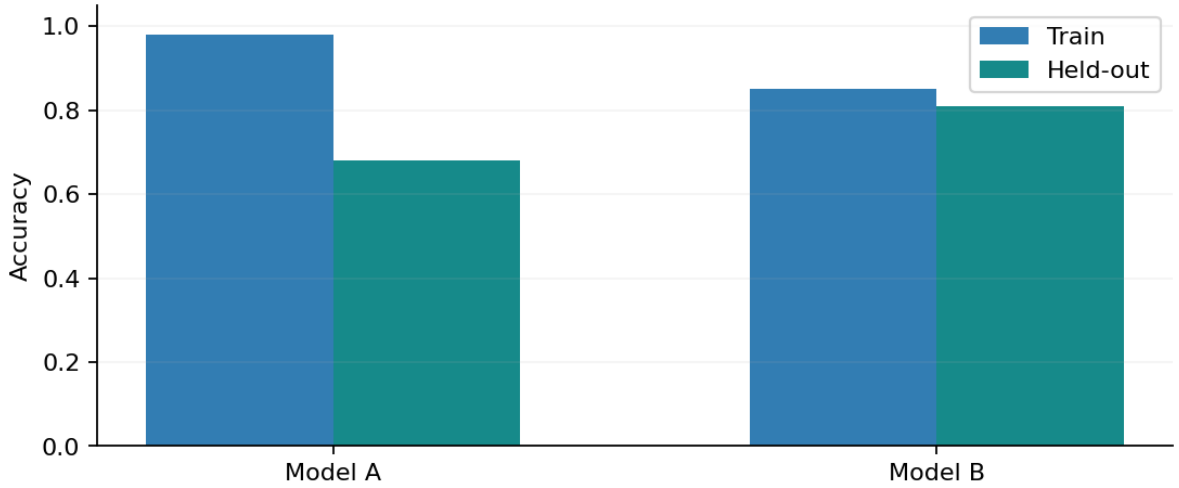
נניח ש־MAE של קו הבסיס הוא 3.2 יחידות זמן, ושל המודל שלנו הוא 1.5. זו ראיה לכך שהמודל עדיף לפי המדד ועל מערך הבדיקה שנבחר. אבל אם מודל חדש מסוכן יותר או יקר יותר להפעלה, השיפור המספרי לבדו אינו מבטיח שזו הבחירה הנכונה.



איור: השוואת *MAE* לדוגמה: תוצאת מודל חייבת להיבחן ביחס לקו בסיס ולמגבלות המדידה.

8.1 מקרה השוואה: דיוק אימון מול יכולת הכללה

לשני מודלים יש ציוני אימון ובדיקה שונים. מודל A משיג Accuracy של 98% באימון אך רק 68% בנתוני בדיקה חיצוניים; מודל B משיג 85% באימון ו־81% בבדיקה. אם שניהם נבחנו על אותן דוגמאות ובאותם תנאים, מודל B עדיף לפי Accuracy של הבדיקה. אך לפני שמכריעים סופית נבדוק אילו טעויות כל מודל עושה ואיזה מחיר יש לכל טעות.



איור: דוגמה להבחנה בין הצלחה באימון לביצועי הכללה. הנתונים באיור מומצאים לצורכי הוראה.

9. ניסוי מדעי במודל: איך משפרים בלי להטעות את עצמנו?

חקר מודלים אינו תחרות של הרצת כמה שיותר אלגוריתמים. ניסוי טוב מתחיל בהשערה ברורה: "אולי קנה המידה גורם לכך שמאפיין אחד משתלט על מרחקי KNN". נשנה גורם אחד, נמדוד תוצאה על אותם קיפולים או אותה קבוצת ולידציה, נתעד את התוצאה ונשקול הסברים חלופיים. שינוי כמה גורמים בבת אחת יקשה עלינו לדעת מה השפיע.



איור: מחזור ניסוי: השערה, שינוי מוגדר, מדידה, תיעוד והסבר מגבלות.

ניסוי	מה משנים?	מה מחזיקים קבוע?	מה מודדים?
1 — קו בסיס	סוג המודל	חלוקת הנתונים	MAE ומשך אימון
2 — תקנון	עם/בלי StandardScaler	אותו מודל ואותם קיפולים	MAE או Accuracy
3 — מורכבות	ערך k או מורכבות אחרת	אותם מאפיינים וקיפולים	Train/Validation
4 — הוגנות	בדיקת קבוצות נפרדות	מדד וסף החלטה	שגיאות לפי קבוצות

יומן ניסויים קצר צריך לכלול תאריך, שם מאגר, מספר דוגמאות, מאפיינים, שיטת חלוקה, seed, היפר־פרמטרים, מדדים והסבר למה למדנו מהתוצאה. בפרויקט גמר דרישות התיעוד אינן קישוט: הן מאפשרות לשחזר את הניסוי ולהגן על החלטות מקצועיות.

10. מעבדת Python מודרכת — קו בסיס לעומת רגרסיה

להלן נתוני הוראה סינתטיים. הקוד מפריד ל־Train/Test, בונה קו בסיס של ממוצע ומודל ליניארי, ומחשב MAE על אותה קבוצת בדיקה. הספר מסביר את התהליך במלואו; במחברת התלמיד הנלווית יהיו ניסויים חדשים להשלים ללא פתרונות.

```
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.dummy import DummyRegressor
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error

X = np.arange(1, 21).reshape(-1, 1)
y = 12 + 4 * X.ravel() + np.array([
    0, 1, -2, 1, 0, -1, 2, -2, 1, 0,
    -1, 1, 2, -1, 0, 1, -2, 2, 0, -1
])
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42
)
baseline = DummyRegressor(strategy="mean")
model = LinearRegression()
baseline.fit(X_train, y_train)
model.fit(X_train, y_train)
for name, estimator in [("Baseline", baseline), ("Linear", model)]:
    prediction = estimator.predict(X_test)
    print(name, mean_absolute_error(y_test, prediction))
```

הסבר: np.arange יוצר ערכי מאפיין מ־1 עד 20; reshape הופך אותם למטריצה של עשרים שורות ועמודה אחת, כי scikit-learn מצפה שמאפיינים יוגשו במבנה דו־ממדי. מערך y מכיל ערך בקירוב ליניארי עם רעש קטן. שתי השיטות מותאמות רק על Train ונבחנות על אותם X_test ו־y_test. השוואת שתי התוצאות היא הוגנת מבחינת קבוצת הבדיקה — אך אם נבחר מודל בעקבותיה, אסור לדווח על אותו Test כאילו היה מבחן סופי עצמאי.

למה השתמשנו במאגר קטן ומומצא? כדי לבודד את הרעיון המתמטי ולבדוק תחילה שקוד הבסיס פועל. בניסוי אמיתי חייבים להבין כיצד הנתונים נאספו, האם דוגמאות עצמאיות זו מזו, מה מייצג כל מאפיין ומהן ההשלכות של טעות חיזוי.

11. מתי מודל מצליח מספרית אך לא מתאים לפרויקט?

נניח שמודל מסווג מחשבים פגומים ומחמיץ מעט תקלות חמורות, אך מסמן כמעט כל מחשב כתקול. Recall עשוי להיות גבוה, אך השירות לא יהיה שימושי. לחלופין, מודל יכול להצליח בבדיקה פנימית אך להיכשל בשבוע הבא בגלל החלפת חומרה, שינויים בסביבת העבודה או מדידות חדשות. לכן פיתוח מערכת AI כולל גם בדיקת איכות הנתונים, שיקולי שימוש, פרטיות ובקרה לאחר ההטמעה.

11.1 ארבע שאלות שמפתחים חייבים לשאול

- האם הנתונים מייצגים את מי או את מה שהמערכת תפגוש בעתיד?
- האם המאפיינים יהיו ידועים באמת בזמן החיזוי?
- מהי עלות טעות מהסוג החמור יותר, והאם המדד שבחרנו משקף אותה?
- האם אדם יוכל להבין מתי המודל אינו בטוח או מתי צריך להעביר החלטה לבדיקה אנושית?

12. קשר ישיר לפרק הבא: פרספטרון ורשתות עצביות

בפרקים הקודמים השתמשנו במסווגים שמנבאים על סמך קרבה של דוגמאות או מרכזי מחלקות. עכשיו אנחנו מוכנים להכיר מודל שמשתמש במשקולות: עבור וקטור מאפיינים x ומשקולות w, מחשבים סכום משוקלל ומוסיפים הטיה b. במסווג בינארי אפשר להעביר את התוצאה דרך כלל החלטה.

$$z = w \cdot x + b$$

הרעיון החדש אינו עצם המכפלה הפנימית, שכבר למדנו בפרק הווקטורים. הרעיון החדש הוא כיצד משנים משקולות בעקבות שגיאות. בפרספטרון נלמד כלל עדכון המיועד לסיווג; בהמשך נלמד אופטימיזציה המבוססת על נגזרות, ובפרקים על רשתות נרחיב את העיקרון למספר שכבות ול־Backpropagation. כך כל נושא חדש נשען על מושגים שכבר הבנו.

13. טעויות נפוצות ותיקונן

כיצד מתקנים?	טענה שגויה
הביצועים באימון אינם מוכיחים הכללה; דרושה הערכה חיצונית.	אם Accuracy באימון הוא 100%, המודל מצוין.
בחלק מהמודלים פרמטרים נלמדים מתוך נתונים; היפר־פרמטרים מכוונים בנפרד.	כל פרמטר במודל נבחר מראש.
שניהם מתארים שגיאה אך מענישים גדלים שונים באופן שונה.	MSE ו־MAE מודדים אותו הדבר.
שימוש ב־Test לבחירה פוגע בעצמאותה של ההערכה הסופית.	המודל שניצח על Test הוא הסופי.
מורכבות עשויה גם להתאים רעש ולפגוע בביצועים מחוץ לאימון.	ככל שמוסיפים מורכבות, המודל תמיד משתפר.
ייתכן ששגיאת ולידציה לא תרד או אף תעלה.	כשיש ירידה בשגיאת אימון, יש שיפור אמיתי.
התאמת התקנון חייבת להתבצע על Train בלבד.	אפשר לתקנן את כל המאגר ואז לחלק.
יש להתחשב גם בעלות, יציבות, הוגנות ודרישות המערכת.	שיפור מספרי קטן מצדיק שימוש במודל.

14. תרגול מדורג — ללא פתרונות

רמה א׳ — מושגים וחישובים

- א1. הסבירו בשתי פסקאות את ההבדל בין מודל לבין אלגוריתם אימון, ובחרו דוגמה מתוך רגרסיה.
- א2. עבור $y=(4,6,8)$ ותחזיות $\hat{y}=(5,5,8)$, חשבו את השאריות, MAE ו־MSE, וציינו את יחידות המדדים.
- א3. סווגו את המונחים הבאים לפרמטר, היפר־פרמטר או מאפיין: מספר שכנים, מספר התקלות, שיפוע רגרסיה, קצב למידה.
- א4. מה ההבדל בין תחזית של מודל לבין הערך האמיתי? מדוע איננו משתמשים בתווית האמיתית בזמן חיזוי של מקרה חדש?

רמה ב׳ — פרשנות, ניסויים וגרפים

- ב1. נתונות שתי עקומות: שגיאת אימון יורדת מ־1.2 ל־0.2, ושגיאת ולידציה יורדת תחילה ל־0.6 ואז עולה ל־0.9. הסבירו מה ייתכן שהתרחש.
- ב2. במאגר יש כמה רשומות לכל מחשב. הציעו חלוקה ל־Train ול־Test שמצמצמת זליגה.

• ב3. הציעו קו בסיס לבעיה של חיזוי זמן תיקון, והסבירו מדוע ממוצע האימון הוא בחירה סבירה להשוואה.

• ב4. מדוע אין לכוון עשרים היפר־פרמטרים על בסיס תוצאה חוזרת של אותה קבוצת Test?

• ב5. תכננו ניסוי המבודד את השפעת התקנון על KNN. אילו דברים חייבים להיות זהים בשתי ההרצות?

רמה ג׳ — תכנון מערכת וחשיבה ביקורתית

• ג1. מערכת חיזוי תקלות נבדקה על מחשבים משנת 2024, וכעת מותקנים מחשבים מדגם חדש. איזה סיכון להכללה נוצר? תכננו בדיקת ניטור.

• ג2. מודל A משיג Accuracy 0.90 אך מחמיץ יותר תקלות קשות ממודל B שמשיג 0.85. מה עוד צריך לדעת כדי לבחור?

• ג3. קוד מתקנן את כל X לפני train_test_split. זהו את הבעיה וכתבו בפסאודו־קוד את סדר הפעולות הנכון.

• ג4. בנו טבלת ניסויים של שלוש גרסאות מודל. כללו השערה, שינוי מבוקר, מדדים, מסקנה ומגבלה.

15. שאלת הערכה מקורית בסגנון בגרות

דודו בונה מערכת שמנבאת את משך התיקון של מחשב חדש. הוא בדק מודל ליניארי על שלוש תצפיות: הערכים האמיתיים הם (10, 12, 16), והתחזיות הן (9, 14, 15). המודל אומן על נתונים אחרים, אך במהלך בנייתו השתמש דודו בקבוצת הבדיקה כדי לבחור את קצב הלמידה.

• א. חשבו את שאריות החיזוי ואת MAE. הציגו את שלבי החישוב.

• ב. חשבו את MSE והסבירו מדוע הוא מעניש יותר שגיאה גדולה.

• ג. הסבירו מדוע הבחירה בקצב למידה בעזרת Test יוצרת בעיה בהערכת המודל.

• ד. הציעו תהליך חלוקה ובחירה מתוקן, וציינו מה ידווח בסוף.

• ה. ציינו סיכון אחד שעשוי להתעורר כאשר משתמשים במודל בבית ספר אחר.

הערה: זו שאלה מקורית שנכתבה עבור הספר לצורכי הכנה, ואינה מועתקת מבחינת בגרות רשמית. תשובות וקריטריונים לניקוד יופיעו בגרסת המורה.

16. מעבדת Colab — מהספר אל ניסוי עצמאי

מחברת התלמיד הנלווית מכילה 12 משימות: זיהוי מאפיינים ומטרות; השוואת תחזיות; חישוב MAE ו־MSE; שינוי משקל; ניסוי בקצב למידה; מודל קו בסיס; הפרדת Train/Test; בדיקת Leakage; השוואת מודלים; בניית יומן ניסויים; ניתוח עקומות; וסיכום להערכה בעל־פה. במקומות המיועדים לעבודה יופיע TODO או תא חסר. אין במחברת פתרונות מלאים למשימות.

התוצר הסופי של המעבדה אינו רק מחברת שרצה ללא שגיאות. על התלמיד להגיש שני גרפים, טבלה אחת של ניסויים ופסקה שבה הוא מסביר מדוע בחר במודל מסוים, מה הראיות לכך ומה אינו יכול להסיק מהנתונים.

17. מפת קשרים בין שלוש התכניות

שימוש עתידי	מה כבר נלמד ומה העומק כאן	מסלול
ניתוח בסיסי והסבר אלגוריתמים	היכרות עם אימון, חיזוי, מדדים והכללה	יחידה 3 — כיתה י׳
הצדקת תהליך מדעי והכנה	פונקציות מרחק, מדדי הצלחה,	נושאי ליבה — יחידה 5

שימוש עתידי	מה כבר נלמד ומה העומק כאן	מסלול
לבגרות	ולידציה וחקר היפר־פרמטרים	
פרספטרון, Gradient Descent, רשתות ופרויקט גמר	תהליך האימון כמערכת של פרמטרים, שגיאה, בחירה ובדיקה	התמחות למידה עמוקה — פרק 8

עוגן התכנית: במפת ההתמחות שבמאגר GitHub מופיע פרק 8 ״מושגי יסוד בלמידת מכונה״ בהיקף 10 שעות, בין KNN לפרספטרון. סדר תתי־הנושאים בפרק זה הוא עיבוד דידקטי שלנו, ולא העתק מאומת של תתי־סעיפי ה־PDF הרשמי.

18. הצעת חלוקת זמן להוראה — 10 שעות

שעות	מוקד	שיעור
2	מודל, נתונים, תחזית, פרמטר והיפר־פרמטר	1
2	שאריות, MAE, MSE, קו בסיס וחישוב ידני	2
2	אימון, שינוי פרמטרים, העמקת Overfit/Underfit וקריאת עקומות	3
2	הכללה, זליגת מידע, ניסוי מבוקר ופרשנות	4
2	מעבדת Python, שאלת הערכה, יומן ניסויים והכנה לפרספטרון	5

זוהי חלוקת שעות מוצעת למורה (שעה לימודית = 45 דקות), לא חלוקה רשמית שהועתקה מהסילבוס.

19. מילון מושגים

מונח	פירוש קצר
Model	פונקציה המפיקה תחזית מקלט
Training	תהליך התאמת פרמטרים מנתוני אימון
Parameter	ערך שהמודל מתאים בהתאם לנתונים
Hyperparameter	הגדרה המכוונת את תהליך האימון או צורת המודל
Loss	פונקציה המודדת את הפער בין תחזית לתוצאה
Epoch	מעבר אחד על נתוני האימון, לפי שיטת האימון
Generalization	יכולת לתפקד היטב על נתונים חדשים
Overfitting	התאמת יתר שעלולה לפגוע בהכללה
Baseline	כלל פשוט המשמש אמת מידה להשוואה
Data Leakage	שימוש במידע שלא אמור להיות זמין בעת האימון או החיזוי
Reproducibility	יכולת לשחזר ניסוי בתנאים מתועדים

20. חקר מקרה מסכם — האם המודל החדש באמת עדיף?

דודו אסף שישה מחשבים ותיעד עבור כל אחד מהם את מספר התקלות ואת זמן התיקון בפועל. כדי להמחיש את הבדיקה בלי להסתבך במאגר גדול, נתוני האימון הם ארבעת המחשבים הראשונים, ונתוני

הבדיקה הם שני מחשבים חדשים. הערכים פשוטים ומומצאים לצורך המחשה. בניסוי אמיתי אסור לבחור חלוקה כזו רק מפני שהיא גורמת למודל להצליח.

קבוצה	מספר תקלות x	זמן תיקון בפועל y
Train	1	20
Train	2	24
Train	3	28
Train	4	32
Test	5	34
Test	6	38

20.1 קו הבסיס — תחזית אחת לכולם

נגדיר Baseline שמנבא תמיד את ממוצע ערכי המטרה של האימון. שימו לב שאיננו משתמשים כלל בשני נתוני Test לחישוב הממוצע. הם שמורים לבדיקת הכללה.

mean(Train) = (20 + 24 + 28 + 32) / 4 = 26

לכן שתי תחזיות קו הבסיס הן 26. עבור המחשב הראשון ב־Test, שהזמן האמיתי שלו הוא 34, הטעות המוחלטת היא 8. עבור המחשב השני, שהזמן האמיתי שלו הוא 38, הטעות היא 12. ממוצע הטעויות המוחלטות הוא:

MAE_baseline = (8 + 12) / 2 = 10

20.2 מודל שלומד את הקשר בנתוני Train

כעת נסתכל על ארבע דוגמאות האימון בלבד. כל תוספת של תקלה אחת מלווה בעלייה של ארבע יחידות זמן. ישר שעובר בכל ארבע הנקודות מקיים את המשוואה הבאה:

ŷ = 4x + 16

בבדיקה החיצונית המודל מנבא 36 למחשב עם חמש תקלות ו־40 למחשב עם שש תקלות. הערכים האמיתיים הם 34 ו־38. לכן בכל אחת משתי הדוגמאות הטעות המוחלטת היא 2:

MAE_model = (2 + 2) / 2 = 2

המודל הצליח יותר מקו הבסיס על שתי דוגמאות הבדיקה האלה. זהו ממצא, אך לא הוכחה שהוא תמיד יצליח. המדגם קטן מאוד, שתי נקודות ה־Test הן מחוץ לטווח ערכי x של Train — כלומר יש כאן גם אקסטרפולציה — והקשר במחשבים אחרים עלול להיות שונה. ההבחנה בין ממצא לבין מסקנה כללית היא חלק מהחשיבה המדעית שאנחנו רוצים לפתח.

20.3 שלוש שאלות של בוחן פרויקט גמר

- מדוע אינך יכול לקבוע שהמודל שלך מדויק תמיד, אף ש־MAE שלו קטן?
- מה היה קורה לו היית משתמש גם בשני מחשבי Test לצורך בחירת השיפוע?
- איזה נתון אמיתי היית אוסף כדי לבדוק אם מספר התקלות מספיק לחיזוי משך התיקון?

21. דף תכנון ניסוי לתלמיד

לקראת מיני־פרויקט או פרויקט גמר, כל תלמיד יוכל להשתמש בטבלה הבאה כדי לתכנן ניסוי. לפני שרצים לקוד, ממלאים את העמודה הראשונה והשלישית. לאחר ההרצה ממלאים את התוצאות והפירוש. כך שומרים על ניסוי שניתן לשחזר ועל שיקול דעת שקוף.

מה אכתוב ביומן הניסוי?	השאלה
למשל: האם StandardScaler משפר את KNN על הנתונים שלי?	מהי מטרת הניסוי?
מקור המאגר, מספר רשומות, מאפיינים, דרך איסוף ורישוי	מהם הנתונים?
קבוצת Test, אותם קיפולים, אותו מדד ואותם מאפיינים	מה קבוע?
גורם יחיד — למשל תקנון עם/בלי	מה אני משנה?
מדד איכות ובמידת הצורך זמן חישוב או עלות	מה אני מודד?
מסקנה מוגבלת לנתונים שנבדקו והסבר חלופי אפשרי	מה למדתי?
בדיקה נוספת, איסוף נתונים, שיפור או דחיית הרעיון	מה הצעד הבא?

דוגמה לניסוח מסקנה מקצועית: ״בשלושה קיפולי ולידציה על נתוני האימון, התקנון שיפר את ממוצע הדיוק של KNN. לכן בחרנו להמשיך לבדוק מודל עם תקנון. עם זאת, איננו יודעים עדיין אם השיפור יישמר עבור נתונים ממחשבים מדגם חדש, ונידרש לבחון זאת במועד מאוחר יותר.״

דוגמה לניסוח שאינו מקצועי: ״התקנון תמיד משפר KNN, ולכן כל המודלים צריכים תקנון״. משפט כזה הופך תוצאה מקומית לכלל אוניברסלי בלי נתונים שמצדיקים זאת.

22. סיכום מושגי — מה חשוב לקחת לפרק הפרספטרון?

בפרק זה ראינו שמודל מפיק תחזית באמצעות פרמטרים, שתהליך האימון מתאים את הפרמטרים לנתונים, ושפונקציית השגיאה מאפשרת להגדיר מה נחשב שיפור. למדנו ששיפור על נתוני האימון אינו מספיק, ולכן מפרידים בין אימון, בחירה והערכה סופית. בדקנו כיצד זליגת מידע, חלוקה לא נכונה וקנה מידה עלולים לגרום לנו להעריך מודל באופן מוטעה.

בפרק הבא נפגוש את הפרספטרון. במקום לזכור רק נוסחת סיווג חדשה, נחפש בו את אותם מרכיבים: מהו וקטור הקלט? מהם המשקולות וההטיה? מהי התחזית? מתי מתרחשת טעות? כיצד מתקנים את המשקולות? באילו נתונים בודקים שהשינוי הועיל? הקישור בין השאלות הללו הופך את הלמידה ממאגר פקודות להבנה של מערכת לומדת.

מקורות מקצועיים ועוגני קריאה

עוגן תכנית: משרד החינוך — התמחות בלמידת מכונה ולמידה עמוקה, פרק 8 (10 שעות), על פי מפת התכנית השמורה במאגר Data-Science-ML-Course; נדרש אימות סופי של תתי־הסעיפים מול PDF הרשמי המעודכן.

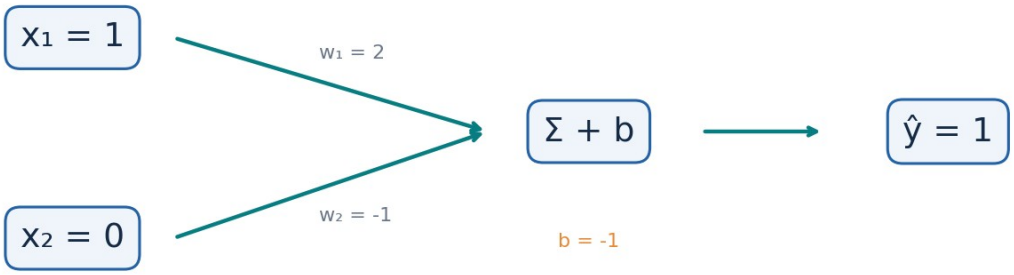
מקורות מרכזיים להשראה מקצועית: Luis Serrano, Grokking Machine Learning; Aurélien Géron, Hands-On Machine Learning; James et al., An Introduction to Statistical Learning with Applications in Python; Deisenroth et al., Mathematics for Machine Learning; Goodfellow, Bengio & Courville, Deep Learning. ההסברים והדוגמאות כאן מקוריים ולא הועתקו מהספרים.

לבחינת המימושים והטעויות: scikit-learn User Guide (Common Pitfalls); Google Machine Learning Crash Course; מאגר הקורס: github.com/samiabo75/Data-Science-ML-Course.

פרק 9

הפרספטרון — ממכפלה פנימית למודל שלומד

ספר לימוד לתלמיד | התמחות בלמידת מכונה ולמידה עמוקה (5 יחידות) | 30 שעות לפי מפת התכנית



איור: הפרספטרון מקבל וקטור מאפיינים, מחשב סכום משוקלל ומחליט על מחלקה.

המטרה שלנו אינה רק לדעת להפעיל פונקציה. נלמד לתאר את ההחלטה שהמודל מקבל, לחשב אותה ביד, להבין מדוע הוא טועה, לעקוב אחר עדכון המשקולות ולהסביר כיצד הוא משתפר לאורך האימון.

מפת למידה

מה אבין ומה אבצע	צעד
מהו נוירון מלאכותי ומה ההבדל בין חיזוי לאימון	1
כיצד פועלים משקולות, הטיה ופונקציית מדרגה	2
כיצד מחשבים z = 0 ותחזית, כולל מקרה z	3
כיצד מעדכנים משקולות בעקבות שגיאה	4
כיצד שינוי משקולות מזיז את קו ההחלטה	5
לא נלמד XOR נלמד ו־ AND למה שער בפרספטרון יחיד	6
כיצד מממשים אלגוריתם עצמאי ומשווים ל־scikit-learn	7
כיצד בודקים יכולת הכללה ומציגים ניסוי בפרויקט	8

ידע קודם: וקטורים, מכפלה פנימית, משוואת ישר, סיווג, NumPy, נתוני X ו־y, Train/Test. ידע על נגזרות אינו תנאי לפרק זה.

1. בעיית הפתיחה: מערכת פשוטה מקבלת החלטה

דודו רוצה לבנות מערכת שתסייע במיון פניות למעבדת המחשבים. לכל פנייה יש שני מאפיינים בינריים: האם המחשב אינו נדלק, והאם הופיעה הודעת שגיאה קריטית. המערכת תחזיר תווית אחת מתוך שתי אפשרויות: 1 — נדרש טיפול דחוף; 0 — אין סימון אוטומטי כדחוף. אלה נתונים מדומים לצורכי לימוד, ולא מדיניות בטיחות אמיתית.

אפשר להתחיל בכתיבת כללים ידניים: אם שני הסימנים מופיעים יחד, סמן דחוף. אבל מערכת למידת מכונה אמורה לקבל דוגמאות שכבר סווגו ולכוונן את הכללים המספריים שלה. הפרספטרון הוא אחד המודלים הפשוטים ביותר שממחישים את העיקרון הזה.

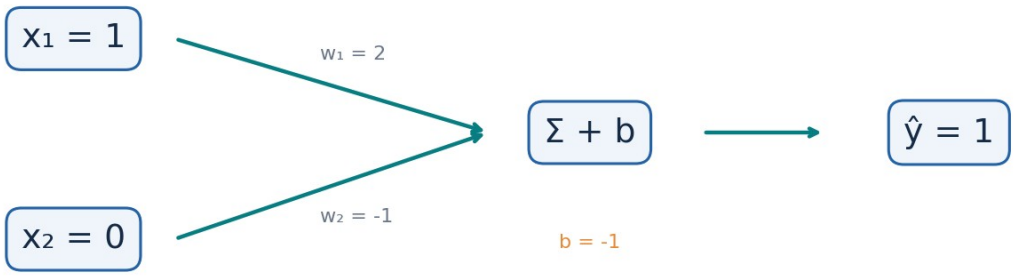
x_1 מחשב לא נדלק	x_2 שגיאה קריטית	y תווית
0	0	0
0	1	0
1	0	0
1	1	1

הטבלה ממחישה את שער AND: רק כאשר שתי התכונות שוות 1, התווית היא 1. בהמשך נשתמש דווקא בדוגמה הפשוטה הזאת כדי לראות כל חישוב וכל תיקון — בלי נתונים רבים שמסתירים את הרעיון.

2. מהו פרספטרון ומה הוא אינו?

פרספטרון הוא מסווג ליניארי בינרי: הוא יוצר ציון על סמך שילוב ליניארי של המאפיינים, ואחר כך ממיר אותו להחלטת מחלקה. חשוב להבחין בין שלושה מושגים: המודל הוא מבנה החישוב; המשקולות וההטיה הם הפרמטרים; אלגוריתם האימון הוא הכלל שמשנה את הפרמטרים לאחר טעויות.

פרספטרון אינו מודל שמפיק הסתברות רק משום שהתוצאה שלו היא 0 או 1. גם אם הציון גדול מאוד, אין משמעות הדבר שהמודל בטוח ב־99%. ציון z הוא ערך של פונקציית החלטה, לא הסתברות מכוילת.



איור: תרשים קלט–חישוב–החלטה; ההטיה מאפשרת להזיז את גבול ההחלטה.

3. מהתכונות לווקטור המאפיינים

בפרק הווקטורים למדנו שלכל תצפית אפשר להצמיד רשימת מספרים באותו סדר. בפרספטרון המספרים נקראים מאפיינים, ולכל מאפיין מוצמד משקל. אם יש שני מאפיינים, נכתוב את הקלט

בתור וקטור $x=(x_1,x_2)$ ואת המשקולות בתור $w=(w_1,w_2)$. אם יש d מאפיינים, שני הווקטורים הם באורך d.

$$x = (x_1, x_2, \ldots, x_d) \qquad w = (w_1, w_2, \ldots, w_d)$$

המשקל מתאר את התרומה הליניארית של המאפיין לציון ההחלטה, כאשר שאר הדברים נשארים קבועים. משקל חיובי מגדיל את הציון כשהמאפיין גדל; משקל שלילי מקטין אותו. אך אין לפרש את המשקל כסיבה בעולם האמיתי או להשוות גדלי משקולות בלי להביא בחשבון את יחידות המדידה.

4. חישוב הציון z — פירוק הנוסחה

$$z = w \cdot x + b = \Sigma(i=1..d)\ w_i x_i + b$$

הסימן · הוא מכפלה פנימית: כופלים רכיבים מתאימים ומחברים את כל המכפלות. האות b היא ההטיה (bias), כלומר מספר שמוסיפים לסכום המשוקלל. ההטיה מאפשרת להזיז את קו ההחלטה גם אם שני המשקולות נשארות ללא שינוי.

דוגמה פתורה: נניח $x=(1,0)$, $w=(2,-1)$, $b=-1$. מציבים כל רכיב במקום שלו:

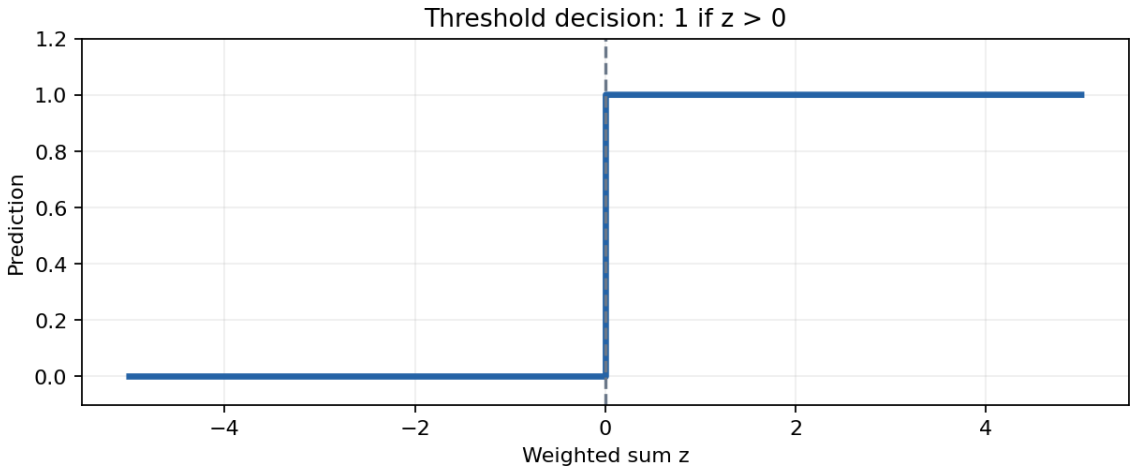
$$z = 2\cdot 1 + (-1)\cdot 0 - 1 = 1$$

קיבלנו $z=1$. זהו ציון חיובי, ולכן לפי כלל ההחלטה שנאמץ בפרק, התחזית תהיה 1. שימו לב: לא שינינו אף משקל. ביצענו חיזוי בלבד; אימון הוא תהליך נפרד.

5. פונקציית מדרגה: איך ציון הופך לתווית?

$$\hat{y} = 1 \ \text{ if } z > 0; \qquad \hat{y} = 0 \ \text{ if } z \le 0$$

בחרנו כלל חד־משמעי גם למקרה הגבולי $z=0$: במקרה זה התחזית היא 0. בחירת הכלל חשובה במיוחד בטבלאות מעקב, משום שהיא משפיעה על השאלה אם נדרשת פעולת עדכון. קיימות הגדרות אחרות בספרים ובספריות, ולכן יש לציין בכל דוגמה איזה כלל משמש אותנו.



איור: פונקציית ההחלטה קופצת מ־0 ל־1 כאשר z עובר מעל אפס.

בדיקת הבנה: אם נשנה רק את ההטיה מ־-1 ל־-3, מה יקרה לציון בדוגמה הקודמת? האם משתנים המאפיינים? האם התווית האמיתית השתנתה?

6. גבול ההחלטה — המשמעות הגיאומטרית

במישור של שני מאפיינים, כל הזוגות המקיימים $z=0$ נמצאים על ישר. מצד אחד של הישר נקבל $z>0$, ומצדו השני $z\le 0$. לכן פרספטרון יחיד יוצר גבול החלטה ליניארי. במרחב של שלושה מאפיינים מתקבל מישור; בממד גבוה יותר — על־מישור.

$$w_1x_1 + w_2x_2 + b = 0$$

$$x_2 = -(w_1/w_2)x_1 - b/w_2 \qquad (\text{when } w_2 \neq 0)$$

כדי לשרטט את הקו, מוצאים שתי נקודות שמקיימות את המשוואה ומחברים ביניהן. אם $w_2=0$, אין לחלק בו: הגבול הוא ישר אנכי בתנאי $w_1\neq 0$. השיפוע הוא $-w_1/w_2$ והחיתוך עם ציר x_2 הוא $-b/w_2$.

עיקרון חשוב: קו ההחלטה אינו חייב לעבור דרך הנקודות של הדוגמאות. תפקידו להפריד מחלקות ולא לבצע רגרסיה על ערכי התווית.

7. מהי טעות סיווג וכיצד מודדים אותה?

לכל דוגמת אימון יש תווית אמיתית y ותחזית $\hat{y}$. טעות מתרחשת כאשר שני הערכים שונים. אפשר לספור את מספר הטעויות בכל מעבר על נתוני האימון. בשונה מרגרסיה, כאן אין מנסים לקרב ערך מספרי רציף אל y באמצעות ישר; מטרתנו היא לשנות את ההחלטה של המודל עבור דוגמאות שסווגו לא נכון.

$$\text{error} = y - \hat{y}$$

פעולה	error	חזוי $\hat{y}$	אמיתי y
לא מעדכנים	0	0	0
לא מעדכנים	0	1	1
מגדילים ציון	+1	0	1
מקטינים ציון	−1	1	0

הטבלה קושרת בין סוג השגיאה לכיוון התיקון. כאשר החמצנו דוגמה חיובית, יש צורך להגדיל את הציון שלה. כאשר סימנו בטעות דוגמה שלילית כחיובית, צריך להקטין את הציון שלה.

8. כלל הלמידה של הפרספטרון

$$w_{new} = w_{old} + \eta(y - \hat{y})x$$

$$b_{new} = b_{old} + \eta(y - \hat{y})$$

האות היוונית η (אֶטָה) מייצגת קצב למידה — מספר חיובי שקובע את גודל הצעד. אם התחזית נכונה, ההפרש $y-\hat{y}$ הוא אפס ולכן לא יהיה שינוי. אם התחזית שגויה, נכפיל את וקטור הקלט ב־η ונוסיף או נחסיר אותו בהתאם לכיוון הטעות.

מדוע מעדכנים גם את b? משום שהגבול אינו תלוי רק בזווית הישר אלא גם במיקומו. שינוי המשקולות לבדו עלול להשאיר את הקו במקום לא מתאים. בהדגמות נשתמש ב־$\eta=1$ כדי לפשט את החישובים; אין פירוש הדבר שזה תמיד קצב הלמידה הטוב ביותר.

דוגמת עדכון יחיד: משקולות התחלתיות $w=(0,0)$, הטיה $b=0$, דוגמה $x=(1,1)$ עם $y=1$. הציון $z=0$ ולכן $\hat{y}=0$, ונוצרה שגיאה $+1$.

$$w_{new} = (0,0) + 1\cdot(+1)\cdot(1,1) = (1,1)$$

$$b_{new} = 0 + 1\cdot(+1) = 1$$

כעת אותה דוגמה תקבל $z=1+1+1=3$ ותסווג כחיובית. אך אסור להסיק מכך שכל הדוגמאות מסווגות נכון: שיפור בדוגמה אחת עלול לפגוע בסיווג דוגמה אחרת.

9. טבלת מעקב מלאה — איך לומדים שער AND?

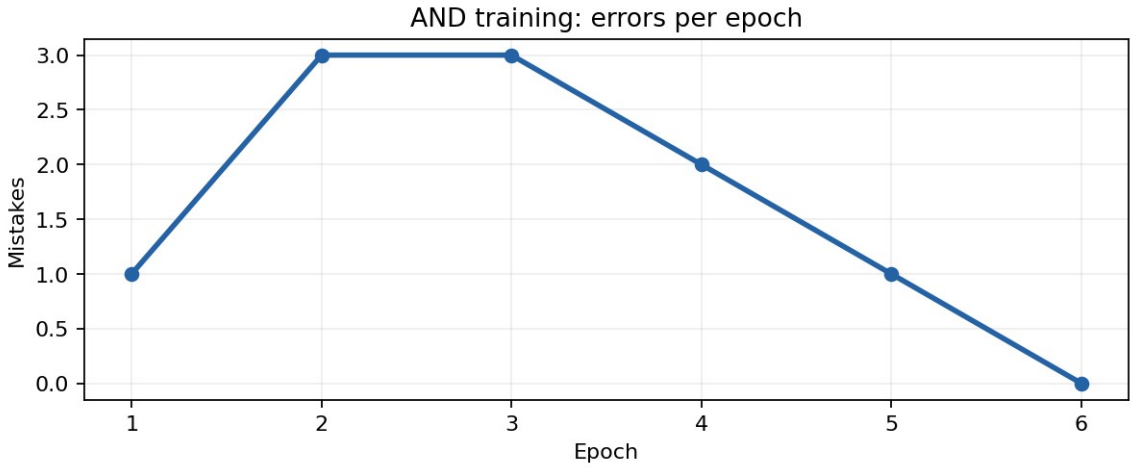
נאמץ סדר מעבר קבוע על ארבע הדוגמאות: (0,0), (0,1), (1,0), (1,1). מתחילים עם w=(0,0), b=0 ו־η=1. בכל שורה מחשבים z באמצעות הערכים הנוכחיים, מפיקים תחזית לפי z>0, מחשבים y−ŷ ורק אז מעדכנים את המשקולות וההטיה. את הערכים החדשים מעבירים לשורה הבאה.

הטבלה הבאה מציגה במלואם את שני המעברים הראשונים. השורה שמסומנת ״לפני״ היא המשקולות וההטיה לפני חיזוי הדוגמה; ״אחרי״ היא המצב לאחר העדכון. המשך התהליך מופיע באיור ובניסוי הקוד.

מעבר	x	y	z	ŷ	y−ŷ	לפני (w₁,w₂,b)	אחרי (w₁,w₂,b)
1	(0, 0)	0	0	0	0	(np.int64(0), np.int64(0), 0)	(0, 0, 0)
1	(0, 1)	0	0	0	0	(np.int64(0), np.int64(0), 0)	(0, 0, 0)
1	(1, 0)	0	0	0	0	(np.int64(0), np.int64(0), 0)	(0, 0, 0)
1	(1, 1)	1	0	0	1	(np.int64(0), np.int64(0), 0)	(1, 1, 1)
2	(0, 0)	0	1	1	-1	(np.int64(1), np.int64(1), 1)	(1, 1, 0)
2	(0, 1)	0	1	1	-1	(np.int64(1), np.int64(1), 0)	(1, 0, -1)
2	(1, 0)	0	0	0	0	(np.int64(1), np.int64(0), -1)	(1, 0, -1)
2	(1, 1)	1	0	0	1	(np.int64(1), np.int64(0), -1)	(2, 1, 0)

פענוח השורה האחרונה במעבר הראשון: מכיוון ש־(1,1) אמורה להיות חיובית אך קיבלה תחזית 0, מעדכנים ל־w=(1,1), b=1. בתחילת המעבר השני הדוגמה (0,0) מקבלת בטעות תחזית 1, ולכן מתקנים את ההטיה ל־0.

חשוב לראות שעדכון אינו מבטיח ירידה מיידית במספר הטעויות בכל שורה או בכל מעבר. האלגוריתם מגיב למקרים בזה אחר זה; כאשר הנתונים ניתנים להפרדה ליניארית, קיימת לו הבטחת התכנסות בתנאים המתאימים.



איור: מספר שגיאות במעברים 1–6 עבור נתוני AND בסדר קבוע; במעבר השישי לא היו טעויות.

תוצאה נבדקת: בסוף התהליך מתקבלות משקולות w=(2,1), הטיה b=−2; כל ארבע הדוגמאות מקבלות את התוויות הנכונות לפי כלל z>0.

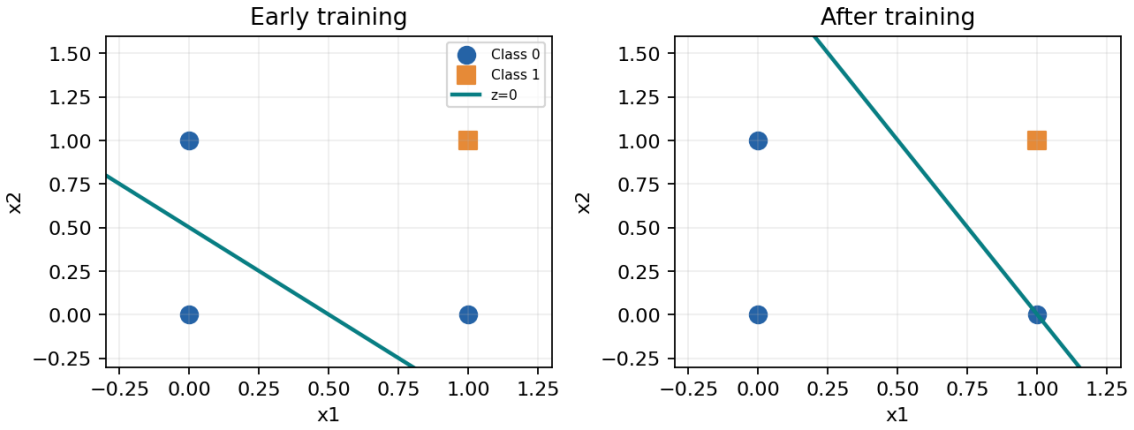
10. מהו מעבר (Epoch), ומה קובע סדר הנתונים?

Epoch הוא מעבר אחד על כל דוגמאות האימון. התהליך יכול לדרוש מספר מעברים, משום שעדכון שבוצע בדוגמה מאוחרת עשוי לשנות את הסיווג של דוגמה שכבר נבדקה במעבר הנוכחי. לכן בודקים שוב את כל הדוגמאות.

סדר הדוגמאות יכול להשפיע על סדר העדכונים ועל המשקולות הסופיות. לשם שחזור מדויק של ניסוי חשוב לשמור סדר קבוע או לקבוע seed כאשר מערבבים את הדוגמאות. הפרספטרון אינו מבטיח למצוא את אותם משקולות בכל ריצה, גם כשקיים יותר מקו אחד שמפריד את המחלקות.

11. כיצד רואים את הלמידה על הגרף?

כל עדכון משנה את המשוואה z=0 ולכן משנה את המיקום או הכיוון של גבול ההחלטה. בתחילת האימון הקו יכול לעבור בצד הלא נכון של חלק מהנקודות. ככל שמתקבלות טעויות ומבוצעים תיקונים, הקו יכול לנוע עד שכל הדוגמאות מופרדות כראוי.



איור: שתי נקודות זמן בהתקדמות האימון. הגבול משתנה עם w ו־b; קו ההפרדה הסופי אינו יחיד בהכרח.

דיון: אם נכפיל את כל המשקולות ואת ההטיה בקבוע חיובי, האם התחזיות ישתנו? לא — סימן z נשמר. אבל אם נכפיל בקבוע שלילי, הסימן מתהפך והמחלקות עלולות להתחלף. זהו חיבור ישיר ללימוד וקטורים וכיוונים.

12. חישוב ידני מאפס: אימון עם שתי תכונות ממשיות

נחזור לסיפור מעבדת המחשבים, אך הפעם נשתמש במאפיינים מספריים שאינם בינריים: שעות מאז התקלה ומספר הודעות שגיאה. נניח שהנתונים הותאמו לקנה מידה סביר, ויש לנו משקולות נתונות w=(0.5,−0.25) והטיה b=−0.5.

עבור מחשב שתכונותיו x=(2,1), הציון הוא 0.5×2 − 0.25×1 − 0.5 = 0.25 ולכן התחזית היא 1. אם התווית האמיתית דווקא 0, טעות החיזוי היא −1. עבור η=0.2 נעדכן את כל רכיבי המשקל באופן יחסי למאפיינים.

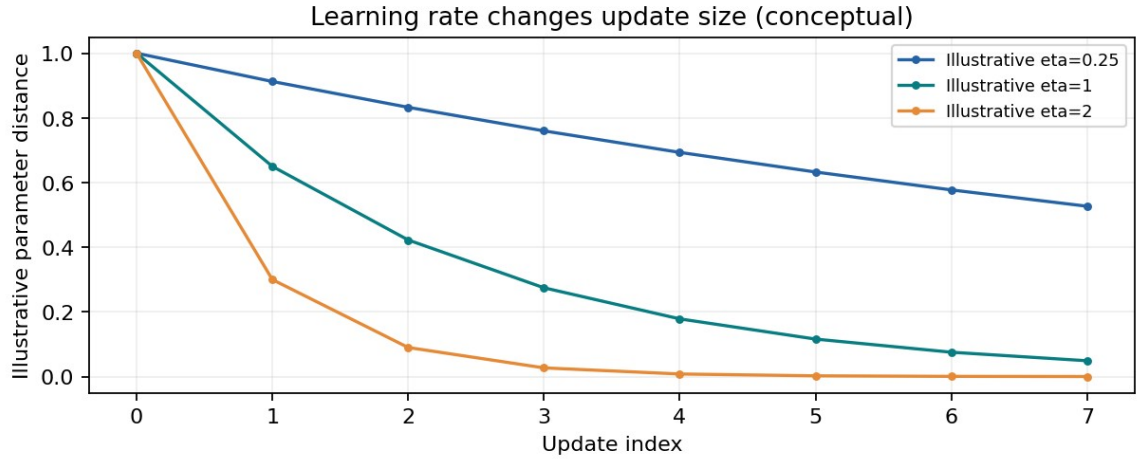
$$w_{new} = (0.5, -0.25) + 0.2(-1)(2,1) = (0.1, -0.45)$$

$$b_{new} = -0.5 + 0.2(-1) = -0.7$$

אם נחשב שוב את הציון לאותה דוגמה, נקבל 0.1×2 − 0.45×1 − 0.7 = −0.95, ולכן התחזית תשתנה ל־0. התיקון בכיוון הנדרש הצליח בדוגמה הזאת; עדיין יש לבדוק את יתר הדוגמאות ואת נתוני הבדיקה.

13. קצב למידה η — מה משפיע עליו?

קצב למידה קטן יוצר צעדים קטנים יותר בכל תיקון, וקצב למידה גדול יוצר צעדים גדולים יותר. חשוב לא להחיל באופן אוטומטי את ההסבר של Gradient Descent על הפרספטרון הקלאסי: כאן התיקון מופעל בדרך כלל בעקבות טעות סיווג, ולא על ידי חישוב נגזרת של MSE.



איור: המחשה רעיונית בלבד של גודל צעד; התרשים אינו ניסוי אמיתי של אימון פרספטרון.

בנתונים הניתנים להפרדה ליניארית, שינוי קצב הלמידה הקבוע והחיובי יכול לשנות את מסלול האימון ואת מקדמי ההפרדה. במימוש מעשי בודקים זאת בניסוי; לא קיימת הבטחה שכל הגדלה של η משפרת את המודל.

14. מבט לתוך הקוד: אימון פרספטרון ללא ספרייה

הקוד הבא מממש בדיוק את כלל ההחלטה שקבענו: תחזית 1 רק כאשר z>0. הוא משתמש ב־NumPy, אבל אינו קורא לפונקציית אימון מוכנה. בכל מעבר סופרים את מספר הטעויות; אם לא היו טעויות, עוצרים.

```
import numpy as np
X = np.array([[0, 0], [0, 1], [1, 0], [1, 1]], dtype=float)
y = np.array([0, 0, 0, 1])
w = np.zeros(X.shape[1], dtype=float)
b, eta = 0.0, 1.0

for epoch in range(20):
```

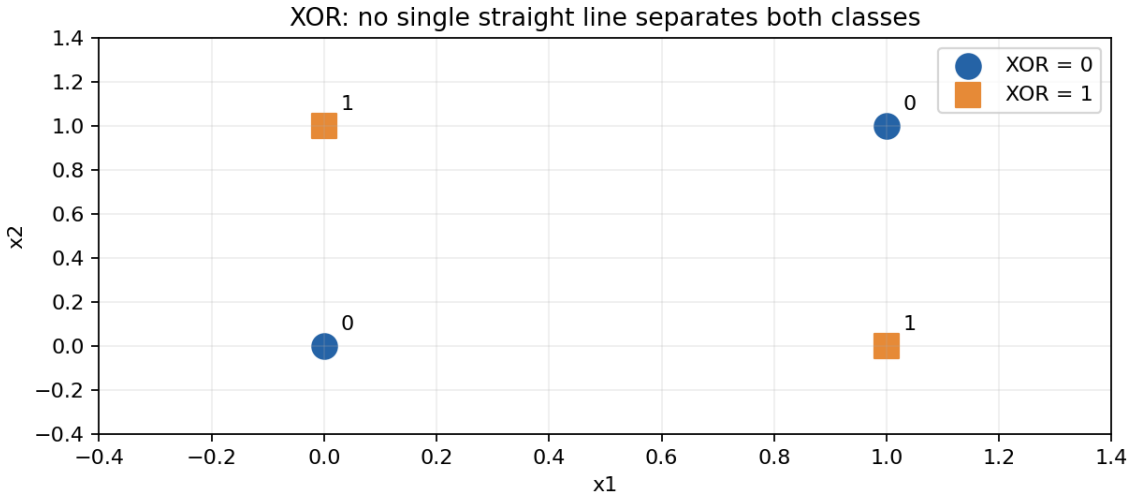
```
    errors = 0
    for i in range(len(X)):
        z = np.dot(w, X[i]) + b
        prediction = int(z > 0)
        delta = int(y[i]) - prediction
        w = w + eta * delta * X[i]
        b = b + eta * delta
        if delta != 0:
            errors += 1
    print(epoch + 1, errors, w, b)
    if errors == 0:
        break
```

שימו לב למיקום השורות: תחילה `z`, אחריו `prediction`, ורק אחר כך שינוי `w` ו־`b`. אם מעדכנים לפני שחושבו התוצאה והטעות, משנים את משמעות האלגוריתם. `X.shape[1]` מחזיר את מספר המאפיינים, ולכן אותו קוד מתאים גם ליותר משני מאפיינים.

תוצאה צפויה לנתוני AND ולסדר שנקבע: שישה מעברים; מספר הטעויות בכל מעבר הוא 1, 3, 3, 2, 1, 0. נסו להסביר את המספרים בעזרת טבלת המעקב, ולא רק לראות את הפלט.

15. למה אי אפשר ללמוד XOR באמצעות פרספטרון יחיד?

הביטוי XOR מחזיר 1 כאשר בדיוק אחד משני הקלטים הוא 1, ומחזיר 0 כאשר שניהם שווים. בארבע הנקודות במישור, שתי המחלקות מסודרות באלכסונים מנוגדים. שום קו ישר אחד אינו מסוגל להפריד את שתי הנקודות החיוביות משתי הנקודות השליליות.



איור: נקודות XOR משני סוגים באלכסונים מנוגדים; אין קו ישר שיפריד את המחלקות.

x_1	x_2	AND	OR	XOR
0	0	0	0	0
0	1	0	1	1
1	0	0	1	1
1	1	1	1	0

אפשר גם להוכיח את הבעיה בעזרת אי־שוויונים, בלי להסתמך רק על הציור. נסמן $z = w_1x_1 + w_2x_2 + b$ ונשתמש בהכרעה 1 כאשר $z > 0$. עבור XOR תנאי ארבע הדוגמאות מחייבים:

$$b \le 0 \;;\quad w_1 + b > 0 \;;\quad w_2 + b > 0 \;;\quad w_1 + w_2 + b \le 0$$

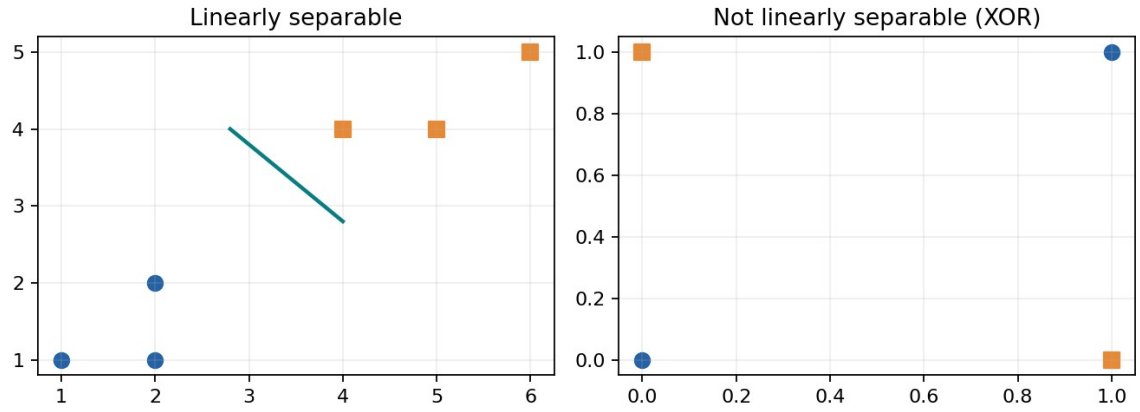
משני האי־שוויונים האמצעיים נקבל $w_1 > -b$ וגם $w_2 > -b$, ולכן $w_1 + w_2 + b > -b$. מאחר ש־$b \leq 0$, נקבל $-b \geq 0$, ולכן $w_1 + w_2 + b > 0$. זו סתירה לתנאי האחרון. לכן לא קיימים משקולות והטיה שיסווגו נכון את ארבע הדוגמאות באמצעות פרספטרון יחיד.

זו הסיבה שבפרק הרשתות העצביות נלמד לשלב מספר יחידות חישוב, כולל שכבות חבויות ואי־ליניאריות. יותר שכבות לבדן אינן מספיקות אם אין פעולות לא־ליניאריות בין השכבות.

16. מה אומר משפט ההתכנסות — ומה לא?

משפט ההתכנסות של הפרספטרון קובע שבתנאים מתאימים, עבור קבוצת אימון סופית שניתן להפריד ליניארית עם שוליים חיוביים ודוגמאות בעלות נורמה חסומה, אלגוריתם הפרספטרון הקלאסי יגיע לאחר מספר סופי של טעויות לגבול שמפריד נכון את דוגמאות האימון.

המשפט אינו מבטיח שהמודל יצליח על דוגמאות חדשות, אינו מחייב שהמשקולות יהיו יחידות ואינו חל על XOR כבעיה דו־ממדית ללא הרחבת מאפיינים. אם הנתונים אינם ניתנים להפרדה ליניארית, האימון עשוי להמשיך לטעות ולכן נהוג לקבוע מספר מעברים מרבי ולעקוב אחרי ביצועים על נתוני ולידציה.



איור: הפרדה ליניארית לעומת דפוס שאינו ניתן להפרדה בקו יחיד.

17. מה ההבדל בין פרספטרון לבין רגרסיה ליניארית?

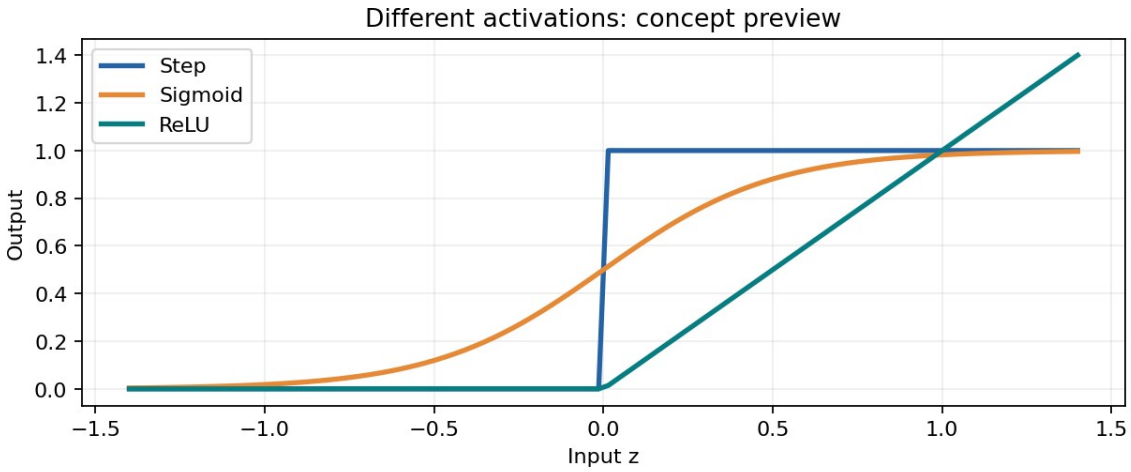
פרספטרון	רגרסיה ליניארית	נושא
סיווג למחלקה בינרית	חיזוי ערך מספרי	**מטרה**
z = w·x + b; threshold(z)	$\hat{y} = w \cdot x + b$	**ביטוי מרכזי**
תווית 0 או 1	מספר רציף	**פלט**
תיקון בעקבות סיווג שגוי	מזעור שגיאה מספרית	**אימון**
Accuracy, Precision, Recall, F1	MAE, MSE, R^2	**מדדי הערכה**

שני המודלים משתמשים באותו סכום משוקלל, אך שונים במשמעות הפלט ובמטרת האימון. זו נקודת חיבור חשובה בין פרק הרגרסיה שכבר למדנו לבין הרשתות העצביות שנלמד בהמשך.

18. פונקציית הפעלה לעומת פונקציית שגיאה

פונקציית הפעלה הופכת ציון ביניים לפלט של יחידת החישוב. בפרספטרון הבסיסי השתמשנו בפונקציית מדרגה. פונקציית שגיאה, לעומת זאת, מודדת עד כמה תוצאת המודל אינה עומדת במטרת האימון. אלו שני תפקידים שונים.

בפרקי הלמידה העמוקה נלמד פעולות כמו Sigmoid ו־ReLU, ונעמיק באימון באמצעות נגזרות ו־Backpropagation. אין לטעון שהפרספטרון הבינרי הקלאסי מתאמן באמצעות נגזרת של פונקציית המדרגה: הפונקציה הזאת אינה גזירה בנקודת המעבר והנגזרת שלה אפס כמעט בכל מקום אחר.



איור: השוואה חזותית של מדרגה, Sigmoid ו־ReLU — טעימה לקראת הרשת העמוקה.

19. מימוש מקצועי בעזרת scikit-learn

אחרי שכתבנו את האלגוריתם בעצמנו נוכל להשתמש במימוש הספרייה. מטרת ההשוואה אינה לשנן פקודות, אלא להבחין בין מה שאנחנו שולטים בו בניסוי לבין מה שהספרייה עושה אוטומטית: אתחול, ערבוב דוגמאות, מספר מעברים ועצירה.

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline
from sklearn.linear_model import Perceptron
from sklearn.metrics import accuracy_score, confusion_matrix

iris = load_iris()
X = iris.data[iris.target != 2, :2]
y = iris.target[iris.target != 2]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, stratify=y, random_state=42
)
model = make_pipeline(StandardScaler(), Perceptron(
    max_iter=1000, tol=1e-3, random_state=42
))
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
print('Accuracy:', accuracy_score(y_test, y_pred))
print(confusion_matrix(y_test, y_pred))
```

מדוע בחרנו שתי מחלקות בלבד? מפני שהדוגמה מסבירה פרספטרון בינרי; ספריות יכולות להרחיב מסווגים ליניאריים גם לריבוי מחלקות, אבל ההרחבה אינה זהה לפרספטרון יחיד בעל פלט בינרי. מדוע שמנו `StandardScaler` בתוך Pipeline? משום שחישוב הממוצע וסטיית התקן חייב להתבצע על נתוני האימון בלבד.

שימו לב: אין לצפות שזהות מלאה של מקדמי המודל או סדר התיקונים תתקבל בין המימוש הידני שלנו לבין `Perceptron` של scikit-learn. חלק מהגדרות האימון שונות, ולכן נשווה תחזיות וביצועים באותו פרוטוקול ניסוי, לא את כל צעדיו הפנימיים.

20. מדוע Accuracy גבוה אינו הוכחה למודל מוצלח?

אם רוב הדוגמאות שייכות למחלקה 0, מודל שמנבא תמיד 0 יכול להשיג Accuracy גבוה, אך להחמיץ את כל האירועים החשובים של מחלקה 1. לכן עלינו לשמור נתוני Test שלא השתתפו באימון או בבחירת ההגדרות ולבדוק מטריצת בלבול, Precision, Recall ו־F1 לפי משמעות סוגי השגיאה.



Select hyperparameters without using Test labels

איור: הפרדה בין התאמת המודל, בחירת הגדרות והערכה סופית על נתונים שלא נחשפו בתהליך הפיתוח.

בתכנון פרויקט גמר יש לתעד את מקור המידע, סוגי המאפיינים, מי סיפק תוויות, כיצד התחלקו הנתונים ומה מחיר טעות מכל סוג. שיפור דיוק על אותם נתוני אימון בלבד אינו הוכחה לשיפור המודל בעולם האמיתי.

21. שלושה כשלים נפוצים ודרכי זיהוי

- בלבול בין חיזוי לאימון: מחשבים z בעזרת w,b קיימים וחושבים שהמודל כבר למד. הפתרון: לתעד לפחות שינוי אחד בפרמטרים בעקבות שגיאה.
- בלבול במקרה z=0: משתמשים פעם אחת ב־0≤ ופעם אחרת ב־0<. הפתרון: לנסח כלל אחד בתחילת השאלה ולדבוק בו לאורך טבלת המעקב.
- שינוי קנה מידה ללא תיעוד: הכפלת מאפיין באלף משנה את תרומתו לציון. הפתרון: להבין יחידות, לבחור תקנון כאשר מתאים ולבצע fit רק על Train.
- הנחה ש־XOR יילמד עם מספיק מעברים: עוד מעבר לא משנה את העובדה שאין גבול ליניארי מתאים. הפתרון: בדקו תחילה היתכנות הפרדה ליניארית.
- הצגת תוצאת Test כמדד לשיפור לאחר שכבר בחרנו לפיה מודל: זו זליגת החלטות. הפתרון: Train/Validation לבחירה ו־Test לבדיקה הסופית.

22. תרגול מדורג — ללא פתרונות

רמה א: הבנה וחישוב

- א. נתון x=(2,1), w=(−1,3), b=−1. חשבו z ואת התחזית לפי הכלל z>0.
- ב. לאותה דוגמה התווית האמיתית היא 0. עבור η=0.5 חשבו w_new ו־b_new.
- ג. הסבירו במילים את ההבדל בין משקל לבין הטיה, וציירו שני גבולות החלטה השונים רק בהטיה.
- ד. נתון z=0. מה תהיה התחזית בספר שלנו? מדוע חייבים לציין זאת מראש?

רמה ב: טבלת מעקב וניתוח

- ה. התחילו ב־$w=(0,0), b=0, \eta=1$ ועברו על OR בסדר הדוגמאות הנתון. רשמו שתי איטרציות לפחות.
- ו. עבור הנקודות $(1,1)$, $(2,2)$ במחלקה 1 ו־$(-1,-1)$,$(-2,-2)$ במחלקה 0, הציעו קו הפרדה ונמקו.
- ז. תלמיד כתב רק $w=w+\eta(y-\hat{y})x$ והשאיר את b קבוע. כיצד זה משפיע על אפשרויות ההפרדה?
- ח. בדקו קוד שבו מבוצע `np.argsort` על z במקום על תחזיות. האם יש כאן בכלל שלב הנחוץ לאימון פרספטרון? נמקו.

רמה ג: חשיבה מדעית ופרויקט

- ט. נסו להציע קו יחיד שמפריד XOR. הסבירו מדוע כל ניסיון נכשל, ובנו טיעון מתמטי.
- י. השוו ניסיון עם נתוני אימון בלבד לניסיון עם Train/Test ושימרו את כל התוצאות. מה ההבדל בין שתי הטענות שאפשר להסיק?
- יא. תכננו ניסיון על נתוני Iris עם שתי מחלקות: אילו מאפיינים תבחרו, איך תבדקו הכללה ואילו טעויות חשוב לנתח?
- יב. תלמיד מצא Accuracy של 97% אך Recall למחלקה החיובית הוא 30%. תארו מקרה שימוש שבו המודל אינו מתאים.

23. שאלת הערכה מקורית בסגנון בגרות

נתונות ארבע דוגמאות של שער AND בסדר שבטבלה שבתחילת הפרק. מתחילים עם $w=(0,0), b=0, \eta=1$. לפי כלל $z>0$ מתקבלת תחזית 1; אחרת 0. השיבו על הסעיפים הבאים בלי להריץ קוד לפני השלמת החישוב הידני:

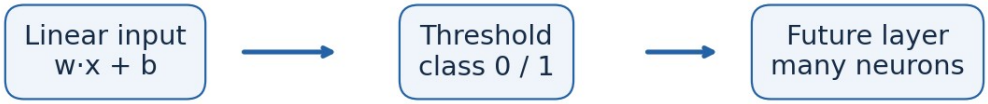
- א. חשבו את z, התחזית והשינוי בפרמטרים עבור הדוגמה הראשונה והדוגמה הרביעית במעבר הראשון.
- ב. הסבירו מדוע תחילת המעבר השני עשויה לכלול שגיאות אף שבסוף המעבר הראשון תוקנה הדוגמה החיובית.
- ג. ציינו מה פירוש Epoch וכתבו כלל עצירה מתאים לנתוני אימון שניתנים להפרדה.
- ד. עבור $w=(2,1), b=-2$, בדקו את כל ארבעת הקלטים והסבירו מדוע התקבלה הפרדה נכונה.
- ה. תלמיד טען שאם נוסיף עוד 100 מעברים אפשר יהיה ללמד את XOR באמצעות פרספטרון יחיד. נמקו מדוע זו טעות.
- ו. הציעו בדיקה שמאפשרת להעריך האם מודל שלמד נתוני אימון מתמודד גם עם דוגמאות שלא ראה.

הערה: זוהי שאלה מקורית לצורכי אימון בסגנון בגרות, לא שאלה שהופיעה בבחינה רשמית.

24. משימת פרויקט גמר: ממודל פשוט למערכת ניתנת לבדיקה

בפרויקט AI אמיתי, הפרספטרון יכול לשמש Baseline — מודל פשוט שמולו בודקים פתרון מורכב יותר. מומלץ להתחיל בבעיה שבה יש תוויות אמינות ומספר קטן של מאפיינים מספריים, להשתמש בחלוקת נתונים הוגנת ולהסביר את משמעות המשקולות ומגבלות הקו הליניארי.

כדי להגיש ניסוי שאפשר לשחזר, הכינו כרטיס ניסוי: תיאור הבעיה, מקור הנתונים וזכויות שימוש, גודל Train/Test, מאפיינים, קנה מידה, הגדרות המודל, מדדים, מטריצת בלבול, דוגמאות שגיאה ותובנות. אל תחשפו פרטי תלמידים אמיתיים או מידע מזהה במאגר ציבורי.



איור: הפרספטרון הוא יחידת בניין רעיונית בדרך לרשתות בעלות מספר נוירונים ושכבות.

25. סיכום מושגים וידע נדרש להמשך

מושג	מה חייבים להסביר
Feature	מספר המייצג מידע קיים על תצפית
Weight	מקדם הקובע תרומה לציון ליניארי
Bias	הטיה שמשנה את מיקום הגבול
z	ציון לפני ההחלטה; לא הסתברות
Threshold	כלל הממיר ציון לתווית
η	גודל צעד בתיקון בעקבות טעות
Epoch	מעבר מלא על כל דוגמאות האימון
Linear separability	קיום גבול ליניארי שמפריד את דוגמאות האימון
Generalization	ביצוע על נתונים שלא שימשו לאימון

אם אתם יכולים לחשב תחזית ידנית, לעדכן משקולות, לשרטט את הגבול ולהסביר את בעיית XOR — אתם מוכנים לעבור לשאלות המרכזיות בפרקי מתמטיקה ללמידה עמוקה ורשתות עצביות: מהי פונקציית שגיאה גזירה, כיצד מחשבים נגזרת, ואיך מאמנים כמה שכבות ביחד?

26. מקורות, עוגן תכנית והבחנה בין חובה להרחבה

עוגן תכנית: התמחות למידת מכונה ולמידה עמוקה, פרק 9 — פרספטרון, 30 שעות לפי מפת ההתמחות במאגר הקורס. חיבור קודם: יחידה 3, שיעור פרספטרון ו־SVM; פרקים 1.1–1.2 (וקטורים ומרחקים); פרקים 4.1–4.4 (סיווג); פרקים 5.1–5.2 (הערכה ולידציה).

ספרי עזר מקצועיים ששימשו השראה לאופן ההצגה: Grokking Machine Learning מאת Luis Serrano; Mathematics for Machine Learning מאת Deisenroth, Faisal ו־Dive into Deep Learning מאת Zhang, Lipton, Li ו־Understanding Deep Learning מאת Simon Prince. הקוד המוכן וההתנהגות של Perceptron נבדקו גם מול התיעוד הרשמי של scikit-learn.

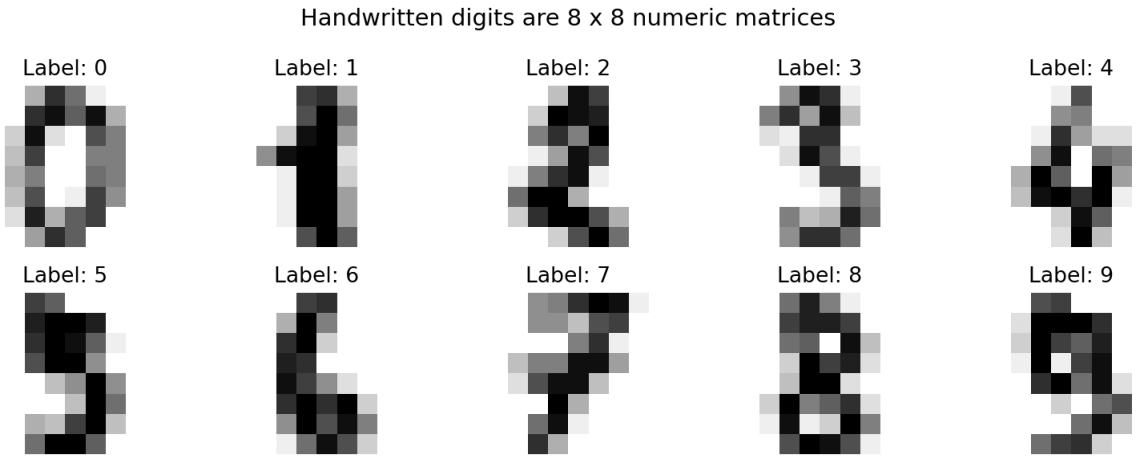
הדוגמאות המספריות, הסברים, איורים ושאלות בפרק נכתבו מחדש לצורכי הוראה; אין כאן העתקה של קטעים מספרים מסחריים. הרחבות כמו הוכחת XOR, משפט ההתכנסות והבחנת פונקציות הפעלה מוצגות כהעמקה פדגוגית. יש להצליב מול מסמך משרד החינוך התקף כדי לקבוע אילו תתי־נושאים מחויבים בפועל בבחינה או בהגשת הפרויקט.

פרק 10

מיני־פרויקט בלמידת מכונה

מספר לימוד לרעיון מחקר ועד למודל שאפשר להסביר, לבדוק ולשפר

התמחות בלמידת מכונה ולמידה עמוקה • 5 יחידות לימוד • פרק 10 • 30 שעות לפי מפת התכנית



איור: האתגר המלווה בפרק: המחשב מקבל מטריצות פיקסלים ומנסה לזהות ספרות בכתב יד.

בפרקים הקודמים למדנו את הכלים לחוד: מטריצות, עיבוד תמונה, KNN, מרכזי מחלקות, מדדי הערכה וולידציה. הפעם נשתמש בהם יחד. לא נשאל רק אם הקוד רץ, אלא האם ביצענו ניסוי אמין, מה למדנו מהטעויות ומה ניתן לומר בביטחון על המודל.

מפת למידה ומטרות ביצועיות

בסיום הפרק תוכלו להגדיר שאלת פרויקט, לתאר את הנתונים, לבחור מדד מתאים, להשוות מודל לקו בסיס, להפריד בין אימון לבחינה, לבצע ולידציה, לאתר טעויות, לתעד ניסוי ולהציג מסקנה זהירה בעל־פה.

הקשר לחומר קודם	התוצר המצופה	שלב
פרק 4.1	שאלה מדידה וקהל יעד	שאלת מחקר
פרקים 2.1–2.5	כרטיס מאגר, EDA וגרפים	הבנת נתונים
4.2, 4.3, 9	Baseline ושני מועמדים	בניית מודל
5.1–5.2	CV, Test, מדדים ושגיאות	בדיקה ושיפור
הכנה לפרויקט הגמר	דוח, מחברת ושחזור ניסוי	תוצר והצגה

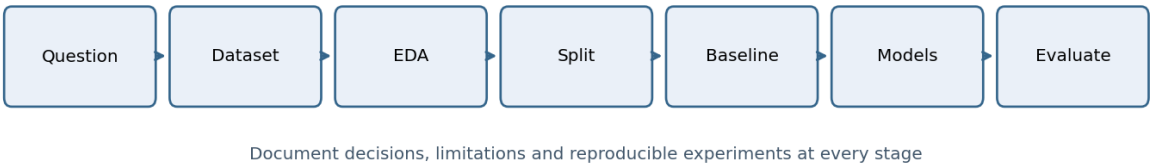
1. מה הופך תרגיל בכיתה למיני־פרויקט?

בתרגיל קצר בדרך כלל ידוע לנו איזו פקודה להפעיל ומה התוצאה שאמורה להתקבל. במיני־פרויקט חלק מההחלטות אינן מוכתבות מראש. עלינו להבין מה הבעיה, לבדוק אם הנתונים מתאימים, להצדיק את האלגוריתם, לבחון חלופות ולכתוב מה למדנו. זו עבודה מחקרית בקנה מידה קטן: המטרה אינה רק ציון טוב אלא תהליך שאפשר לשחזר ולבקר.

דודו רוצה לסייע למורה לקלוט טפסים שבהם תלמידים כתבו ספרות. במקום להקליד את הספרה בכל פעם, הוא רוצה לבדוק אם מערכת למידת מכונה תוכל להציע ספרה משוערת. הוא יודע כבר איך לייצג

תמונה כמטריצה ואיך מסווג KNN משתמש בשכנים קרובים. אך האם המודל יצליח גם על כתב יד שלא ראה? אילו ספרות יתבלבלו זו בזו? ומה תהיה הדרך הנכונה לבדוק זאת?

שאלת המחקר המנחה: עד כמה מודלים המבוססים על מרחק מצליחים לזהות ספרות בכתב יד במאגר digits, ומהם דפוסי הטעות שלהם? השאלה אינה מבטיחה הצלחה מראש. היא מזמינה ניסוי שבו נדע מה מדדנו, על אילו נתונים ומול איזה קו בסיס.



איור: תהליך העבודה הוא מחזורי: לאחר ניתוח שגיאות חוזרים לשאלת המחקר ולשיטות עיבוד הנתונים.

2. ניסוח בעיה טובה: מי המשתמש, מהו הקלט ומהי התחזית?

מערכת שאינה יודעת לומר במדויק מה היא מנבאת, אינה מוכנה לאימון. במקרה שלנו כל דוגמה היא תמונה אחת של ספרה; הקלט הוא 64 עוצמות פיקסלים, והיעד הוא ספרה אחת בין 0 ל־9. זהו סיווג רב־מחלקתי, ולא רגרסיה: הפלט 8 אינו כמות גדולה יותר מהפלט 3, אלא שם של מחלקה.

צריך להגדיר גם את תנאי השימוש. תמונות המאגר קטנות ומסודרות באותו פורמט; תמונות של טלפון נייד באור משתנה שונות מאוד. לכן הצלחה על המאגר לא מוכיחה יכולת לזהות כל כתב יד בעולם. כבר בשלב הגדרת הבעיה יש לציין את גבולות המערכת.

מה על התלמיד להחליט בפרויקט שלו	תשובה בדוגמת הספר	שאלה
מה מייצגת כל שורה?	תמונה אחת	מה יחידת התצפית?
אילו תכונות זמינות לפני התחזית?	64 ערכי פיקסלים	מהו X?
מה רוצים לדעת?	תווית הספרה 0–9	מהו y?
מה תיחשב תוצאה ראשונית?	ניחוש המחלקה הנפוצה באימון	מהו קו בסיס?
אילו טעויות חשובות?	Accuracy וגם Macro F1	מהו מבחן ההצלחה?

3. כרטיס מאגר — לומדים להכיר את הנתונים לפני אימון

מאגר digits הזמין בתוך scikit-learn מכיל 1,797 תמונות של ספרות בכתב יד. כל תמונה היא בגודל 8×8 פיקסלים, כלומר 64 תכונות מספריות, וערכי הפיקסלים נעים בין 0 ל־16. זהו פורמט המאפיינים של המאגר המסוים הזה; אין לבלבל אותו עם תמונות RGB רגילות שבהן מקובל להשתמש בטווח 0–255 לכל ערוץ.

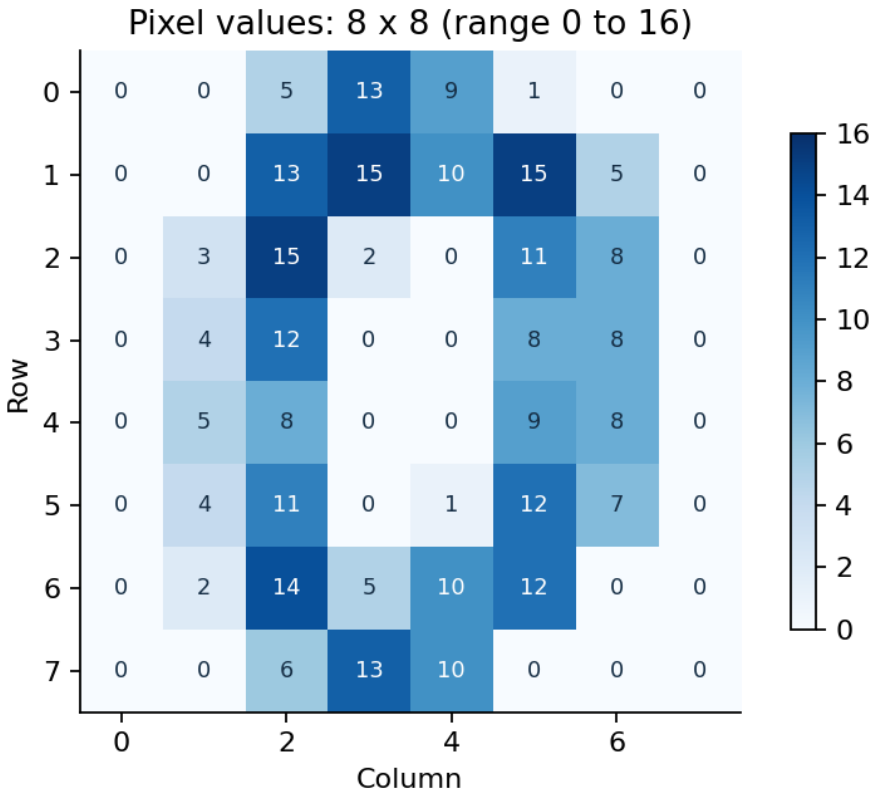
כרטיס המאגר צריך לתעד שם ומקור, משמעות התכונות, סוג המטרה, מספר הדוגמאות, מגבלות הייצוג והרשאות שימוש. במחקר אמיתי נוסיף גם מי אסף את הנתונים, באילו תנאים נאספו ומי אינו מיוצג בהם. מידע זה עוזר לקבוע אם המודל מתאים לשימוש שאליו אנו מייעדים אותו.

```
import numpy as np
from sklearn.datasets import load_digits

digits = load_digits()
X = digits.data
```

```
Y = digits.target
print(X.shape)      # (1797, 64)
print(digits.images.shape)  # (1797, 8, 8)
```

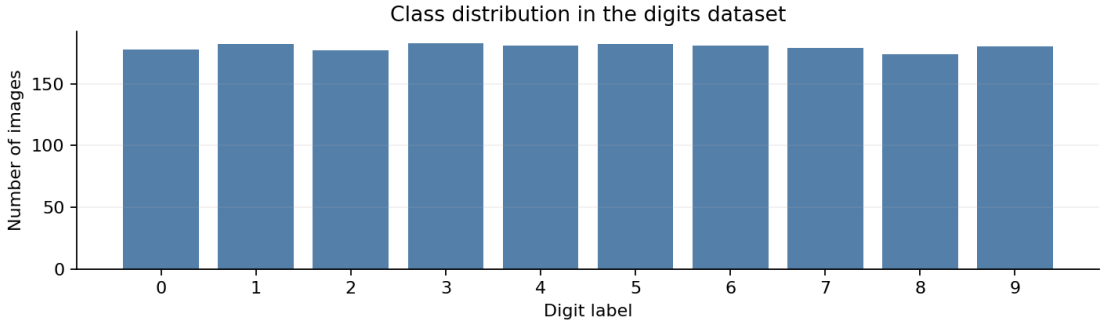
מבחינה מתמטית, המטריצה X מכילה 1,797 שורות ו־64 עמודות. כל שורה היא תמונה ש״נפרשה״ לווקטור; אותה תמונה עצמה נמצאת גם ב־digits.images כמטריצה של 8 שורות ו־8 עמודות. התוויות Y נשמרות בנפרד, משום שהתלמיד צריך להבין שהמודל לא רשאי לקבל את התשובה כחלק מהקלט.



איור: דוגמה למטריצת פיקסלים: מיקום התא ועוצמתו הם חלק מהתיאור המספרי של התמונה.

4. חקר נתונים — מה שואלים לפני שמאמנים?

שלב חקר הנתונים אינו קישוט. התלמיד בודק אם כל הספרות מיוצגות, האם קיימות דוגמאות חריגות, מהו טווח עוצמות הפיקסלים והאם הדוגמאות נראות ברורות לעין. באמצעות גרף שכיחויות אפשר לזהות אם אחת המחלקות נדירה מאוד; באמצעות תצוגה של מספר תמונות אפשר להבין את רמת השונות בכתב היד.



איור: שכיחות הספרות במאגר לדוגמה: התפלגות כמעט מאוזנת אינה פוטרת מבדיקת ביצועים לכל מחלקה.

אין להסיק מהיסטוגרמה בלבד שאין הטיה בנתונים. ייתכן שהמאגר מאוזן בין התוויות אבל פחות מייצג סגנונות כתיבה מסוימים או תנאי צילום חדשים. צריך להבחין בין איזון מספרי לבין ייצוג הולם של מצב שימוש אמיתי. בדוגמה שלפנינו לא נבצע טענות כלליות על אוכלוסיות של כותבים בלי מידע שמבסס זאת.

5. נתונים חסרים, כפולים ונתונים חשודים

לפני עיבוד נתונים יש להבחין בין ערך אפס בפיקסל לבין ערך חסר. במאגר digits אפס הוא עוצמת פיקסל תקינה; אין להחליפו בממוצע כאילו פירושו ״לא נמדד״. אם מאגר אחר מכיל NaN, נסביר כיצד נוצר החסר ורק אז נחליט על טיפול. גם ״תמונה שנראית מוזרה״ אינה בהכרח טעות — ייתכן שזה כתב יד שקשה לסווג.

יש לבדוק כפילויות או כמעט־כפילויות: אם צילומים של אותו טופס נמצאים גם באימון וגם בבדיקה, המודל עלול להצליח בזכות דמיון חריג בין הדוגמאות. בפרויקט עם כמה תמונות לכל אדם או מסמך, החלוקה הנכונה עשויה להיות לפי אדם או מסמך, ולא לפי תמונה בודדת. נתונים מקבוצות או מתאריכים שונים דורשים חלוקה המותאמת לשאלת הפרויקט.

6. קובעים מבחן הוגן: Train, Validation ו־Test

נתוני האימון הם הדוגמאות שמותר למודל ללמוד מהן. קבוצת הוולידציה משמשת לבדוק החלטות במהלך הפיתוח, למשל איזה k מתאים ל־KNN. קבוצת הבדיקה נשמרת מחוץ לתהליך הבחירה כדי לאפשר הערכה מסכמת אמינה ככל האפשר. אם נשנה את המודל שוב ושוב לפי ציוני Test, אותה קבוצה מפסיקה להיות מבחן חיצוני אמיתי.

בפרויקט הכיתה נשתמש בפיצול חד־פעמי ובשכבות לפי התווית כדי לשמור בקירוב על שיעור הספרות בכל חלק. בתוך נתוני האימון נוכל לבצע Stratified K-Fold לבחירת היפר־פרמטרים. כך אין צורך בקבוצת Validation נפרדת וקבועה: כל קיפול ממלא את תפקידה בתורו.

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(
    X, Y, test_size=0.20,
    random_state=42, stratify=Y
)
print(X_train.shape, X_test.shape)
```

חשוב לזכור: קביעת **random_state** מאפשרת לשחזר את החלוקה, אך אינה מוכיחה שהחלוקה מייצגת כל שימוש עתידי. תמיד צריך לבדוק את מבנה המאגר והקשר שלו לשאלת המחקר.

7. מדוע חייבים להתחיל ב־Baseline?

לפני השקעה במודל מורכב קובעים ניבוי פשוט שמולו משווים. בסיווג אפשר למשל לבחור תמיד את המחלקה הנפוצה בנתוני האימון. זהו מודל בסיס חלש מאוד, אך מועיל: אם האלגוריתם החדש אינו משיג תוצאה טובה יותר, ייתכן שיש שגיאה בקוד, בנתונים או בעצם ההנחה שהמאפיינים מאפשרים את החיזוי.

חשוב להתאים גם את קו הבסיס על Train בלבד. חיזוי ״הספרה הנפוצה בכל המאגר״ משתמש גם במידע מקבוצת הבדיקה ולכן אינו נקודת ייחוס נקייה. כמו כן, נתון Accuracy גבוה אינו בהכרח מספיק כאשר המחלקות אינן מאוזנות או כאשר מחיר הטעות שונה ממקרה למקרה.

```
from sklearn.dummy import DummyClassifier
from sklearn.metrics import accuracy_score

baseline = DummyClassifier(strategy="most_frequent")
baseline.fit(X_train, y_train)
base_pred = baseline.predict(X_test)
print(accuracy_score(y_test, base_pred))
```

8. מאמנים מועמדים — לא רק את המודל האהוב עלינו

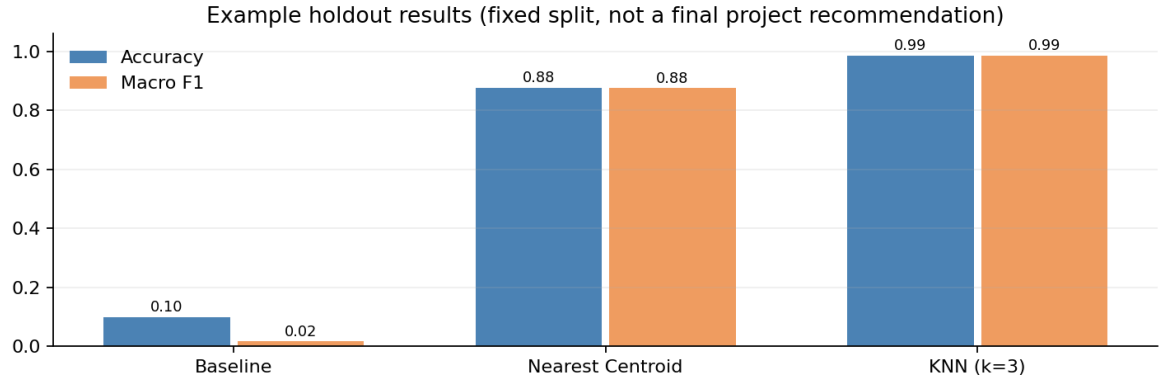
לאחר שבנינו קו בסיס, נבחן שני אלגוריתמים שכבר למדנו: Nearest Centroid ו־KNN. הראשון מייצג כל מחלקה על ידי מרכז יחיד ומסווג לפי מרחק למרכזים. השני משתמש בדוגמאות אימון הקרובות לנקודת

השאילתה. זהו ניסוי מעניין משום ששני המודלים מבוססי מרחקים אך מניחים דברים שונים על מבנה המחלקות.

על אותן דוגמאות אימון ועל אותן דוגמאות בדיקה אפשר להדגים השוואה ראשונית. יש לזכור שזו הדגמה אחת בחלוקה קבועה; לפני בחירת מודל סופית נרצה להגדיר את הליך הבחירה באמצעות Validation או Cross-Validation, ולא להתבסס על טבלת ה־Test כדי להמשיך לשפר את הפרמטרים.

```
from sklearn.neighbors import KNeighborsClassifier, NearestCentroid

model_centroid = NearestCentroid()
model_knn = KNeighborsClassifier(n_neighbors=3)
model_centroid.fit(X_train, y_train)
model_knn.fit(X_train, y_train)
```

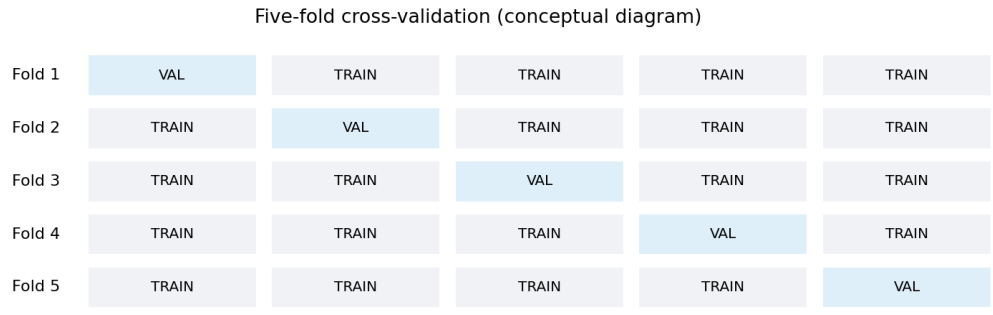


איור: השוואת הדגמה על פיצול יחיד: הבסיס, *Nearest Centroid* ו־*KNN*. זו אינה תחרות מחקרית סופית.

המספרים בגרף מבוססים על דוגמה ניתנת לשחזור מתוך מאגר digits עם random_state=42. גם כאשר KNN נראה טוב יותר כאן, אסור להסיק שמודל זה מנצח תמיד. השוואה מקצועית מחייבת לבדוק באילו נתונים, איזה מדד חושב, כיצד נבחרו ההיפר־פרמטרים ומהי רמת אי־הוודאות סביב התוצאה.

9. בוחרים היפר־פרמטרים בתוך נתוני האימון

מספר השכנים k אינו נלמד אוטומטית מתוך הנתונים על ידי KNN; אנחנו קובעים אותו לפני ההתאמה. לכן הוא היפר־פרמטר. הבחירה ב־k=3 בדוגמה הקודמת נועדה להמחשה בלבד. בפרויקט אמיתי נבדוק כמה ערכים בעזרת קיפולי ולידציה, ונשתמש באותו מדד בכל ההשוואות.



איור: בכל קיפול מאמנים על חלק אחר מהנתונים ובודקים על החלק הנותר; את *Test* החיצוני שומרים בצד.

נניח שקיבלנו עבור k=1 שלושה ציוני ולידציה: 0.92, 0.90, 0.94. ממוצעם הוא 0.92. עבור k=3 התקבלו 0.95, 0.93, 0.94, ולכן הממוצע הוא 0.94. לפי דוגמה מספרית זו k=3 עדיף, אך ההבדל הקטן דורש זהירות: בסט נתונים קטן גם חלוקה אחרת עשויה לשנות את המסקנה.

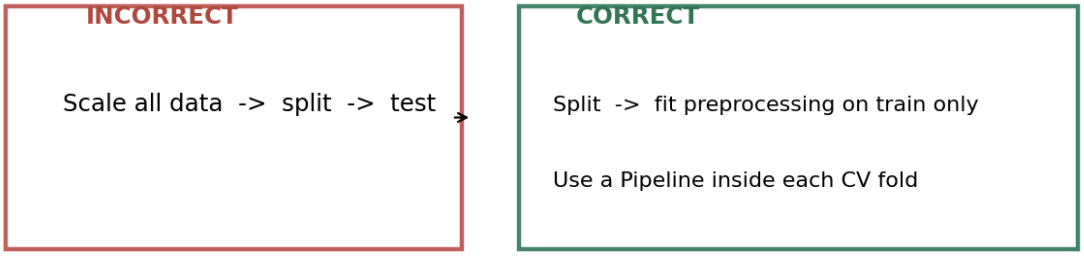
$$\text{mean}(k=1) = (0.92 + 0.90 + 0.94) / 3 = 0.92$$

$$\text{mean}(k=3) = (0.95 + 0.93 + 0.94) / 3 = 0.94$$

נוח להשתמש ב־GridSearchCV או בלולאה עם cross_val_score, אך חשוב להבין את ההיגיון לפני הפקודה. בשני המקרים מקפידים שהמודל מאומן מחדש בתוך כל קיפול, ואין מעבר מידע מהוולידציה אל נתוני האימון של אותו קיפול.

10. מניעת Data Leakage — שגיאה שמייצרת אשליה של הצלחה

זליגת מידע מתרחשת כאשר מידע שאינו אמור להיות זמין בעת החיזוי משפיע על אימון המודל או על בחירתו. דוגמה פשוטה: חישוב ממוצע וסטיית תקן לכל המאגר לפני חלוקת Train/Test. גם אם אין שימוש ישיר בתוויות, ערכי Test השפיעו על שלב ההכנה. לכן תקנון נלמד על Train בלבד ומוחל על Test באמצעות transform.



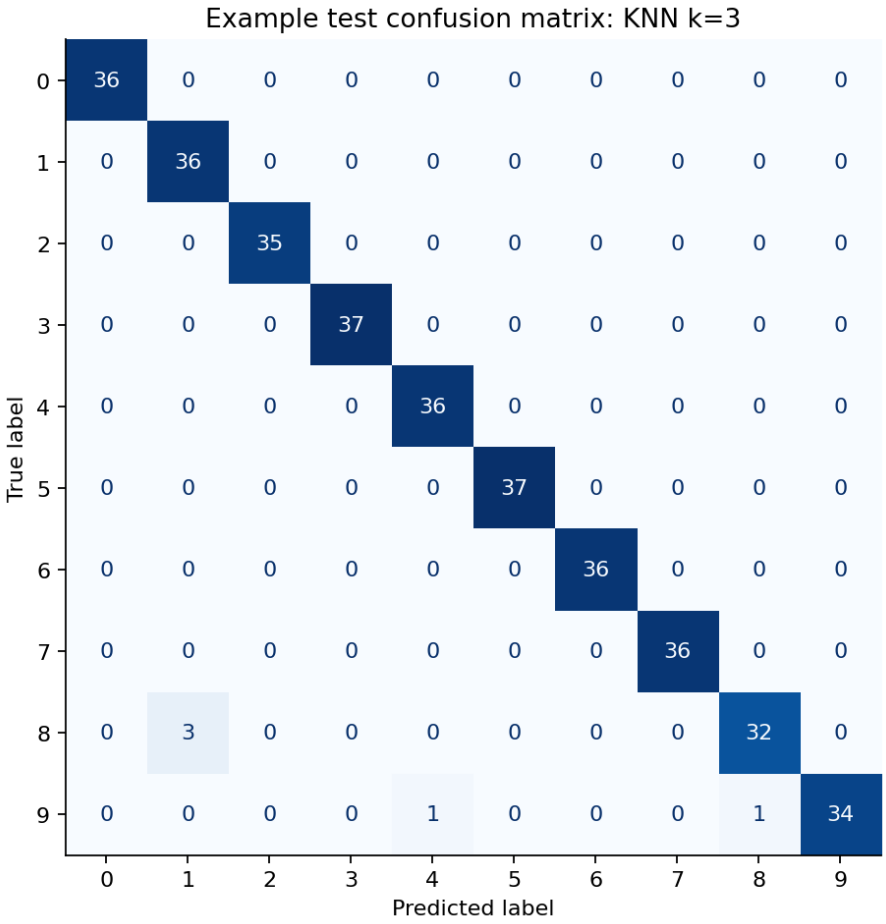
איור: סדר הפעולות קובע: קודם מפרידים נתונים, ורק אחר כך מתאימים טרנספורמציות למידע שמותר ללמוד ממנו.

בתהליך Cross-Validation חשוב עוד יותר לכלול את שלבי ההכנה בתוך Pipeline, כדי שהתקנון יתאים את עצמו מחדש לנתוני האימון של כל קיפול. בפרויקט ספרות שבו כל הפיקסלים כבר באותו סולם, אפשר לבדוק אם תקנון אכן משפר ביצועים; אין להניח שכל שלב הכנה משפר כל מודל.

11. כיצד מודדים הצלחה בסיווג עם עשר מחלקות?

בבעיה בינרית למדנו TP, FP, FN ו־TN. כאן יש עשר ספרות אפשריות, ולכן מטריצת בלבול בגודל 10×10: שורה מייצגת תווית אמיתית ועמודה תווית חזויה. התא באלכסון הראשי מציג חיזוי נכון, ותאים שמחוץ לאלכסון מציגים בלבולים בין ספרות.

Accuracy הוא אחוז התחזיות הנכונות מכלל הדוגמאות. Macro F1 מחשב F1 נפרד לכל ספרה וממצע את עשרת הערכים במשקל שווה, ולכן הוא מסייע כאשר מעוניינים לתת לכל מחלקה חשיבות דומה. Weighted F1 שוקל לפי גודל מחלקה; אלה מדדים שונים, ואין להחליף ביניהם בלי הסבר.



איור: דוגמת מטריצת בלבול ל־*KNN*. חשוב להצביע על ספרות שמבלבלות את המודל, ולא רק לדווח על מספר סופי.

12. חישוב ידני של מדדים לפרויקט — "האם זו הספרה 8?"

כדי לחבר את מטריצת הבלבול הרב־מחלקתית למונחים שלמדנו, נבחר ספרה אחת בתור מחלקה חיובית: 8. נניח שבדוגמה לימודית של 40 תמונות התקבלו 7 זיהויים נכונים של 8, שלוש אזעקות שווא, שתי החמצות ו־28 דוגמאות של ספרות אחרות שזוהו כלא־8. זו טבלה מומצאת לצורך חישוב, לא תוצאה של הניסוי על digits.

פירוש	מספר	מצב
הספרה 8 זוהתה נכון	7	TP
ספרה אחרת זוהתה בטעות כ־8	3	FP
הספרה 8 הוחמצה	2	FN
ספרה אחרת לא זוהתה כ־8	28	TN

$$\text{Accuracy} = (7 + 28) / 40 = 0.875$$

$$\text{Precision} = 7 / (7 + 3) = 0.70$$

$$\text{Recall} = 7 / (7 + 2) = 0.778$$

$$F1 = 2 * 0.70 * 0.778 / (0.70 + 0.778) \approx 0.737$$

למרות Accuracy של 87.5%, מדד Precision כאן הוא 70% בלבד. במערכת שבה סיווג שגוי של טופס עלול לגרום לאיבוד מידע, גם מקרה כזה דורש בדיקה. המדד החשוב אינו רק מספר גדול, אלא מדד שמתאים לנזק של שגיאה ולמטרת השימוש.

13. ניתוח טעויות — החלק שבו הפרויקט הופך למחקר

לאחר ההערכה נציג תמונות שהמודל טעה בהן, נבדוק מה הייתה התווית האמיתית ומה המודל חזה, ונשאל מה משותף למקרים. האם מדובר בצורה חריגה של הספרה? האם קיימות תכונות שאינן מספיקות להבדיל בין 1 ל־7? האם מגבלת הרזולוציה של 8×8 פיקסלים מקשה על הזיהוי?



איור: מדגם טעויות מתוך קבוצת הבדיקה בדוגמה — כל מקרה הוא הזדמנות להציע השערה חדשה.

אסור לבחור רק את הדוגמאות שהמודל הצליח בהן כדי להציגו בצורה מחמיאה. חשוב גם לא לשנות את המודל לפי אותן טעויות Test שוב ושוב ולדווח על אותה קבוצה כהערכה בלתי תלויה. דפוסי טעויות יכולים להוליד רעיון למחקר המשך; לצורך בחירה נוספת נידרש למסגרת הערכה חדשה או נשתמש בנתוני האימון והוולידציה.

14. ניתוח משאבים ומגבלות — מה עובד בכיתה ומה יעבוד במערכת?

מיני־פרויקט אינו מסתיים בבחירת אלגוריתם בעל ציון גבוה. עלינו לשאול גם מה זמן החיזוי, האם כל דוגמאות האימון חייבות להישמר, כמה זיכרון נדרש, ומה יקרה אם יתקבלו תמונות בגדלים שונים. KNN שומר דוגמאות אימון לצורך חיזוי; Nearest Centroid שומר לכל מחלקה ייצוג תמציתי. ההבדל עשוי להיות משמעותי בסביבת עבודה עם הרבה נתונים.

יש לשאול גם מה לא אפשר להסיק. מאגר digits הוא מאגר לימודי של תמונות קטנות, ולכן הוא מתאים לבדיקת תהליך אך לא מהווה אישור למערכת לזיהוי מסמכים משפטיים, פיננסיים או אישיים. במערכת אמיתית נידרש לבדיקת דוגמאות חדשות, פרטיות, אבטחה ודרכי טיפול כאשר המודל אינו בטוח בתשובתו.

15. כיצד הופכים מחברת ניסויים לתוצר שאפשר להדגים?

הפרויקט יכול להסתיים במחברת מסודרת שמקבלת תמונה אחת ומציגה את הספרה שהמודל בחר, לצד התמונה עצמה. אפשר לבנות ממשק פשוט שמאפשר לתלמיד לבחור דוגמה מתוך מאגר הבדיקה. אין חובה בשלב זה לבנות יישום נייד או שירות אינטרנט; אלה יהיו אפשרויות להעמקה בפרויקט הגמר.

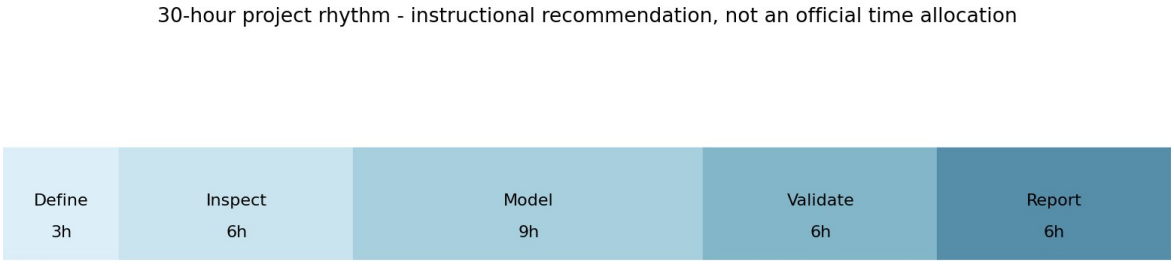
הדגמה אמינה צריכה להראות גם מקרה שבו המודל טועה, להסביר מה ההשלכות של הטעות ולתאר איזה מידע נוסף יהיה דרוש לפני שימוש אמיתי. במסגרת הדגמה לא צריך להמציא ״רמת ביטחון״ אם המודל אינו מספק הסתברות מכוילת. מספר השכנים שהצביעו למחלקה אינו תמיד הסתברות מהימנה.

16. לוח ביצוע: 30 שעות, עשרה מפגשים של שלוש שעות

חלוקת השעות בטבלה היא הצעה דידקטית שלנו למימוש היקף 30 השעות בפרק ההתמחות. זו אינה חלוקת זמן רשמית של משרד החינוך. כל מפגש מסתיים בתוצר שאפשר לבדוק, ולא רק ב״התקדמנו בקוד״.

מפגש	מוקד	תוצר להגשה
1	בעיית מחקר ומטרת שימוש	שאלת מחקר + שלושה קריטריוני הצלחה
2	מקור, איכות ורישוי נתונים	כרטיס מאגר והצהרת מגבלות

מפגש	מוקד	תוצר להגשה
3	EDA, שכיחויות ותמונות	שלושה גרפים עם פרשנות
4	חלוקה ושימוש ב־Baseline	פרוטוקול פיצול וקו בסיס
5	בניית שני מודלים	קוד אימון ותיאור הבדלים
6	ולידציה ובחירת פרמטרים	טבלת ניסויים ומסקנה
7	הערכה חיצונית וניתוח מדדים	מטריצת בלבול ודוח מדדים
8	ניתוח שגיאות ומגבלות	שתי השערות וממצאים
9	הדגמה, ריפרודוקציה ובדיקות	מחברת עובדת והוראות הרצה
10	דוח, מצגת והגנה	דוח סופי, הצגה ומשוב עמיתים



איור: חלוקה מוצעת של 30 שעות לשלבי עבודה; בתכנית הכיתה המפורטת מפורטים עשרה מפגשים בני 3 שעות.

17. מחוון הערכה לימודי — כיצד יודעים שהפרויקט שלם?

כדי להעריך פרויקט באופן הוגן נגדיר מראש מה נבדק. המחוון הבא הוא הצעת הוראה עצמאית לספר ואינו מוצג כמחוון הרשמי של משרד החינוך. ניתן להתאים את האחוזים בהתאם לצורכי הכיתה, תוך שמירה על כך שהערכת המודל ותיעוד ההחלטות מקבלים משקל משמעותי.

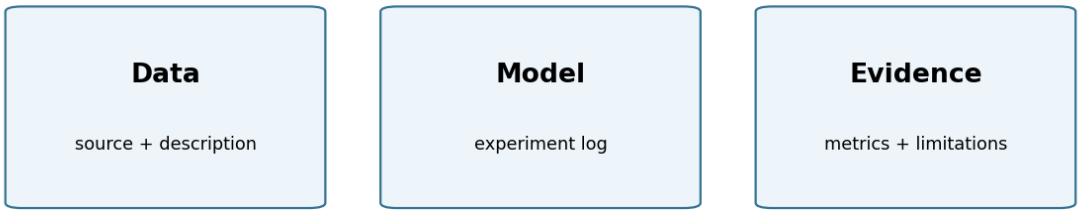
רכיב	ניקוד מוצע	ראיות נדרשות
הגדרת בעיה ונתונים	20	שאלה, כרטיס מאגר, בדיקות איכות
EDA והצגת ממצאים	15	גרפים, פירושים ומגבלות
מודלים ו־Baseline	20	שני מודלים לפחות וניסוי משוחזר
ולידציה והערכה	20	CV, Test ומדדים מותאמים
ניתוח שגיאות	15	מקרי טעות וניתוח מנומק
תיעוד והגנה	10	מחברת, מקורות, הצגה ומענה לשאלות
סה״כ	100	

18. דוח סופי מקצועי — מבנה אחיד להגשה

דוח טוב אינו תמליל של כל פקודה. הוא מספר מה הייתה הבעיה, מדוע הנתונים מתאימים באופן חלקי או מלא, אילו החלטות התקבלו ומה הראיות התומכות בהן. יש להבחין בין ממצא מדוד לבין הסבר

אפשרי. למשל "KNN" השיג ציון גבוה יותר בניסוי המסוים" הוא ממצא; "KNN" תמיד טוב יותר" אינו מסקנה מוצדקת.

מה צריך להופיע	פרק בדוח
שאלה, שיטה ותוצאה עיקרית בשלושה משפטים	תקציר
מה מתכוונים לחזות ולמי זה מועיל	בעיה ויעד
מקור, רישוי, יחידת תצפית ומגבלות	נתונים
פיצול, עיבוד נתונים, מודלים והיפר־פרמטרים	שיטה
טבלת החלטות, גרפים ונתוני ולידציה	ניסויים
Test, מדדים ומקרי שגיאה	תוצאות
מגבלות, סיכונים, רעיונות למחקר המשך	דיון
סביבת הרצה, גרסאות, זרע אקראי, קישורים	שחזור ומקורות



איור: שלושת סוגי הראיות המרכזיים: נתונים מתועדים, ניסויים משוחזרים ומסקנות מוגבלות למה שנמדד.

19. בדיקות עצמיות למפתח — אל תסמכו על קוד שרק "רץ"

פרויקט גמר מקצועי מחייב בדיקות גם לקוד. לפני אימון נבדוק שמספר התוויות שווה למספר השורות במטריצה, שאין ערכי NaN בלתי צפויים, שמספר המאפיינים מתאים לצורה שקבענו, ושאין חפיפה לא מכוונת בין דוגמאות האימון לבדיקה. לאחר חיזוי נבדוק שהפלט כולל תחזית לכל תמונה ושכל תווית נמצאת בטווח האפשרי.

```
assert X.shape[0] == len(Y)
assert X.shape[1] == 64
assert not np.isnan(X).any()
assert set(np.unique(Y)) == set(range(10))
```

הבדיקות הללו מתאימות למאגר המסוים שלנו ואינן "מתכון כללי" לכל מאגר. בפרויקט אחר ייתכן שמספר המאפיינים שונה, חסרים הם חלק מן הבעיה, או שהמחלקות הן טקסט ולא מספרים. המטרה היא ללמוד לכתוב בדיקות שמגנות על ההנחות של המודל, ולא להעתיק אותן בלי חשיבה.

20. אתיקה, פרטיות ושימוש אחראי

לפני שימוש בנתונים שנאספו בכיתה או באפליקציה יש לבחון אם יש מידע אישי, האם התקבלה הרשאה מתאימה, מה משך השמירה ומה עושים כאשר תלמיד מבקש להוציא נתונים מהמאגר. לצורך מיני־פרויקט לימודי עדיף להתחיל במאגר פומבי המיועד להוראה או בנתונים מדומים שאינם מזהים אנשים.

גם פרויקט בעל כוונה טובה עשוי להיכשל אם מתעלמים מהקשר שימוש. אין להסיק מתוצאות זיהוי ספרות שום דבר על יכולות לימודיות של תלמידים, ואין להשתמש בתחזית אוטומטית יחידה כדי לקבל החלטה בעלת השלכה משמעותית על אדם ללא בדיקה מתאימה.

21. שאלות מדורגות לתרגול — ללא פתרונות

רמה א׳ — הבנה וביצוע

1. מה ההבדל בין מאפיין לתווית במאגר digits? מדוע מספר התכונות הוא 64?

2. הסבירו מדוע בחירת הספרה הנפוצה בקבוצת האימון היא קו בסיס תקין, ומדוע אין לבחור את הספרה הנפוצה בכל המאגר.

3. למה אי אפשר להסיק ממספר תמונות כמעט שווה לכל ספרה שאין הטיה במאגר?

רמה ב׳ — חישוב, ניתוח ותכנון

1. בהערכה של 50 תמונות נמצאו 43 תחזיות נכונות. חשבו Accuracy והסבירו מה המדד אינו מספר לכם.

2. עבור ספרה 7 התקבלו TP=12, FP=3, FN=4. חשבו Precision, Recall ו־F1. הסבירו מה משמעות כל אחד.

3. הציעו כיצד לבחור k ב־KNN באמצעות Cross-Validation מבלי לגעת ב־Test.

4. נמצא שזמן החיזוי ב־KNN ארוך יותר מאשר ב־Nearest Centroid. הציעו שני הסברים אפשריים שמחייבים ניסוי נוסף.

רמה ג׳ — חשיבה ביקורתית בסגנון בגרות והגנה

1. תלמיד תקנן את כל הנתונים, בחר k לפי Test ואז דיווח על אותו Test כציון הסופי. זהו שתי שגיאות שונות: הסבירו כל אחת ותקנו את הליך המחקר.

2. מודל א׳ קיבל Accuracy גבוה יותר, אך נכשל במספר רב יותר של ספרות 9. אילו מדדים וראיות נוספים תדרשו לפני החלטה?

3. אחרי צפייה בחמש טעויות Test הוחלט לשנות את עיבוד התמונה. כיצד ניתן להמשיך במחקר בלי להציג באופן מטעה את תוצאת Test הראשונית?

4. דונו בהבדל בין הצלחה על תמונות 8×8 לבין הצלחה על תמונות שנלקחו במצלמת טלפון, והציעו תכנית בדיקה מתאימה.

22. מחברת המעבדה העצמאית — הוראות לתלמיד

לצד פרק זה מצורפת מחברת Colab עצמאית. התלמידים יטענו מאגר, יבדקו את מאפייניו, יתעדו שאלת מחקר, יבנו קו בסיס, ישוו מסווגים, יבצעו ולידציה ויכתבו מסקנות. תאי הקוד שיש להשלים מסומנים TODO; אין במחברת פתרון מלא מוכן להעתקה. הדוגמאות הפתורות שבספר מיועדות להבנת הרעיונות ולבדיקות חלקיות בלבד.

מומלץ לשמור גרסאות מסודרות של הניסוי, לא לשנות את מספר הזרע האקראי רק כדי לקבל ציון טוב יותר, ולרשום מה נוסה ומה לא. תוצאה שלא הצליחה יכולה להיות תוצר לימודי משמעותי, אם ברור כיצד בדקנו אותה ומה למדנו ממנה.

23. שאלות הגנה בעל־פה: האם אתם מבינים את מה שבניתם?

הבוחן יכול לבקש מהתלמיד להסביר את מהלך העבודה בלי לראות את הקוד. לכן נתרגל הצגה של שתי דקות: מה הבעיה, מה הנתונים, מה המודל, כיצד בחרנו אותו, מה תוצאת הבדיקה ומה המגבלה

החשובה ביותר. לאחר מכן נשיב לשאלות שינוי כמו ״מה יקרה אם יופיעו תמונות באיכות אחרת?״ או ״כיצד תזהו זליגת מידע?״.

משפט סיכום טוב לפרויקט הוא לא ״המערכת מזהה ספרות ב־99%״ ללא הקשר. ניסוח מקצועי יותר: ״בחלוקת הבדיקה המסוימת ובתנאים שתוארו התקבל ציון X; נדרשות בדיקות נוספות לפני שימוש בתמונות שמקורן מחוץ למאגר״. כך מציגים יכולת בלי להגזים בהסקת מסקנות.

24. מה נרצה לקחת מפרק 10 לפרויקט הגמר בכיתה י״ב?

כאן תרגלנו את השיטה השלמה: מגדירים בעיה, מתארים מאגר, בוחנים קו בסיס, בונים מודל, בודקים ולידציה, מנתחים טעויות ומציגים ראיות. בפרויקט הגמר נוסיף עומק רב יותר במרכיב המודל, במערכת התוכנה ובניהול הפרויקט. לכן התוצר החשוב של המיני־פרויקט הוא לא רק מודל שמחזיר תחזית, אלא תיק חקר מסודר שמוכיח כיצד הגענו אליה.

הפרק הבא, מתמטיקה ללמידה עמוקה, יאפשר לנו להבין כיצד מודלים מעדכנים משקולות בעזרת נגזרות וגרדיאנטים. את המיני־פרויקט הנוכחי נוכל לשמור כנקודת השוואה לרשת עצבית שנבנה בהמשך.

נספח א׳ — תיק הפרויקט: תבנית לתכנון והחלטות

גם תלמיד שיודע לתכנת עלול להתקשות בהסבר של החלטותיו כמה שבועות לאחר הניסוי. לכן נשמור תיק פרויקט קצר לצד המחברת. תיק זה אינו מוסיף אלגוריתם חדש; הוא מלמד להפוך עבודה חד־פעמית לתהליך שחוקר אחר יכול לשחזר. יש למלא אותו תוך כדי התקדמות, ולא לכתוב את כולו רק ערב ההגשה.

דוגמה לסוג תשובה	מה נרשום	שדה בתיק
סיווג ספרה בתמונה קטנה	איזו החלטה רוצים לקבל בעזרת המודל	גרסת השאלה
מאגר פתוח מתוך ספרייה מוכרת	מקור מדויק, רישוי והגבלות שימוש	מקור הנתונים
תמונה אחת, לא תלמיד שלם	מה כל שורה מייצגת	יחידת תצפית
חלוקה משוכבת לפי תווית	איך נוצרו Train ו־Test ומדוע	חלוקה לנתונים
ניבוי מחלקת הרוב באימון	מהי רמת הביצוע של פתרון נאיבי	קו בסיס
השוואה של מודל מרכזים ושכנים	מועמדים והסיבה לבחירתם	בחירת מודל
Accuracy־ו Macro F1	מדד, CV וקבוצת Test עצמאית	שיטת הערכה
תמונות ממצלמות אחרות	מה עוד לא בדקנו	החלטה פתוחה

נספח ב׳ — יומן ניסויים: דוגמה לפירוש תוצאות

נניח שתלמיד בדק שלוש גרסאות של מודל על אותה מסגרת ולידציה. כדי להבין אם נוצר שיפור, נרשום לא רק את הציון אלא גם שינוי אחד שנעשה בכל גרסה. אם משנים גם את הסולם, גם את סוג המודל וגם את מספר השכנים באותה ריצה, קשה לדעת מה תרם לשינוי בתוצאה. בניסוי מבוקר משנים משתנה אחד בכל פעם ככל האפשר, תוך התחשבות בכך שלפעמים יש תלות בין גורמים.

ניסוי	מה השתנה	Macro F1 ב־CV (דוגמה מומצאת)	מסקנה זהירה
A	KNN, k=1	0.91	נקודת התחלה

ניסוי	מה השתנה	Macro F1 ב־CV (דוגמה מומצאת)	מסקנה זהירה
B	KNN, k=3	0.93	נראה שיפור קטן, לא הוכחה כללית
C	KNN, k=5	0.92	נמוך מעט מ־B בחלוקה זו

המספרים בטבלה מומצאים לצורכי לימוד; הם אינם תוצאות הניסוי על digits שבאיורים. בחקר אמיתי כדאי לציין גם פיזור ציוני הקיפולים, תאריך ריצה, גרסאות ספריות וזרע אקראי. אם השינוי קטן מאוד, ייתכן שההבדל נובע מהחלוקה או ממספר הדוגמאות. אין לתאר הבדל זעיר כ״שיפור משמעותי״ בלי עדויות מתאימות.

בכל רשומת ניסוי יש לכתוב השערה לפני ההרצה, שינוי שבוצע, תוצאה שנמדדה, הסבר אפשרי ומסקנה לגבי הצעד הבא. בדרך זו גם ניסוי שלא שיפר את הציון תורם להבנת המודל ומונע חזרה על אותן טעויות.

נספח ג׳ — דוגמת הסבר לבוחן: ממצא לעומת טענה

בוחן שואל: ״מדוע בחרתם ב־KNN ולא במודל של מרכזי מחלקות?״ תשובה חלשה תהיה: ״כי הוא יצא הכי טוב״. תשובה מקצועית צריכה לתאר את ההשוואה, להראות איזה מדד נקבע מראש, להסביר כיצד הופרדו הנתונים, ולציין מגבלה. לדוגמה: ״בחרנו שני מודלים שכבר למדנו, ביצענו ולידציה על קבוצת האימון ובחרנו את ההגדרה המועדפת לפי המדד שהוגדר. בבדיקת Test שלא שימשה לבחירה קיבלנו את התוצאה המדווחת; איננו טוענים שהתוצאה תישמר באותה רמה על תמונות ממקורות אחרים.״

שאלת המשך אפשרית היא: ״איך תדעו שהמודל אינו פשוט זוכר את תמונות האימון?״ כאן יש להדגיש שהערכת Test משתמשת בדוגמאות שלא היו באימון, שזיהוי כפילויות עשוי להיות חשוב ושדוגמאות חדשות באמת יכולות להיות שונות יותר מנתוני המאגר. התלמיד צריך לדעת להבחין בין יכולת הכללה בתוך מאגר מסוים לבין יכולת ההעברה לסביבה חדשה.

נספח ד׳ — רשימת בדיקה לפני הגשה

בדקו שכל ההחלטות מתועדות: הגדרת משתנה המטרה; מקור נתונים חוקי; רישום מספר תצפיות ותכונות; גרפים עם כותרות ויחידות; חלוקת Train/Test שנעשתה לפני ההתאמות; השוואה ל־Baseline; בחירת פרמטרים שאינה משתמשת ב־Test; מדד מתאים; ניתוח לפחות כמה טעויות; תיאור מגבלות; הוראות הרצה ושחזור; והפרדה בין קוד תלמיד לבין מקורות שהסתמכתם עליהם.

מבחן עצמי לסיום: תלמיד אחר מקבל רק את קובצי הפרויקט וההוראות שלכם. האם הוא יכול להריץ את הניסוי, לזהות אילו נתונים היו זמינים למודל, לשחזר את ההערכה ולגלות כיצד התקבלה ההחלטה? אם התשובה אינה חיובית, הפרויקט עדיין אינו מוכן להגשה גם אם המספר הסופי מרשים.

25. עוגן בתכנית ומקורות מקצועיים

מיקום בתכנית: מסלול התמחות למידת מכונה ולמידה עמוקה, פרק 10 — מיני־פרויקט, 30 שעות, לפי מפת התכנית במאגר GitHub של הקורס. חלוקת 30 השעות לעשרת המפגשים, שאלת הפרויקט ומחוון ההערכה שבספר הם תכנון דידקטי מקורי; אין לראות בהם חלוקה רשמית או מחוון רשמי של משרד החינוך.

מקורות מקצועיים להעמקת ההסבר: Géron — Hands-On Machine Learning (עבודה שלמה עם מאגר, השוואת מודלים והערכה); James ואחרים — An Introduction to Statistical Learning with Applications in Python (רגרסיה, סיווג, הערכה וולידציה); Serrano — Grokking Machine Learning (הסברים אינטואיטיביים). ההמחשות, התרגילים והסיפור בספר נכתבו באופן עצמאי.

קריאה ותיעוד — קישורים למקורות:

```
https://github.com/samiabo75/Data-Science-ML-Course/tree/main/03-Deep-Learning-5-Units/10-Mini-Project
https://scikit-learn.org/stable/common_pitfalls.html
https://www.statlearning.com/
https://scikit-learn.org/stable/modules/generated/sklearn.datasets.load_digits.html
```

מפת שימוש במקורות — מה למדנו מכל ספר?

מה נכתב בספר שלנו	תרומה לפרק זה	מקור מקצועי
מקרה לימודי בעברית, ניסויים ואחריות תיעודית	מחזור עבודה: נתונים, אימון, מדידה ושיפור	Hands-On Machine Learning — Géron
חישובים ידניים וטבלאות שמתאימות לתיכון	השוואת מודלים, מדדים, ולידציה ופרשנות	An Introduction to Statistical Learning — James ואחרים
סיפור דודו, שאלות הבנה ודוגמה מפורקת	הסבר אינטואיטיבי של מושגים לפני קוד	Grokking Machine Learning — Serrano
קוד קצר שאפשר להריץ ומדיניות Test ברורה	הגדרות של API, מאגר digits ואזהרות Data Leakage	scikit-learn Documentation

השימוש במקורות מיועד לאימות מושגים, למידה של שיטות הוראה והפניית התלמיד להמשך העמקה. האיורים, התרגילים והדוגמאות המספריות בפרק נוצרו באופן מקורי; אין כאן תרגום של טקסט מוגן מספר מסחרי.

מעמד התוכן ביחס לתכניות הלימודים

פרק 10 מופיע בהיקף של 30 שעות במפת תכנית ההתמחות הרשומה במאגר הקורס. המטרות של תיעוד ניסויים, ניתוח טעויות ושחזור הן התאמות מקצועיות לקראת עבודת הגמר. חלוקת השעות הפנימית, המחוון המוצע והפרויקט על מאגר digits הם בחירות פדגוגיות מקוריות; אין להציגם כציטוט מחייב ממחוון משרד החינוך שלא נבדק במלואו.

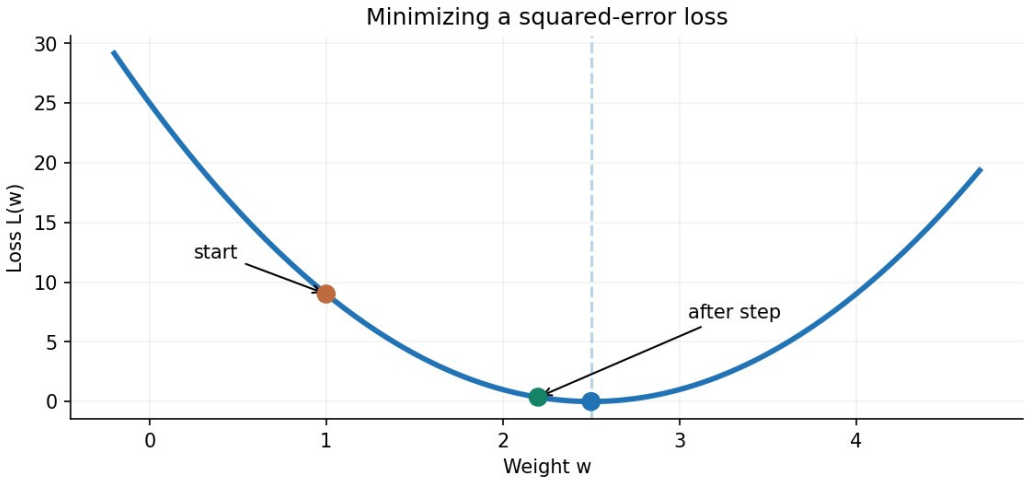
קישור רוחבי בספר המאוחד: יחידה 3 — קריאת נתונים ו־KNN בסיסי; יחידה 5 — וקטורים, מרחקים, הערכה ולידציה; התמחות למידה עמוקה — תהליך ניסוי מלא, תיעוד וביסוס לקראת רשתות עצביות ופרויקט גמר.

פרק י״א — מתמטיקה ללמידה עמוקה

כיצד מחשב לומד באמצעות שיפועים, נגזרות וגרדיאנטים?

מסלול התמחות בלמידת מכונה ולמידה עמוקה • כיתות י״א–י״ב • שלושים שעות לפי מפת תכנית ההתמחות

ספר תלמיד | פרק עצמאי בסדרה המאוחדת | נוסחאות, חישובים, גרפים, קוד ותרגול



מה נראה בפרק? מודל משנה משקל כדי להקטין את פונקציית השגיאה. נלמד לחשב ולהסביר את התהליך, ולא רק להריץ קוד.

הפרק בנוי לפי שיטת הספר שלנו: בעיה אמיתית → אינטואיציה → מתמטיקה מדורגת → חישוב ידני ב־Python → ניסוי → משמעות ל־AI → תרגול עצמאי.

מפת מטרות הלמידה

הוכחת הבנה	בסיום הפרק התלמיד יוכל…
לנתח מודל חיזוי ולקבוע מה נלמד ומה מחושב	לזהות קלט, פלט, פרמטר ופונקציית שגיאה
לחשב ולהסביר סימן וגודל נגזרת	לפרש שיפוע ונגזרת
לפתור תרגילי פונקציות בשני פרמטרים	לחשב נגזרות, נגזרות חלקיות וגרדיאנט
לבצע כמה צעדי עדכון ולפרש השפעת קצב למידה	לעקוב אחר Gradient Descent
לחשב את השפעת משקל על שגיאה	להפעיל כלל שרשרת במסלול חישוב
להשוות חישוב ידני, NumPy והפרשים סופיים	לממש ולבדוק חישובים ב־Python
לצייר גרף חישוב ולפרט מה מתקדם קדימה ומה אחורה	להסביר את הקשר ל־Backpropagation

ידע קודם וחיבור למסלול

דרוש: אלגברה בסיסית, חזקות, פונקציה ליניארית, משוואת ישר, מכפלה פנימית, NumPy, רגרסיה ליניארית ומושגי משקולת, הטיה ושגיאה מפרקים 5 , 8 ו־ 9 . איננו מניחים שהלומד כבר למד חשבון דיפרנציאלי באופן שיטתי; כל סימן יוסבר בעת הופעתו.

הפרק מעוגן בפרק י״א — מתמטיקה ללמידה עמוקה (0 3 שעות) שבמפת תכנית ההתמחות. החלוקה הפנימית, דוגמאות Gradient Descent ובדיקת נגזרות הן תכנון דידקטי שלנו. התוכן המדויק של כל תת־נושא יתוקף מול הסילבוס הרשמי המלא לפני קביעת מעמדו כחומר חובה.

11.1 סיפור מסגרת: דודו רוצה לשפר תחזית

דודו בונה מערכת המסייעת למעבדת המחשבים להעריך כמה דקות נדרשות לתיקון תקלה. בתחילה הוא יוצר נוסחה פשוטה המבוססת על מספר התקלות הקודמות. המודל שלו מפיק תחזית, אבל התחזית אינה זהה לזמן האמיתי. מה צריך לשנות כדי להשתפר?

דודו יכול לנסות משקל חדש באופן מקרי, לבדוק את השגיאה ולנסות שוב. אבל כאשר למודל יש אלפי משקולות, ניסוי מקרי לכל אחת מהן אינו דרך יעילה. עלינו להבין לאיזה כיוון לשנות כל משקל, וכמה לשנות אותו.

זה תפקיד המתמטיקה בפרק: הנגזרת מתארת שינוי מקומי, הגרדיאנט מאגד נגזרות לפי פרמטר, וכלל השרשרת מאפשר להעביר מידע על השגיאה דרך רצף החישובים.

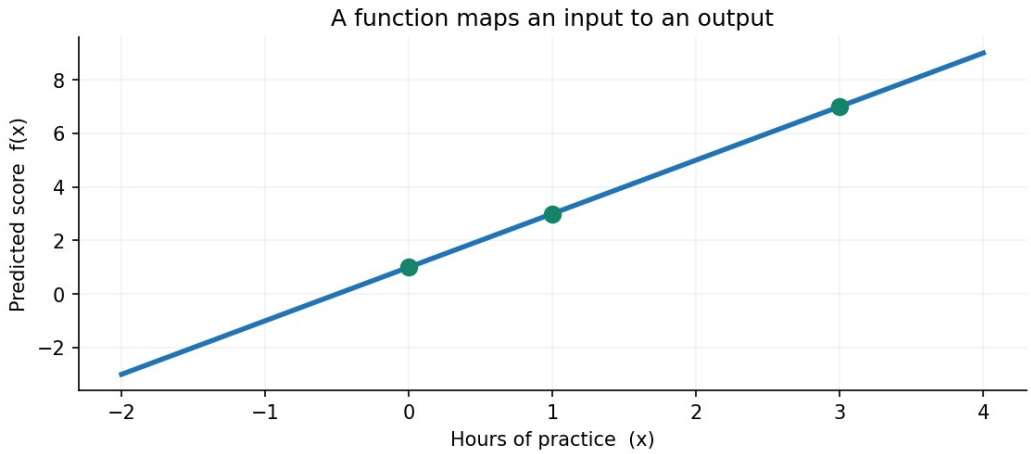
11.2 פונקציה — מכונה מתמטית המקבלת קלט ומחזירה פלט

פונקציה היא כלל המייחס לכל ערך קלט בתחום ערך פלט אחד. לדוגמה, 21 מתארת קשר בין משתנה x לערך מספרי מחושב. אם מציבים $x=3$ מקבלים 7 . אין פירוש הדבר שהעולם באמת מתנהג על פי הפונקציה הזאת; היא מודל אפשרי של הקשר.

במודל רגרסיה אנחנו כותבים $\hat{y}=wx+b$. הסימון $\hat{y}$ נקרא ״וואי כובע״ ומציין תחזית. x הוא מאפיין נתון, w הוא משקל ו־b הוא הטיה. במהלך האימון מנסים להתאים את w ואת b. בעת החיזוי, מאפייני הקלט אינם משקולות.

$$f(x)=2x+1 \quad ; \quad f(3)=2\cdot 3+1=7$$

כדי להבין מודל, עלינו תמיד לשאול: מה המשתנה שמשנים? מה מוחזק קבוע? ומה נמדד בציר האנכי?



דוגמה לקשר בין קלט לפלט. נקודות על הישר אינן הוכחה שהקשר בחיים האמיתיים הוא ליניארי.

11.3 שינוי ממוצע — השיפוע בין שתי נקודות

נניח שהתלמיד רוצה לדעת עד כמה התחזית משתנה כאשר מגדילים את ערך הקלט. נחשב תחילה שינוי ממוצע: לוקחים הפרש בין שני ערכי הפונקציה ומחלקים בהפרש בין ערכי הקלט.

$$\text{Average slope} = [f(x_2) - f(x_1)] \,/\, (x_2 - x_1)$$

עבור $f(x)=x^2$, בין $x=1$ לבין $x=2$ נקבל $f(1)=1$ ו־$f(2)=4$. לכן השיפוע הממוצע הוא $(4-1)/(2-1)=3$. משמעות התוצאה: לאורך הקטע הזה, הפונקציה גדלה בממוצע בשלוש יחידות פלט עבור יחידת קלט אחת.

אבל האם השיפוע הוא 3 גם ממש ליד $x=1$? לא. ככל שמקטינים את המרחק בין הנקודות מקבלים מידע מקומי מדויק יותר.

11.4 ממוצע שינוי לנגזרת — הרעיון של גבול

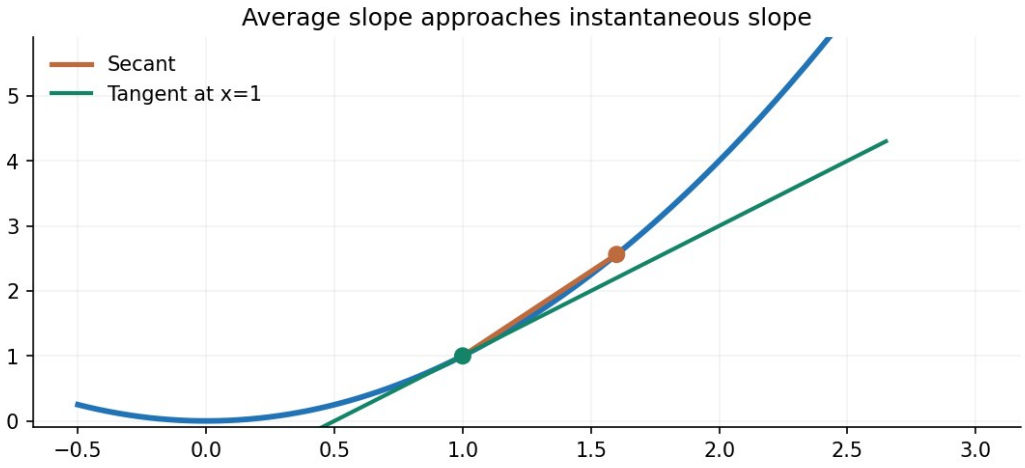
נרצה למדוד את קצב השינוי בנקודה אחת. נבחר נקודה x ונסתכל על נקודה קרובה $x+h$. כאשר h נעשה קטן יותר, מיתר המחבר את שתי הנקודות מתקרב למשיק לעקומה. אם הגבול קיים, שיפוע המשיק נקרא הנגזרת בנקודה.

$$f'(x) = \lim_{h \to 0} [f(x+h) - f(x)] \,/\, h$$

בסימון זה $\lim$ פירושו גבול; h הוא שינוי קטן בקלט; והסימן f' מבטא נגזרת. הגבול איננו הצבה ישירה של $h=0$, מפני שאז נקבל חילוק באפס. הוא מתאר למה מתקרב הביטוי כאשר h קרוב לאפס.

בדוגמה $f(x)=x^2$, כאשר $x=1$ נכתוב: $[(1+h)^2 - 1]/h = 2+h$, לכל h שאינו אפס. לכן כאשר h מתקרב לאפס, הנגזרת מתקרבת ל־2. זהו השיפוע המיידי בנקודה $x=1$.

שימו לב להבחנה: הפונקציה מחזירה גובה או ערך; הנגזרת מתארת את קצב השינוי של אותו ערך.



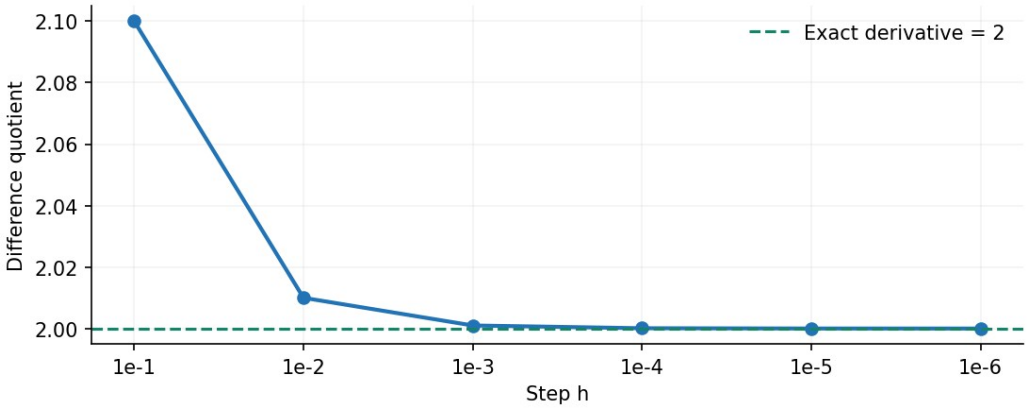
המיתר מודד שיפוע ממוצע; המשיק מתאר את השיפוע המקומי בנקודה $x=1$.

11.5 בדיקה מספרית — מאמתים נגזרת בעזרת Python

נחשב שוב את שיפוע ההפרש עבור $f(x)=x^2$ ליד $x=1$. כאשר $h=0.1$ נקבל 2.1, כאשר $h=0.01$ נקבל 2.01, וכאשר $h=0.001$ נקבל 2.001. הערכים מתקרבים לתוצאה המדויקת 2.

המחשב מאפשר לנו לחקור את הרעיון הנובע מן הנוסחה. עם זאת, מספרים במחשב מיוצגים בדיוק מוגבל. h קטן באופן קיצוני עלול דווקא ליצור שגיאת עיגול. לכן בדיקה מספרית מספקת ראיה מעשית, לא תחליף להוכחה מתמטית.

התנסות מודרכת: כתבו פונקציה המחזירה את $f(x)$, ואז חשבו את הביטוי עבור כמה ערכי h. לפני ההרצה נסו לנבא את כיוון ההתקרבות.



הפרש המנות מתקרב לערך הנגזרת המדויק. סדר הדוגמאות בגרף הוא מגודל $h=0.1$ ועד $h=0.000001$.

```
def f(x):
    return x ** 2

for h in [0.1, 0.01, 0.001, 0.0001]:
    slope = (f(1 + h) - f(1)) / h
    print(h, round(slope, 6))
```

11.6 איך מפרשים נגזרת חיובית, שלילית או אפס?

כאשר $f(x)>0$ הפונקציה עולה באופן מקומי. כאשר $f'(x)<0$ היא יורדת באופן מקומי. נגזרת אפס מתארת משיק אופקי, אך לא תמיד נקודת מינימום: היא יכולה להיות גם מקסימום או נקודה אחרת, למשל $f(x)=x^3$ בנקודה $x=0$.

גודל הנגזרת מלמד על רגישות מקומית: נגזרת 6 פירושה שינוי בקירוב של 6 יחידות פלט לכל שינוי קטן של יחידת קלט, כאשר נשארים מספיק קרובים לנקודה. זהו קירוב מקומי ולא הבטחה לשינוי לאורך מרחק גדול.

$$\Delta f \approx f'(x) \cdot \Delta x$$

למידת מכונה משתמשת בפירוש הזה כדי להעריך מה יקרה לשגיאה אם נשנה מעט את המשקל. אבל יש לבדוק את גודל הצעד: קירוב מקומי עשוי לא להספיק לצעד גדול.

11.7 כללי גזירה בסיסיים — כלי עבודה למודל

במקום לחשב גבול מההתחלה בכל פעם, אפשר להשתמש בכללי גזירה. חישוב הנגזרת של קבוע הוא אפס, משום ששינוי בקלט לא משנה אותו. הנגזרת של x היא 1, כי כל שינוי בקלט מועבר ישירות לפלט.

כלל החזקה קובע שהנגזרת של x בחזקת n היא n כפול x בחזקת $n-1$. אם יש מקדם קבוע, הוא נשאר כגורם כפל. סכום של פונקציות גוזרים רכיב אחר רכיב.

$$d(c)/dx=0 \;;\; d(x^n)/dx=n\cdot x^{n-1} \;;\; d[c\cdot f(x)]/dx=c\cdot f'(x)$$

דוגמה: אם $f(x)=3x-4x+7$, נגזור $3x^2$ לקבלת $6x$, נגזור $-4x$ לקבלת -4, ונגזור את הקבוע 7 לקבלת 0. לכן $f'(x)=6x-4$. בנקודה $x=2$, הנגזרת היא 8.

כללי הכפל והמנה יופיעו כהעשרה בסוף הפרק. לבניית ההבנה של אימון רשתות חשובה במיוחד שליטה בגזירת הרכבה באמצעות כלל השרשרת.

פונקציה	נגזרת	מה חשוב להבין
c	0	קבוע אינו משתנה עם הקלט
x	1	שינוי יחידה יוצר שינוי יחידה
x^2	$2x$	השיפוע תלוי במיקום
$3x^2-4x+7$	$6x-4$	גוזרים כל איבר בנפרד
e^x	e^x	פונקציה מעריכית שומרת צורה בנגזרת
$\ln(x)$	$1/x \ (x>0)$	לוגריתם טבעי בתחום החיובי

11.8 הקשר בין נגזרת לנקודות קיצון

במודל אנו מחפשים ערכי משקולות שמקטינים את השגיאה. כאשר מציירים את פונקציית השגיאה, מחפשים מקום שבו הערך שלה נמוך. בפונקציה חלקה, נקודת מינימום פנימית מקיימת לעיתים קרובות תנאי של נגזרת אפס.

אבל חשוב לא להפוך זאת לכלל שגוי: נגזרת אפס אינה מבטיחה מינימום, ולא כל פונקציה חלקה היא בעלת מינימום יחיד. לדוגמה, גם בשיא של גבעה הנגזרת יכולה להיות אפס.

בפונקציה $L(w)=(2w-5)^2$ נקבל מינימום כאשר $2w-5=0$, ולכן $w=2.5$. נגזור ונראה שהנגזרת מתאפסת בדיוק שם.

11.9 פונקציית שגיאה — מה מודדים במהלך האימון?

מודל מקבל מאפיין x ומחשב תחזית $\hat{y}$. להשוואה לערך הנכון y נקרא שארית או טעות חיזוי $e=\hat{y}-y$. כדי שהשגיאה החיובית והשלילית לא יתבטלו, אפשר לרבע אותה. פונקציית שגיאה אחת לדוגמה בודדת היא $L=(y-\hat{y})^2$.

כאשר יש כמה דוגמאות, מדד Mean Squared Error הוא הממוצע של ריבועי השגיאות. לעתים מגדירים הפסד עם גורם $1/2$ כדי לפשט את הנגזרת; יש לשמור על אותה הגדרה בזמן חישוב והשוואה.

$$e = \hat{y} - y \qquad ; \qquad L = e^2 \qquad ; \qquad MSE = (1/n)\ \Sigma_i(\hat{y}_i - y_i)^2$$

המטרה של אימון היא להפחית את השגיאה על נתוני האימון בלי לפגוע ביכולת ההכללה. הפחתה אינסופית של שגיאת אימון אינה יעד עצמאי: היא עלולה להוביל להתאמת יתר.

11.10 גוזרים את השגיאה ביחס למשקל אחד

נניח שבתרגיל מפושט $x=2$, התשובה הנכונה היא $y=5$, והמודל מנבא $\hat{y}=2w$ ללא הטיה. לכן הפונקציה המתארת את שגיאת המודל היא $L(w)=(2w-5)^2$.

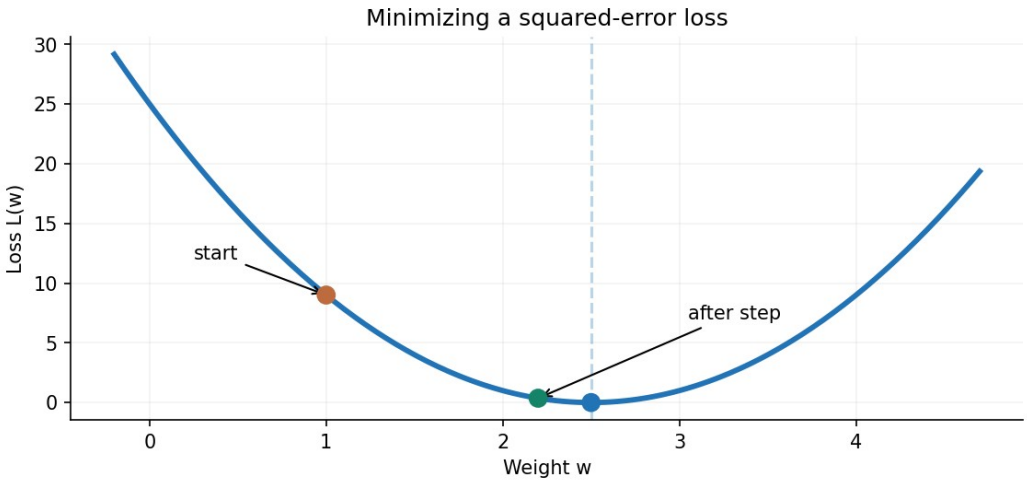
תחילה נגזור את הביטוי החיצוני: ריבוע של משהו. לפי כלל השרשרת, הנגזרת היא פעמיים הביטוי הפנימי כפול הנגזרת של הביטוי הפנימי.

$$L(w) = (2w - 5)^2$$

$$dL/dw = 2(2w-5)\cdot 2 = 8w-20$$

אם $w=1$, התחזית היא 2, והשגיאה היא 9. הנגזרת היא $8\cdot(1-20)=-12$. הסימן השלילי אומר שהגדלה קטנה של המשקל צפויה להקטין את השגיאה בנקודה הזאת. הוא לא אומר שהשגיאה עצמה שלילית.

אם $w=3$, הנגזרת היא 4, כלומר הגדלה קטנה של המשקל דווקא תעלה את השגיאה. כך ניתן לקבוע כיוון פעולה.



פונקציית השגיאה כתלות במשקל. הנגזרת מראה לאיזה כיוון לשנות אותו.

צעד עדכון מלא — Gradient Descent 11.11

ב־Gradient Descent משנים את המשקל בכיוון ההפוך לנגזרת, משום שזהו בדרך כלל הכיוון שבו השגיאה יורדת כאשר הצעד מספיק קטן. קצב הלמידה מסומן באות היוונית η (אֶטָה).

$$w_{new} = w_{old} - \eta \cdot (dL/dw)$$

בדוגמה שלנו $w=1$, $\eta=0.1$ והנגזרת היא -12. תחילה מכפילים קצב למידה בנגזרת: $0.1 \cdot (-12) = -1.2$. כעת מחסרים את הערך השלילי מהמשקל: $w_{new} = 1 - (-1.2) = 2.2$.

אחרי העדכון התחזית היא $2.2 \cdot 2 = 4.4$. השגיאה החדשה היא $(4.4-5)^2 = 0.36$, במקום 9. שימו לב: לא "בחרנו" מראש 2.2; הוא התקבל מכלל עדכון שהיה תלוי בנגזרת.

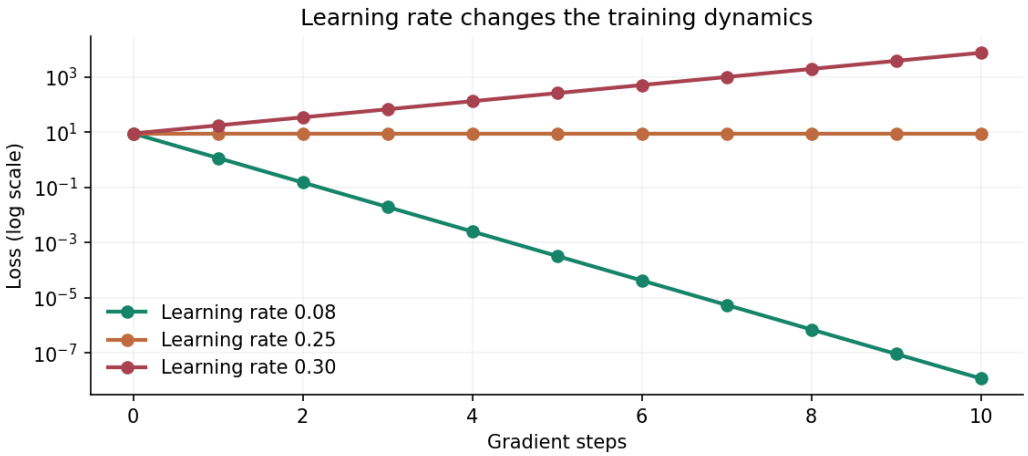
אין הבטחה שכל צעד ובכל קצב למידה יפחית את השגיאה. זוהי השערה שצריך לאמת על פונקציית השגיאה בפועל.

קצב למידה קטן מדי או גדול מדי 11.12

אם η קטן מאוד, ייתכן שהמודל יתקדם לעבר המינימום לאט. אם הוא גדול מדי, העדכון עשוי לדלג מעל המינימום ולהגדיל את השגיאה שוב ושוב. ערכי קצב הלמידה המתאימים תלויים בצורת פונקציית השגיאה.

בדוגמה הריבועית $L(w) = (2w-5)^2$, נגזרת השגיאה היא $8w - 20$. כאשר $\eta=0.1$, משקל התחלתי 1 מתקרב אל 2.5. לעומת זאת, כאשר $\eta=0.3$, צעד העדכון עלול להרחיק אותנו מן המינימום ולגרום לעלייה בשגיאה.

בתרגול נציג שני מסלולים ונבקש מתלמידים לנבא מה יקרה לאחר עוד שלושה צעדים לפני הרצת הקוד.



השוואת קצבי למידה באותה פונקציית שגיאה; הגרף משתמש בסולם לוגריתמי בציר האנכי.

```
def loss(w):
    return (2*w - 5)**2

def gradient(w):
    return 8*w - 20

w = 1.0
eta = 0.1
for step in range(6):
    print(step, round(w, 4), round(loss(w), 5))
    w = w - eta * gradient(w)
```

11.13 ממשקל אחד לשני פרמטרים: w ו־b

מודל אמיתי יכול להשתמש גם בהטיה b. כעת השגיאה תלויה בשני מספרים, ולכן נוכל לשאול בנפרד: כיצד היא משתנה כשמגדילים מעט את w ומשאירים את b קבוע? וכיצד היא משתנה כשמגדילים את b ומשאירים את w קבוע?

נגזרת ביחס למשתנה אחד כאשר האחרים קבועים נקראת נגזרת חלקית. סימונה ∂. אם $L(w,b)=(w+b-4)^2$, נגזור לפי w ו־b ונקבל בשני המקרים $2(w+b-4)$, מפני שכל אחד מהמשתנים מופיע כאן במקדם 1.

$$\partial L/\partial w = 2(w+b-4) \quad ; \quad \partial L/\partial b = 2(w+b-4)$$

אם w=1 ו־b=0, השגיאה היא 9, וכל אחת משתי הנגזרות שווה −6. בקצב למידה 0.1, שני הערכים גדלים ב־0.6. כעת w=1.6, b=0.6 והתחזית 2.2; השגיאה החדשה היא $(2.2-4)^2 = 3.24$.

בדוגמה הזאת יש יותר מזוג אחד של משקולת והטיה שנותנים אותה תחזית. כך אנו מבחינים בין הקטנת שגיאה לבין פתרון יחיד לפרמטרים.

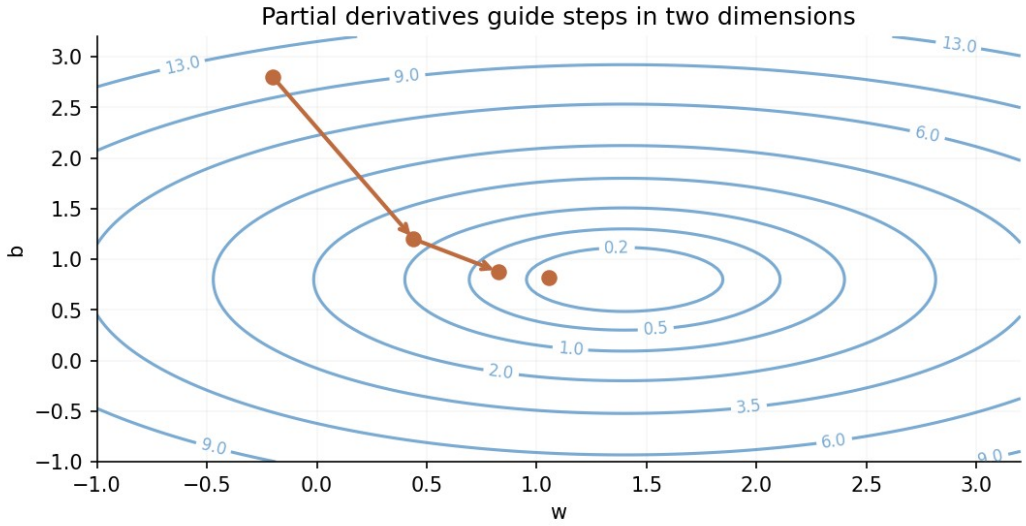
11.14 גרדיאנט — וקטור נגזרות חלקיות

כאשר לפונקציית השגיאה יש כמה פרמטרים, אוספים את הנגזרות החלקיות שלה לווקטור אחד הקרוי גרדיאנט (Gradient). כל רכיב בו מתאר רגישות של השגיאה לשינוי קטן בפרמטר מסוים.

$$\nabla L(w,b) = (\partial L/\partial w,\ \partial L/\partial b)$$

בכל איטרציה אפשר לעדכן את וקטור הפרמטרים כולו: $\theta_{new} = \theta_{old} - \eta \nabla L(\theta_{old})$. זהו אותו רעיון של צעד אחד, כעת במספר ממדים.

באזור חלק של הפונקציה, הגרדיאנט מצביע לכיוון העלייה המהירה ביותר לפי המרחק האוקלידי. לכן הכיוון השלילי שלו משמש לתנועה מקומית לכיוון ירידה. שוב: מדובר במידע מקומי, ולא בהבטחה למצוא מינימום גלובלי.



קווי גובה של פונקציית שגיאה בשני פרמטרים; החצים מדגימים תנועה לכיוון ערכי שגיאה קטנים.

11.15 דוגמה מלאה: גרדיאנט של MSE על שתי תצפיות

נשתמש בשתי דוגמאות אימון: $x=1$ עם $y=3$, ו־$x=2$ עם $y=5$. המודל הוא $\hat{y}=wx+b$. נתחיל במשקל $w=1$ ובהטיה $b=0$. התחזיות יהיו 1 ו־2, ולכן השאריות הן -2 ו־-3.

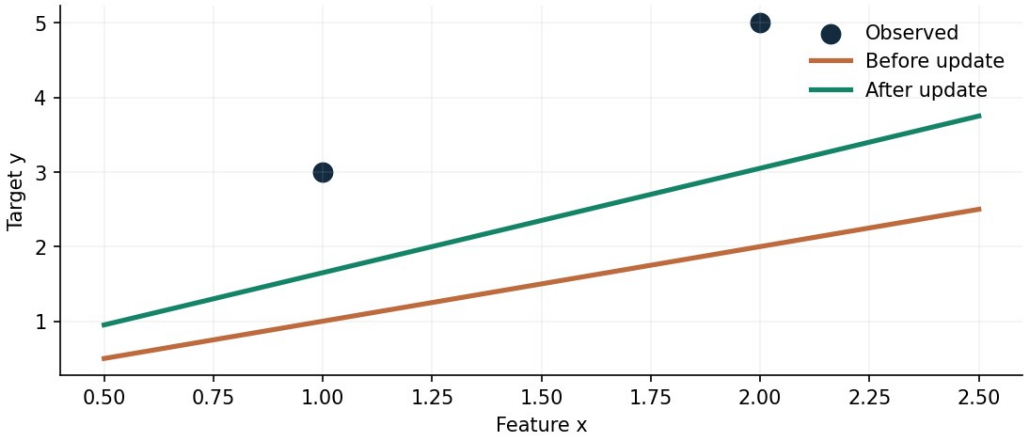
ממוצע ריבועי השגיאות הוא $(4+9)/2 = 6.5$. כאשר גוזרים את MSE ביחס למשקל, כל שארית מוכפלת גם בערך המאפיין שלה; ביחס להטיה מכפילים ב־1. כך מקבלים:

$$\partial \mathrm{MSE}/\partial w = (2/n)\,\Sigma_i\,(\hat{y}_i - y_i)\cdot x_i = -8$$

$$\partial \mathrm{MSE}/\partial b = (2/n)\,\Sigma_i\,(\hat{y}_i - y_i) = -5$$

בקצב למידה 0.05 המשקל החדש הוא 1.4 וההטיה החדשה 0.25. התחזיות הופכות ל־1.65 ול־0.35. השגיאה החדשה היא $(1.8225+3.8025)/2 = 2.8125$. ירדנו מ־$6.5$ ל־2.8125 בצעד אחד.

מדוע חישבנו את ממוצע השגיאות ולא את סכומן? משום שבדוגמה הזאת בחרנו במפורש ב־MSE. אם היינו מגדירים פונקציית שגיאה אחרת, גם הנגזרת הייתה משתנה.



שתי דוגמאות אימון, קו חיזוי לפני עדכון וקו חיזוי אחריו. הקווים הם המחשה לשינוי פרמטרים, לא להכללה.

שלב	לפני	אחרי עדכון אחד
משקל w	1.00	1.40
הטיה b	0.00	0.25
תחזית ל־$x=1$	1.00	1.65
תחזית ל־$x=2$	2.00	3.05
MSE	6.5000	2.8125

11.16 כלל השרשרת — איך נגזרת עוברת דרך שלבים?

חישוב של מודל אינו פעולה אחת. תחילה מחשבים $z=wx+b$; אחר כך ייתכן שמפעילים פונקציה; לאחר מכן מחשבים שגיאה. אם משנים את w, התוצאה הסופית משתנה דרך כל השלבים. כלל השרשרת מאפשר למדוד את ההשפעה הכוללת.

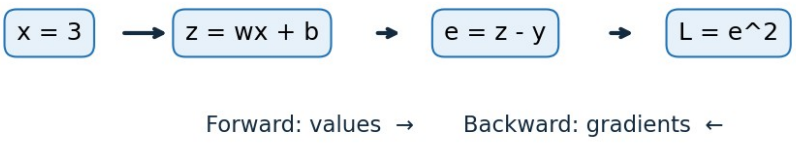
למשל, נניח $x=3,\ w=2,\ b=1$ ו־$y=10$. תחילה $z=2\cdot3+1=7$. השארית היא $e=z-y=-3$, ופונקציית השגיאה $L=e^2=9$.

$$\frac{dL}{dw} = \frac{dL}{de}\cdot\frac{de}{dz}\cdot\frac{dz}{dw}$$

$$\frac{dL}{de} = 2e = -6 \quad ; \quad \frac{de}{dz}=1 \quad ; \quad \frac{dz}{dw}=x=3$$

$$\frac{dL}{dw} = (-6)\cdot1\cdot3 = -18$$

בדומה, $dL/db = -6$ משום שנגזרת z ביחס ל־b היא 1. סימון קשתות בגרף החישוב מאפשר לעקוב אחרי ההשפעה בלי לאבד אף שלב.



גרף חישוב מקורי: הערכים מחושבים קדימה, והנגזרות מחושבות אחורה באמצעות כלל השרשרת.

11.17 עדכון משקולת והטיה אחרי כלל השרשרת

נמשיך את הדוגמה הקודמת ונבחר קצב למידה $\eta=0.01$. המשקל החדש הוא $2-0.01\cdot(-18)=2.18$. ההטיה החדשה היא $1-0.01\cdot(-6)=1.06$. עכשיו נחשב תחזית מחדש.

$$z_{new} = 2.18\cdot 3 + 1.06 = 7.60$$

$$L_{new} = (7.60 - 10)^2 = 5.76$$

השגיאה ירדה מ־9 ל־5.76. בניגוד לעדכון פרספטרון על טעות סיווג, כאן העדכון מבוסס על נגזרות של פונקציית שגיאה רציפה. ההבחנה הזאת חיונית בדרך להבנת אימון רשת עצבית.

בפונקציה מרובת שכבות, כל שכבה תלויה בקודמתה. לכן אי אפשר לחשב את השפעת המשקולות הראשונות על השגיאה הסופית בלי להעביר מידע לאורך כל שרשרת החישוב.

11.18 פונקציות הפעלה: מדוע לא מספיק חיבור ליניארי?

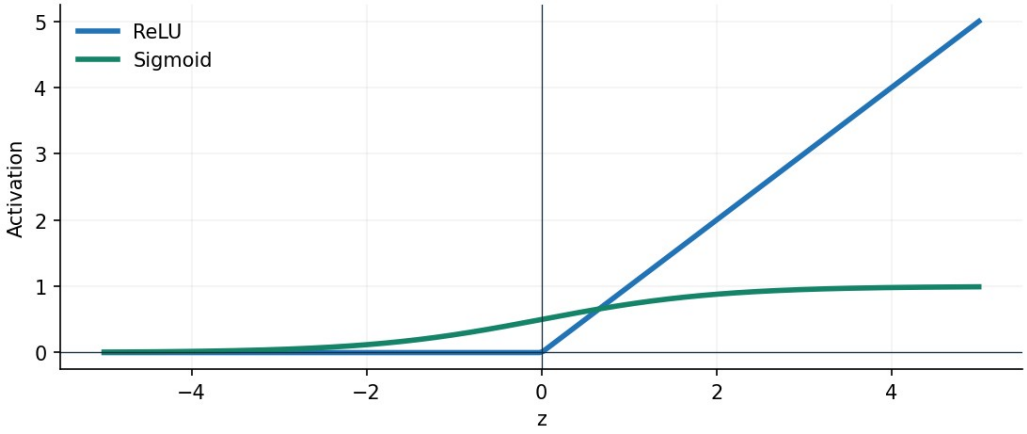
אם מחברים שכבות המבצעות רק צירופים ליניאריים, הרכב השכבות עדיין יכול להיות מוצג כפעולה אפינית אחת. כדי ללמוד דפוסים מורכבים כגון XOR, מוסיפים לרשת פונקציות הפעלה לא־ליניאריות.

פונקציית ReLU מחזירה 0 לקלט שלילי ואת הקלט עצמו לקלט חיובי. פונקציית Sigmoid מחזירה ערך בין 0 ל־1, ולכן משמשת לעתים כשלב פלט בסיווג בינארי. הפלט שלה לבדו אינו מבטיח הסתברות מכוילת.

$$\mathrm{ReLU}(z)=\max(0,z) \quad ; \quad \mathrm{sigmoid}(z)=1/(1+e^{-z})$$

נגזרת ReLU היא 0 לערכים שליליים ו־1 לערכים חיוביים; בנקודה 0 הנגזרת המתמטית אינה קיימת, ובמימושים נהוג לבחור ערך תת־נגזרת או מוסכמה. נגזרת Sigmoid היא $\sigma(z)\cdot[1-\sigma(z)]$.

בפרק זה אנו מבינים את הרעיון המתמטי. את אימון הרשת המלא נלמד בפרק 12.



שתי פונקציות הפעלה נפוצות: *ReLU* ו־*Sigmoid*. הן יוצרות אי־ליניאריות במודל.

11.19 דוגמת העמקה — כלל השרשרת דרך Sigmoid

נבנה יחידה אחת: $z=wx+b$, אחר כך $p=\sigma(z)$, ולבסוף $L=0.5(p-y)^2$. המספר 0.5 הוא בחירה טכנית שמפשטת את הנגזרת; בדוגמה זו איננו משתמשים ב־Cross Entropy.

ניקח $x=2, y=1, w=0, b=0$. נקבל $z=0$ ולכן $p=0.5$. השגיאה ההתחלתית היא $(0.5-1)^2=0.125$.

$$dL/dw = (p-y)\cdot p(1-p)\cdot x$$

כעת נציב: $dL/dw=(-0.5)\cdot 0.25\cdot 2=-0.25$. עבור ההטיה הנגזרת היא $(0.5-1)\cdot 0.25=-0.125$.

בקצב למידה 0.4 נקבל $w=0.1$ ו־$b=0.05$. הסכום המשוקלל החדש יהיה $z=0.25$, ולכן $p\approx 0.5622$. השגיאה החדשה תהיה בקירוב 0.0959, כלומר נמוכה מן השגיאה הקודמת.

זהו גשר מפורש אל Backpropagation, אך אינו מימוש מלא של רשת בעלת שכבות רבות.

11.20 Gradient Checking — כיצד בודקים נגזרת שחושבה ביד?

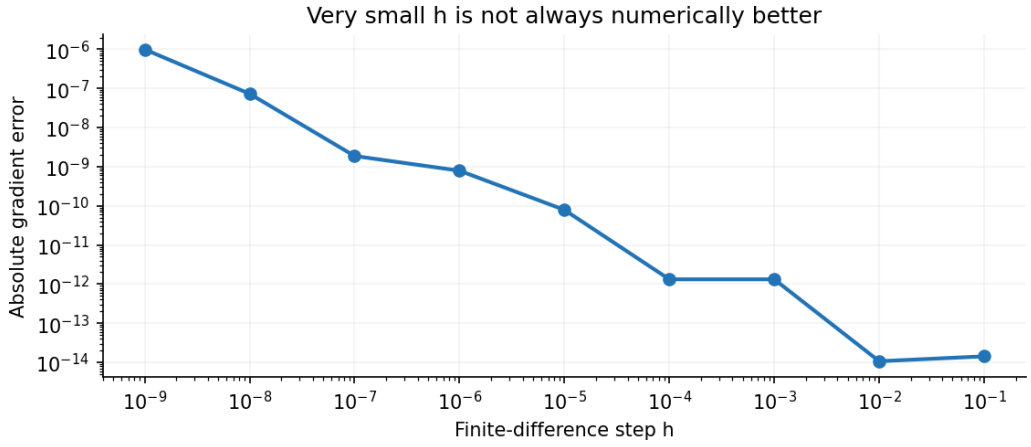
כאשר מפתחים פונקציית שגיאה מורכבת, טעות באחד מגורמי כלל השרשרת עלולה לגרום לאימון לא נכון. דרך אבחון מועילה היא להשוות את הנגזרת האנליטית לנגזרת מספרית המחושבת באמצעות הפרשים סופיים.

$$f'(w) \approx [f(w+h) - f(w-h)] / (2h)$$

הדוגמה נקראת הפרש מרכזי. נבחר h שאינו גדול מדי ואינו זעיר מדי. הנוסחה נותנת קירוב בדרך כלל מדויק יותר מהפרש חד־צדדי עבור פונקציה חלקה, אך אינה עובדת באותה צורה בנקודות לא־גזירות.

כדאי לבדוק הבדלים מוחלטים ויחסיים, להתייחס לסולם המספרים ולוודא שהפונקציה הדטרמיניסטית נבדקת באותם ערכי קלט.

בדיקת גרדיאנט היא כלי פיתוח ובקרה. במהלך אימון רגיל של רשת עמוקה נעדיף מנגנוני גזירה אוטומטיים ולא נחשב הפרשים סופיים לכל משקל.



דוגמה מספרית להשפעת גודל צעד ההפרש על שגיאת קירוב הנגזרת; מספרים זעירים מאוד עלולים לפגוע בדיוק.

```
def loss(w):
    return (2*w - 5)**2

w = 1.0
h = 1e-5
numerical = (loss(w+h) - loss(w-h)) / (2*h)
analytical = 8*w - 20
print('numerical:', numerical)
print('analytical:', analytical)
```

11.21 Gradient Descent במאגרי נתונים: Batch, Mini-batch ו־SGD

כאשר יש מאגר גדול, מחשבים גרדיאנט של שגיאה ממוצעת על מספר דוגמאות. ב־Batch Gradient Descent משתמשים בכל דוגמאות האימון לחישוב צעד אחד. ב־Stochastic Gradient Descent הקלאסי משתמשים בדוגמה אחת בכל צעד. ב־Mini-batch משתמשים בקבוצה קטנה של דוגמאות.

Mini-batch מאפשר עבודה וקטורית יעילה ויוצר גרדיאנט שהוא אומדן המבוסס על הקבוצה. גודל הקבוצה משפיע על עלות חישוב ועל תנודתיות העדכונים. אין מספר קסם אחד המתאים לכל משימה.

חשוב להבחין בין Epoch — מעבר על כלל דוגמאות האימון לפי שיטת האימון — לבין Step — פעולת עדכון אחת של הפרמטרים. בתקופה אחת עשויים להיות הרבה צעדים.

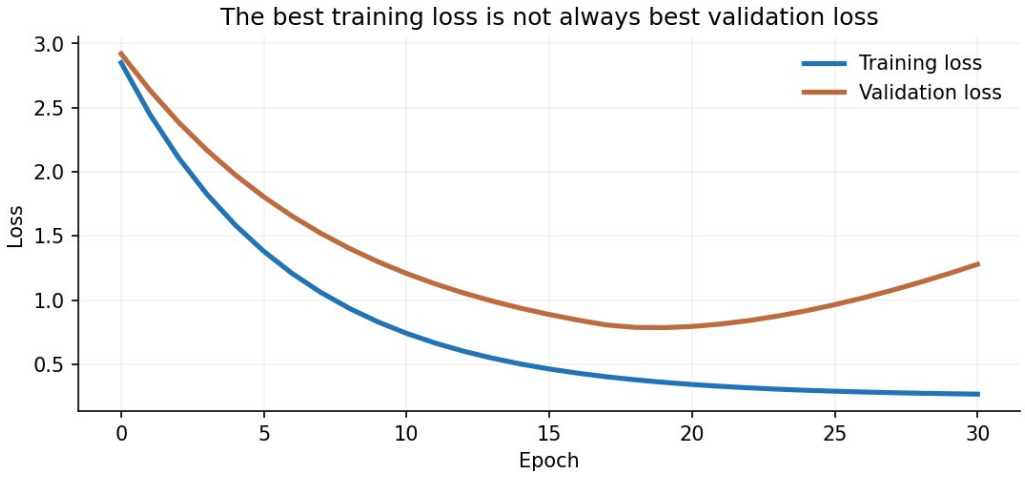
בדוגמה בעלת 100 תצפיות וקבוצות בגודל 20, מעבר מלא כולל בדרך כלל חמישה צעדי עדכון, אם אין דוגמה שנגרעת והקבוצות אינן חופפות.

11.22 האם ירידה בשגיאת אימון מספיקה?

לא. כמו בפרק 2 . 5 , מודל נדרש לבצע תחזיות גם על נתונים שלא שימשו להתאמת המשקולות. לכן נעקוב בנפרד אחרי שגיאת האימון ושגיאת הוולידציה. אם שגיאת האימון ממשיכה לרדת ושגיאת הוולידציה עולה, זו יכולה להיות אינדיקציה להתאמת יתר.

זו אינה הוכחה יחידה: צריך לבדוק איכות נתונים, מדדי הערכה, חלוקה תקינה, ייצוג אוכלוסיות ושונות ניסוי. לפעמים שגיאת ולידציה מתנדנדת בגלל מאגר קטן.

בפרויקטים תלמידים יתעדו ערכי קצב למידה, מספר Epochs, חלוקת הנתונים וניסוי השוואה. לא בוחרים פרמטרים באמצעות קבוצת Test שנועדה להערכה הסופית.



דוגמת עקומות אימון וולידציה סכמטיות. התלמיד צריך לזהות בעיית הכללה, לא רק את ערך השגיאה הנמוך ביותר.

11.23 מתי הנגזרת אינה מספר פשוט?

קיימות פונקציות שאין להן נגזרת בחלק מהנקודות. דוגמה מוכרת היא הערך המוחלט $|x|$ ב־$x=0$. בפונקציות בעלות פינות משתמשים לעיתים בכלים מתמטיים נוספים כגון תת־גרדיאנט, או בוחרים מוסכמה חישובית כפי שנהוג ב־ReLU.

יש גם אזורים שבהם הנגזרת קטנה מאוד או גדולה מאוד. ברשתות עמוקות תופעות של גרדיאנטים נעלמים ומתפוצצים עלולות להקשות על אימון. כאן נציג את הרעיון בלבד: אם גרדיאנט כמעט אפס, שינוי הפרמטר עשוי להיות זעיר; אם הוא עצום, צעד עלול להיות לא יציב.

את ההתמודדות המקצועית עם התופעות נבחן בפרקי הרשתות, לרבות בחירת פונקציות הפעלה, אתחול משקולות ואופטימייזרים מתקדמים.

11.24 הקשר המדויק בין פרק 11 לפרק 12

בפרק 11 למדנו כיצד מחשבים תחזית, שגיאה, נגזרת ושינוי משקולת. בפרק הבא נראה כיצד רשת שלמה מפעילה שכבות רבות: תחילה Forward Pass מחשב תוצאות, אחר כך Loss מודד שגיאה, ואז Backward Pass משתמש בכלל השרשרת כדי לחשב את הגרדיאנט לכל משקל.

יש להבדיל בין שלוש פעולות: Backpropagation מחשבת גרדיאנטים; אופטימייזר כגון SGD משתמש בהם לעדכון פרמטרים; והערכת ביצועים בודקת אם המודל מכליל לנתונים חדשים.

חשיבה במבנה הזה מונעת ערבוב בין פעולה מתמטית, כלי תוכנה ותוצאה לימודית.

שאלות עצירה ובדיקת הבנה

- אם הנגזרת של פונקציית שגיאה חיובית בנקודה מסוימת, מה המשמעות של הגדלה קטנה במשקל? האם ירידה בשגיאה מובטחת גם לצעד גדול?
- מדוע $f'(x)=0$ אינה הוכחה לכך שמצאנו מינימום?

- מה ההבדל בין פונקציית שגיאה לבין הגרדיאנט שלה?
- במודל $\hat{y}=wx+b$, מה מחזירה הנגזרת החלקית לפי w ומה מחזירה הנגזרת לפי b?
- כיצד יודעים מהו כיוון החישוב של הערכים ומהו כיוון החישוב של הגרדיאנטים בגרף חישוב?
- מה ההבדל בין Backpropagation לבין Gradient Descent?
- מדוע בדיקת ירידה בשגיאת אימון אינה מספיקה כדי להכריז על מודל מצליח?

תרגול מדורג — שלוש רמות

רמה א׳: מיומנויות יסוד

- הציבו $x=-2,\ x=0$ ו־$x=3$ בפונקציה $f(x)=3x-1$. פירשו משמעות של כל פלט.
- חשבו שיפוע ממוצע של $f(x)=x^{2}$ בין $x=2$ ל־$x=3$. השוו לנגזרת בנקודה $x=2$.
- גזרו $f(x)=4x-6x+5$ וחשבו $f'(1)$. הסבירו במילים את הסימן.
- נתון $L(w)=(w-4)^{2}$, $w=1$, $\eta=0.1$. חשבו נגזרת וצעד עדכון אחד.

רמה ב׳: חיבור מתמטיקה לאימון

- נתון $\hat{y}=wx+b$, $x=2,\ y=7,\ w=1,\ b=0$. חשבו תחזית, שגיאה וריבועה, ואז את שתי הנגזרות החלקיות.
- בדוגמה $L(w)=(2w-5)^{2}$ השוו צעד עדכון עם $\eta=0.05$ ו־$\eta=0.3$. איזו מסקנה אפשר להסיק מן התוצאה ואיזו לא?
- כתבו קוד המשווה בין נגזרת אנליטית לקירוב מספרי בשלוש נקודות שונות. הציגו טבלה ופרשנות.
- שרטטו גרף חישוב של $z=3w+1$ ושל $L=(z-10)^{2}$. ציינו מה ידוע בכל צומת ומהי הנגזרת המקומית.

רמה ג׳: חשיבה של מפתח AI

- במודל עם שני מאפיינים הגדירו MSE עבור שלוש דוגמאות. חשבו ידנית צעד שלם עבור שני משקולות והטיה.
- ניסוי מציג ירידה עקבית ב־Training Loss ועלייה ב־Validation Loss. הציעו שתי השערות ניתנות לבדיקה וניסוי מבוקר.
- מדוע לא נכון לפרש את פלט Sigmoid תמיד כהסתברות אמינה? הציעו דרך לבחון את האמינות.
- השוו חישוב גרדיאנט באמצעות הפרשים סופיים לבין גזירה אוטומטית: יתרון, מגבלה ושימוש מתאים לכל שיטה.

שאלת הערכה מקורית בסגנון בגרות

במערכת של דודו מתקבל מודל $\hat{y}=wx+b$. נתונים שני זוגות אימון: $(x=1,y=3)$, $(x=2,y=5)$. מגדירים MSE כממוצע ריבועי השאריות. בתחילת הניסוי $w=1$ ו־$b=0$, וקצב הלמידה הוא $\eta=0.05$.

סעיף	דרישה
א׳	חשבו את שתי התחזיות, השאריות וערך MSE ההתחלתי. (שמונה נקודות)
ב׳	גזרו את MSE לפי w ולפי b, והסבירו מה משמעות כל נגזרת. (שתים־עשרה נקודות)
ג׳	בצעו צעד Gradient Descent אחד וחשבו מחדש את שתי התחזיות. (עשר נקודות)
ד׳	חשבו את MSE החדש. נמקו אם הצעד הפחית את שגיאת האימון. (שש נקודות)
ה׳	הסבירו מדוע אין להסיק מכך שהמודל יצליח גם על מחשבים שלא נראו באימון. (ארבע נקודות)

זו שאלה מקורית לתרגול, ברוח ניתוח מתמטי ואלגוריתמי. אין מדובר בשאלת בגרות רשמית, והמחוון המוצע אינו מחוון משרד החינוך.

תכנית הוראה מוצעת — שלושים שעות

תוצר מרכזי	שעות	מפגש
תרשים פונקציה וחישוב שינוי ממוצע	3	1 — פונקציה ושיפוע
חישוב נגזרת ביד ובדיקה מספרית	3	2 — נגזרת וגבול
דף תרגול ונימוק סימן נגזרת	3	3 — כללי גזירה
טבלת עדכון משקולות	3	4 — Loss ו־Gradient Descent
גרף שגיאה עבור כמה קצבי למידה	3	5 — קצב למידה וניסויים
חישוב בשני פרמטרים	3	6 — נגזרות חלקיות וגרדיאנט
צעד אימון מלא מוקטָן	3	7 — MSE על כמה תצפיות
גרף חישוב והעברת נגזרות	3	8 — כלל השרשרת
בדיקת גרדיאנט וקשר ל־Sigmoid	3	9 — הפעלות ובדיקת נגזרות
מעבדה עצמאית ומבחן מסכם	3	10 — סיכום ויישום

מילון מונחים

מונח	הגדרה לתלמיד
נגזרת — Derivative	קצב השינוי המקומי של פונקציה
נגזרת חלקית — Partial derivative	קצב שינוי ביחס לפרמטר אחד כאשר אחרים קבועים
גרדיאנט — Gradient	וקטור הנגזרות החלקיות של פונקציה
פונקציית שגיאה — Loss	מספר המודד פער בין תחזית לערך מטרה לפי כלל מוגדר
קצב למידה — Learning rate	גודל צעד העדכון היחסי
Epoch	מעבר דרך נתוני האימון לפי שיטת האימון
כלל השרשרת — Chain rule	כלל לחישוב נגזרת של הרכבת פונקציות
Backpropagation	הליך יעיל לחישוב גרדיאנטים ברשת באמצעות כלל השרשרת
גזירה אוטומטית — Autodiff	מערכת המחשבת נגזרות של פעולות מתוך גרף חישוב

מקורות מקצועיים והבחנה בין חובה להרחבה

מקור מסגרת: מאגר Data-Science-ML-Course של מסלול ההתמחות בלמידת מכונה ולמידה עמוקה, פרק י״א — מתמטיקה ללמידה עמוקה, 30 שעות. שייכות הפרק וההיקף אומתו לפי קובץ מפת המסלול; סעיפי החובה המפורטים דורשים הצלבה נוספת עם מסמך המקור המלא של משרד החינוך.

מקורות השראה מתמטיים: Deisenroth, Faisal & Ong, Mathematics for Machine Learning (Cambridge University Press, 2020), בעיקר חשבון וקטורי ואופטימיזציה; Zhang, Lipton, Li & Smola, Dive into Deep Learning, פרקי Calculus ו־Automatic Differentiation; וכן Understanding Deep Learning של Simon J. D. Prince להעמקה של קשר הנגזרות לרשתות.

גישת הוראה מעשית: Hands-On Machine Learning של Aurélien Géron; Grokking Machine Learning של Luis G. Serrano. ההסברים, החישובים, השאלות והגרפים בפרק זה נכתבו ונוצרו באופן מקורי; לא הועתקו איורים או קטעי ספרים.

קישורים לקריאה נוספת: https://mml-book.github.io/ | https://d2l.ai/chapter_preliminaries/calculus.html | https://d2l.ai/chapter_preliminaries/autograd.html

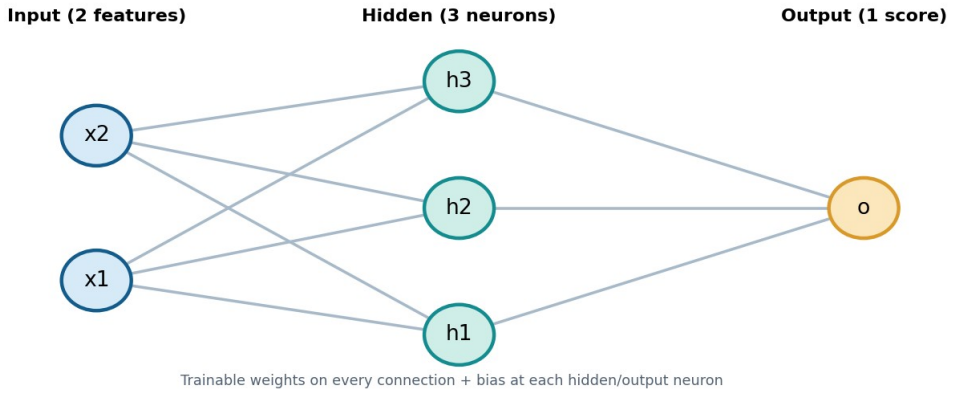
סימון פדגוגי: נגזרות וגרדיאנטים הם בסיס הפרק; Gradient Checking, בחינת יציבות מתמטית ופירוש מפורט של Sigmoid מופיעים כאן כהרחבה מקצועית וכהכנה לפרקים מתקדמים. מעמד החובה המדויק נקבע רק לפי תכנית הלימודים העדכנית.

התמחות למידת מכונה ולמידה עמוקה — 5 יחידות לימוד

פרק 12 | רשתות נוירונים עמוקות

מרשת של נוירונים עד לאימון מודל שמזהה דפוסים מורכבים

מהדורת תלמיד • ספר מאוחד יחידה 5 + התמחות • היקף פרק התמחות במפת התכנית: 20 שעות לימוד • כל הדוגמאות והאיורים מקוריים.



איור: מבנה רשת קדמית: שכבת קלט, שכבה חבויה ושכבת פלט.

כיצד להשתמש בפרק

הפרק בנוי כך שכל פעולה מתמטית מקבלת תחילה משמעות מילולית והמחשה. אחר כך נחשב ביד, נבדוק את התוצאה באמצעות NumPy ולבסוף נאמן רשת באמצעות ספריית למידת מכונה. ליד הנוסחאות נפרש כל סימן, ולא נצפה שתלמיד יבין רשת עצבית מתוך קוד בלבד.

הידע הקודם הדרוש: וקטורים ומטריצות, מכפלה פנימית, תקנון נתונים, רגרסיה ליניארית, הפרספטרון, נגזרות, כלל השרשרת ו־Gradient Descent. הפניות לפרקים קודמים מחליפות הוראה כפולה.

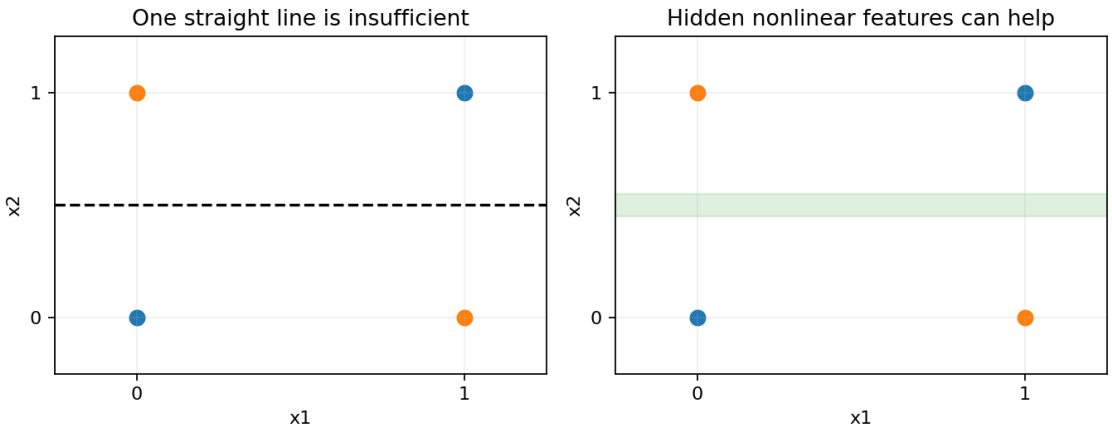
בסיום הפרק התלמיד יוכל...	ראיה לשליטה
להסביר מדוע רשת עם שכבות חבויות שונה ממסווג ליניארי	להמחיש אי־ליניאריות בעזרת XOR
לחשב מעבר קדימה של רשת קטנה	טבלת z, h ופלט עם משקולות נתונות
לחשב נגזרות בצעד Backpropagation	להציג נגזרות למשקולות ולהטיות
לאמן ולבדוק רשת על נתונים חדשים	מחברת עם חלוקה נכונה לנתוני אימון ובדיקה
לפרש גרף שגיאה ולהצביע על מגבלות	פסקת מסקנה מקצועית ומבוססת תוצאות

1. סיפור הפתיחה — מדוע פרספטרון יחיד אינו מספיק?

דודו בונה מערכת המזהה על סמך שני מאפיינים אם קיימת תופעה מסוימת במעבדה. בחלק מהמקרים אפשר להפריד את הדוגמאות בקו ישר: נתונים בצד אחד שייכים למחלקה א׳, ובצד השני למחלקה ב׳. אבל קיימים דפוסים שבהם אותו קו לא יצליח להפריד נכון את כל הנתונים. אם מציירים את ארבע

הדוגמאות של XOR, שתי הנקודות החיוביות נמצאות בפינות נגדיות. אין קו ישר יחיד שיפריד אותן משתי השליליות.

הבעיה איננה שהפרספטרון "התאמן מעט מדי". המגבלה נמצאת בצורה המתמטית של המודל: חישוב סכום משוקלל ולאחריו החלטה באמצעות סף. כדי לתאר גבולות מורכבים יותר נרצה ליצור ייצוגי ביניים חדשים, המתאפשרים באמצעות שכבות חבויות ופונקציות הפעלה לא־ליניאריות.



איור: דוגמת *XOR*: המבנה הגיאומטרי אינו ניתן להפרדה בקו ישר יחיד.

הבהרה מדעית: רשת עם כמה שכבות ליניאריות בלבד עדיין שקולה להעתקה ליניארית או אפינית אחת. אי־הליניאריות של פונקציית ההפעלה היא רכיב חיוני ביכולת לייצג דפוסים לא־ליניאריים.

2. מהי רשת נוירונים רב־שכבתית?

רשת נוירונים היא פונקציה שמורכבת מחישובים רבים המחוברים זה לזה. כל נוירון בשכבה מקבל מספרים מהשכבה הקודמת, מחשב סכום משוקלל ומוסיף הטיה. לאחר מכן הוא מפעיל פונקציית הפעלה. הפלטים של כל הנוירונים בשכבה נעשים לקלט של השכבה הבאה. כך נוצר רצף של ייצוגים מספריים של אותה תצפית.

השכבה הראשונה מקבלת את המאפיינים המקוריים. שכבה חבויה אינה מקבלת ישירות את התווית הנכונה, אלא יוצרת מאפייני ביניים הנלמדים במהלך האימון. שכבת הפלט מחזירה מספר לחיזוי ברגרסיה, או ציונים/הסתברויות בסיווג — בהתאם למבנה הרשת ולפונקציית הפלט.

מבחינת המונח "עמוקה": מקובל לתאר רשת כבעלת מספר שכבות חישוב נלמדות. רשת בעלת שכבה חבויה אחת היא דוגמת ההוראה הפשוטה ביותר לרשת רב־שכבתית; במהלך ההמשך נלמדות גם רשתות שבהן כמה שכבות חבויות.

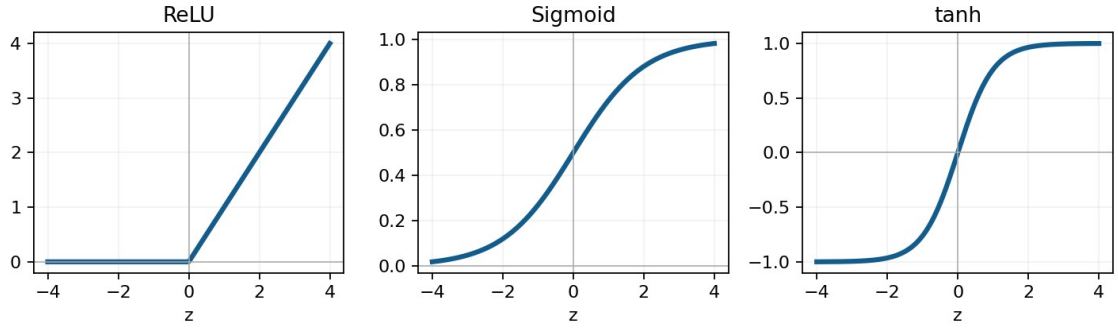
מונח	מה פירושו?	ממה חשוב להיזהר?
נוירון (Neuron)	יחידת חישוב המקבלת קלטים ומחזירה ערך	אין זה תא עצב ביולוגי
משקל (Weight)	מקדם נלמד המגדיל, מקטין או הופך השפעת מאפיין	המשקל אינו בהכרח חשיבות סיבתית
הטיה (Bias)	מספר נוסף המחושב בכל נוירון	אין לבלבל עם הטיה חברתית בנתונים
שכבה חבויה	ייצוג ביניים שנלמד מן הדוגמאות	אין הכרח שאפשר לתת לכל

מונח	מה פירושו?	ממה חשוב להיזהר?
		נוירון שם אנושי
Epoch	מעבר מלא על נתוני האימון	אינו שקול תמיד לצעד עדכון אחד
Batch	קבוצת דוגמאות לעדכון משקולות	גודל ה־Batch משפיע על חישוב ואימון

3. מדוע נדרשת פונקציית הפעלה?

נניח שהשכבה הראשונה מחשבת $z_1 = W_1x + b_1$, והשכבה השנייה מחשבת $z_2 = W_2z_1 + b_2$, בלי שום פעולה לא־ליניארית באמצע. אם מציבים את הביטוי הראשון בשני, מתקבלת עדיין נוסחה מהצורה $Wx + b$. למעשה אפשר לאחד את כל המשקולות למטריצה אחת ואת ההטיות לווקטור אחד. לכן הוספת שכבות ללא פונקציית הפעלה אינה משחררת אותנו מהמגבלה הליניארית.

פונקציית ההפעלה שוברת את הפשטות הזאת. ערכים מסוימים משתנים באופן שונה מאחרים, והרשת מסוגלת ליצור צורות החלטה מורכבות יותר. נדגים שלוש פונקציות נפוצות ונבחין היכן משתמשים בכל אחת.



איור: שלוש פונקציות הפעלה: *ReLU, Sigmoid* ו־*tanh*.

```
ReLU(z) = max(0, z)
```

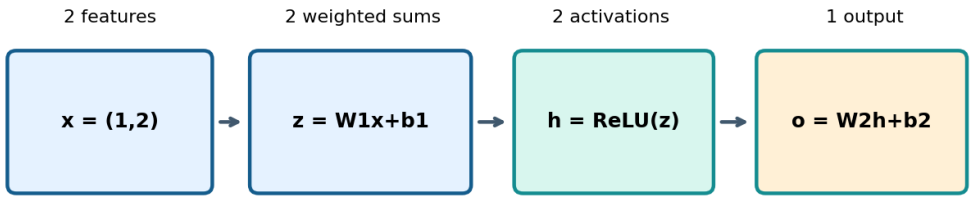
```
sigmoid(z) = 1 / (1 + exp(-z))
```

ReLU מחזירה אפס לכל קלט שלילי ומעבירה קלט חיובי כמו שהוא. Sigmoid ממפה מספר ממשי לערך בין אפס לאחד; כאשר משתמשים בה כפלט של מסווג בינארי ומתאימים פונקציית אובדן מתאימה, התוצאה יכולה להתפרש כהערכת הסתברות. tanh מפיקה ערכים בין מינוס אחד לאחד. הבחירה בפונקציית הפעלה קשורה לתפקיד השכבה וליציבות האימון, לא רק למראה הגרף.

ל־ReLU נגזרת אפס בחלק השלילי ונגזרת אחת בחלק החיובי; בנקודת האפס יש אי־גזירות במובן הקלאסי, וכלי תוכנה בוחרים שם כלל מעשי מתאים. ייתכן מצב שבו נוירונים נשארים כבויים זמן ממושך, ולכן קיימות גם גרסאות אחרות כגון Leaky ReLU — נושא העמקה.

4. כיצד מחשבים מעבר קדימה (Forward Pass)?

המעבר הקדמי הוא סדר החישובים שמפיק תחזית ממשקולות שכבר נקבעו. הוא איננו אימון: במהלך מעבר קדימה רגיל עדיין לא שינינו את המשקולות. נתחיל בדוגמה אחת עם שני מאפיינים, שני נוירונים חבויים ונוירון פלט יחיד. בדוגמה זו הפלט רציף, ולכן נשתמש בו להמחשת רגרסיה.



Forward pass: values flow from left to right

איור: סדר החישוב: קלט → סכום משוקלל → $ReLU$ → פלט.

```
z1 = W1 @ x + b1        h = ReLU(z1)
o = W2 @ h + b2
```

בכתיב זה x הוא וקטור קלט, W1 מטריצת משקולות לשכבה החבויה, b1 וקטור הטיות, z1 הסכומים לפני ההפעלה, h הפלטים אחרי ההפעלה, ו־o תחזית הרשת. סימן @ מציין מכפלה מטריציונית או מכפלה פנימית בהתאם לממדים.

5. דוגמה ידנית מלאה — שני נוירונים ושכבת פלט

דודו מודד שני מאפיינים של מחשב: $x_1=1$ ו־$x_2=2$ (ערכים שכבר הותאמו ליחידות המשמשות בניסוי). נבחר משקולות התחלתיות כדי ללמוד את הפעולה; במודל אמיתי הן נלמדות בזמן האימון ולא נקבעות ידנית לפי התוצאה הרצויה.

```
x = (1, 2)
W1 = [[0.2, 0.4], [-0.3, 0.5]]
b1 = (0.1, -0.2)
W2 = (0.6, -0.4)      b2 = 0.2
```

נוירון חבוי ראשון: מכפילים כל מאפיין במשקל המתאים, מחברים ומוסיפים הטיה. לכן $z_{11}=0.2\times1+0.4\times2+0.1=1.1$. מאחר שהתוצאה חיובית, ReLU מחזירה 1.1.

נוירון חבוי שני: $z_{12}=-0.3\times1+0.5\times2-0.2=0.5$. גם כאן התוצאה חיובית ולכן $h_2=0.5$.

שלב	פעולה	תוצאה
נוירון חבוי 1	0.1 + 2×0.4 + 1×0.2	1.1
נוירון חבוי 2	0.2 − 2×0.5 + 1×0.3−	0.5

שלב	פעולה	תוצאה
ReLU	$\max(0, 1.1), \max(0, 0.5)$	$h = (1.1, 0.5)$
שכבת פלט	$0.2 + 0.5 \times 0.4 - 1.1 \times 0.6$	$o = 0.66$

אם הערך האמיתי הוא $y=1$, פער החיזוי הוא $1-0.66=0.34$. הסימן השלילי אומר שהמודל חזה ערך נמוך מן הערך האמיתי; אין פירושו שמדובר בהסתברות שלילית, שכן בדוגמת הרגרסיה שלנו הפלט אינו מוגבל לטווח 0–1.

6. איך מודדים שגיאה? Loss לעומת מדד הערכה

לפני שנשנה משקולות, צריך להגדיר מה נחשב לטעות. ברגרסיה מקובל להשתמש, בין היתר, בשגיאה ריבועית. בחישוב ידני של תצפית אחת נוח לבחור במחצית ריבוע השגיאה; גורם המחצית מקל על הנגזרת ואינו משנה את נקודת המינימום.

```
L = 0.5 * (o - y)^2
```

```
L = 0.5 * (0.66 - 1)^2 = 0.0578
```

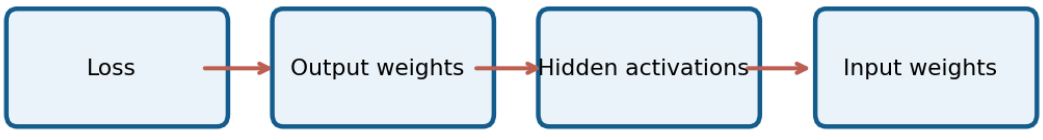
ה־Loss הוא מספר המשמש את תהליך האופטימיזציה. מדדי דיווח כגון MAE, Accuracy ו־F1 אינם בהכרח אותה פונקציה שעליה מתבצע עדכון המשקולות. בסיווג בינארי עם פלט Sigmoid נפוץ להשתמש ב־Binary Cross-Entropy, ולא רק בריבוע הפרש פשוט.

```
BCE = -[ y*ln(p) + (1-y)*ln(1-p) ]
```

בנוסחה זו p היא ההסתברות החזויה למחלקה החיובית, ו־y היא התווית האמיתית שערכה אפס או אחד. כאשר המודל בטוח בתחזית שגויה, פונקציית האובדן מטילה עליו קנס גדול. אין לחשב לוגריתם של אפס ללא טיפול מספרי מתאים; ספריות מקצועיות מבצעות חישובים יציבים יותר.

7. העברת נגזרות לאחור — Backpropagation

עד עכשיו ידענו לחשב תחזית, אבל עדיין לא ידענו אילו משקולות לשנות. Backpropagation הוא שיטה יעילה לחישוב הנגזרת של פונקציית האובדן ביחס לכל פרמטר ברשת. הוא מתקדם לאחור דרך פעולות הרשת תוך שימוש בכלל השרשרת, ומנצל תוצאות שנשמרו במעבר הקדמי. לאחר שחושבו הנגזרות, האלגוריתם המעדכן את המשקולות — למשל Gradient Descent — מבצע את השינוי בפועל.



Backward pass: chain-rule gradients propagate towards earlier layers

איור: כיוון חישוב הנגזרות: מן השגיאה אל שכבות הקלט.

בפרק 11 למדנו את כלל השרשרת: הנגזרת של פונקציה מורכבת היא מכפלת הנגזרות המקומיות לאורך המסלול. עכשיו נראה כיצד הכלל הופך לתהליך עבודה מסודר בנוירונים אחדים.

dL/do = o - y = -0.34

ראשית מחשבים את הנגזרת ביחס לפלט. במקרה שלנו היא שלילית, ולכן הגדלת הפלט מעט צפויה להקטין את השגיאה בסביבת המצב הנוכחי. נגזרת זו מועברת אל המשקולות בשכבת הפלט דרך הקשר בין הפלט לערכי השכבה החבויה.

dL/dW2 = (o-y) * h = (-0.374, -0.170)

dL/db2 = o-y = -0.34

המשקל הראשון בשכבת הפלט קיבל נגזרת −0.374 כי ערך הנוירון המתאים הוא 1.1. המשקל השני קיבל −0.170 כי ערך הנוירון השני 0.5. זוהי דוגמה לכך שהנוסחה נגזרת מהתלות של כל משקל בחישוב, ולא מכלל שרירותי של שינוי מספרים.

delta_hidden = (o-y) * W2 * ReLU'(z1)

delta_hidden = (-0.204, +0.136)

שני הערכים z_1 חיוביים, ולכן נגזרת ReLU היא אחת בכל אחד מהם. המשקל השני בשכבת הפלט שלילי, ולכן סימן הנגזרת המוחזרת לנוירון החבוי השני מתהפך. הדבר חשוב: לא כל משקל צריך להשתנות באותו כיוון.

dL/dW1 = outer(delta_hidden, x)

dL/dW1 = [[-0.204, -0.408], [0.136, 0.272]]

dL/db1 = (-0.204, 0.136)

סיימנו לחשב את הגרדיאנטים. יש לנו נגזרת לכל משקל ולכל הטיה, ומכאן ניתן לבצע עדכון. המילה outer מציינת כאן מכפלה חיצונית בין וקטור נגזרות הנוירונים לבין וקטור המאפיינים.

8. עדכון אחד מלא של המשקולות

נבחר קצב למידה $\eta = 0.1$. בכל פרמטר מחסרים את קצב הלמידה כפול הנגזרת שלו. אם הנגזרת שלילית — הערך יגדל; אם הנגזרת חיובית — הערך יקטן. העדכון אינו מבטיח ירידה בשגיאה בכל צעד ובכל קצב למידה, אך בדוגמה הקטנה שלנו נוכל לחשב ולבדוק את השפעתו.

parameter_new = parameter_old - 0.1 * gradient

פרמטר	לפני העדכון	נגזרת	לאחר העדכון
W2[0]	0.6000	-0.3740	0.6374
W2[1]	-0.4000	-0.1700	-0.3830
b2	0.2000	-0.3400	0.2340
W1[0,0]	0.2000	-0.2040	0.2204
W1[0,1]	0.4000	-0.4080	0.4408
W1[1,0]	-0.3000	+0.1360	-0.3136
W1[1,1]	0.5000	+0.2720	0.4728

פרמטר	לפני העדכון	נגזרת	לאחר העדכון
$b1[0]$	0.1000	-0.2040	0.1204
$b1[1]$	-0.2000	+0.1360	-0.2136

כאשר מחשבים מעבר קדימה נוסף אחרי העדכון, מקבלים תחזית בקירוב 0.8529, ושגיאה חדשה 0.01082 במקום 0.0578. זאת הדגמה של צעד למידה אחד על תצפית אחת בלבד. עדיין לא נוכל להסיק ממנה שהמודל מצליח על נתונים שלא ראה.

9. מעבר מחישוב ידני לקוד NumPy

נכתוב קוד הניתן להרצה רצופה הממחיש את אותם חישובים. הקוד מציג את הנגזרות ולא מסתיר אותן בפעולת fit. בהמשך, ספריית למידה עמוקה תחשב אותן אוטומטית, אבל ההבנה המתמטית שכאן מאפשרת להסביר מה הכלים עושים.

```
import numpy as np
x = np.array([1.0, 2.0])
y = 1.0
W1 = np.array([[0.2, 0.4], [-0.3, 0.5]])
b1 = np.array([0.1, -0.2])
W2 = np.array([0.6, -0.4])
b2 = 0.2
eta = 0.1

z1 = W1 @ x + b1
h = np.maximum(0, z1)
o = W2 @ h + b2
loss_before = 0.5 * (o - y)**2

delta_o = o - y
gW2 = delta_o * h
gb2 = delta_o
delta_h = delta_o * W2 * (z1 > 0)
gW1 = np.outer(delta_h, x)
gb1 = delta_h

W2 -= eta * gW2
b2 -= eta * gb2
W1 -= eta * gW1
b1 -= eta * gb1

h_after = np.maximum(0, W1 @ x + b1)
o_after = W2 @ h_after + b2
loss_after = 0.5 * (o_after - y)**2
print(round(loss_before, 5), round(loss_after, 5))
```

שימו לב לשלושה רעיונות בקוד: הווקטור z1 נוצר לפני ההפעלה; הביטוי `(z1 > 0)` מחזיר מסכה המתאימה לנגזרת ReLU במקרה הנלמד; ולבסוף מעדכנים את כל המשקולות רק אחרי שחושבו הנגזרות מתוך אותו מצב התחלתי. שינוי משקל מוקדם מדי במהלך החישוב עלול לערבב מצבים שונים ולשבש את התוצאה.

10. חישוב ממדים — כמה פרמטרים יש לרשת?

הצורה של מטריצת משקולות נקבעת לפי מספר הנוירונים בשכבות. אם בשכבה קודמת 3 נוירונים ובשכבה הבאה 4 נוירונים, דרושים 12 משקולות לחיבורים ועוד 4 הטיות. לכן מספר הפרמטרים של מעבר בין שכבות הוא מכפלת גדלי השכבות בתוספת מספר הנוירונים בשכבה הבאה.

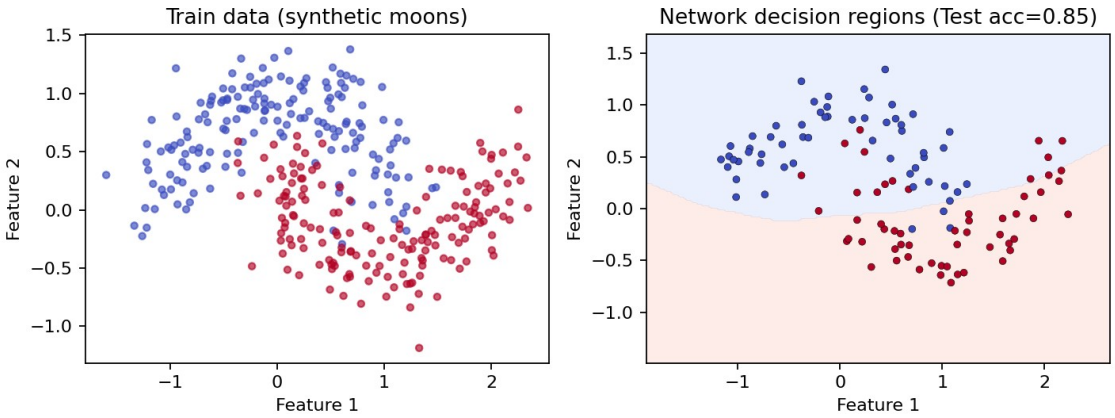
Parameters = n_in * n_out + n_out

לרשת בעלת 4 מאפייני קלט, 5 נוירונים חבויים, 3 נוירונים חבויים בשכבה השנייה ו־2 נוירוני פלט: בין קלט לשכבה הראשונה יש 25 פרמטרים; בין השכבות החבויות 18; ובין השנייה לפלט 8. סך הכול 51 פרמטרים. חשבו זאת ביד לפני הפעלת תוכנה.

יותר פרמטרים מאפשרים לייצג פונקציות מורכבות יותר, אך אינם מבטיחים ביצועים טובים יותר. רשת גדולה דורשת יותר נתונים, חישוב ואמצעי בקרה למניעת התאמת יתר.

11. מרשת קטנה למימוש מקצועי ב־scikit-learn

כדי לראות רשת לומדת על נתונים מורכבים מעט יותר, נשתמש במאגר סינתטי שנראה כשתי קשתות. זהו מאגר לימודי, ולא נתונים אמיתיים על בני אדם או על תהליך עסקי. נפריד את הנתונים לאימון ולבדיקה, ונבנה Pipeline המתקנן את המאפיינים באמצעות נתוני האימון בלבד.



איור: דוגמת שתי קשתות: רשת עם שכבות חבויות יכולה ללמוד אזורי החלטה שאינם קו ישר.

```
from sklearn.datasets import make_moons
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score

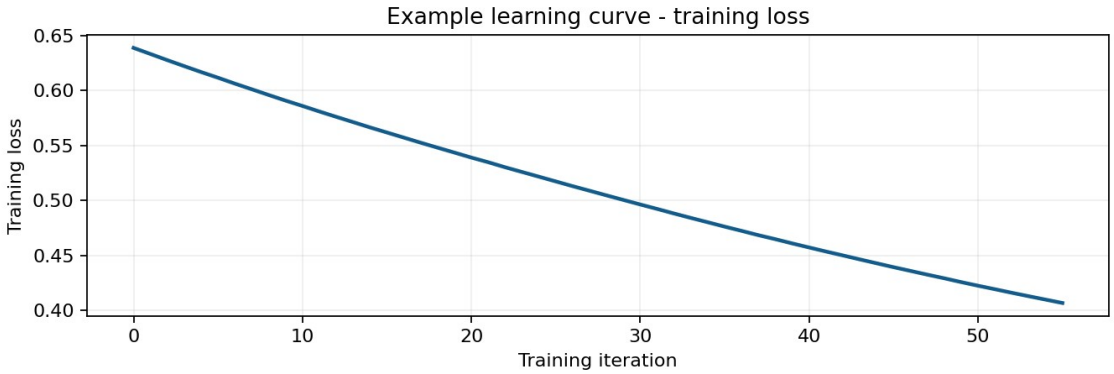
X, y = make_moons(n_samples=450, noise=0.22, random_state=8)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, stratify=y, random_state=15
)
model = make_pipeline(
    StandardScaler(),
    MLPClassifier(hidden_layer_sizes=(8, 8),
                  max_iter=600, early_stopping=True,
                  n_iter_no_change=25, random_state=12)
)
model.fit(X_train, y_train)
```

```
y_pred = model.predict(X_test)
print("Test accuracy:", accuracy_score(y_test, y_pred))
```

הארכיטקטורה `(8, 8)` פירושה שתי שכבות חבויות, שבכל אחת שמונה נוירונים. המשתנה `max_iter` מגביל את מספר איטרציות האימון לפי המימוש של הספרייה. `early_stopping=True` מאפשר עצירה מוקדמת על בסיס ולידציה פנימית. הדוגמה מיועדת להבנת הצינור המקצועי; היא אינה מחליפה בחירה מושכלת של ארכיטקטורה והערכת תוצאות על בעיה אמיתית.

12. כיצד קוראים עקומת למידה?

במהלך האימון המודל מציג לעיתים את שינוי פונקציית האובדן לאורך איטרציות. אם השגיאה באימון יורדת, סימן שהתהליך מצליח להתאים טוב יותר את הדוגמאות ששימשו לאימון. אך אין בכך הוכחה ליכולת הכללה. נדרשת גם בדיקה של ביצועים על נתוני ולידציה ובסוף על קבוצת Test שלא שימשה לבחירות.

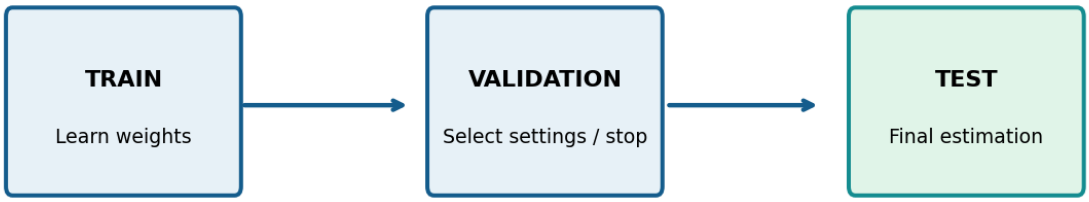


איור: עקומת שגיאת אימון מתוך הניסוי הסינתטי; לא להסיק ממנה לבדה על הכללה.

למידת חסר מתאפיינת לעיתים בשגיאה גבוהה גם באימון וגם בוולידציה, בעוד התאמת יתר יכולה להתבטא בפער גדל ביניהן. יש להיזהר מלהפוך סימנים אלה לכללים מוחלטים: אופי הנתונים, מדד השגיאה ומידת הרעש משפיעים על צורת העקומות.

13. Train, Validation ו־Test — מדוע לא משתמשים באותם נתונים לכל דבר?

אימון משמש לעדכון משקולות. ולידציה משמשת לבחירת ארכיטקטורה, קצב למידה, מספר Epochs או נקודת עצירה. Test מיועד להערכה סופית אחרי שהתקבלו ההחלטות. אם בוחרים את מבנה הרשת שוב ושוב לפי תוצאות Test, המבחן אינו בלתי תלוי עוד, והתוצאה המדווחת עלולה להיות אופטימית.



איור: שלוש מטרות שונות לשימוש בנתונים. *Pipeline* מפחית סיכון לדליפת מידע.

דוגמה לזליגה: מחשבים ממוצע וסטיית תקן על כלל המאגר לפני חלוקתו, ואז משתמשים בהם בתקנון נתוני האימון. כך מידע על נתוני הבדיקה השפיע בעקיפין על האימון. דרך נכונה היא להתאים את התקנון לנתוני האימון בלבד. בניסויי Cross-Validation יש להתאים את התקנון מחדש בתוך כל קיפול אימון.

14. אופטימיזציה, Batch ו־Epoch

כאשר מאגר הנתונים גדול, חישוב נגזרות על כל התצפיות בכל צעד עלול להיות יקר. לכן נהוג לעבד קבוצות קטנות (mini-batches). בכל Batch מחשבים תחזיות, אובדן, גרדיאנטים ועדכון. Epoch הוא מעבר מלא על דוגמאות האימון; אם גודל האימון 1,000 וגודל Batch הוא 100, יהיו בדרך כלל עשרה צעדי Batch בכל Epoch, אם לא משנים את דוגמת החלוקה.

קצב הלמידה קובע את אורך העדכון. אם הוא קטן מאוד, השיפור עשוי להיות איטי; אם הוא גדול מדי, המודל עלול להחמיץ אזורי שגיאה נמוכה או להפוך את האימון לבלתי יציב. שיטות כגון SGD ו־Adam משנות את אופן ביצוע העדכונים, אך אינן מחליפות את העיקרון של חישוב גרדיאנטים.

15. מה הקשר בין Backpropagation ל־Automatic Differentiation?

ברשתות מודרניות אין גוזרים ביד מאות אלפי משקולות. מסגרות כגון PyTorch בונות גרף חישובי ומשתמשות בגזירה אוטומטית לחישוב גרדיאנטים. הפעולה `loss.backward()` מחשבת נגזרות; היא אינה מעדכנת לבדה את המשקולות. לאחריה פועלה כגון `optimizer.step()` מבצעת עדכון. לפני צעד האימון הבא יש לאפס גרדיאנטים צבורים, למשל באמצעות `optimizer.zero_grad()`.

> הרחבה מקצועית: דוגמת *PyTorch* היא גשר להמשך ולא דרישה שכל תלמיד יכתוב את תשתית *Autograd* בעצמו. השימושים בקוד מקצועי מחייבים הסבר של מבנה המודל והחלטות האימון.

```
# A short professional pattern (PyTorch installed separately)
# optimizer.zero_grad()
# prediction = model(features)
# loss = loss_fn(prediction, target)
# loss.backward()       # compute gradients
# optimizer.step()      # update parameters
```

16. תקלות נפוצות וכיצד מאבחנים אותן

תצפית	מה ייתכן שקרה?	כיצד נבדוק?
שגיאת אימון אינה יורדת	קצב למידה, תקנון, נתונים או באג בקוד	בדיקת טווחים, גרדיאנטים וקו בסיס
אימון מצוין, Test חלש	התאמת יתר או זליגת מידע בבחירת מודל	השוואת עקומות ואופן חלוקת נתונים
פלט כמעט קבוע	ארכיטקטורה לא מתאימה או רשת שאינה לומדת	בדיקת משקולות, קלטים ופלטים
שגיאה מספרית NaN	ערכים קיצוניים, קצב למידה או לוגריתם אפס	ניטור חישובים ותקנון
מדדים משתנים בין הרצות	אקראיות באתחול ובחלוקה	זרעים קבועים וניסוי חוזר
פלט בינארי אינו 0/1	המודל מחזיר ציון או הסתברות לפני סף	הגדרת כלל החלטה מפורש

17. מדוע גם רשת מצליחה יכולה להיות בעייתית?

רשת עצבית היא כלי חיזוי סטטיסטי, ולא מכשיר הקובע אמת מוחלטת. אם הנתונים אינם מייצגים אוכלוסיות מסוימות, אם נאספו באופן לא מוסכם, או אם התוויות אינן אמינות — איכות גבוהה במדד מספרי יחיד אינה מספיקה כדי להצדיק שימוש. בפרויקט נדרש לתאר את מקור הנתונים, את מגבלותיהם, את השגיאות ואת הסיכון הנובע מן השימוש במודל.

בתחומים רגישים, כגון בריאות, מיון תלמידים או קבלת החלטות בעלות השלכות משמעותיות, יש צורך בבקרה מקצועית ובשמירה על פרטיות. את פרק 12 ניישם בבעיות לימודיות שניתן לבקר ולהסביר, ולא נציג תחזית אוטומטית כאבחנה או כפסיקה.

18. תרגול מדורג — מהבנה לעומק

רמה א — מושגים וחישוב: (א) הסבירו במילים שלכם את ההבדל בין שכבת קלט לשכבה חבויה. (ב) חשבו עבור ReLU $2, 0$− ו־3. (ג) ברשת בעלת 2 מאפיינים, 3 נוירונים חבויים ו־1 פלט — כמה משקולות וכמה הטיות יש? הציגו שלבי חישוב.

רמה ב — העברה ונימוק: (ד) נתונה רשת שבה כל השכבות ליניאריות. הסבירו למה הוספת שכבה לא בהכרח תפתור XOR. (ה) במודל שבדוגמה הידנית, מה יקרה לערך הנוירון השני אם נשנה את אחת המשקולות? נמקו בחישוב. (ו) מדוע אי־אפשר לפרש תחזית 0.66 בהכרח כהסתברות?

רמה ג — פיתוח וביקורת: (ז) גזרו מחדש נגזרת של משקל אחד בשכבה החבויה באמצעות כלל השרשרת. (ח) הציעו ניסוי שמשווה רשת בעלת שכבה חבויה אחת לרשת בעלת שתי שכבות, באותה חלוקת נתונים. (ט) תארו שלושה צעדים למניעת זליגת מידע בפרויקט אמיתי.

19. שאלת הערכה מקורית בסגנון בגרות / הגנה

מערכת AI מקבלת שני מאפיינים של מוצר ומפיקה תחזית מספרית. המודל כולל שכבה חבויה אחת בת שני נוירונים עם ReLU ושכבת פלט ליניארית. נתונים המשקולות, ההטיות והקלט מהדוגמה בסעיף 5. התלמיד מתבקש: (א) לחשב את פלט השכבה החבויה; (ב) לחשב תחזית ושגיאה; (ג) להסביר איזה שלב

נדרש כדי לעדכן משקולות; (ד) להציע בדיקה אחת שתבחן יכולת הכללה; (ה) לנמק מדוע הרשת עשויה להיות מתאימה יותר ממודל ליניארי פשוט עבור תופעה לא־ליניארית.

הערה: זוהי שאלה מקורית בדפוס הערכה עיוני והגנה בעל־פה, ואינה שאלה רשמית מתוך בחינת משרד החינוך.

20. רצף הוראה מוצע — 20 שעות

מפגש	שעות הוראה	מוקד	ראיה ללמידה
1	2	בעיה אמיתית, פרספטרון לעומת MLP, XOR	המחשת מגבלת ליניאריות
2	2	שכבות, משקולות, הטיות וממדים	שירטוט והערכת פרמטרים
3	2	פונקציות הפעלה ו־ReLU	חישוב ערכי הפעלה
4	2	Forward Pass ידני	טבלת עקיבה
5	2	פונקציות אובדן ורגרסיה/סיווג	חישוב Loss ופירוש
6	2	Backpropagation וכלל השרשרת	טבלת גרדיאנטים
7	2	עדכון משקולות ו־NumPy	מימוש חישוב ידני
8	2	אימון רשת וספריית sklearn	ניסוי עם שני ארכיטקטורות
9	2	Evaluation, Validation, Leakage	דוח ניסוי מנומק
10	2	תרגול עצמאי, שאלת הערכה והכנה לפרויקט	מחברת, מסקנה והסבר בעל־פה

חלוקת עשר יחידות לשתי שעות היא הצעה דידקטית מתוך היקף 20 השעות של פרק ההתמחות, ולא חלוקה רשמית של משרד החינוך.

21. מילון מושגים קצר

מושג	פירוש
MLP	רשת נוירונים קדמית בעלת כמה שכבות
Forward Pass	מעבר חישוב מהקלט אל הפלט
Backpropagation	חישוב גרדיאנטים לאחור באמצעות כלל השרשרת
Activation	פונקציה שמשנה את פלט הנוירון
Loss	פונקציית שגיאה לאימון
Gradient	וקטור הנגזרות של פונקציית האובדן
Optimizer	כלל עדכון פרמטרים

מושג	פירוש
Epoch	מעבר מלא על נתוני אימון
Batch	קבוצת תצפיות בתוך האימון
Early Stopping	עצירה לפי התקדמות מדד ולידציה

22. הקשר בין שלוש תכניות הלימוד

תכנית	מה כבר נלמד?	מה מוסיף פרק זה?
יחידה 3	Python, וקטורים בסיסיים, סיווג והערכה	חיבור לרשת כאמצעי ייצוג חדש — גשר בלבד
יחידה 5 נושאי ליבה	וקטורים, מרחקים, סטטיסטיקה ואימות מודלים	ניצול המתמטיקה שכבר נלמדה, ללא ייחוס רשתות כחובת יחידה 5
התמחות DL	רגרסיה, פרספטרון, נגזרות ו־Gradient Descent	מעבר קדימה, Backpropagation ורשת מאומנת

23. ספרות מקצועית ומקורות

עוגן תכנית: משרד החינוך, התמחות למידת מכונה ולמידה עמוקה, פרק 12 — רשת נוירונים עמוקה (20 שעות), על פי מפת התכנית השמורה במאגר GitHub. פירוט הדוגמאות, תרגילי ההעשרה והחלוקה למפגשים הם פיתוח מקורי; האימות הסופי של תתי־סעיפי חובה מחייב בדיקת מסמך התכנית הרשמי המעודכן.

Zhang, A., Lipton, Z., Li, M. & Smola, A. — Dive into Deep Learning, פרק Multilayer Perceptrons ובפרט Forward / Backpropagation: https://d2l.ai/chapter_multilayer-perceptrons/backprop.html

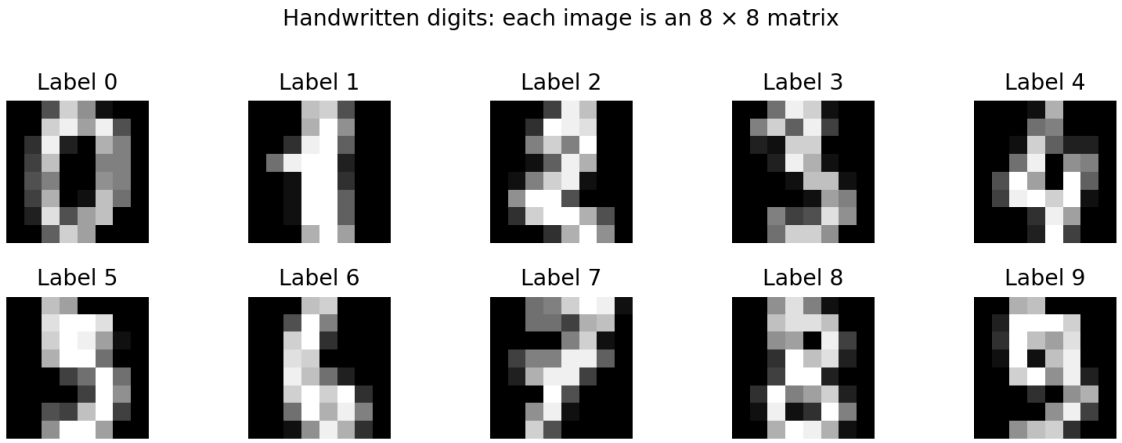
Prince, S. J. D. — Understanding Deep Learning, MIT Press, 2023. Deisenroth, Faisal & Ong — Mathematics for Machine Learning, Cambridge University Press, 2020. Géron — Hands-On Machine Learning. המקורות משמשים לעקרונות, לרמת ההסבר ולביקורת, ולא הועתקו מהם איורים או קטעי טקסט.

scikit-learn — User Guide: Neural Network Models (Supervised): https://scikit-learn.org/stable/modules/neural_networks_supervised.html. PyTorch — Neural Networks Tutorial: https://docs.pytorch.org/tutorials/beginner/blitz/neural_networks_tutorial

פרק 13 — רשתות קונבולוציה (CNN)

מפיקסלים ומסננים לרשת עצבית שמזהה דפוסים בתמונה

ספר הקורס המאוחד | מסלול התמחות למידת מכונה ולמידה עמוקה, 5 יחידות | היקף פרק בתכנית: 50 שעות | מהדורת תלמיד



איור: דוגמאות מתמונות ספרות. כל תמונה היא מערך של 64 עוצמות אפור.

לפני שמתחילים — מטרות, ידע קודם ומעמד התוכן

בסיום הפרק תוכלו להסביר מדוע רשת צפופה אינה מנצלת את המבנה המרחבי של תמונה באופן מובנה; לחשב מסנן על חלון תמונה; לחזות את ממדי התוצר של שכבות Convolution ו־Pooling; לתאר כיצד מסננים נלמדים באימון; ולבנות רשת CNN קטנה, להעריך אותה ולנתח טעויות.

הפרק ממשיך את פרק 6 (עיבוד תמונה), פרק 11 (נגזרות וגרדיאנטים) ופרק 12 (רשתות עצביות). מומלץ לחזור על מטריצה דו־ממדית, כפל וחיבור, מכפלה פנימית, ReLU, Cross-Entropy ונתוני Train/Validation/Test. לא נניח שידיעת Python לבדה מספיקה להבנת הרשת.

ידע קודם	איפה השתמשנו בו	איך ישמש בפרק
מטריצות ו־RGB	פרק 6	חלונות, מסננים וערוצים
מכפלה פנימית ומשקולות	פרקים 1 ו־9	כל תא במפת מאפיינים
כלל השרשרת ו־Gradient Descent	פרק 11	למידת משקולות המסננים
Forward / Backward	פרק 12	אימון רשת מקצה לקצה
הערכה וולידציה	פרקים 5.1 ו־5.2	בדיקה הוגנת של CNN

1. סיפור פתיחה — כיצד טלפון מזהה ספרה בכתב יד?

דודו מפתח יישומון לסריקת מספרי חדרים בבית הספר. לפעמים התמונה מוארת, לפעמים היא מעט מוזזת, ולפעמים הספרה נכתבת בקו עבה. אם נצמיד לכל פיקסל משקל שונה ברשת צפופה, הרשת

תצטרך ללמוד בנפרד שמאפיין מסוים יכול להופיע גם למעלה וגם למטה. אבל קצה, קו או פינה הם אותם דפוסים חזותיים גם כשהם זזים מעט.

מכאן עולה רעיון מרכזי: במקום לתת לכל מיקום בתמונה מערכת משקולות נפרדת, נסרוק את התמונה בעזרת מסנן קטן שמשקולותיו משותפות לכל המיקומים. כך נלמד לזהות דפוס מקומי, נשמור על קשרים בין פיקסלים שכנים ונקטין לעיתים קרובות את מספר הפרמטרים. זו אחת מאבני היסוד של CNN.

לבדיקה עצמית: אם אותו קו אנכי נמצא פעם בצד שמאל ופעם בצד ימין, מדוע כדאי להשתמש באותו מסנן בשני המקומות?

2. לפני הרשת: תמונה היא טנזור ולא ציור מסתורי

תמונת גווני אפור אפשר לראות כמטריצה בגובה H וברוחב W. ערך בכל תא מתאר עוצמת אור. בתמונת RGB יש שלוש מטריצות — אחת לכל ערוץ צבע. במהלך אימון הרשתות אנו עובדים בדרך כלל עם אוסף תמונות (Batch), ולכן למערך הנתונים יש ארבעה ממדים.

יש להבחין בין שמות הממדים לבין סדרם בספרייה. ב־PyTorch סדר ברירת המחדל הוא N,C,H,W: מספר תמונות, מספר ערוצים, גובה ורוחב. במערכות אחרות תיתכן צורה N,H,W,C. בלבול בסדר הערוצים גורם לאחת משגיאות הקוד השכיחות בתחום.

Grayscale: (H, W) | RGB: (H, W, 3) | PyTorch batch: (N, C, H, W)

למשל, אצווה של 32 תמונות צבע בגודל 64×64 תוצג ב־PyTorch בממדים (32,3,64,64). מספר הערכים הגולמיים הכולל הוא 32×3×64×64. רשת CNN לומדת להפוך את הפיקסלים למפות מאפיינים, ובהמשך להחלטה על מחלקת התמונה.

```
import numpy as np
from sklearn.datasets import load_digits

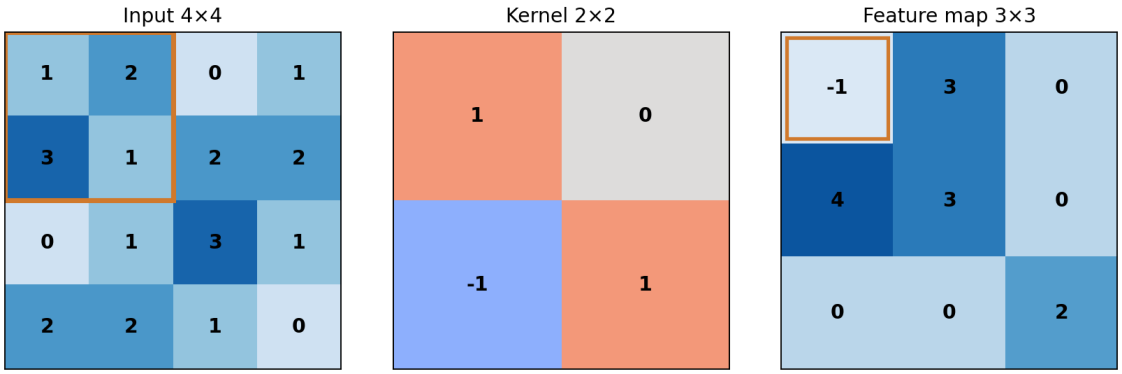
digits = load_digits()
image = digits.images[0]
print(image.shape)         # (8, 8)
print(image.min(), image.max())  # intensities in [0, 16]
print(image[0, :])         # first row of pixels
```

לבדיקה עצמית: מה ההבדל בין מספר הפיקסלים בתמונה לבין מספר המאפיינים המופקים בשכבת CNN?

3. מהו מסנן (Kernel) ומה הוא עושה?

מסנן הוא מטריצה קטנה של משקולות, לדוגמה 2×2 או 3×3. מניחים אותו מעל אזור מקומי בתמונה, מכפילים כל ערך תמונה במשקל המתאים במסנן ומחברים. התוצאה היא מספר יחיד: תגובת המסנן באזור הזה. לאחר מכן מזיזים את המסנן למיקום הבא וחוזרים על החישוב. כל התוצאות יחד יוצרות מפת מאפיינים (Feature Map).

במדעי התמונה נעשה שימוש במסננים קבועים, למשל להדגשת קצוות. ברשת CNN משקולות המסנן יכולות להשתנות בזמן האימון. לכן ההבדל החשוב אינו רק הפעולה של החלקת חלון, אלא העובדה שהמערכת לומדת איזה מסנן מועיל למשימה.



איור: חלון 2×2 עובר על מטריצה 4×4 ומפיק מפת מאפיינים 3×3.

4. חישוב ידני מלא — ערך ראשון במפת המאפיינים

ניקח חלון תמונה בגודל 2×2 והמסנן הבא. בכל תא במכפלה יש כפל אחד, ולאחריו מחברים את ארבע התוצאות. שימו לב לסימן השלילי במשקל השמאלי־תחתון של המסנן: הוא יכול להפחית מן התגובה כאשר הערך באזור הזה גבוה.

Window = [[1, 2], [3, 1]] Kernel = [[1, 0], [-1, 1]]

Output[0,0] = 1×1 + 2×0 + 3×(-1) + 1×1 = -1

כאשר המסנן זז פיקסל אחד ימינה, החלון הבא הוא [[2,0],[1,2]]. כעת מחשבים 2×0+1×1+0×(-1)+1×2=3. יש להשלים ידנית את יתר המיקומים כדי להבין מדוע גודל מפת התוצאה הוא 3×3. זו דוגמה של קורלציה צולבת (Cross-Correlation): המסנן אינו מתהפך לפני החישוב. בספריות CNN מקובל לקרוא לפעולה Convolution, אף שבמתמטיקה הקונבולוציה הפורמלית מבצעת היפוך של המסנן.

Y = [[-1, 3, 0], [4, 3, 0], [0, 0, 2]]

```
import numpy as np

image = np.array([[1, 2, 0, 1],
                  [3, 1, 2, 2],
                  [0, 1, 3, 1],
                  [2, 2, 1, 0]])
kernel = np.array([[1, 0], [-1, 1]])

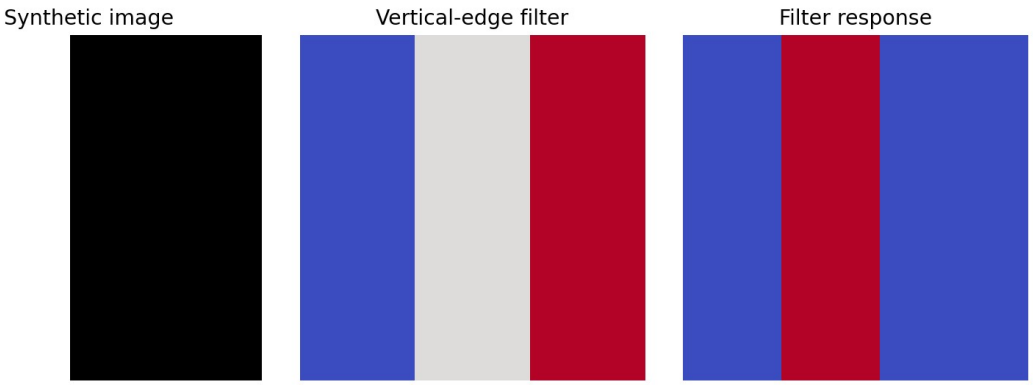
out = np.zeros((3, 3), dtype=int)
for row in range(3):
    for col in range(3):
        window = image[row:row+2, col:col+2]
        out[row, col] = np.sum(window * kernel)
print(out)
```

דיוק מתמטי: הקוד והדוגמה מבצעים *Cross-Correlation* ולא קונבולוציה עם היפוך מסנן. זו אותה מוסכמה שבה משתמשת *nn.Conv2d* ב*PyTorch*.

5. איך מסנן מזהה קצה או קו?

נניח שאזור שמאלי בתמונה כהה ואזור ימני בהיר. שינוי חד בין האזורים עשוי להצביע על קצה של עצם. מסנן שקובע משקלים שליליים בצד שמאל וחיוביים בצד ימין מגיב למעבר כזה. אין למסנן הבנה של המושג ״דלת״ או ״ספרה״; הוא רק מחבר הפרשים מקומיים על פי משקולות.

מסנן יחיד עשוי להגיב לקצה אנכי. מסננים אחרים עשויים להגיב לקווים אופקיים או לטקסטורות. בשכבות עמוקות יותר הרשת יכולה לשלב תגובות מקומיות ולהבחין בדפוסים מורכבים. התיאור הוא אינטואיטיבי: בפועל לא כל מסנן לומד תכונה שקל לתת לה שם.



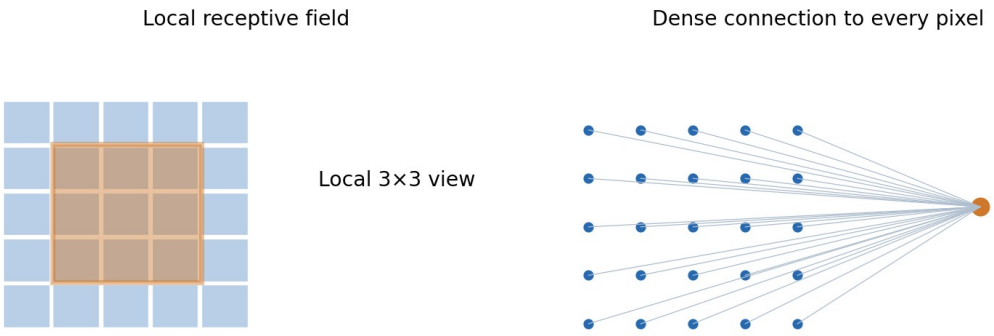
איור: דימוי של מסנן המדגיש מעבר בין אזור כהה ובהיר.

לבדיקה עצמית: אם נכפיל את כל ערכי התמונה ב־2, מה יקרה לתגובת המסנן הזה, בהנחה שאין **Bias** ואין **Clipping**?

6. מדוע שיתוף משקולות כל כך חשוב?

ברשת צפופה (Dense) כל נוירון יכול להיות מחובר לכל ערכי הקלט, ולכן ניתן ללמוד משקל שונה לכל מיקום. ב־CNN המסנן הקטן מחובר רק לסביבה מקומית, אך משתמש באותן משקולות כשהוא עובר על מיקומים שונים. השילוב בין קישוריות מקומית לשיתוף משקולות מאפשר לנצל את המבנה המרחבי.

לדוגמה, אם תמונה מכילה 64×64×3 ערכים ולנוירון צפוף יחיד יש משקל לכל ערך, דרושים 12,288 משקולות ועוד הטיה. לעומת זאת מסנן 3×3 המקבל שלושה ערוצי קלט דורש 3×3×3=27 משקולות ועוד הטיה; הוא מוחל שוב ושוב לאורך התמונה. ההשוואה היא בין נוירון צפוף יחיד למסנן יחיד, ולא בין שתי ארכיטקטורות שלמות.



איור: קישוריות מקומית לעומת קישוריות מלאה למאפייני הקלט.

One 3×3 RGB filter: 3 × 3 × 3 + 1 bias = 28 parameters

7. ממדי הפלט: Kernel Size, Stride, Padding

אחת המיומנויות החשובות לבגרות ולפרויקט היא היכולת לחזות את ממדי המערך לאחר שכבת עיבוד. יש שלושה גדלים מרכזיים: גודל המסנן K, גודל הצעד S (Stride) ומספר ערכי הריפוד P (Padding) שמוסיפים בכל צד.

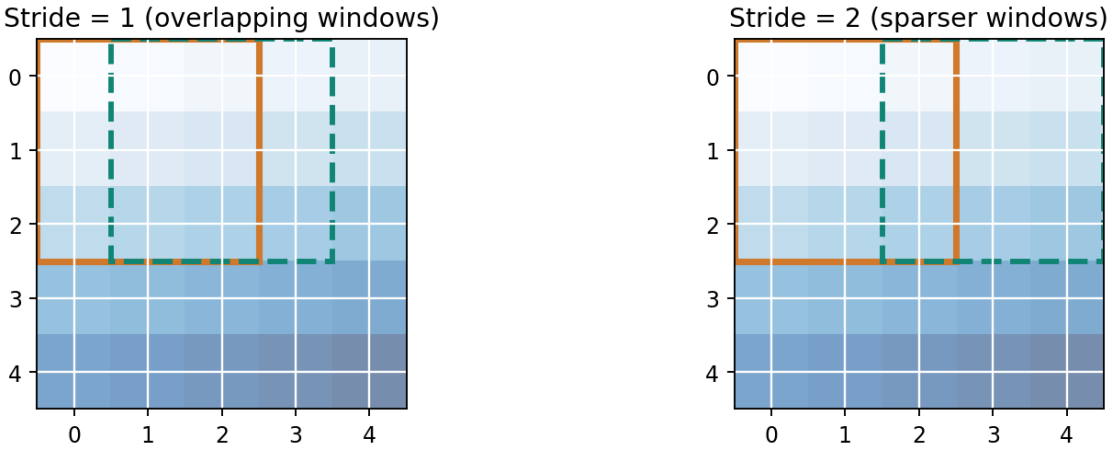
כאשר המסנן נע בצעדים גדולים יותר, מספר המקומות שבהם הוא נסרק קטן. כאשר מוסיפים ריפוד אפסים מסביב לתמונה, אפשר לחשב תגובות סמוך לשוליים ולשמר טוב יותר את הממדים. חשוב לזכור: Padding אינו משחזר מידע שלא היה בתמונה; הוא מוסיף ערכים לפי כלל שנקבע.

$$\mathbf{H_out = floor((H + 2P - K) / S) + 1 \quad [dilation = 1]}$$

$$\mathbf{W_out = floor((W + 2P - K) / S) + 1 \quad [dilation = 1]}$$

דוגמה 1: תמונה 8×8, מסנן 3×3, צעד 1, בלי ריפוד: H_out=(8−3)/1+1=6 ולכן הפלט 6×6. דוגמה 2: אותה תמונה עם ריפוד 1 בכל צד וצעד 1: H_out=(8+2−3)+1=8. דוגמה 3: תמונה 8×8, מסנן 3×3, ריפוד 1, צעד 2: 7214 . בכל שלוש הדוגמאות מספר ערוצי הפלט נקבע לפי מספר המסננים, לא לפי מספר ערוצי הקלט.

קלט H×W	K	Stride	Padding בכל צד	פלט H×W
8×8	3	1	0	6×6
8×8	3	1	1	8×8
8×8	3	2	1	4×4
28×28	5	1	0	24×24



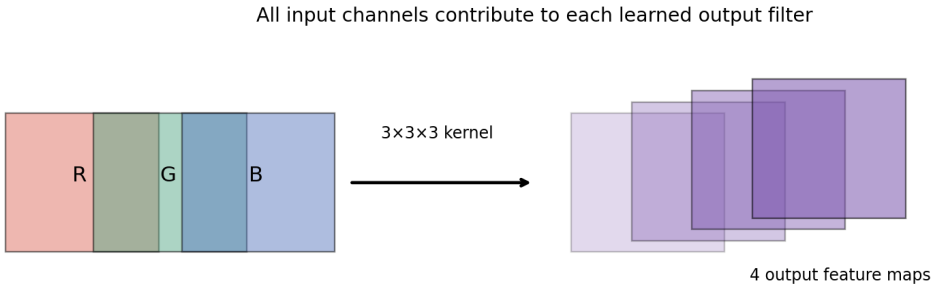
איור: *Stride* גדול יותר מדלג על מיקומי סריקה ומקטין את ממדי הפלט.

לבדיקה עצמית: חישבו את גודל הפלט של תמונה 32×32 עם מסנן 5×5, ריפוד 2 וצעד 2. נמקו את פעולת הרצפה.

8. לאן נעלמים ערוצי הצבע? Convolution רב־ערוצית

תמונת RGB מכילה שלושה ערוצים. מסנן CNN אחד אינו מסנן של ערוץ בודד בלבד: בדרך כלל הוא מכיל קבוצת משקולות לכל ערוץ קלט. מחשבים תגובה מכל ערוץ, מחברים את שלושת התרומות ומוסיפים Bias; התוצאה היא מפת מאפיינים אחת.

אם נבחר שמונה מסננים שונים, נקבל שמונה מפות מאפיינים. כל מסנן יכול ללמוד דפוס אחר. אם לקלט יש 3 ערוצים ולשכבה 8 מסננים בגודל 3×3, מספר הפרמטרים הוא 8×(3×3×3+1)=224. ממדי הפלט יהיו N,8,H_out,W_out.



איור: כל מסנן פועל על כל ערוצי הקלט ויוצר ערוץ פלט אחד.

Conv parameters = C_out × (C_in × K_h × K_w + 1 bias)

לצורך ההבנה, חשבו ידנית פיקסל פלט של מסנן 1×1 כאשר תרומת שלושת הערוצים היא R×0.2+G×0.5+B×(0.1)+0.3. אין משמעות הדבר שהרשת מפרשת כל ערוץ כמשתנה עצמאי ללא הקשר; המשקולות נלמדות יחד בתהליך האימון.

9. פונקציית הפעלה — מדוע Convolution לבדה אינה מספיקה?

שכבת קונבולוציה מחשבת צירוף ליניארי מקומי של הערכים. אם נחבר רק שכבות ליניאריות, ללא פעולות לא־ליניאריות ביניהן, המערכת עדיין תהיה צירוף ליניארי כולל (בהתעלם מפעולות כמו Pooling). לכן מוסיפים לרוב פונקציית הפעלה כגון ReLU: ערכים שליליים הופכים לאפס וערכים חיוביים נשארים כפי שהם.

ReLU מאפשרת לשלב דפוסים בצורה לא־ליניארית. בגרף, החצי השמאלי נשאר באפס והחצי הימני עולה בקו ישר. אולם ReLU לבדה אינה פותרת כל בעיה, ויש לדון גם במקרים שבהם נוירונים נשארים לא פעילים לאורך האימון.

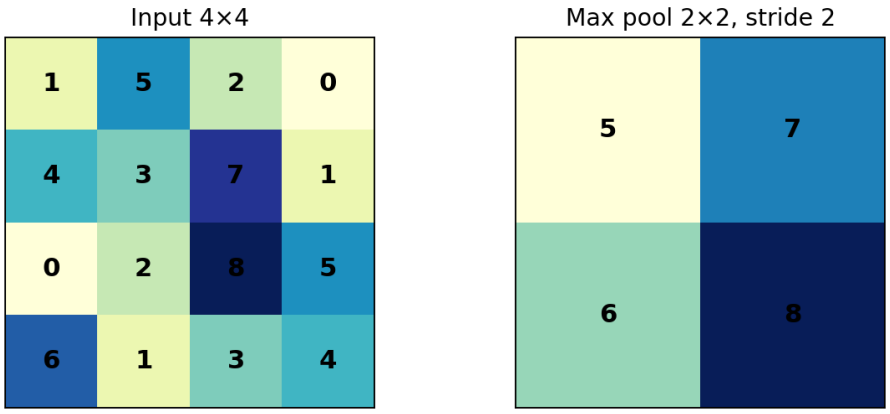
ReLU(z) = max(0, z)

ReLU(-2)=0 ReLU(0)=0 ReLU(3.5)=3.5

10. Pooling — כיצד מצמצמים מפה בלי לאבד כל דפוס מקומי?

Pooling מסכמת אזור קטן במפת מאפיינים לערך יחיד. ב־Max Pooling לוקחים את הערך המרבי בכל חלון. פעולה זו מקטינה את הרזולוציה ויכולה להקטין רגישות לשינויים מקומיים קטנים, אך עלולה גם למחוק מידע מרחבי מדויק. אין להתייחס אליה כאל פעולה שמשפרת כל מודל בכל מצב.

נחשב לדוגמה Max Pooling בגודל 2×2 עם Stride 2: נפרק מטריצה 4×4 לארבעה בלוקים לא חופפים. מכל בלוק נבחר את הערך הגבוה ביותר. התוצאה היא מטריצה 2×2. אין כאן משקולות שהשכבה לומדת; כלל הבחירה מוגדר מראש.



איור: *Max Pooling* בוחר את הערך הגדול בכל חלון 2×2.

MaxPool([[1,5],[4,3]]) = 5

```
import torch
import torch.nn.functional as F

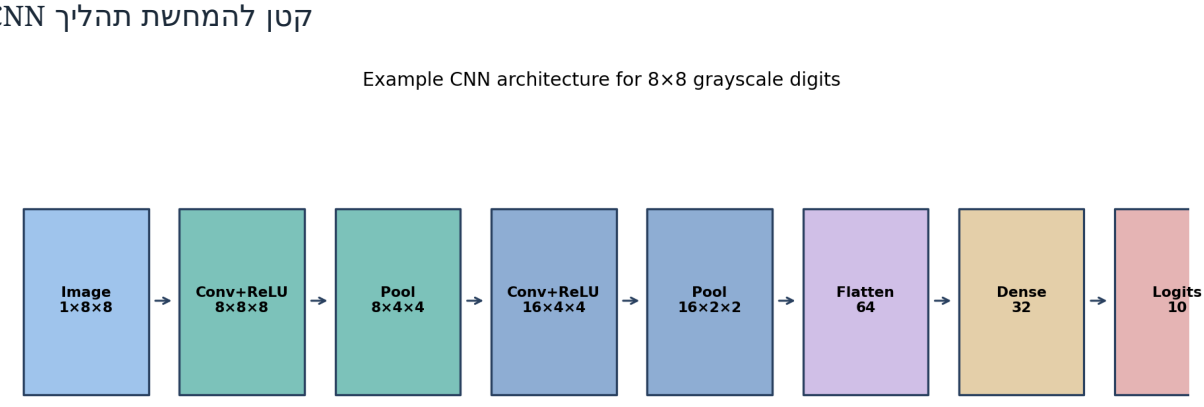
x = torch.tensor([[[[1., 5., 2., 0.],
                    [4., 3., 7., 1.],
                    [0., 2., 8., 5.],
                    [6., 1., 3., 4.]]]])
print(F.max_pool2d(x, kernel_size=2, stride=2))
```

לבדיקה עצמית: אם היינו משתמשים ב־**Average Pooling** במקום **Max Pooling**, איזה מידע היה מודגש יותר? חשבו את הבלוק הראשון.

11. מהי ארכיטקטורת CNN מלאה?

CNN היא לא שכבה אחת אלא שרשרת פעולות. בשלב הראשון הקלט הוא תמונה. שכבות Convolution לומדות דפוסים מקומיים, פונקציות ReLU מוסיפות אי־ליניאריות, ו־Pooling עשוי להקטין את המפות. בסוף ניתן לשטח את מפות המאפיינים לווקטור ולחבר שכבות Dense שמפיקות ערכי תחזית.

בדוגמה שנשתמש בה נבנה רשת קטנה לסיווג ספרות: 1×8×8 → קונבולוציה ל־8 מפות → Pooling ל־8×4×4 → קונבולוציה ל־16 מפות → Pooling ל־16×2×2 → Flatten באורך 64 → שכבה חבויה בגודל 32 → 10 ערכי פלט. המודל אינו נחשב בהכרח ״עמוק״ לפי כל סף מחקרי; זהו מבנה לימודי קטן להמחשת תהליך CNN.

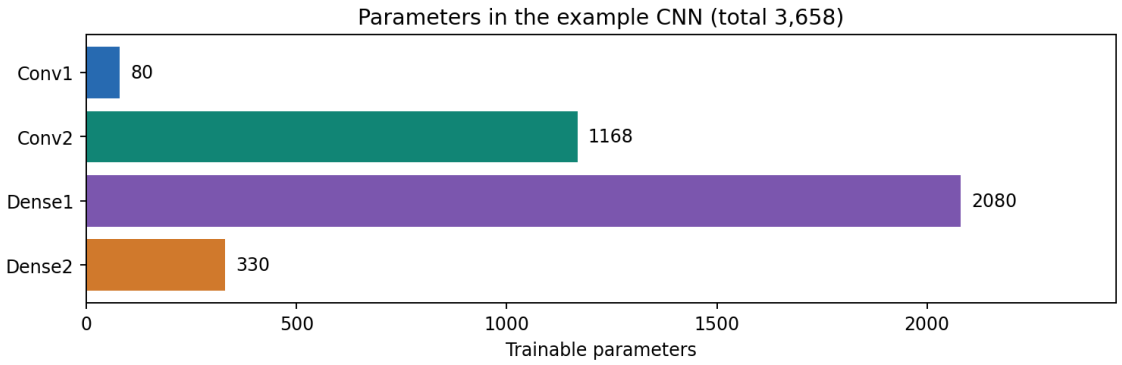


איור: ממדי המערכים ברשת *CNN* לימודית לספרות 8×8.

12. סופרים פרמטרים — יכולת שחזור והבנת הארכיטקטורה

ספירת פרמטרים מאפשרת לזהות טעויות בתכנון שכבות. בשכבת Conv2d המשקולות הן בגודל (מספר מסנני פלט, מספר ערוצי קלט, גובה המסנן, רוחב המסנן), ולכל מסנן יש בדרך כלל Bias אחד. בשכבת Dense מספר המשקולות הוא מספר הקלטים כפול מספר הנוירונים, ועוד Bias לכל נוירון.

שכבה	נוסחת חישוב	מספר פרמטרים
Conv1: 1→8, 3×3	(3+1×3×1)×8	80
Conv2: 816, 3×3	(3+1×3×8)×16	1,168
Dense1: 64→32	32+32×64	2,080
Dense2: 3210	10+10×32	330
סך הכל	80+1,168+2,080+330	3,658



איור: השוואת מספר הפרמטרים בארכיטקטורה הלימודית.

13. Forward Pass, Loss, Backpropagation — כיצד רשת לומדת?

במעבר קדימה (Forward Pass) התמונה עוברת דרך כל השכבות ונוצרים עשרה מספרים הנקראים Logits: מספר אחד לכל מחלקת ספרה. אלה אינם הסתברויות בפני עצמם. פונקציית Softmax יכולה להמיר אותם לערכים בין 0 ל־1 שסכומם 1, אך בזמן אימון מסווג רב־מחלקתי משתמשים לרוב ב־CrossEntropyLoss שמקבלת את ה־Logits ישירות.

לאחר השוואת ה־Logits לתווית האמיתית, פונקציית האובדן מספקת מספר המייצג עד כמה התחזית אינה מתאימה לתווית. באמצעות Backpropagation וכלל השרשרת מחשבים את נגזרת האובדן ביחס למשקל בכל שכבה. האופטימייזר משנה מעט את המשקולות בכיוון שנועד להקטין את השגיאה. זהו אותו עקרון מתמטי שתרגלנו בפרק 11, הפעם על מספר רב של מסננים ושכבות.

$$\text{Loss} = -\log\left(\frac{\exp(z_y)}{\sum_j \exp(z_j)}\right)$$

נוסחת Cross-Entropy לדוגמה אחת משתמשת ב־z_y שהוא ה־Logit של המחלקה האמיתית. אם למודל שלוש מחלקות וה־Logits הם [2,1,0], עבור תווית 0 ההסתברות אחרי Softmax היא בקירוב 0.6652 ולכן האובדן הוא בערך 0.4076. אם תווית האמת הייתה 2, האובדן היה גדול יותר. החישוב נועד לבנות אינטואיציה, לא לדרוש פיתוח של כל נגזרות Softmax בשלב זה.

$$P(y=0) = \frac{e^2}{e^2+e^1+e^0} \approx 0.6652$$

$$\text{CrossEntropy}(y=0) = -\ln(0.6652) \approx 0.4076$$

```
import torch
```

```
import torch.nn as nn

logits = torch.tensor([[2.0, 1.0, 0.0]])
labels = torch.tensor([0])
loss = nn.CrossEntropyLoss()(logits, labels)
print(round(loss.item(), 4))  # approximately 0.4076
```

14. מימוש רשת CNN אמיתית באמצעות PyTorch

במימוש הבא נשתמש ב־PyTorch, משום שהקוד מאפשר לזהות באופן מפורש את שכבות הרשת ואת תהליך האימון. זהו קוד לימודי, לא מתכון יחיד לפרויקט. נשתמש במאגר `digits` הזמין ב־scikit-learn ללא הורדה חיצונית. תמונה היא 8×8 בגווני אפור, הערכים המקוריים בין 0 ל־16, והתווית היא אחת מעשר הספרות.

נחלק את הנתונים באופן מדורג ל־Train, Validation ו־Test, עם שמירה בקירוב על התפלגות המחלקות. את קבוצת ה־Validation נשתמש כדי לבחור למשל מספר Epochs ואת Test נשמור להערכה סופית. הנרמול לפי תחום מדידה ידוע [0,16] אינו מחשב סטטיסטיקה מכל המאגר, ולכן אין כאן התאמת Scaler על Test.

```
import numpy as np
import torch
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split

torch.manual_seed(7)
np.random.seed(7)
digits = load_digits()
X = digits.images.astype(np.float32) / 16.0
y = digits.target.astype(np.int64)

X_dev, X_test, y_dev, y_test = train_test_split(
    X, y, test_size=0.15, stratify=y, random_state=7)
X_train, X_val, y_train, y_val = train_test_split(
    X_dev, y_dev, test_size=0.18, stratify=y_dev, random_state=7)

train_x = torch.tensor(X_train[:, None, :, :])
train_y = torch.tensor(y_train)
val_x = torch.tensor(X_val[:, None, :, :])
val_y = torch.tensor(y_val)
print(train_x.shape, val_x.shape)
```

השימוש בממד חדש `None` מוסיף ערוץ צבע יחיד, כדי לקבל את סדר הממדים המתאים ל־Conv2d: מספר הדוגמאות, מספר הערוצים, גובה ורוחב. אין לשטח תמונה לווקטור לפני שכבת הקונבולוציה, מפני שהמטרה היא לנצל את השכנות המרחבית.

```
import torch.nn as nn

class DigitCNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.features = nn.Sequential(
            nn.Conv2d(1, 8, kernel_size=3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2),
            nn.Conv2d(8, 16, kernel_size=3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2)
        )
        self.classifier = nn.Sequential(
            nn.Flatten(),
            nn.Linear(16 * 2 * 2, 32),
            nn.ReLU(),
            nn.Linear(32, 10)
```

```
        )

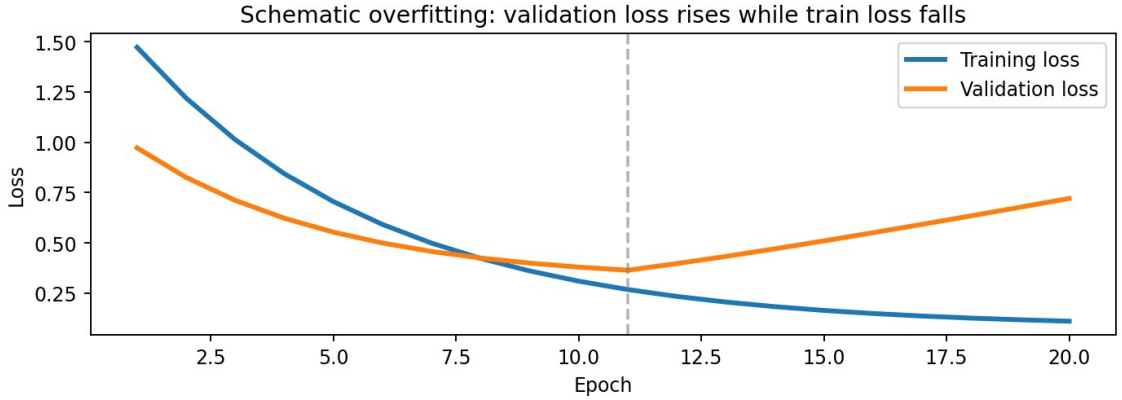
    def forward(self, x):
        return self.classifier(self.features(x))

model = DigitCNN()
print(model(torch.zeros(2, 1, 8, 8)).shape)  # (2, 10)
print(sum(p.numel() for p in model.parameters()))  # 3658
```

15. אימון מבוקר: מדוע צריך להפריד בין אימון לוולידציה?

בכל Epoch הרשת עוברת על דוגמאות האימון, מחשבת תחזיות, מודדת אובדן ומעדכנת משקולות. אחרי הסיום בודקים את המודל על קבוצת Validation ללא חישוב נגזרות. אם האובדן על Train ממשיך לרדת, אבל האובדן על Validation עולה, זו אינדיקציה אפשרית להתאמת יתר. אין להסיק מהתנהגות של Epoch יחיד מסקנה גורפת.

בדוגמת הקוד אנחנו בודקים את שתי הקבוצות אחרי כל Epoch. בספר נריץ מעט Epochs, כדי לשמור על זמן מעבדה סביר גם ללא GPU. בפרויקט גמר ניתן לשלב Batch קטן, Early Stopping ושמירת הגרסה הטובה ביותר לפי Validation. רצוי לא לבחור את המודל לפי Test, גם אם הדבר מפתה אחרי ניסוי ראשון.



איור: עקומות סכמטיות — אינן תוצאות אימון אמיתיות.

```
from torch.utils.data import TensorDataset, DataLoader

loader = DataLoader(TensorDataset(train_x, train_y),
                    batch_size=64, shuffle=True)
criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=0.003)

for epoch in range(5):
    model.train()
    running = 0.0
    for xb, yb in loader:
        optimizer.zero_grad()
        loss = criterion(model(xb), yb)
        loss.backward()
        optimizer.step()
        running += loss.item() * len(yb)

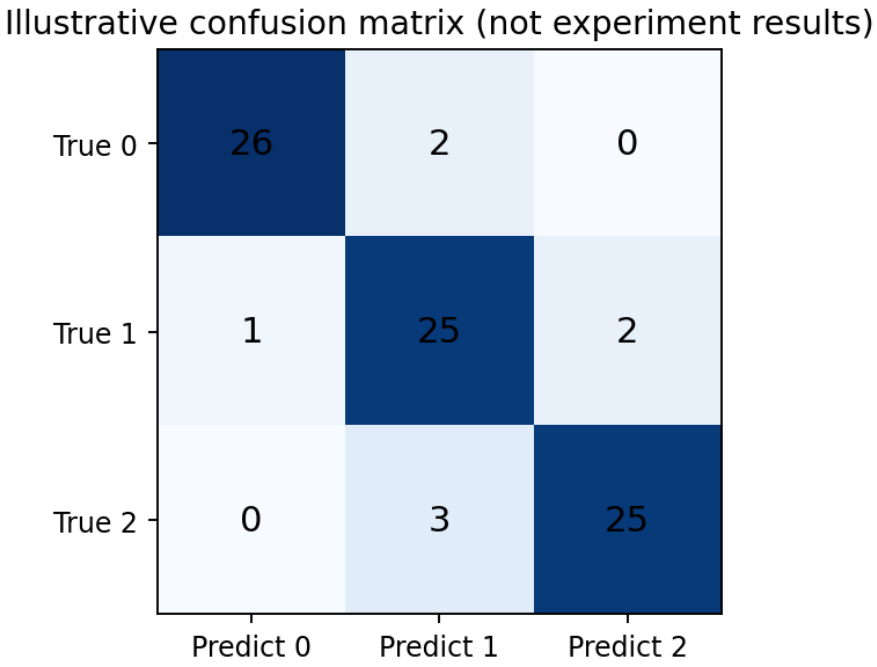
    model.eval()
    with torch.no_grad():
        val_logits = model(val_x)
        val_loss = criterion(val_logits, val_y).item()
        val_acc = (val_logits.argmax(1) == val_y).float().mean().item()
    print(epoch + 1, round(running / len(train_y), 4),
          round(val_loss, 4), round(val_acc, 3))
```

פלט המדדים תלוי באתחול, בסביבת הביצוע ובבחירות האימון; אין להבטיח תוצאה מסוימת. בדוגמת הפרק אין שימוש ב־*Test* לצורך בחירת *Epochs*.

16. כיצד נבדוק ביצועי CNN בצורה מדעית?

Accuracy מודד את שיעור התחזיות הנכונות, אבל אינו אומר אילו ספרות התבלבלו זו בזו. לכן נציג גם מטריצת בלבול, ונסתכל על כמה דוגמאות שסווגו בטעות. אם המודל מתקשה בספרות 3 ו־8, למשל, כדאי לשאול האם מדובר בכתב יד דומה, בעובי קו או בייצוג לא מספיק טוב.

מטריצת הבלבול שבתמונה הבאה היא דוגמה המחשה בלבד, ולא תוצאה שמדדנו במודל שלנו. במעבדה התלמיד יחשב מטריצה אמיתית מהתחזיות שלו, לאחר שהחליט על המודל. חשוב גם לחשוב על מדדי Macro F1 כאשר יש חוסר איזון משמעותי בין מחלקות.



איור: מטריצת בלבול מדגימה כיצד מנתחים את סוגי הטעויות.

```
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

# Inspect errors on validation during model development.
model.eval()
with torch.no_grad():
    val_pred = model(val_x).argmax(1).cpu().numpy()
print('CNN validation accuracy:', accuracy_score(y_val, val_pred))
print(confusion_matrix(y_val, val_pred))
print(classification_report(y_val, val_pred, zero_division=0))
```

17. השוואה הוגנת למודל בסיס — האם CNN תמיד מנצחת?

לא. על מאגרים קטנים מאוד, מודל KNN עשוי להיות תחרותי ואף להצליח יותר מרשת CNN מסוימת. CNN דורשת בחירות ארכיטקטורה, אימון ומשאבי חישוב, ואילו KNN מבוסס על דוגמאות שמורות ומרחקים. השוואה מקצועית אינה מתחילה במסקנה אלא בניסוי הוגן: אותן קבוצות נתונים, אותן דרכי מדידה, ושימוש ב־Validation לצורך בחירת פרמטרים.

בדוגמת הספר שלנו נשווה תחזיות KNN לרשת CNN רק אחרי שהרשת עברה אימון. נשווה ביצועים, זמן הרצה, מספר פרמטרים וקשיים בניתוח טעויות. נדגיש שאין למדוד את ביצועי המודלים על Train ולדווח עליהם כיכולת הכללה.

```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

knn = KNeighborsClassifier(n_neighbors=3)
knn.fit(X_train.reshape(len(X_train), -1), y_train)
knn_val_pred = knn.predict(X_val.reshape(len(X_val), -1))
knn_val_accuracy = accuracy_score(y_val, knn_val_pred)
cnn_val_accuracy = accuracy_score(y_val, val_pred)
print('KNN validation:', round(knn_val_accuracy, 3))
print('CNN validation:', round(cnn_val_accuracy, 3))

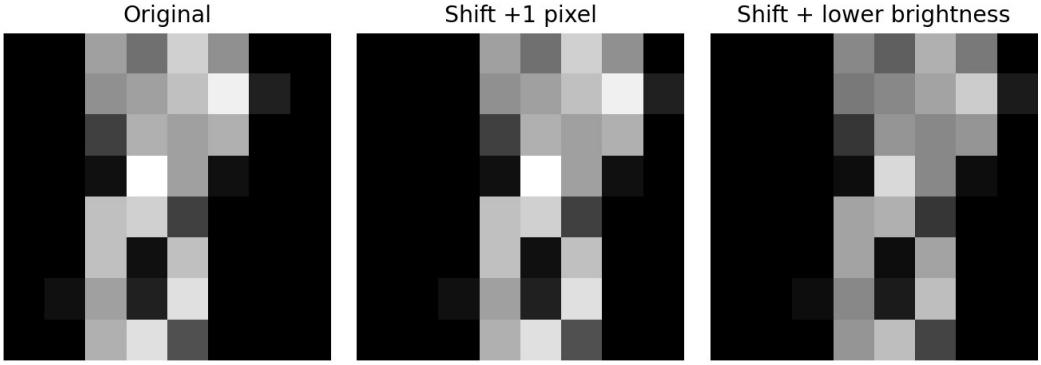
# Make the decision before looking at Test.
if knn_val_accuracy >= cnn_val_accuracy:
    selected = 'KNN'
    final_pred = knn.predict(X_test.reshape(len(X_test), -1))
else:
    selected = 'CNN'
    model.eval()
    with torch.no_grad():
        test_x = torch.tensor(X_test[:, None, :, :])
        final_pred = model(test_x).argmax(1).cpu().numpy()

print('Selected by validation:', selected)
print('Final test accuracy:', accuracy_score(y_test, final_pred))
print(confusion_matrix(y_test, final_pred))
```

18. Data Augmentation — למה להציג גרסאות נוספות של תמונה?

בפרויקט לזיהוי ספרות, צילום של אותה ספרה יכול להיות מעט מוזז או בעל בהירות שונה. אפשר להוסיף בזמן האימון שינויים ששומרים על התווית, כגון הזזה קטנה. זו שיטת Augmentation: יצירת דוגמאות אימון חדשות מתוך דוגמאות קיימות.

אסור להפעיל Augmentation על Test כדי להגדיל את מספר הדוגמאות המדווחות, ואסור לבחור שינויים שמשנים את משמעות הספרה. לדוגמה, סיבוב חד או שיקוף עלול ליצור ספרה אחרת או להפוך אותה ללא קריאה. על תלמיד בפרויקט להסביר מדוע הטרנספורמציות שבחר תואמות את המציאות הצפויה.



איור: המחשה של שינויי קלט אפשריים — לא כל שינוי מתאים לכל מאגר.

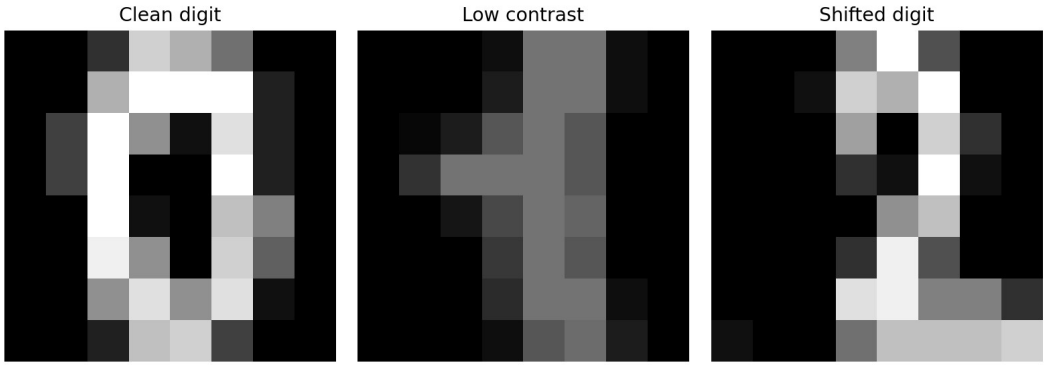
19. טעויות נפוצות שחייבים לזהות

טעות	מדוע זו טעות	כיצד מתקנים
שיטוח התמונה לפני Conv2d	נעלם המבנה המרחבי שהשכבה מנצלת	להשתמש ב־(N,C,H,W)
הוספת Softmax לפני CrossEntropyLoss	CrossEntropyLoss מקבלת Logits	להעביר ערכי Logits ישירים
שימוש ב־Test לבחירת Epoch	מידע מהבדיקה משמש לכיוון המודל	לבחור באמצעות Validation
בלבול בין מספר ערוצים לגודל התמונה	מספר מסננים קובע ערוצי פלט	לחשב בנפרד C_out ו־H_out,W_out
הנחה ש־CNN תמיד עדיפה על KNN	הצלחה תלויה בנתונים ובאימון	לבצע השוואה הוגנת
Augmentation שמשנה תווית	המודל לומד דוגמאות סותרות	לבחון כל טרנספורמציה בהקשר
שכחת model.eval() בזמן בדיקה	התנהגות של Dropout/BatchNorm יכולה להשתנות	להגדיר מצב בדיקה

20. חקר מקרה — מדוע המודל נכשל על תמונות שלא ראה?

נניח שרשת זיהתה תמונות ספרות מסריקות ברורות, אך במבחן על תמונות מטושטשות מאפליקציה חדשה הביצועים ירדו. לפני שמוסיפים שכבות יש לבדוק האם השוני נובע מתאורה, מרזולוציה, משיעור הספרות במאגר, מתהליך Crop, או מתוויות לא מדויקות. זהו שינוי התפלגות (Distribution Shift).

פתרון אפשרי אינו רק ״לאמן יותר״. ייתכן שצריך להגדיר מחדש תהליך עיבוד, להרחיב את נתוני האימון, לבצע הערכה לפי קבוצות או לשנות את הבעיה. בפרויקט גמר נדרשת יכולת לזהות את הסיבה האפשרית לכישלון ולהציע ניסוי מבוקר שיאמת אותה.



איור: שינויים בבהירות ובהזזה ממחישים מדוע יש לבדוק עמידות לשינויי קלט.

21. תרגול מדורג — הבנה, חישוב, ניתוח ונימוק

רמה א — הבנת מושגים

1. הסבירו בשפה שלכם מה ההבדל בין Kernel לבין Feature Map. מה מהם נלמד באימון?

2. הגדירו מהו Stride. הסבירו כיצד שינוי מ־1 ל־2 עשוי להשפיע על מספר תאי הפלט.

3. מהו ההבדל בין Max Pooling לבין Convolution מבחינת קיומן של משקולות נלמדות?

4. היכן מופיעה פונקציית ReLU ברשת, ומה קורה לפלט שלילי?

רמה ב — חישוב ידני

5. עבור מטריצה 5×5 ומסנן 3×3, Stride=1, Padding=0: חשבו את ממדי הפלט והסבירו את המספר.

6. עבור קלט 16×16×3 ו־12 מסננים בגודל 3×3, Padding=1: חשבו את מספר המשקולות ואת ממדי הפלט.

7. חשבו את ארבעת ערכי Max Pooling למטריצה 4×4 שבראש סעיף 10. השוו ל־Average Pooling.

8. לפלט Logits=[0,2,1] ולתווית אמת 1: בטאו את ה־Softmax של התווית ואת ה־Cross-Entropy. אפשר להיעזר במחשבון.

רמה ג — ניתוח מקצועי והכנה להגנה בעל־פה

9. תלמיד דיווח על Accuracy 99% בנתוני אימון ו־76% ב־Test. הציעו לפחות שלוש בדיקות שצריך לבצע לפני שינוי הארכיטקטורה.

10. השוו בין CNN ל־KNN בזיהוי ספרות: אילו יתרונות מעשיים יש לכל אחת, ומה אי אפשר להסיק ללא ניסוי?

11. מפתח הוסיף סיבוב של 180° לכל תמונת אימון. האם זה תמיד Augmentation תקין לספרות? נמקו.

12. בחרו בעיית תמונה לפרויקט גמר. הגדירו מאגר, תוויות, Baseline, חלוקת נתונים, שלוש בדיקות סיכון ושני מדדי הערכה.

22. תכנית הוראה מוצעת ל־50 שעות

זהו תכנון כיתתי מוצע של 10 יחידות בנות חמש שעות לימוד, לא חלוקה רשמית של מפגשי ההשתלמות.

יחידה	שעות	תוכן ותוצר
1	5	תמונות, טנזורים, קלט וידע קודם; מפת מושגים
2	5	Kernel וקורלציה צולבת; מפת מאפיינים בחישוב ידני
3	5	Padding, Stride, ערוצים וחישובי ממדים; בוחן קצר
4	5	Pooling, ReLU ואדריכלות; מעבדת NumPy
5	5	ארכיטקטורת CNN וספירת פרמטרים; פירוק שכבות
6	5	Forward, Loss, Backprop; ניסוי Logits
7	5	מימוש PyTorch, DataLoader ואימון מבוקר

תוכן ותוצר	שעות	יחידה
Validation, השוואה ל־KNN, מטריצת בלבול	5	**8**
Augmentation, ניתוח שגיאות ועמידות	5	**9**
מיני־פרויקט CNN, תיעוד והצגה בעל־פה	5	**10**
פרויקט למידה ותיק ניסויים	50	**סה״כ**

23. נספח לפרויקט הגמר — דף ניסוי CNN

מומלץ להדפיס או להעתיק את כרטיס הניסוי הבא לכל שינוי מהותי ברשת. מטרתו למנוע מצב שבו תלמיד משנה כמה רכיבים בבת אחת ואינו יודע מה גרם לשיפור. ניסוי איכותי מגדיר השערה, שומר את חלוקת הנתונים, מתעד את השינוי ומפריד בין תוצאות Validation לבין Test.

מה התלמיד מתעד	שדה
איזו בעיה אמיתית רוצים לפתור?	**שאלת חקר**
מקור, גודל, רישוי, איכות ואופן חלוקה	**גרסת מאגר**
שכבות, מסננים, Pooling ומספר פרמטרים	**ארכיטקטורה**
Learning rate, Epochs, Batch size, Seed	**Hyperparameters**
מודל פשוט להשוואה	**Baseline**
Accuracy, F1, Loss וניתוח טעויות	**מדדי Validation**
מה השתנה, מה נלמד ומה מגבלה	**החלטה מקצועית**
תוצאה לאחר סיום כל בחירות המודל	**בדיקת Test סופית**

24. הקשר לשלוש תכניות הלימוד

הקשר לחומר המקושר	מה התלמיד כבר יודע / לומד	מסלול
תשתית קוד ו־CNN; EDA אינה דרישת חובה מזוהה ביחידה 3	ייצוג נתונים, וקטורים, גרפים, KNN והערכת מודלים	**יחידה 3 — כיתה י׳**
תשתית מתמטית ומושגית; CNN אינה מוכרזת כאן כחומר חובה ביחידה 5	וקטורים, תקנון, מדדים ו־Validation	**יחידה 5 — נושאי ליבה**
יישום בתמונות, פיתוח מודל והכנה לפרויקט גמר	פרק 6 עיבוד תמונה; פרק 11 מתמטיקה; פרק 12 DNN; פרק 13 CNN	**התמחות למידה עמוקה**

25. מקורות מקצועיים והבחנה בין חובה להעשרה

מפת תכנית ההתמחות במאגר GitHub מגדירה את פרק 13 — CNN ואת היקפו (50 שעות). ההסברים מתבססים על רעיונות מקצועיים המופיעים ב־Dive into Deep Learning (פרקי CNN), Understanding

Deep Learning מאת Simon Prince (פרק 10), Hands-On Machine Learning מאת Aurélien Géron, ועל תיעוד PyTorch. הדוגמאות, המשימות והאיורים בספר מקוריים, ולא הועתקו מספרים מסחריים.

נושאים חיוניים ללמידה כאן: ייצוג טנזורים, מסננים, Convolution, Stride/Padding, Pooling, ארכיטקטורה, אימון בסיסי והערכת מודל. כהעמקה מוצעים חישובי Cross-Entropy ידניים, מורכבות פרמטרים, Augmentation וניתוח Distribution Shift. רק נוסח התכנית הרשמי המעודכן מוסמך לקבוע אילו פרטים הם חובה בבחינה או בפרויקט.

https://github.com/samiabo75/Data-Science-ML-Course/tree/main/03-Deep-Learning-5-Units/13-Convolutional-Neural-Networks

https://en.d2l.ai/chapter_convolutional-neural-networks/conv-layer.html

https://en.d2l.ai/chapter_convolutional-neural-networks/padding-and-strides.html

https://docs.pytorch.org/docs/stable/generated/torch.nn.Conv2d.html

https://scikit-learn.org/stable/modules/generated/sklearn.datasets.load_digits.html

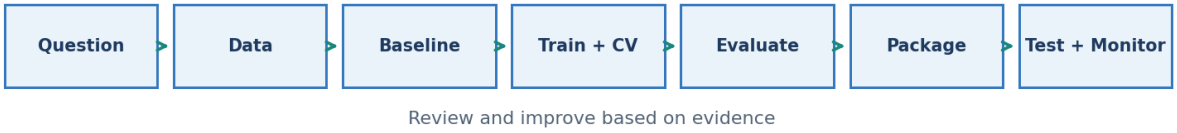
https://mitpress.mit.edu/9780262377102/understanding-deep-learning/

ספר הלימוד המאוחד במדעי הבינה המלאכותית

פרק 14 — ניהול פרויקט במדעי הנתונים

מן המחברת ב־Colab למערכת AI שאפשר לשחזר, לבדוק, להסביר ולהציג

מהדורת תלמיד | כיתות י״א–י״ב | התמחות למידת מכונה ולמידה עמוקה | תכנון לימודי ל־25 שעות



איור 1 — מחזור החיים של פרויקט למידת מכונה: התקדמות עם בדיקות וחזרה לתיקון.

מה נלמד בפרק?

עד כה למדנו כיצד לטעון נתונים, לבצע חקר, לאמן מסווגים ורשתות עצביות ולחשב מדדי הצלחה. אבל במעבדה אמיתית אין די בתא Colab שמפיק Accuracy גבוה. מפתח AI צריך לדאוג שהנתונים יישארו מזוהים, שהקוד ניתן להרצה חוזרת, שהמודל יעבוד גם מחוץ למחברת, ושהחלטות יהיו מתועדות ומבוססות ראיות.

בפרק זה נפתח את עבודתו של דודו: מערכת זיהוי ספרות בכתב יד. לא נבנה רשת עצבית חדשה — כבר למדנו לעשות זאת בפרקים קודמים. הפעם נלמד לעטוף מודל קיים בתהליך הנדסי ומחקרי אחראי. כל שלב ילווה בתוצר שאפשר להציג למורה ולבוחן.

עוגן התכנית וידע קודם

מפת ההתמחות במאגר הקורס מייעדת לפרק 14 — "ניהול פרויקט במדע הנתונים: היבטים מעשיים" — 25 שעות. תתי־הנושאים והמימושים המוצעים כאן הם הרחבה דידקטית מקצועית על סמך ספרי המקור והנחיות הנדסת ML; אין להציג את חלוקת השעות והכלים הספציפיים כציטוט מתוך הסילבוס הרשמי המלא, שעדיין מצריך הצלבה פרטנית.

ידע קודם: Python, NumPy, Pandas, תיעוד Notebook, KNN, חלוקת נתונים, Pipeline, Validation, Test, Accuracy, F1־ו, מטריצת בלבול ועקרונות עיבוד תמונה.

14.1 — סיפור המסגרת: למה קוד שעובד אינו תמיד מוצר?

דודו בנה מחברת שמזהה ספרות באימון. המודל קיבל ציון טוב, ולכן הוא שלח את המחברת לחברה שעובדת במחשב אחר. התברר שהקובץ לא מצא את נתוני האימון, המשתנים נוצרו בסדר הרצה לא ברור, ובמחשב החדש הותקנה גרסת ספרייה אחרת. בשלב הבא ניסו לחזות תמונה חדשה, אבל היא הייתה בגודל 28×28 ולא 8×8. שלוש בעיות שונות נחשפו: ניהול סביבת עבודה, חוזה קלט, ושחזור תוצאות.

הלקח הראשון הוא שיש להבחין בין אב־טיפוס מחקרי לבין מערכת אמינה. אב־טיפוס נועד להוכיח שרעיון עשוי לעבוד. מערכת מסודרת צריכה להגדיר מראש קלט ופלט, לבדוק כשלים, לתעד מגבלות ולאפשר לאדם אחר להפעיל אותה.

14.2 — ניסוח בעיה וקריטריון הצלחה

לפני כתיבת קוד מנסחים מה המערכת מקבלת, מה היא מחזירה, למי היא מיועדת וכיצד נחליט שהצליחה. בדוגמת הספר, הקלט הוא תמונה בגווני אפור של ספרה יחידה בגודל 8×8. הפלט הוא ספרה חזויה מ־0 עד 9. זו משימת סיווג רב־מחלקתי. הנתונים הם תמונות שכבר תויגו, ולכן מדובר בלמידה מונחית.

נגדיר את היעד במחקר: להשוות מסווג פשוט למודל מבוסס מרחק ולבדוק ביצועים על דוגמאות שלא שימשו לבחירת המודל. אחוז הצלחה לבדו אינו מספיק: נדרוש גם Macro F1, ניתוח דוגמאות שגויות וזמן סביר להפקת תחזית. ערכי סף מספריים יתאימו לצורכי הפרויקט ולא ייבחרו לאחר צפייה בתוצאות.

Success criteria = metric + threshold + test protocol + limitations

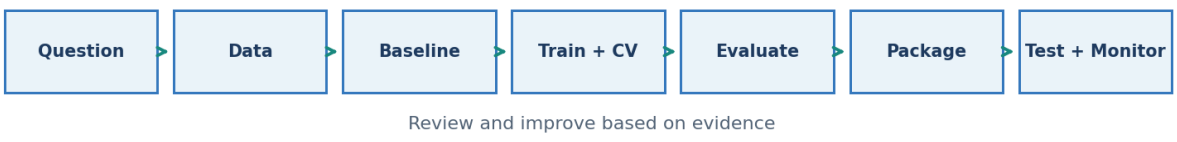
לדוגמה, אפשר להציע מראש כי המודל יעבור מבחן רק אם Macro F1 יהיה לפחות 0.90 על קבוצת Test שמורה, והמערכת תדחה קלט שאינו במבנה המוגדר. זהו קריטריון הוראה מוצע ולא תקן רשמי או התחייבות לביצועים.

14.3 — מסמך דרישות קצר (Project Brief)

למה צריך להגדירו?	החלטת הפרויקט	רכיב
מבהיר למי מיועדת המערכת	תלמיד המעוניין לבדוק ספרה כתובה	משתמש
מונע אי־התאמה למאגר	64 ערכים מספריים, טווח 0–16	קלט
מונע ערבוב עם הסתברות	תווית חזויה 0–9	פלט
מונע הסתמכות על מדד יחיד	Macro F1 לצד מטריצת בלבול	איכות
מבהיר אי־התאמה אפשרית לתמונות טלפון	תמונות 8×8 שדומות למקור	מגבלות
ניהול נתונים אחראי	לא משתמשים בתמונות אישיות ללא אישור	פרטיות

14.4 — שלבי הפרויקט וקריטריוני סיום

אפשר לראות בפרויקט רצף של תוצרים: כרטיס בעיה, כרטיס נתונים, מחברת EDA, Baseline, ניסוי ולידציה, מודל שמור, בדיקות קלט, דוח מסכם והדגמה. לכל שלב נגדיר תנאי סיום. כך התלמיד יודע מתי עבודה הושלמה, והמורה יכול לזהות בעיות לפני שלב מאוחר שבו קשה לתקן אותן.

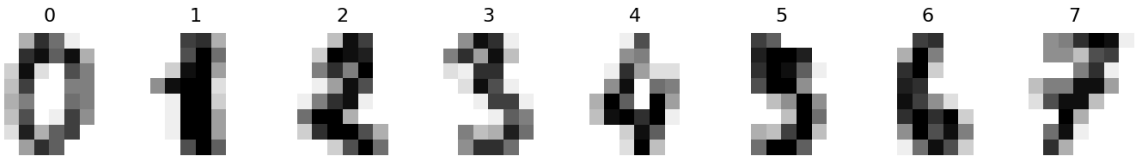


איור 2 — בכל מעבר שלב נדרש תוצר לבדיקה: נתון, קוד, מדד או הסבר.

14.5 — כרטיס נתונים (Data Card)

בכל פרויקט יש לתעד את מקור הנתונים, הרישוי, תאריך האיסוף, משמעות המאפיינים, אופן יצירת התוויות, נתונים חסרים, ייצוג קבוצות אפשרי וסיכונים. כרטיס כזה אינו בירוקרטיה: בלעדיו קשה לדעת אם אפשר להשתמש בנתונים באופן חוקי, האם מותר לפרסם אותם, והאם נתון מסוים יהיה זמין בזמן חיזוי אמיתי.

בדוגמת הספר נשתמש במאגר digits שמגיע עם scikit-learn: 1,797 תמונות, כל תמונה 8×8 פיקסלים בגווני אפור, והערכים במאגר נעים בין 0 ל־16. כל תמונה מיוצגת גם כווקטור באורך 64. המטרה היא ספרה מסומנת. מאגר זה מתאים להדגמת תהליך מבלי לאסוף מידע אישי על תלמידים.

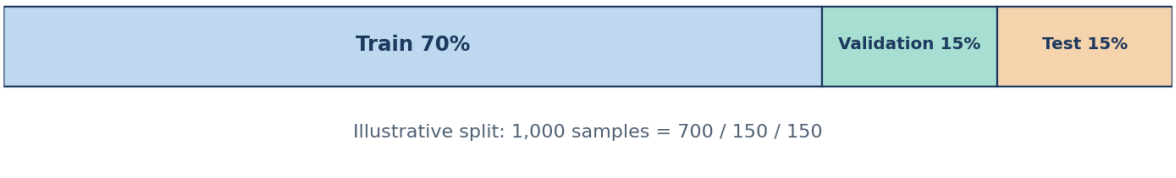


איור 3 — דוגמאות מתוך מאגר *digits* של *scikit-learn*. מספר מעל כל תמונה הוא התווית.

אם נרצה להחליף בהמשך את המאגר בצילומים מהטלפון, אי אפשר להניח שהמודל יעבוד באותה איכות. צילום כזה עשוי להוסיף צבע, צל, סיבוב ורקע, וקיים פער בין תחום נתוני האימון לבין הנתונים החדשים.

14.6 — הפרדת Train, Validation ו־Test

קבוצת Train משמשת ללמידת פרמטרים. קבוצת Validation או קיפולי Cross-Validation משמשים לבחירת מודל והיפר־פרמטרים. קבוצת Test נועדה לאומדן סופי של הכללה אחרי שכל ההחלטות התקבלו. שימוש ב־Test כדי לבחור k ואחר כך דיווח על אותה תוצאה כאילו הייתה עצמאית הוא שגיאה מתודולוגית.



איור 4 — חלוקה מספרית להמחשה בלבד: מתוך 1,000 דוגמאות, 700 לאימון, 150 לוולידציה ו־150 לבדיקה.

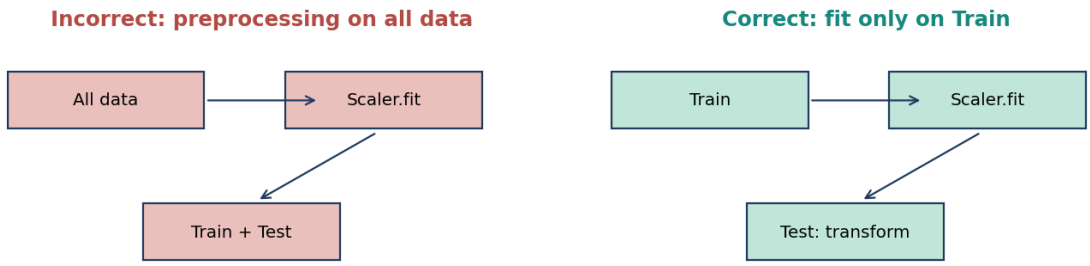
ב־digits נשתמש בדוגמה מודרכת בהפרדה חיצונית של 20% ל־Test, ולאחר מכן נבצע Cross-Validation רק על נתוני Train. נשמור random_state קבוע ונשתמש ב־stratify לשמירה בקירוב על חלקה של כל ספרה בחלוקה. במאגר זמנים או כשיש תמונות רבות מאותו אדם, חלוקה אקראית פשוטה עלולה להטעות; במקרה כזה יש לתכנן חלוקה לפי זמן או לפי קבוצות.

$$1{,}000 \times 0.70 = 700 \;;\; 1{,}000 \times 0.15 = 150 \;;\; 1{,}000 \times 0.15 = 150$$

14.7 — מניעת זליגת מידע כתנאי לאמינות

דוגמה לשגיאה: התלמיד מבצע תקנון על כל המאגר ורק אחר כך מחלק ל־Train ול־Test. כך אפילו ממוצע פשוט חושף לתהליך האימון מידע מנתוני הבדיקה. פתרון נכון: מחלקים קודם, וכל שלב שלומד

נתונים — למשל StandardScaler, בחירת מאפיינים או השלמת חסרים — מותאם ל־Train בלבד. בתוך Cross-Validation, שלבי ההכנה צריכים להילמד מחדש בכל קיפול אימון.



איור 5 — משמאל: פעולה לא נכונה על כל הנתונים; מימין: התאמה לנתוני האימון ושימוש ב־*transform* לנתונים חדשים.

Pipeline מסייע למנוע שגיאה זו משום שהוא כולל באותו אובייקט את שלבי ההכנה והמסווג. פעולה fit מתאימה את השלבים על נתוני האימון, ואילו predict מפעילה על קלט חדש את אותם שלבים בסדר הנכון. Pipeline אינה מתקנת אוטומטית כל דליפה, למשל תווית עתידית שהכנסנו לתכונה.

14.8 — ארגון תיקיות: מחברת אינה כל הפרויקט

כשכל הקוד מצוי במחברת אחת קשה לנהל שינוי, לבדוק רכיב בנפרד או להציג למפתח אחר מה צריך להריץ. לכן נפריד בין נתונים, מחברות חקר, קוד משותף, מודלים שמורים, בדיקות ותיעוד. מבנה מוצע לפרויקט תלמיד:

```
handwritten-digit-project/
    README.md
    requirements.txt
    .gitignore
    data/
        README.md
    notebooks/
        01_exploration.ipynb
        02_experiments.ipynb
    src/
        train.py
        predict.py
    models/
        model_info.json
    reports/
        experiment_log.csv
        model_card.md
    tests/
        test_prediction.py
```

העיקרון חשוב יותר מהשמות: תלמיד אחר צריך לדעת מה הקלטים, מה פותחים קודם, אילו קבצים נדרשים ומהו התוצר. README טוב יכלול את שאלת הפרויקט, המקורות, הוראות התקנה, שיטת הערכה ומגבלות ידועות.

14.9 — ניהול גרסאות עם Git

Git עוקב אחר שינויים בקוד ובתיעוד. כל Commit הוא צילום מצב מוסבר של העבודה. אם ניסוי חדש נשבר, אפשר להבין אילו קבצים השתנו. רצוי לכתוב הודעות Commit בעלות משמעות, כגון "Add train-test split" או "Document prediction input validation", במקום "final final 3".

אין להעלות אוטומטית נתונים אישיים, סיסמאות, מפתחות API וקובצי מודל גדולים ל־Git. ב־gitignore. נציין קבצים זמניים ורגישים. כאשר נדרשת גרסת נתונים גדולה, אפשר ללמוד על כלים כגון DVC כהרחבה — אך אין חובה להתקינם בפרויקט הראשון.

```
# .gitignore - example
__pycache__/
.ipynb_checkpoints/
.env
*.joblib
private_data/
```

14.10 — סביבת עבודה ושחזור (Reproducibility)

ניסוי משוחזר הוא ניסוי שאדם אחר מסוגל להריץ לפי אותן הוראות ולקבל תוצאה עקבית במידה סבירה. כדי לשחזר, יש לשמור את גרסאות הספריות, גרסת המאגר, קוד התהליך, פרמטרים שנבחרו וזרעים אקראיים. random_state מסייע לשלוט בחלק מהאקראיות, אך הוא לא מבטיח שתוצאה תהיה זהה על כל חומרה, גרסת ספרייה או מימוש מקבילי.

requirements.txt־ב מומלץ לתעד גרסאות שתואמות להרצה שנבדקה. אין להעתיק מספרי גרסאות באופן שרירותי; אפשר לייצר רשימת ספריות מתוך הסביבה הפעילה ולסנן אותה לתלויות הנדרשות. רצוי גם לציין גרסת Python ותאריך בדיקה.

14.11 — מודל בסיס (Baseline) לפני סבך של ניסויים

לפני KNN או CNN נבדוק מה משיגה שיטה פשוטה שאינה לומדת קשר בין פיקסלים לספרה, למשל תחזית של הספרה השכיחה ביותר ב־Train. אם יש עשר מחלקות בכמויות דומות, נקבל בערך עשירית מהתחזיות נכונות; התוצאה המדויקת תלויה בחלוקה. בלי Baseline אי אפשר לדעת אם המודל שלנו מוסיף ערך משמעותי.

לפעמים פתרון המבוסס על כללים פשוטים מתאים יותר מפרויקט מסובך. לכן על התלמיד להסביר לא רק איזה מודל טוב יותר, אלא גם מהו מחיר התחזוקה שלו, כיצד נבדק ומהן מגבלותיו.

14.12 — תהליך אימון מודרך ב־Python

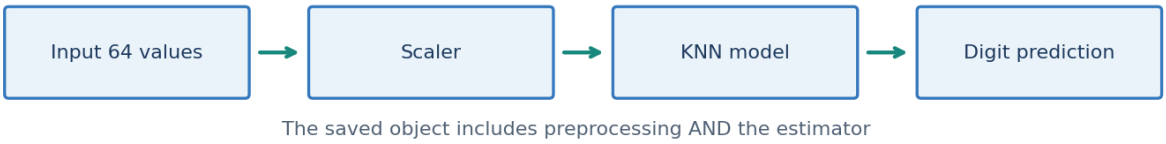
נבצע ניסוי בסיסי על digits. בחרנו להשתמש ב־StandardScaler וב־KNN בתוך Pipeline. הקוד שלהלן ניתן להרצה לפי הסדר; בהמשך הפרק נתרגל הפרדה בין קובץ אימון לקובץ חיזוי. אין לפרש את התוצאה כיכולת לזהות כל תמונה בעולם האמיתי.

```
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, f1_score


digits = load_digits()
X, y = digits.data, digits.target
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, stratify=y, random_state=42
)
model = make_pipeline(
    StandardScaler(), KNeighborsClassifier(n_neighbors=3)
)
model.fit(X_train, y_train)
```

```
pred_train = model.predict(X_train)
print('Train Accuracy:', accuracy_score(y_train, pred_train))
print('Train Macro F1:', f1_score(y_train, pred_train, average='macro'))
```

זהו ניסוי הדגמה לקוד, ולא פרוטוקול בחירת מודל סופי: לבחירת מספר השכנים נשתמש ב־Cross-Validation על Train בלבד, ורק אחריה נמדוד ביצועים סופיים על Test שמור.

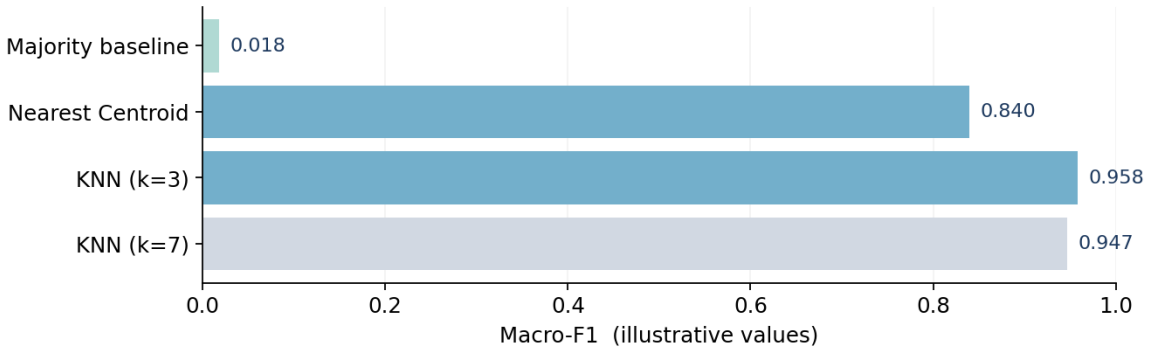


איור 6 — ה־*Pipeline* משמר את אותו עיבוד גם בזמן חיזוי של נתונים חדשים.

14.13 — יומן ניסויים: מה שינינו, ומה למדנו?

ניסוי טוב משנה גורם מתוכנן, מודד תוצאות ומסביר אותן. אם נחליף גם מאפיינים, גם מודל, גם שיטת חלוקה וגם מדד בבת אחת, לא נדע מה גרם לשינוי. לכן נתעד גרסת נתונים, מאפיינים, מודל, היפר־פרמטרים, שיטת ולידציה, מדדים, זמן ריצה ומסקנה.

ניסוי	מה נבדק	מדד CV	החלטה
E01	Baseline של מחלקת הרוב	המחשה — 0.018	נקודת ייחוס
E02	Nearest Centroid	המחשה — 0.840	השוואה ראשונית
E03	KNN עם k=3	המחשה — 0.958	מועמד מוביל
E04	KNN עם k=7	המחשה — 0.947	השוואה נוספת



איור 7 — ערכי הדגמה מומצאים להוראת יומן ניסויים; אינם תוצאות הרצה על *digits*.

בתוך מחברת התלמיד יתועדו תוצאות ההרצה האמיתיות. יש להבדיל בין מספר בדוגמה פדגוגית לבין מספר שהתקבל מריצה, ולשמור את שיטת המדידה כדי שנוכל לפרש פערים.

14.14 — ניתוח תוצאות ולא רק מספר אחד

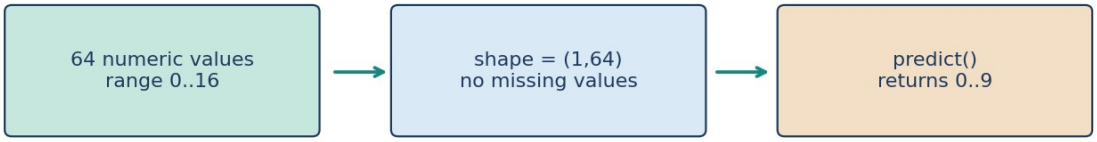
Accuracy שווה למספר התחזיות הנכונות מחולק במספר הבדיקות. אם מתוך 100 דוגמאות 92 סווגו נכון, נקבל 92/100 = 0.92. עם זאת, בסיווג כמה מחלקות חשוב לבדוק גם Macro F1, שמחשב את ממוצע ציוני F1 של המחלקות ללא שקלול לפי גודל המחלקה.

Accuracy = correct predictions / all predictions = 92 / 100 = 0.92

מטריצת בלבול מאפשרת לגלות אילו זוגות ספרות מתבלבלים. אם המודל טועה לעיתים בין 3 ל־8, ייתכן שהתמונות דומות או שמאגר האימון אינו מייצג דיו כתב יד מסוים. הממצא מחייב בדיקה של דוגמאות ולא הסקת סיבה חד־משמעית מתוך המטריצה לבדה.

14.15 — חוזה קלט ופלט (Inference Contract)

כשמעבירים מודל ממחברת למערכת, חובה להגדיר מה מותר להעביר אליו. לדוגמה: מערך של 64 מספרים בטווח 0 עד 16, ללא ערכים חסרים. לאחר הבדיקה נבנה שורה אחת במבנה (1,64) ונפעיל predict. התוצאה היא תווית ספרה ולא הסתברות. אם רוצים להציג הסתברות, נדרש לבדוק מה מחזיר המודל ואת מידת הכיול שלו; אין להפוך מרחק או הצבעת רוב אוטומטית ל"אחוז ביטחון" אמין.



Prediction service contract: validate before calling the model

איור 8 — חוזה הקלט והפלט מונע קריאות שגויות לפני שהן מגיעות למודל.

```
import numpy as np

def predict_digit(values, trained_model):
    arr = np.asarray(values, dtype=float)
    if arr.shape != (64,):
        raise ValueError('Expected exactly 64 values')
    if not np.isfinite(arr).all():
        raise ValueError('All values must be finite')
    if np.any(arr < 0) or np.any(arr > 16):
        raise ValueError('Pixel values must be in 0..16')
    return int(trained_model.predict(arr.reshape(1, -1))[0])
```

הקוד בודק את מבנה קלט הדוגמה, אך מערכת אמיתית זקוקה גם לטיפול מסודר בשגיאות ולהצגת הודעות מובנות למשתמש. שימו לב: תמונת טלפון אינה קלט תואם עד שתעבור תהליך עיבוד מוגדר ומאומת.

14.16 — שמירת מודל ושחזור תחזית

בסיום אימון נשמור את אובייקט ה־Pipeline, כדי שלא נשכח את שלב התקנון בעת הפעלת המודל. אפשר להשתמש ב־joblib עבור קובץ שנוצר על ידינו בסביבה מהימנה. חובה להזהיר כי טעינת קובצי joblib או pickle ממקור לא מהימן עלולה להפעיל קוד; אסור לטעון מודל אקראי שהגיע מאינטרנט או מאדם לא מוכר.

```
from pathlib import Path
import joblib

path = Path('digit_pipeline.joblib')
joblib.dump(model, path)       # save your own trusted model
restored = joblib.load(path)   # only from a trusted source
sample = X_train[0]
assert int(restored.predict([sample])[0]) == int(model.predict([sample])[0])
```

בנוסף למודל נשמור קובץ metadata בפורמט JSON ובו גרסת מודל, שם המאגר, רשימת המאפיינים, גרסאות תוכנה, תאריך ניסוי, שיטת בדיקה ומגבלות. קובץ JSON מכיל תיעוד, לא את המודל עצמו. אין להניח שקובץ מודל ישן יעבוד באופן נתמך אחרי שדרוג ספריות גדול.

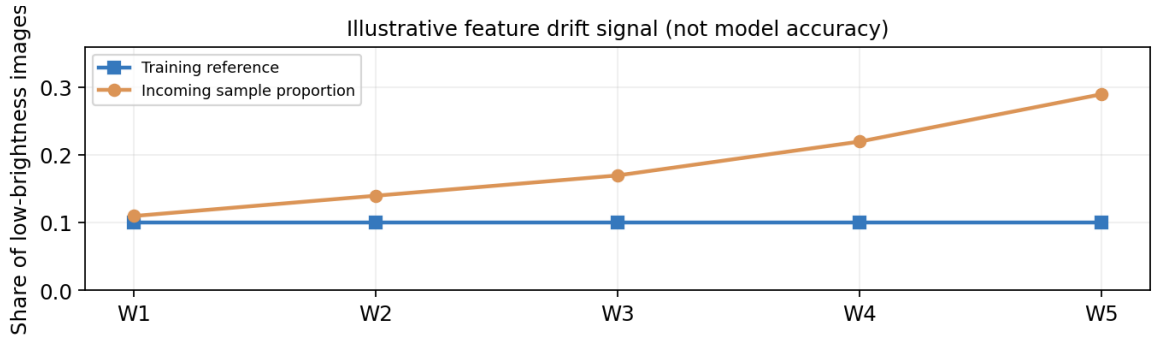
14.17 — בדיקות תוכנה: האם המערכת מתנהגת כמצופה?

בחינת מודל אינה זהה לבדיקת תוכנה. ייתכן שמודל משיג ציון מצוין אך היישום קורס אם משתמש שולח 63 ערכים במקום 64. נבנה בדיקות חיוביות ושליליות: קלט תקין, אורך שגוי, ערך חסר, ערך מחוץ לטווח, קובץ מודל חסר ואי־התאמה בין גרסאות. את הבדיקות נריץ מחדש לאחר שינוי בקוד.

בדיקה	קלט	תוצאה צפויה
חיזוי תקין	64 מספרים בטווח 0–16	ספרה בין 0 ל־9
צורה שגויה	63 מספרים	הודעת שגיאה מובנת
ערך לא חוקי	פיקסל שלילי	דחיית קלט
ערך חסר	NaN	דחיית קלט
שחזור	מודל שמור מהימן	אותה תחזית לדוגמת בדיקה

14.18 — ניטור, שינוי נתונים ותחזוקה

מודל שאומן על כתב יד מסוג אחד עלול לפעול אחרת כשהמשתמשים משתנים. יש להבחין בין שינוי בהתפלגות הקלט (Data Drift) לבין ירידה אמיתית באיכות (Performance Degradation). ניתן לזהות שינוי בסטטיסטיקות של הקלט גם לפני שיש לנו תוויות אמת, אך כדי למדוד Accuracy או F1 צריך תוויות אמת אמינות.



איור 9 — גרף מומצא של אות לשינוי נתונים, לא של איכות המודל. עלייה באות מחייבת בדיקה, לא מסקנה אוטומטית שהמודל כשל.

במערכת כיתתית אין צורך להקים תשתית ניטור עננית. מספיק להגדיר בדיקה תקופתית: האם טווחי הקלט השתנו? האם מופיעים ערכים חסרים? האם יש דוגמאות חדשות שעליהן המודל נכשל? כל שינוי במודל יחייב גרסה חדשה, תיעוד והערכה הוגנת. אין לשמור נתונים אישיים לצורך ניטור ללא בסיס ואישור מתאימים.

14.19 — פרטיות, רישוי ואחריות

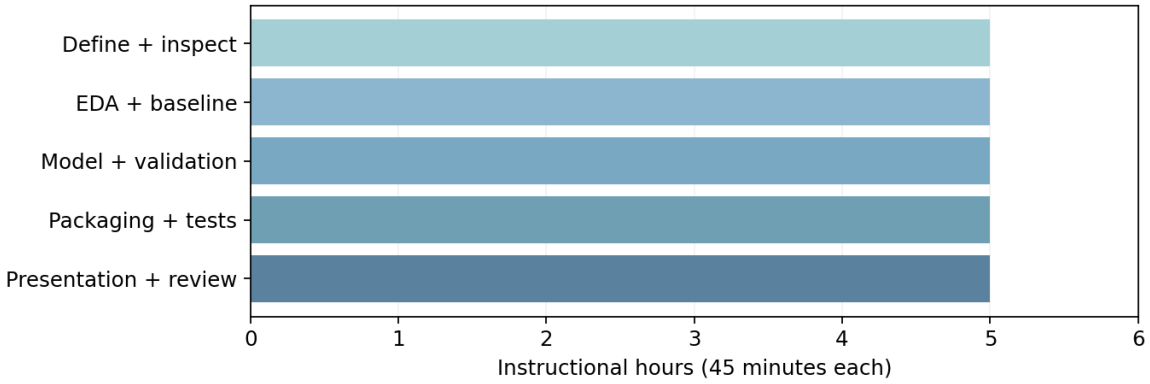
אם הפרויקט משתמש בתמונות של תלמידים, יש לשאול האם בכלל צריך לשמור אותן. אפשר לעבוד תחילה על מאגר ציבורי מתאים ולמנוע איסוף מיותר. כשהפרויקט נוגע לאנשים יש לבדוק ייצוג

אוכלוסיות, נזק אפשרי מטעות, זכויות שימוש ורישוי. אין להבטיח שהמערכת "נטולת הטיה" רק משום שהצליחה על Test אחד.

בפרויקט גמר יש לציין במפורש אם הנתונים מקוריים או פתוחים, מה מותר לעשות בהם, האם נדרש אישור צילום, אילו פרטים נמחקו או הוסוו ואילו מגבלות נשארו. תחזית מודל אינה החלטה מוסמכת בנושאים רפואיים, משפטיים או אחרים בעלי סיכון גבוה.

14.20 — תכנית עבודה מעשית ל־25 שעות

הטבלה שלהלן היא הצעת הוראה מקורית: 25 שעות לימוד לפי חלוקה לחמישה מקטעים בני חמש שעות (שעה לימודית היא 45 דקות). אפשר לפרוס את המקטעים לעשרה מפגשים קצרים יותר. בכל מקטע יש תוצר שמוגש ונבדק.



איור 10 — חלוקת שעות מוצעת, לא חלוקה רשמית של משרד החינוך.

שעות	משימות מובילות	תוצר להגשה
5–1	הגדרת בעיה, כרטיס נתונים, בדיקת שימושיות	Project Brief + Data Card
10–6	EDA, חלוקה, Baseline, הסברים לגרפים	מחברת EDA
15–11	Pipeline, CV, השוואת מודלים, ניתוח טעויות	יומן ניסויים
20–16	שמירת מודל, חוזה קלט, בדיקות, תיעוד	קובץ מודל + בדיקות
25–21	סקירת סיכונים, הצגת תוצר, שיפור והגנה	דוח ופרזנטציה

14.21 — חקר מקרה: האם המודל שבחרנו מתאים למשתמש?

נניח שמודל א׳ משיג Macro F1 של 0.95 אבל זמן חיזוי ממוצע של 400 מילישניות, בעוד מודל ב׳ משיג 0.94 וזמן חיזוי של 8 מילישניות. אין תשובה אוטומטית לשאלה מי טוב יותר. אם המערכת פועלת על טלפון חלש, זמן ההמתנה ומשאבי המכשיר עשויים להיות חשובים. אם טעות סיווג גוררת נזק גדול, הבדל המדד עשוי להיות משמעותי יותר. יש לבדוק שגם דרך המדידה זהה ושאותה קבוצת Test שימשה להשוואה לאחר סיום בחירת המודלים.

המספרים בסיפור זה מומצאים להמחשה. בשאלת הערכה, תלמיד צריך להציע בדיקה שתאפשר לבחור בצורה מקצועית ולא רק לבחור לפי תחושת בטן. אפשר לבקש מדד נוסף, בדיקת קבוצות משתמשים, ניסוי על נתונים חדשים או בדיקת זמן ריצה מדודה.

14.22 — תרגול מדורג בסגנון הערכה

רמה א׳ — הבנה ותיאור

משימה 1: הסבירו במילים שלכם את ההבדל בין תהליך אימון לבין תהליך חיזוי בסביבת משתמש.

משימה 2: ציינו ארבעה פרטים שחייבים להופיע בכרטיס נתונים, והסבירו מה עלול להשתבש כשאחד מהם חסר.

משימה 3: נתון פרויקט שבו כל הקוד נמצא במחברת אחת. הציעו חלוקה לתיקיות והסבירו את התועלת בכל תיקייה.

רמה ב׳ — חישוב, בדיקה והחלטה

משימה 4: מתוך 240 דוגמאות, המודל סיווג 216 נכון. חשבו Accuracy ופירשו את משמעות המדד והמגבלה שלו.

משימה 5: לפרויקט יש 1,000 דוגמאות. חלקו אותן ל־70% אימון, 15% ולידציה ו־15% בדיקה. הסבירו מתי מותר להשתמש בכל חלק.

משימה 6: תלמיד ביצע StandardScaler.fit על כל המאגר לפני החלוקה. הסבירו מדוע זה פוגע באמינות ואיך יש לתקן.

משימה 7: כתבו שלושה מקרי בדיקה לפונקציה שקולטת וקטור בן 64 ערכים. לפחות שניים צריכים להיות מקרי שגיאה.

רמה ג׳ — חקר והגנה על החלטות

משימה 8: בנו תכנית השוואה הוגנת בין KNN ל־Nearest Centroid על digits. פרטו Baseline, ולידציה ומדדי Test.

משימה 9: נסחו תכנית ניטור מצומצמת למערכת המזהה ספרות וכתבו כיצד תבחינו בין Data Drift לבין ירידה מוכחת באיכות.

משימה 10: הציעו שני סיכוני פרטיות או רישוי בפרויקט שמבוסס על תמונות של ילדים, והסבירו כיצד לצמצם אותם.

משימה 11: הגדירו קריטריון הצלחה מדיד לפרויקט אחר: זיהוי פגמים במוצרים או ניבוי זמני משלוח. נמקו בחירת מדד.

משימה 12: כתבו פסקת הגנה: מה הייתם משנים במערכת אם עלתה איכות האימון אבל איכות הבדיקה לא השתפרה?

14.23 — הכנה להסבר בעל־פה ולפרויקט גמר

בעת הגנה על פרויקט אין די במשפט "הרצתי מודל וקיבלתי 96%". יש לתאר את שאלת המחקר, מקור הנתונים, מה הוזן למודל, כיצד נבחרו הפרמטרים, אילו מדדים נמדדו, כיצד נמנעה זליגת מידע ומהן המגבלות. על התלמיד להיות מסוגל להדגים שינוי קטן בקוד, להציג בדיקה ולהסביר את תוצאתה.

דוגמאות לשאלות בוחן: מדוע בחרתם דווקא בנתונים האלה? כיצד אתם יודעים שמודלכם לא ראה את Test בזמן האימון? האם תוכלו לשחזר ניסוי ישן? מה קורה אם המשתמש שולח תמונה בפורמט אחר? האם שמירת מודל מבטיחה שהוא יפעל במחשב אחר? מה תעשו אם התפלגות הנתונים תשתנה?

מחוון כיתתי מוצע לפרק — שאינו מחוון משרד החינוך: הגדרת בעיה ונתונים 20 נקודות; תהליך ניסוי מבוקר 25; בדיקות וקוד מסודר 20; תיעוד ושחזור 15; ניתוח מגבלות ופרטיות 10; הצגה והגנה בעל־פה 10.

14.24 — מפת ההקשר בין שלוש תכניות הלימוד

תפקיד בהמשך	ידע שנדרש או נלמד	מסלול
בסיס לניתוח ולהסבר מודל	Pandas, גרפים, KNN ומדדי הצלחה	יחידה 3 — כיתה י׳
ביסוס מתמטי והערכה מדעית	מרחקים, K-Means, הערכה וולידציה	יחידה 5 — נושאי ליבה
הכנה ישירה לתוצר תוכנה ולעבודת גמר	ארגון קוד ונתונים, שמירת מודל, בדיקות ויישום	למידה עמוקה — התמחות

הקשר בין המסלולים הוא מיפוי דידקטי שלנו. כל דרישה פורמלית נבחנת בהתאם לתכנית ולמחוון הרלוונטיים, ואין להעתיק אוטומטית עומק של מסלול אחד לאחר.

14.25 — סיכום הפרק: מה הופך ניסוי לפרויקט מקצועי?

בשלב זה התלמיד כבר יודע לא רק לאמן מודל, אלא לחשוב כמו מפתח AI: להגדיר בעיה, לבחור נתונים, לתעד ניסויים, למנוע זליגה, לשמר את תהליך העיבוד, להגדיר חוזה קלט ולבדוק מקרים צפויים וחריגים. פרויקט איכותי אינו מדד יחיד ואינו רק Notebook ארוך; הוא רצף החלטות שאפשר לשחזר ולנמק.

בפרק 15 נלמד רשתות חוזרות (RNN). בהמשך, בפרק 16, נשתמש בכל תהליכי הניהול, התיעוד והבדיקה שנלמדו כאן לצורך עבודת גמר המבוססת על מודל למידה עמוקה ועל יישום תוכנה עצמאי.

נספח א׳ — דף תכנון פרויקט למילוי

זהו טופס עבודה לתלמיד. אין צורך למלא את כל הסעיפים לפני המפגש הראשון: יש לחזור אליו לאחר כל ניסוי ולתקן החלטות לפי הראיות. בכל שדה יש לכתוב תשובה קונקרטית שניתן לבדוק ולא סיסמה כללית.

מה נכתוב בפרויקט שלנו	השאלה שעלינו לתעד
_____	איזו בעיה נפתור, עבור מי ולמה?
_____	מה מקור הנתונים ומה תנאי השימוש?
_____	מהו מאפיין ומהי תווית המטרה?
_____	כיצד נחלק ל־Train, Validation ו־Test?
_____	מהו קו הבסיס ומה המדד הראשי?
_____	איזה ניסוי נבצע ראשון ומדוע?

מה נכתוב בפרויקט שלנו	השאלה שעלינו לתעד
_____	כיצד נבדוק קלט חדש למודל?
_____	מהם סיכון אחד ומגבלה אחת?

נספח ב׳ — יומן ניסויים למילוי

רשמו כל הרצה כראיה הניתנת לשחזור. אין למחוק תוצאה חלשה: גם כישלון מתועד עוזר לבחור ניסוי המשך. הקפידו לא לערבב ציוני Train, CV ו־Test באותה עמודה ללא ציון סוג המדידה.

מזהה ניסוי	מה שיניתי?	מדד + סוג בדיקה	מה הסקתי?
E01	_____	_____	_____
E02	_____	_____	_____
E03	_____	_____	_____
E04	_____	_____	_____

נספח ג׳ — רשימת בדיקה לפני הדגמה לבוחן

לפני ההגשה, הריצו את המחברת מההתחלה בסביבה נקייה וודאו שכל התאים החיוניים עובדים לפי הסדר. פתחו README במחשב אחר או בסביבה חדשה ובדקו שאפשר להבין כיצד מתקינים ומפעילים. הציגו דוגמה שהמודל מסווג נכון ודוגמה שנכשל בה, עם ניתוח ולא עם הסתרה. בדקו שניתן לשחזר חיזוי ממודל שמור ושבדיקת קלט לא חוקי מחזירה שגיאה מובנת.

מומלץ לשמור בפרויקט תמונת מסך של גרף בקרה, קובץ מדדים ברור, פירוט גרסאות ספריות, כרטיס נתונים וכרטיס מודל. אם הקובץ מכיל מידע רגיש יש להסירו לפני שיתוף. על כל תלמיד להיות מסוגל להצביע על קוד שחיבר בעצמו ולהסביר את ההיגיון שלו גם ללא עזרת כלי AI.

נספח ד׳ — כרטיס מודל קצר להגשה

כרטיס מודל הוא דף תיעוד שמסביר איזה מודל נבנה, לאיזו מטרה, על אילו נתונים, כיצד הוערך ומתי אסור להסתמך עליו. זהו כלי תיעוד מקצועי שמאפשר למישהו אחר להבין את ההחלטות שלכם, ולא קובץ שנוצר אוטומטית על ידי התוכנה.

שדה בכרטיס המודל	דוגמה שמראה מה נדרש
שם וגרסה	DigitClassifier, version 1.0
מטרת המודל	סיווג ספרה בודדת בתמונה 8×8
מקור נתונים	sklearn digits + תנאי שימוש
קלט / פלט	64 ערכי פיקסל 0–16 / ספרה 0–9
פרוטוקול בדיקה	Train 80% + CV פנימי, Test חיצוני 20%

דוגמה שמראה מה נדרש	שדה בכרטיס המודל
Accuracy, Macro F1 ומטריצת בלבול	מדדי איכות
לא נבדק על תמונות טלפון ברזולוציה שונה	מגבלות חשובות
גרסת Python ו־scikit-learn, קובץ Pipeline מהימן	גרסאות וקבצים

בוחן יכול לשאול: האם זו מערכת לזיהוי ספרות באופן כללי? התשובה המדויקת היא שהיא נבדקה על נתונים הדומים ל־digits; טענה רחבה יותר דורשת בדיקה עצמאית על אוכלוסיית תמונות חדשה. זהו ההבדל בין טענה מגובה לבין הבטחה שאיננה מבוססת ניסוי.

מקורות לקריאה ולהעמקה

עוגן מסלול: מאגר הקורס — פרק 14, 25 שעות (מפת התכנית; דורש הצלבה מול הסילבוס הרשמי המלא): https://github.com/samiabo75/Data-Science-ML-Course/tree/main/03-Deep-Learning-5-Units/14-Data-Science-Project-Management

ארגון ניסויים, אימון והערכה. — Géron, Hands-On Machine Learning with Scikit-Learn and PyTorch — James et al., An Introduction to Statistical Learning with Applications in Python — Zhang et al., Dive into Deep Learning. ניסויים ויישום. דוגמאות בחירת מודלים והערכת ביצועים. והסברים בפרק זה מקוריים.

scikit-learn — Common Pitfalls and Recommended Practices: https://scikit-learn.org/stable/common_pitfalls.html

scikit-learn — Model Persistence: https://scikit-learn.org/stable/model_persistence.html

Google for Developers — Rules of Machine Learning: https://developers.google.com/machine-learning/guides/rules-of-ml

Google for Developers — Productionization: https://developers.google.com/machine-learning/managing-ml-projects/production

DVC — User Guide (העשרה בלבד): https://doc.dvc.org/user-guide

הערת מקור: הספר אינו תרגום של ספר מסחרי. ההסברים, הדוגמאות, השאלות והתרשימים נוצרו לצורכי הוראה. אין לייחס לכלי כגון DVC, Git או joblib מעמד של דרישת חובה רשמית ללא אימות מול התכנית.

פרק 15 | רשתות עצביות חוזרות (RNN)

מהרצף והמצב החבוי ועד לאימון **RNN, GRU** ו־**LSTM** — הבנה מתמטית, קוד ופרויקט גמר

מסלול התמחות למידת מכונה ולמידה עמוקה | 5 יחידות לימוד | 30 שעות לפי מפת התכנית במאגר GitHub

גרסת תלמיד מורחבת • המחשות מקוריות • פתרונות הדוגמאות המודרכות בלבד • מעבדה עצמאית ללא פתרונות



אותם סוגי מדידות, אבל הסדר בזמן משנה את משמעותן.

מה נלמד ומדוע?

עד כה למדנו מודלים שמקבלים תמונה, טבלה או וקטור ומתייחסים אליהם כאל דוגמה אחת. בפרק זה נפגוש נתונים שהסדר שלהם חיוני: קריאות חיישן, מילים במשפט, תנועת אצבע על מסך ונתוני מכירות לאורך ימים. נלמד כיצד רשת עצבית יכולה לשמור סיכום פנימי של העבר ולעדכן אותו בכל צעד זמן.

הפרק נשען על הידע מהפרקים הקודמים: וקטורים ומטריצות, פונקציות הפעלה, נגזרות, גרדיאנטים, Backpropagation והערכת מודלים. כאן נוסיף את ממד הזמן ואת הבעיה של תלות בין צעדים רבים.

מטרות ביצועיות

- לזהות סדרה, צעד זמן, חלון קלט, אופק חיזוי ותלות בזמן; להבחין בין Sequence-to-One לבין Sequence-to-Sequence.
- לחשב ידנית שלושה מצבים חבויים, להסביר את השימוש החוזר במשקולות, ולזהות את ממדי המערכים.
- להסביר כיצד BPTT גוזר דרך זמן, לחשב עדכון קטן ולהבחין בין גרדיאנט נעלם לגרדיאנט מתפוצץ.
- לתאר את תפקידי השערים ב־LSTM וב־GRU, ולבחור מודל לפי בעיה וניסוי ולא לפי שם מרשים.
- לבנות ניסוי חיזוי סדרת זמן עם הפרדה כרונולוגית, קו בסיס, אימון, הערכה ותיעוד לקראת פרויקט גמר.

היכן הפרק משתלב בשלוש התכניות?

הקשר לפרויקט	ידע והיקף ההוראה	מסלול
פירוש סדרת מדידות והצעת חקר	קריאת טבלאות וגרפים, חלוקה ל־Train/Test ומודלים בסיסיים	**יחידה 3 — כיתה י׳**
הסבר מתמטי של קלט וגרדיאנט; אין לייחס RNN כחובת יחידה 5	וקטורים, תקנון, הערכה ולידציה כידע קודם	**יחידה 5 — נושאי ליבה**
בניית מסווג או חזאי סדרתי, תיעוד והגנה	RNN — פרק התמחות בהיקף 30 שעות במפת המאגר	**התמחות DL — פרק 15**

שימו לב: ה־GitHub. פירוש התוכן 30 שעות והשיוך לפרק 15 אומתו מול מפת תכנית ההתמחות ב־PDF. הרשמי התקף עדיין נדרש. והחלוקה הפנימית כאן הם עיבוד דידקטי; אימות מלא מול נוסח ה־

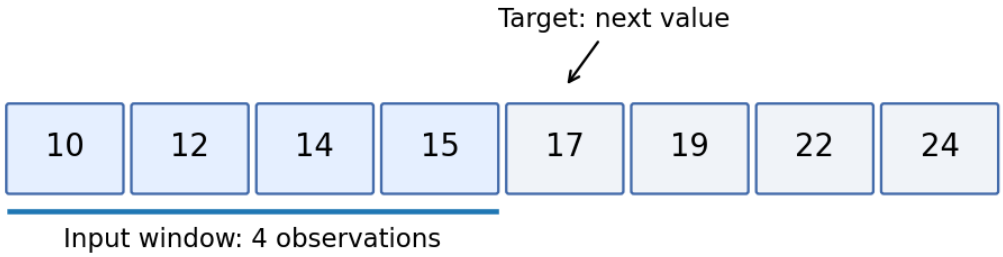
15.1 | מדוע הסדר חשוב? מסיפור על חיישן לשאלת חקר

דודו מנהל מעבדת מחשבים בבית הספר. בכל דקה חיישן מודד את טמפרטורת אחד המחשבים. אם נקבל את המדידות 28, 31, 34, 37 ו־31 מעלות, נחשוד בהתחממות מתמשכת. אך אם אותן מדידות מופיעות בסדר הפוך — 37, 34, 31, 28 — המחשב דווקא מתקרר. הממוצע, המינימום והמקסימום זהים, אולם הסיפור על התהליך שונה לחלוטין.

בנתון טבלאי רגיל, סדר השורות אינו בהכרח משנה את הדוגמה. בסדרה, כל מדידה תלויה במיקום שלה בזמן ובמה שקדם לה. לכן עלינו לשמר את הסדר, ולהימנע משימוש במידע מן העתיד כאשר אנו מבקשים לחזות אותו.

מושגים חדשים

מושג	פירוש	דוגמה
סדרה — Sequence	רצף תצפיות בעלות סדר משמעותי	קריאות חיישן על פני 20 דקות
צעד זמן — Time step	עמדה אחת ברצף, לא בהכרח שנייה אחת	דקה 5 ברצף
חלון — Window	מספר מדידות עוקבות המשמשות קלט	12 הדקות האחרונות
אופק חיזוי — Horizon	כמה צעדים קדימה חוזים	הטמפרטורה בדקה הבאה
Sequence-to-One	רצף קלט לתחזית אחת	האם תופיע תקלה?
Sequence-to-Sequence	פלט לכל צעד או רצף פלט	תווית לכל מילה במשפט



חלון של ארבע מדידות משמש לחיזוי המדידה הבאה. יש לשמור על סדר זמנים.

15.2 | כיצד הופכים סדרת מדידות לדוגמאות אימון?

נניח שקיבלנו שמונה מדידות ברצף. נבחר חלון באורך ארבע. דוגמת האימון הראשונה תכלול את המדידות במקומות 1 – 4 ותחזית היעד שלה תהיה המדידה במקום 5 . הדוגמה השנייה תכלול את המקומות 2 – 5 , והיעד יהיה המקום 6 . כך אנו יוצרים מאותו רצף מספר דוגמאות מבלי לשבור את סדר הזמן.

$$X_1 = [10, 12, 14, 15] \rightarrow y_1 = 17$$

$$X_2 = [12, 14, 15, 17] \rightarrow y_2 = 19$$

חלונות עוקבים חופפים זה לזה. עובדה זו חשובה מאוד בעת החלוקה לקבוצות אימון ובדיקה: אם נערבב חלונות שכנים בין הקבוצות, נתוני הבדיקה עלולים להיות דומים מדי לנתוני האימון ואף לשתף את רוב המדידות. הדרך הבטוחה בדוגמתנו היא לחלק קודם את הרצף לפי זמן, ואז ליצור חלונות בכל קבוצה בנפרד.

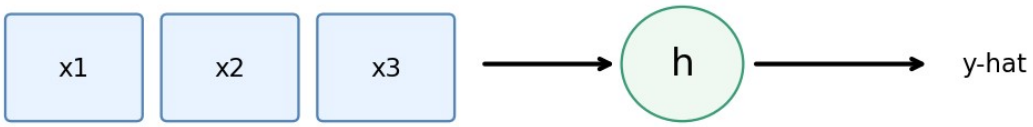
```
def create_windows(values, window=4):
    X, y = [], []
    for end in range(window, len(values)):
        X.append(values[end-window:end])
        y.append(values[end])
    return np.array(X), np.array(y)

values = np.array([10, 12, 14, 15, 17, 19, 22, 24])
X, y = create_windows(values, window=4)
print(X.shape, y.shape)   # (4, 4) (4,)
```

נסו להסביר מדוע מספר החלונות הוא מספר המדידות פחות אורך החלון, כאשר מנבאים את הערך הבא בצעד אחד. מה ישתנה אם נרצה לחזות שני צעדים קדימה?

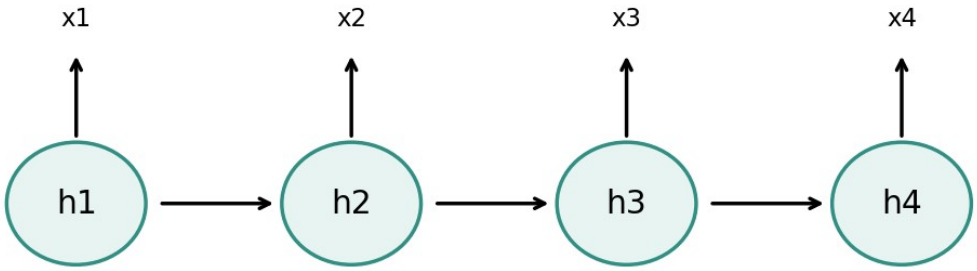
15.3 | מרשת רגילה לרשת עם זיכרון

ברשת Feedforward רגילה החישוב מתקדם מן הקלט לפלט, בלי שמצב פנימי עובר מדוגמה אחת לבאה. ברשת חוזרת קיימת יחידה שמקבלת בכל זמן גם קלט חדש וגם מצב חבוי מן הצעד הקודם. המצב החבוי אינו זיכרון מילולי כמו אצל אדם; זהו וקטור מספרי שנלמד לשמש סיכום שימושי של הרצף שנצפה עד כה.



Sequence model summarizes the ordered inputs

משימת *Sequence-to-One*: כל מדידות החלון מובילות לתחזית אחת.



The same W_x and W_h are reused at each time step

פריסה לאורך זמן: אותו תא *RNN* מופעל כמה פעמים עם אותן משקולות.

הנוסחה הכללית

$$h_t = \tanh(W_x\, x_t + W_h\, h_{t-1} + b_h)$$

$$\hat{y}_t = W_y\, h_t + b_y$$

לנוסחה יש שני חלקים. המכפלה הראשונה מודדת את תרומת הקלט הנוכחי; המכפלה השנייה מודדת את תרומת המצב הקודם; ההטיה מאפשרת הזזה של סכום התרומות. פונקציית tanh מוסיפה אי־ליניאריות ומחזירה ערכים בתחום שבין -1 לבין 1. אותם W_x, W_h ו־b_h משמשים בכל צעדי הזמן — זהו שיתוף משקולות לאורך הזמן.

שימו לב: שימוש באותן משקולות אינו אומר שאותו מצב חבוי מתקבל בכל פעם. הקלט והמצב הקודם משתנים, ולכן גם התוצאה משתנה.

15.4 | דוגמה פתורה: שלושה צעדים וחישוב כל מצב חבוי

כדי שנוכל לחשב ביד, נשתמש במצב חבוי מספרי יחיד. במערכת אמיתית הוא בדרך כלל וקטור בעל כמה רכיבים. נבחר את הפרמטרים הבאים:

$$h_0 = 0, \quad W_x = 0.5, \quad W_h = 0.4, \quad b_h = 0.1$$

$$x_1 = 0.2, \quad x_2 = 0.6, \quad x_3 = 0.4$$

בצעד הראשון אין זיכרון קודם ולכן h_0 שווה אפס. מחשבים את הסכום המשוקלל, מפעילים tanh ושומרים את התוצאה. בצעד השני משתמשים ב־h_1 שחושב בפועל; אין להתחיל בכל צעד מהתחלה.

$$z_1 = 0.5 \cdot 0.2 + 0.4 \cdot 0 + 0.1 = 0.2000$$

$$h_1 = \tanh(0.2000) \approx 0.1974$$

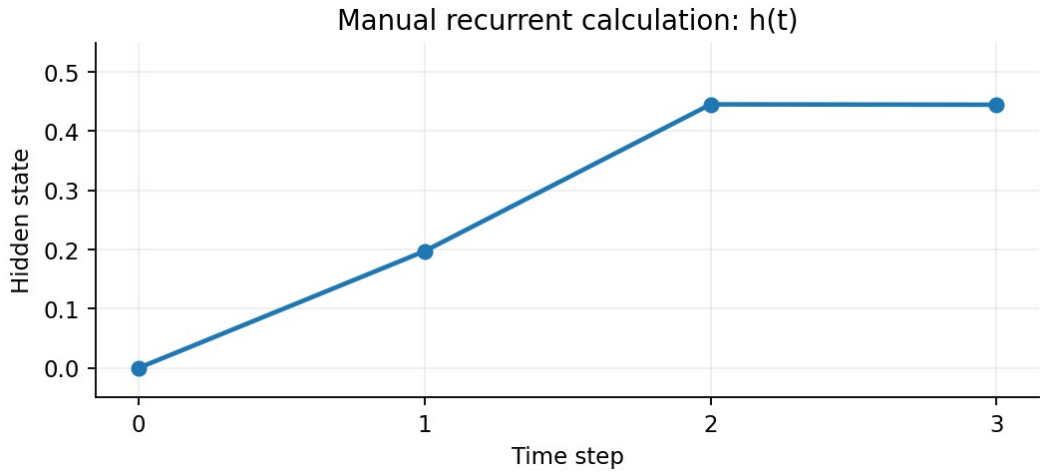
$$z_2 = 0.5 \cdot 0.6 + 0.4 \cdot 0.1974 + 0.1 \approx 0.4790$$

$$h_2 = \tanh(0.4790) \approx 0.4454$$

$$z_3 = 0.5 \cdot 0.4 + 0.4 \cdot 0.4454 + 0.1 \approx 0.4782$$

$$h_3 = \tanh(0.4782) \approx 0.4448$$

צעד	x_t	z_t לפני tanh	h_t אחרי tanh
1	0.2000	0.2000	0.1974
2	0.6000	0.4790	0.4454
3	0.4000	0.4782	0.4448



המצב החבוי משתנה בהדרגה בעקבות הקלט החדש והמצב הקודם.

נוסיף שכבת פלט ליניארית לחיזוי ערך מספרי. נגדיר משקל פלט, הטיה וערך מטרה, ונכתוב אותם בנפרד כדי למנוע בלבול בין הסימונים:

$$W_y = 0.8, \quad b_y = 0.05, \quad y = 0.35$$

$$\hat{y} = 0.8 \cdot h_3 + 0.05 \approx 0.4058$$

$$L = \tfrac{1}{2}(\hat{y} - 0.35)^2 \approx 0.00156$$

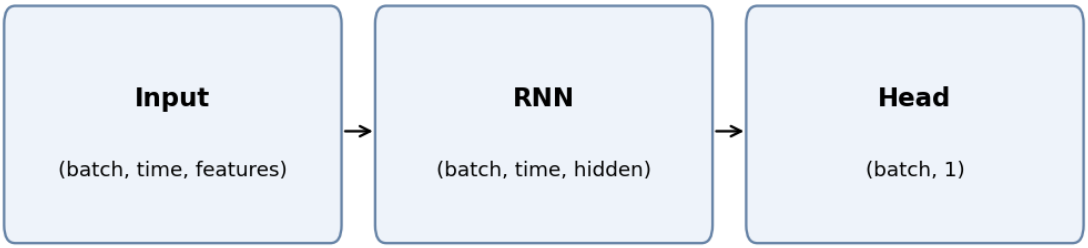
למדנו לבצע Forward Pass שלם. אך בשלב זה המשקולות עדיין קבועות. כדי שהרשת תלמד, נצטרך להבין כיצד שינוי במשקל בכל אחד מצעדי הזמן עשוי לשנות את השגיאה הסופית.

15.5 | ממדים וצורות מערכים — איך המחשב מארגן סדרה?

כאשר מאמנים רשת על כמה סדרות יחד, הקלט אינו וקטור יחיד ואינו מטריצה רגילה של תצפיות. הוא טנזור בשלושה ממדים: מספר דוגמאות באצווה, מספר צעדי זמן, ומספר המאפיינים בכל צעד. כאשר כל רצף מכיל שתים־עשרה דקות ובכל דקה נמדדים שני מאפיינים, דוגמה אחת היא מטריצה ואצווה של דוגמאות היא טנזור. הצורות המדויקות מופיעות בנוסחאות להלן.

$$\texttt{X.shape} = (\text{batch}, \text{sequence_length}, \text{input_size})$$

$$\text{One sample: } (12, 2) \qquad \text{Batch of 32: } (32, 12, 2)$$



כאשר *batch_first=True*, הקלט והשכבות פועלים בסדר (*batch, time, features*).

```
import torch
from torch import nn

X_batch = torch.randn(32, 12, 2)
rnn = nn.RNN(input_size=2, hidden_size=8, batch_first=True)
outputs, last_hidden = rnn(X_batch)
print(outputs.shape)       # (32, 12, 8)
print(last_hidden.shape)   # (1, 32, 8)
```

ממד `8` מציין את מספר הרכיבים במצב החבוי, ולא את מספר המחלקות. הממד הראשון של last_hidden מציין מספר שכבות כפול מספר הכיוונים; כאשר זו שכבה יחידה חד־כיוונית הוא שווה ל־ `1` . תכונת batch_first משנה את צורת הקלט והפלט הסדרתי, אך לא את סדר ממדי המצב החבוי.

15.6 | מהי למידה לאורך זמן? Backpropagation Through Time

ברשת רגילה עקבנו אחרי שגיאה מן הפלט לאחור דרך שכבות. ברשת חוזרת אותו פרמטר מופיע שוב ושוב בצעדי זמן שונים. לכן כאשר מחשבים נגזרת של השגיאה לפי משקל חוזר, יש להביא בחשבון את כל הדרכים שבהן המשקל השפיע על הפלט דרך הצעדים הקודמים. פעולת גזירה זו נקראת Backpropagation Through Time, בקיצור BPTT.

אין צורך לדמיין שיש לנו משקולות חדשות לכל צעד: בפריסה החזותית אנו מציירים עותקים של אותו תא לצורך הבנה, אך המשקולות משותפות. הגרדיאנטים מכל הופעותיהן מצטברים אל אותו פרמטר.

חישוב ידני מפושט של BPTT

כדי לחשב בלי נגזרות ארוכות של tanh, נשתמש בדוגמה לימודית עם הפעלה זהותית. זהו מודל ליניארי פשוט להמחשת כלל השרשרת, לא מודל מומלץ לאימון אמיתי.

$$h_1 = 0.5, \qquad h_2 = w \cdot h_1, \qquad h_3 = 0.5 + w \cdot h_2$$

$$\hat{y} = h_3 = 0.5 + 0.5w^2, \qquad L(w) = \tfrac{1}{2}(\hat{y} - 1)^2$$

נבחר משקל חוזר התחלתי $w=0.2$. נחשב קדימה: $h_2=0.1$, תחזית 0.52 והשגיאה 0.1152. כעת נגזור לאחור. הנגזרת של השגיאה לפי התחזית היא $\hat{y}-1$, והנגזרת של התחזית לפי w היא w. מכפילים אותן לפי כלל השרשרת.

$$dL/dw = (\hat{y} - 1) \cdot w = (0.52 - 1) \cdot 0.2 = -0.096$$

$$w_{new} = w - \eta(dL/dw) = 0.2 - 0.5(-0.096) = 0.248$$

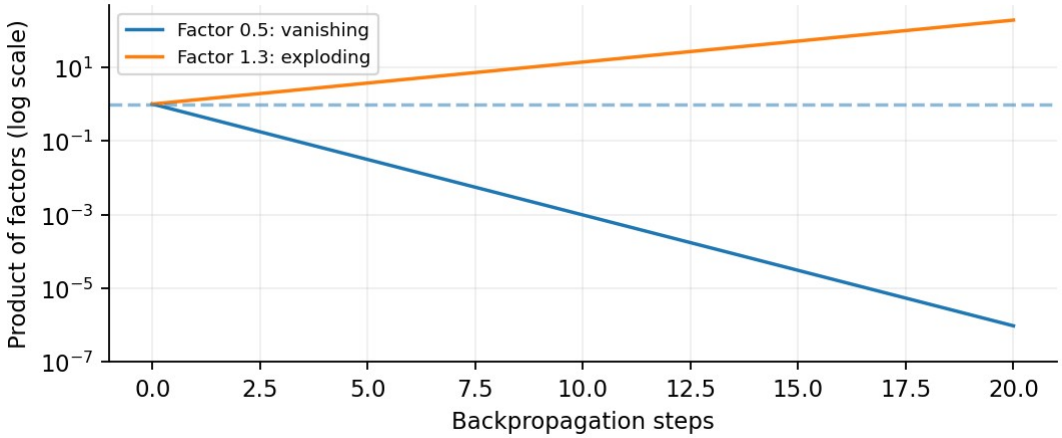
$$\hat{y}_{new} = 0.5 + 0.5 \cdot (0.248)^2 = 0.530752$$

$$L_{new} \approx 0.11010 < 0.11520$$

החישוב מלמד שעדכון פרמטר חוזר משפיע על כמה צעדי זמן יחד. הירידה בשגיאה תקפה בדוגמה ולצעד הזה; אין הבטחה שכל קצב למידה או כל צעד יקטינו תמיד את השגיאה.

15.7 | מדוע קשה לאמן רצפים ארוכים?

ב־BPTT מכפילים נגזרות רבות. כאשר הגורמים קטנים מ־ 1 לאורך צעדים רבים, המכפלה עשויה להתקרב לאפס: זהו גרדיאנט נעלם (Vanishing Gradient). במקרה כזה צעדים מוקדמים מקבלים אות עדכון חלש מאוד, וקשה ללמוד תלות רחוקה. כאשר המכפלות גדולות, הגרדיאנט עלול לגדול מאוד ולגרום לעדכונים לא יציבים: גרדיאנט מתפוצץ (Exploding Gradient).



המחשה מתמטית של מכפלות חוזרות: 0.5 בחזקת t דועך, ואילו 1.3 בחזקת t גדל.

$$0.5^{10} \approx 0.00098, \qquad 1.3^{10} \approx 13.79$$

הדוגמה המספרית מתארת מכפלות פשוטות, ולא טוענת שכל נגזרת בכל RNN היא בדיוק 0.5 או 1.3. ברשת אמיתית התמונה תלויה במשקולות, בפונקציות ההפעלה ובקלט לאורך זמן.

שתי דרכי התמודדות מרכזיות הן שימוש בתאי זיכרון עם שערים (LSTM/GRU), ובמקרה של גרדיאנט מתפוצץ שימוש ב־Gradient Clipping. קיצוץ גרדיאנטים יכול לייצב את הגודל שלהם, אך אינו פותר כשלעצמו בעיית גרדיאנט נעלם.

```
# Training step: clipping happens AFTER backward and BEFORE step.
optimizer.zero_grad()
prediction = model(X_train_batch)
loss = loss_fn(prediction, y_train_batch)
loss.backward()
torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
optimizer.step()
```

15.8 | LSTM — תא זיכרון עם שערים

רשת LSTM נועדה לאפשר למידה יעילה יותר של תלות ארוכה. לצד המצב החבוי h_t היא מחזיקה מצב תא c_t המשמש מסלול זיכרון נפרד. שערים לומדים אילו חלקים מן הזיכרון הקודם לשמר, איזה מידע חדש להוסיף ומה לחשוף כפלט. המילה "שער" אינה מתג מכני; זהו וקטור של מספרים בין 0 ל־ 1 , המחושב באמצעות Sigmoid.



השוואה רעיונית: *RNN* פשוט, *GRU* עם שערים, *LSTM* עם מצב תא.

$$f_t = \sigma(W_f[x_t, h_{t-1}] + b_f) \quad \text{(forget gate)}$$

$$i_t = \sigma(W_i[x_t, h_{t-1}] + b_i) \quad \text{(input gate)}$$

$$g_t = \tanh(W_g[x_t, h_{t-1}] + b_g) \quad \text{(candidate)}$$

$$c_t = f_t \odot c_{t-1} + i_t \odot g_t$$

$$o_t = \sigma(W_o[x_t, h_{t-1}] + b_o) \quad \text{(output gate)}$$

$$h_t = o_t \odot \tanh(c_t)$$

הסימון ⊙ פירושו כפל רכיב ברכיב. הסימון [x,h] פירושו שרשור שני וקטורים. הנוסחאות כאן הן ייצוג חינוכי שמאחד קלט ומצב קודם; מימושי ספרייה מסוימים מחזיקים שתי מטריצות נפרדות והטיות נפרדות לכל שער. אין להסיק ש־LSTM תמיד מצליחה או שאין בה לעולם בעיות גרדיאנט.

דוגמה קצרה לשער שכחה

נניח שלמצב התא הקודם ערך 0.8, שער השכחה נתן 0.75, שער הקלט נתן 0.20 והמועמד החדש הוא 0.50. נחשב את הזיכרון המעודכן:

$$c_{new} = 0.75 \cdot 0.8 + 0.20 \cdot 0.5 = 0.60 + 0.10 = 0.70$$

הערך 0.70 משלב מידע שנשמר מן העבר עם מידע חדש. זהו חישוב של צעד אחד בלבד; בפועל הערכים מחושבים על ידי משקולות שנלמדות מהנתונים.

15.9 | GRU — שערים במבנה פשוט יותר

GRU משתמשת בדרך כלל במצב חבוי יחיד ובשני שערים מרכזיים: Reset, המשפיע על האופן שבו העבר נכנס למועמד למצב חדש, ו־Update, הקובע את היחס בין המצב הישן למצב החדש. המבנה פשוט יותר מן המבנה הסטנדרטי של LSTM מבחינת סוגי המצבים, אך אף אחת מהשיטות אינה מנצחת תמיד.

$$h_t = (1 - z_t) \odot n_t + z_t \odot h_{t-1}$$

שימו לב: בנוסחה של PyTorch, ערך z_t גדול נותן משקל גדול יותר לשימור המצב הישן. מקורות אחרים עשויים להגדיר את השער המשלים בכיוון הפוך; לכן יש לקרוא את הנוסחה ולא רק לזכור תיאור מילולי של שער עדכון.

מאפיין	RNN פשוט	GRU	LSTM
מצב עיקרי	h בלבד	h ושערים	h ו־c ושערים

מאפיין	RNN פשוט	GRU	LSTM
התמודדות עם תלות ארוכה	לעיתים קשה	משופרת לעיתים	משופרת לעיתים
מורכבות יחידה	נמוכה יחסית	בינונית	גבוהה יחסית
בחירה מקצועית	Baseline לימודי	ניסוי והשוואה	ניסוי והשוואה

15.10 | משימות סדרתיות נפוצות והחלטת מודל

בבעיית חיזוי סדרת זמן אפשר לקבל חלון של מדידות ולהחזיר ערך אחד — למשל טמפרטורה בדקה הבאה. בעיית Sequence-to-Sequence תחזיר פלט עבור כל זמן, למשל סימון קטעים לא תקינים ברצף. בניתוח טקסט אפשר לסווג משפט שלם או לתייג כל מילה. הבחירה בין משימות תלויה בשאלה שבאמת רוצים לענות עליה, ולא רק בטכנולוגיה הזמינה.

רשת דו־כיוונית יכולה להתחשב הן בעבר והן בעתיד *בתוך רצף שכבר זמין כולו*. אין להשתמש בה בלי זהירות בתחזית בזמן אמת כאשר בעת קבלת ההחלטה המידע מן העתיד עדיין אינו ידוע. זו דוגמה לשגיאה מערכתית שאינה נפתרת על ידי Accuracy גבוה.

15.11 | תהליך ניסוי הוגן לסדרות זמן

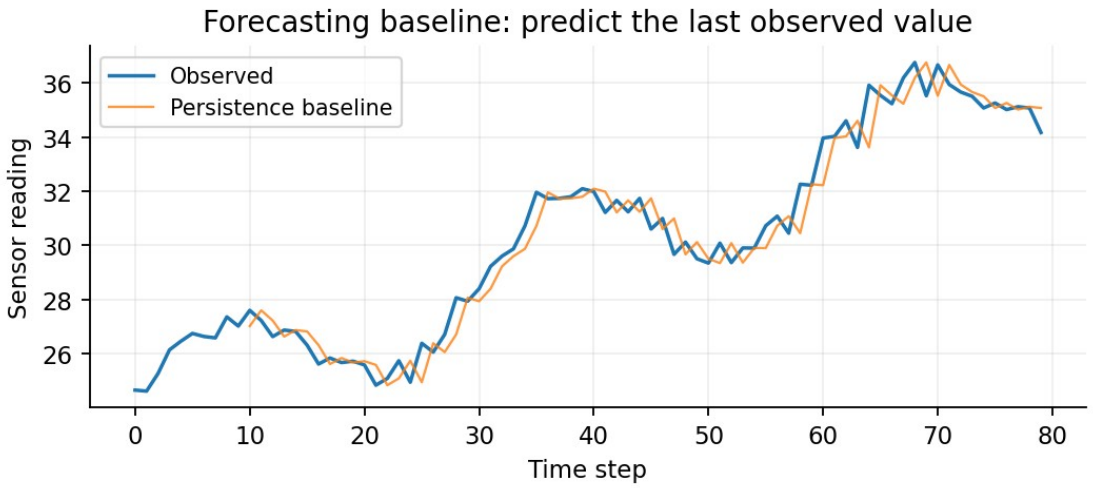
בסדרות זמן סדר החלוקה חשוב במיוחד. לדוגמה, מודל שאמור לפעול בחודש דצמבר צריך להיבחן על תקופה מאוחרת מזו ששימשה לאימונו. אם נערבב נתונים מכל השנה, הסטטיסטיקות של התקופה העתידית עלולות לחדור לאימון. גם תקנון, השלמת חסרים והתאמת פרמטרים צריכים להיעשות רק על נתוני האימון המתאימים.



דוגמה לחלוקה כרונולוגית: אימון בעבר, ולידציה לאחריו, בדיקה סופית בעתיד.

כאשר המערכת אמורה להשתמש במדידות העבר שקדמו לתחילת התקופה החדשה, אפשר לבנות חלונות בהקשר תפעולי מתאים; אך אסור להכניס לחלון קלט ערכים שלא היו ידועים בזמן התחזית. לצורך התרגול נבחר כלל פשוט וזהיר: בונים חלונות בנפרד בכל חלוקה כרונולוגית.

גם אחרי שבנינו RNN, חייבים להשוות אותה לקו בסיס. בחיזוי הערך הבא, אחת האפשרויות הפשוטות היא לנבא שהערך הבא יהיה דומה לערך האחרון שנצפה. מודל מורכב שאינו משפר באופן עקבי את ה־Baseline אינו מוצדק רק משום שהוא רשת עמוקה.



קו בסיס של התמדה (*Persistence*): חיזוי הערך הבא לפי המדידה הקודמת.

15.12 | מודל PyTorch מלא לחיזוי הערך הבא

ניצור נתוני חיישן מדומים בעזרת Python. הנתונים אינם קריאות אמיתיות ממחשבים; כך נוכל ללמוד את התהליך בלי צורך להוריד קובץ חיצוני. תחילה נחלק כרונולוגית, נתקנן באמצעות ממוצע וסטיית תקן מנתוני האימון בלבד, וניצור חלונות באורך 12. 1 2

```
import numpy as np
import torch
from torch import nn

np.random.seed(7)
torch.manual_seed(7)
t = np.arange(600)
sensor = 30 + 0.004*t + 2*np.sin(t/12) + np.random.normal(0, 0.25, len(t))
train_raw = sensor[:390]
val_raw = sensor[390:480]
test_raw = sensor[480:]
mu = train_raw.mean()
sigma = train_raw.std()

def make_windows(raw, window=12):
    scaled = (raw - mu) / sigma
    X, y = [], []
    for end in range(window, len(scaled)):
        X.append(scaled[end-window:end])
        y.append(scaled[end])
    X = torch.tensor(np.array(X), dtype=torch.float32).unsqueeze(-1)
    y = torch.tensor(np.array(y), dtype=torch.float32).unsqueeze(-1)
    return X, y

X_tr, y_tr = make_windows(train_raw)
X_va, y_va = make_windows(val_raw)
X_te, y_te = make_windows(test_raw)
print(X_tr.shape, y_tr.shape)
```

הפונקציה unsqueeze מוסיפה ממד אחד של מאפיינים. כך צורת הקלט הופכת מ־(n_windows,12) לצורה (n_windows,12,1), כפי ש־RNN עם batch_first=True מצפה לקבל.

הגדרת המודל ואימון בסיסי

```
class SensorRNN(nn.Module):
    def __init__(self, hidden_size=12):
        super().__init__()
        self.rnn = nn.RNN(input_size=1, hidden_size=hidden_size,
                          batch_first=True)
        self.head = nn.Linear(hidden_size, 1)
```

```
    def forward(self, x):
        sequence_output, hidden = self.rnn(x)
        last_step = sequence_output[:, -1, :]
        return self.head(last_step)

model = SensorRNN(hidden_size=12)
loss_fn = nn.MSELoss()
optimizer = torch.optim.Adam(model.parameters(), lr=0.01)

for epoch in range(25):
    model.train()
    optimizer.zero_grad()
    pred = model(X_tr)
    loss = loss_fn(pred, y_tr)
    loss.backward()
    torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
    optimizer.step()
    model.eval()
    with torch.no_grad():
        val_loss = loss_fn(model(X_va), y_va).item()
    if (epoch + 1) % 5 == 0:
        print(epoch + 1, round(loss.item(), 4), round(val_loss, 4))
```

אנחנו מאמנים דוגמה לימודית על אצווה אחת בכל Epoch כדי שקוד האימון יהיה קריא. בפרויקט גדול נשתמש לרוב ב־DataLoader ובאצוות קטנות. תוצאת המודל תלויה גם במימוש, בזרע האקראי ובנתונים; אסור לטעון מראש שהוא טוב יותר מקו בסיס.

הערכה ביחידות המקוריות

```
model.eval()
with torch.no_grad():
    pred_scaled = model(X_te).squeeze().numpy()
actual_scaled = y_te.squeeze().numpy()
pred_real = pred_scaled * sigma + mu
actual_real = actual_scaled * sigma + mu

rmse_model = np.sqrt(np.mean((pred_real - actual_real)**2))
baseline_real = X_te[:, -1, 0].numpy() * sigma + mu
rmse_baseline = np.sqrt(np.mean((baseline_real-actual_real)**2))
print('RNN RMSE:', round(rmse_model, 3))
print('Persistence RMSE:', round(rmse_baseline, 3))
```

מדוע מחזירים את התחזיות ליחידות המקוריות? משום שקל יותר להסביר לתלמיד או למשתמש שגיאת חיזוי של 0.6 מעלות מאשר 0.4 יחידות תקן. ההשוואה הוגנת רק אם שני המודלים נבדקו על אותם יעדים בזמן. אל תחזרו להתאים את המודל על Test לאחר צפייה בתוצאות.

15.13 | השוואת RNN, GRU ו־LSTM באופן מבוקר

כדי לבדוק האם תאי זיכרון משפרים את התוצאות, נשאיר את אותו מאגר, חלוקה, חלון, מדד ומספר Epochs. נשנה רק את סוג התא. כך נוכל להסביר מה הייתה השאלה הניסויית ואיזה גורם שינינו. מספר הפרמטרים עשוי להיות שונה ולכן אין לומר שהשוואה כזו ״משווה אותו מספר פרמטרים״; אם דרושה בדיקה מדויקת יותר יש להתאים את המודלים בהתאם.

```
def make_recurrent(kind, in_features=1, hidden=12):
    layers = {'RNN': nn.RNN, 'GRU': nn.GRU, 'LSTM': nn.LSTM}
    recurrent = layers[kind](input_size=in_features, hidden_size=hidden,
                             batch_first=True)
    return recurrent

for kind in ['RNN', 'GRU', 'LSTM']:
    net = make_recurrent(kind)
    print(kind, sum(p.numel() for p in net.parameters()))
```

הקטע מציג השוואה של מספר פרמטרים בתא בלבד. בפרויקט המלא צריך להוסיף גם שכבת פלט, לבצע אימון של כל מודל באותם תנאים ולשמור תוצאות מסודרות. המודל שהשיג את ציון הולידציה הטוב ביותר לא בהכרח ינצח בכל מאגר אחר.

15.14 | טעויות נפוצות, אתיקה והגנה בעל־פה

- לערבב את כל חלונות הזמן באקראי בלי לבחון תלות כרונולוגית; במקרה זה ייתכן שהערכת הביצועים אופטימית מדי.
- לחשב StandardScaler על כלל המדידות לפני חלוקת Train/Validation/Test; זו זליגת מידע.
- לבלבל בין ממד האצווה לממד הזמן, או לקחת את hidden האחרון בצורה שאינה תואמת למספר השכבות/הכיוונים.
- להניח ש־LSTM תמיד טובה יותר מ־RNN/GRU, בלי בדיקה מול Baseline וביצועי הולידציה.
- להשתמש בתווית או במדד מן העתיד כמפתח לחיזוי העבר; לעיתים המקור ל״מודל מצוין״ הוא מידע שלא היה זמין בפועל.
- לשמור רק Accuracy או RMSE בלי לתעד תאריכים, פרמטרים, גרסאות והחלטות מקצועיות.

בפרויקטים על נתוני אנשים יש להימנע מהסקת מסקנות לא מוצדקות על אדם מתוך דפוס התנהגות קצר. בסדרות ניטור רפואיות, למשל, הפלט צריך לעבור בחינה מקצועית והערכת סיכונים. גם במעבדת בית ספר יש להבהיר שהמודל מסייע לקבלת החלטה ואינו מחליף אבחון תקלה אמיתי.

15.15 | תרגול מדורג — ידע, חישוב וחשיבה

רמת יסוד

- 1 . הסבירו כיצד שינוי סדר המדידות יכול לשנות את המשמעות בלי לשנות את הממוצע.
- 2 . עבור רצף בן 5 1 מדידות וחלון באורך 4 , כמה דוגמאות לחיזוי הצעד הבא ניתן ליצור? נמקו.
- 3 . ציינו מה מייצג כל ממד בטנזור שצורתו (3 , 1 0 , 2 0), כאשר batch_first=True.
- 4 . הגדירו מהו מצב חבוי והסבירו מדוע הוא אינו שווה בהכרח לתצפית האחרונה.

רמת תרגול מתמטית

- 5 . חשבו שלושה צעדי RNN עם h0=0, Wx=0.2, Wh=0.5, b=0, x=[1,0,1] והפעלה זהותית; השוו ל־tanh.
- 6 . עבור $L(w)=\frac{1}{2}(0.5+0.5w^2-1)^2$ ו־w=0.4, מצאו נגזרת ובצעו עדכון בקצב 1 . 0 .
- 7 . חשבו 0 . 8 בחזקת 1 2 ופירשו מה היא ממחישה על מכפלות גרדיאנטים.
- 8 . ב־LSTM: c_prev=0.4, f=0.6, i=0.2, g=0.5. חשבו c_new ופרשו את שני האיברים.

רמת בגרות, חקר ופרויקט

- 9 . קיבלתם נתוני חיישן לפי תאריך. תכננו חלוקת Train/Validation/Test והסבירו היכן תקנון צריך להתבצע.

- 10. מודל RNN השיג RMSE=0.8 ו־Baseline השיג RMSE=0.6 על Test. האם ההחלטה לבחור ב־RNN מוצדקת? הציעו שני ניסויי המשך.
- 11. דוגמה מציגה ביצועי אימון מעולים אך ולידציה חלשה. הסבירו אפשרויות: התאמת יתר, שינוי התפלגות ומידע חסר.
- 12. הציעו תכנית למערכת לניטור מחשבים: מאפיינים, חלון, יעד, מדד הצלחה, תוצר תוכנה וקריטריון בטיחות.

15.16 | מעבדת Colab — משימות עצמאיות

לצד פרק זה קיימת מחברת Jupyter עם 12 משימות הדרגתיות. חלק מהתאים כוללים סימוני TODO או סימני שאלה כקוד להשלמה — אלה מקומות שבהם התלמיד אמור לכתוב בעצמו. פתרונות מלאים נשמרים לגרסת המורה בלבד.

תוצר ההגשה כולל מחברת פועלת, השוואת Baseline ו־RNN על Test כרונולוגי, גרף תחזיות, טבלת ניסויים ופסקת מסקנות הכוללת לפחות מגבלה אחת. אין חובה לקבל ציון מסוים; חובה להצדיק את התהליך.

15.17 | תכנית הוראה מומלצת ל־30 שעות

מפגש	מוקד	תוצר ביצועי
1 — 3 שעות	רצף, חלון, יעד והבדל מטבלה	טבלת חלונות ידנית
2 — 3 שעות	ארכיטקטורה, שיתוף משקולות ומצב חבוי	תרשים RNN מחושב
3 — 3 שעות	Forward Pass ו־NumPy	שלושה צעדים מחושבים
4 — 3 שעות	BPTT וכלל השרשרת	נגזרת ועדכון ידניים
5 — 3 שעות	Vanishing/Exploding ו־Clipping	גרפים ופירוש
6 — 3 שעות	LSTM ושערים	חישוב מצב תא
7 — 3 שעות	GRU והשוואת ארכיטקטורות	טבלת בחירה מנומקת
8 — 3 שעות	חלונות, חלוקה בזמן ו־Baseline	Pipeline נתונים
9 — 3 שעות	אימון RNN ב־PyTorch והערכה	מודל וגרף תחזיות
10 — 3 שעות	ניסוי מבוקר, כתיבה והגנה	מחברת ודוח קצר

זו חלוקה פדגוגית מוצעת לעשרה מפגשים בני שלוש שעות. את שעות ההתמחות והיקף הדרישה בפועל יש לקבוע על פי תכנית בית הספר והמסמך הרשמי העדכני.

15.18 | הכנה לעבודת הגמר — תיק חקר מינימלי

שאלה שהתלמיד חייב לענות עליה	רכיב
מהם הנתונים, למי הם שייכים ומתי נאספו?	מקור נתונים

שאלה שהתלמיד חייב לענות עליה	רכיב
איזו תחזית מבצעים ומה היה ידוע בזמן התחזית?	**הגדרת יעד**
מדוע נבחר אורך הרצף הזה?	**בחירת חלון**
איזה פתרון פשוט חייבים לנצח?	**Baseline**
מה נבדק: RNN, GRU או LSTM, ומה נשאר קבוע?	**מודלים**
כיצד נבחרו פרמטרים בלי לראות Test?	**ולידציה**
מה גודל הטעות במונחים של המשתמש?	**מדדים**
כיצד מזינים רצף חדש למערכת?	**תוכנה**
מה עלול להשתבש בפעילות בעולם האמיתי?	**מגבלות**

15.19 | מילון מושגים

הגדרה בעברית	מונח
רשת המעדכנת מצב חבוי לאורך רצף	**Recurrent Neural Network**
וקטור פנימי המסכם מידע מהעבר	**Hidden State**
פריסה חזותית של אותו תא לאורך זמן	**Unrolling**
גזירת השגיאה לאחור לאורך צעדי זמן	**BPTT**
היחלשות אות העדכון כאשר נגזרות מוכפלות	**Vanishing Gradient**
גדילת גרדיאנט עד לחוסר יציבות	**Exploding Gradient**
תא חוזר עם מצב זיכרון ושערים	**LSTM**
תא חוזר עם שערי איפוס ועדכון	**GRU**
חלון הקלט ואופק התחזית	**Window / Horizon**
זליגת מידע שאינו זמין בזמן התחזית	**Temporal Leakage**
ניבוי הערך הבא לפי הערך האחרון שנצפה	**Persistence Baseline**

15.20 | מבחן מסכם — האם אתם מבינים רשת סדרתית?

להלן מבחן עצמי קצר לפני המעבדה האחרונה. התשובות אינן מופיעות בגרסת התלמיד; מומלץ להסביר כל בחירה במילים ולא רק לכתוב מספר. אפשר להגיש תשובות בכתב או להציגן בהסבר בן חמש דקות מול תלמיד אחר.

כיצד תראו הבנה	שאלה מסכמת	מוקד
הסבר לפי מצב חבוי	שני רצפים מכילים אותם ערכים בסדר שונה. האם ה־RNN חייב	**סדר ונתונים**

כיצד תראו הבנה	שאלה מסכמת	מוקד
	להחזיר אותו פלט?	
נימוק לכל אחד מהממדים	.X.shape=(32,12,2), hidden_size=8 מה צורת הפלט הסדרתי?	**ממדיות**
חיבור ל־BPTT	מה המשמעות של שיתוף המשקולות לאורך 1 2 צעדים?	**אימון**
ניסוי הוגן ובסיס השוואה	מדוע לא נכון להבטיח ש־LSTM תמיד תנצח GRU?	**מודלים**
מסקנה מקצועית ומגבלה	הדיוק על Test היה חלש יותר מ־Baseline. מה תדווחו?	**בדיקה**

משימת מעבר לפרויקט: הכינו כרטיס ניסוי בן חצי עמוד הכולל הגדרת בעיה, סוג רצף, אורך חלון, חלוקה כרונולוגית, Baseline, מדד מרכזי ומשפט אחד המסביר מתי השימוש במודל לא יהיה אחראי.

15.21 | מקורות קריאה והבחנה בין חובה להרחבה

עוגן תכנית ההתמחות: מאגר Data-Science-ML-Course, פרק 1 5 — רשתות סדרתיות חוזרות (RNN), 3 0 שעות. הקישור למיקום בפרק מאמת את שם הפרק והיקפו במפה; הוא אינו מחליף קריאה של מסמך משרד החינוך הרשמי המלא.

ספרות מקצועית: Zhang, Lipton, Li & Smola — Dive into Deep Learning, פרקי RNN ו־BPTT; Simon Prince — Understanding Deep Learning; Goodfellow, Bengio & Courville — Deep Learning; Aurélien Géron — Hands-On Machine Learning. שיטות ההסבר הותאמו ונכתבו מחדש באופן מקורי לתלמידי תיכון.

תיעוד יישום: PyTorch nn.RNN, nn.GRU, nn.LSTM ו־torch.nn.utils.clip_grad_norm_. יש לקרוא את הגרסה המותקנת כאשר עובדים עם נתוני תלמידים וגרסאות תוכנה שונות.

קישור לחומרי המקור במאגר: https://github.com/samiabo75/Data-Science-ML-Course/tree/main/03-Deep-Learning-5-Units/15-Recurrent-Neural-Networks

D2L: https://en.d2l.ai/chapter_recurrent-neural-networks/index.html

PyTorch RNN: https://docs.pytorch.org/docs/stable/generated/torch.nn.RNN.html

PyTorch GRU: https://docs.pytorch.org/docs/stable/generated/torch.nn.GRU.html

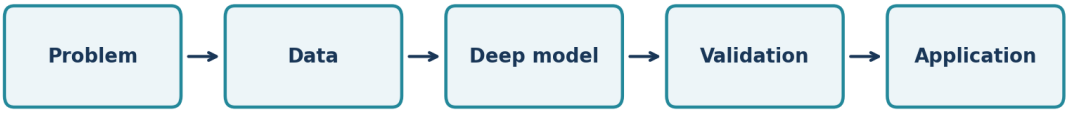
PyTorch LSTM: https://docs.pytorch.org/docs/stable/generated/torch.nn.LSTM.html

מדעי הבינה המלאכותית

פרק 16 — עבודת הגמר

אמיתית למערכת עובדת, תיק פרויקט והגנה בעל־פה AI מבעיית

התמחות למידת מכונה ולמידה עמוקה | 5 יחידות לימוד | מסגרת של 80 שעות



Document decisions, test assumptions and report limitations at every stage

הפרויקט הוא תהליך שלם. איכות התוצאה תלויה גם במידע, בבדיקות ובהסבר — לא רק במודל.

מהדורת תלמיד מורחבת. הדוגמאות, הגרפים והמשימות נכתבו לצורכי הוראה. נתונים מספריים לדוגמה אינם תוצאות אמתיות של פרויקט תלמיד. הבחנה בין הוראות משרד החינוך לבין המלצות פדגוגיות מפורטת בפרק.

מפת הפרק — מה צריך לדעת בסיום?

תוצר הלמידה איננו רק מחברת שפועלת. התלמיד יוכל להציג מערכת המשתמשת במודל למידה עמוקה שהוא בנה ואימן, להסביר את שלבי העבודה, להראות כיצד בדק ושיפר את הביצועים, להוכיח שהיישום מפיק תחזית תקינה, ולהגן על בחירותיו.

יחידה בפרק	שאלת מפתח	ראיה להצלחה
בחירת בעיה	מה בדיוק צריך לחזות, ולמי?	מסמך אפיון עם מדד הצלחה
נתונים	האם אפשר לסמוך על הנתונים?	כרטיס מאגר ודוח EDA
מודל	מה הרשת לומדת וכיצד?	ארכיטקטורה ויומן אימון
הערכה	האם ההצלחה אמיתית?	Test נפרד וניתוח טעויות
יישום	האם משתמש מסוגל להיעזר במודל?	מערכת ומבחני קלט
הגנה	האם התלמיד מבין את עבודתו?	הדגמה והסברים בעל־פה

16.1 מה ההבדל בין תרגיל, מיני־פרויקט ועבודת גמר?

תרגיל כיתה בוחן בדרך כלל מיומנות אחת: למשל חישוב Precision או אימון רשת שמספר שכבותיה נתון. במיני־פרויקט כבר מתקבלת בעיה רחבה יותר, אבל למורה עדיין יש תפקיד משמעותי בבחירת המאגר, הניסוי והארכיטקטורה. בעבודת הגמר התלמיד נדרש לקבל החלטות רבות בעצמו, לתעד אותן ולהסביר אותן. החלטות אלו הן חלק מן התוצר, ולא פעילות נסתרת שמתרחשת מאחורי הקלעים.

לדוגמה, כאשר רשת מזהה ספרות בדיוק של 96%, אין זה מספיק להציג צילום מסך של המספר. על התלמיד לדעת מאין הגיעו התמונות, כיצד חולקו לקבוצות, האם קיימות תמונות דומות מדי בין אימון לבדיקה, מה היה מודל הבסיס, מדוע נבחרו שכבות מסוימות, אילו טעויות בולטות קיימות, וכיצד היישום מטפל בתמונה חדשה שאינה מתאימה לקלט הצפוי.

16.2 העוגן הרשמי — שלושת רכיבי הפרויקט

בהנחיות משרד החינוך לכתיבת תיק פרויקט בחלופת למידת מכונה (סמל 883589) מופיעים שלושה רכיבים מרכזיים: איסוף, הכנה וניתוח הנתונים; בנייה ואימון של מודל למידה עמוקה; ויישום תוכנה המשתמש במודל. נדרשת גם הרחבה והעמקה באחד מן הרכיבים. פרק ההתמחות במפת המאגר שלנו מקצה לעבודת הגמר 80 שעות.

רכיב	מה התלמיד צריך להראות בפועל	דוגמת ראיות
נתונים	מקור, מבנה, חקר והכנה לאימון	Dataset Card, גרפים, פיצול, נרמול
מודל עמוק	רשת שנבנתה ואומנה, הסבר על שכבות ואופטימיזציה	קוד, תרשים שכבות, עקומות Loss, יומן ניסויים
יישום תוכנה	קלט חדש → קדם־עיבוד → חיזוי → תוצאה	מסכים, קוד יישום, בדיקות שגיאה
העמקה	עבודה משמעותית נוספת באחד	החלטת העמקה, השוואה

דוגמת ראיות	מה התלמיד צריך להראות בפועל	רכיב
ותוצאות מתועדות	משלושת הרכיבים	

ההנחיות הרשמיות שנמצאו מפורטות לגבי מבנה הספר, מדריך למפתח, מדריך למשתמש ורפלקציה. מסמך מחוון תשפ״ג שזמין לציבור מציג חלוקה היסטורית של 30% לתיק פרויקט ו־70% לבחינה בעל־פה. אין להניח שחלוקה זו עדיין תקפה במחזור ההגשה הנוכחי; לפני הגשה יש לבדוק מחוון עדכני דרך הפיקוח והמרחב הפדגוגי.

נקודה למחשבה — מסמך זה הוא ספר לימוד, לא תחליף למסמך ההנחיות או למחוון התקפים בשנת ההגשה. המבנים, הטבלאות והמשימות המוצעים כאן הם כלי הוראה, אלא אם צוין במפורש כי מקורם בהנחיות הרשמיות.

16.3 בחירת בעיה מציאותית — לפני שבוחרים רשת

בשלב הראשון מתארים בעיה של משתמש אמיתי ולא סוג אלגוריתם. המשפט "אני רוצה להשתמש ב־CNN" מתאר טכנולוגיה, אך אינו עונה על השאלה מי צריך אותה. לעומתו, המשפט "אני רוצה לאפשר לילדים להזין ספרה כתובה ביד ולקבל פענוח נגיש" מתאר צורך, קלט ותוצאה אפשרית. רק לאחר מכן נשאל אם CNN היא פתרון מתאים מבחינת נתונים, איכות ומאמץ פיתוח.

מסמך אפיון קצר חייב לענות על שש שאלות:

- מי המשתמש? מה הפעולה שהוא רוצה לבצע?
- מהו הקלט המדויק (יחידות, טווחים, פורמט, גודל)?
- מהו הפלט? מחלקה, מספר רציף, מסכה או סדרה?
- מהי טעות מסוכנת או יקרה ומהי טעות נסבלת?
- אילו נתונים נדרשים, והאם מותר להשתמש בהם?
- איזה מדד וערך יעד יסמנו הצלחה? מדוע הם סבירים?

בדוגמת זיהוי הספרות, משתמש מצייר תמונה ולא מעלה תמונה גדולה מן הטלפון. הקלט הצפוי עשוי להיות מטריצת 8×8, והפלט אחת מעשר ספרות. בהמשך נצטרך להגדיר מה קורה אם התמונה ריקה, מטושטשת או כתובה בפורמט אחר. החלטות מסוג זה משפיעות על האפליקציה ועל תוצאות המודל לא פחות מהארכיטקטורה שלו.

16.4 סינון רעיונות והתאמת היקף

לפני בחירת נושא בודקים היתכנות. רעיון עשוי להיות מעניין מבחינה חברתית אך בלתי אפשרי במסגרת שעות הלימוד בגלל קושי להשיג מאגר מתאים, פגיעה בפרטיות, דרישה לתשתיות כבדות או תלות בשירות חיצוני שאי אפשר להפעיל בבחינה. יש לבחור פרויקט שמאפשר למידה עמוקה אמיתית, בניית יישום ועבודה עצמאית.

סיכון שדורש פתרון	רשת אפשרית	נתונים	רעיון
פער בין ציור חדש לבין המאגר	MLP או CNN	תמונות ספרות מתויגות	**זיהוי ספרות בכתב יד**
רקע שמסגיר את	CNN	תמונות עם רשות שימוש	**זיהוי סוג פסולת**

סיכון שדורש פתרון	רשת אפשרית	נתונים	רעיון
התווית			
זליגת מידע ממדידות עתידיות	RNN / LSTM	מדידות מסודרות בזמן	**תחזית מדידות חיישן**
פרטיות, איכות הקלט ומורכבות	מודל רצפי / עמוק	הקלטות עם הסכמה ורישוי	**זיהוי קולי**

16.5 תכנון נתונים ובניית כרטיס מאגר

נתונים הם הבסיס של כל מודל. יש לתעד מי יצר אותם, מתי, מה משמעות כל שדה, מה מותר לעשות איתם ואילו אוכלוסיות או מצבים אינם מיוצגים. לדוגמה, מאגר של ספרות סרוקות וממורכזות לא בהכרח מייצג צילומים בתאורה חלשה או כתב יד על דף מחברת. איכות המודל נמדדת גם ביחס לאוכלוסיית השימוש, ולא רק ביחס למאגר נוח שנמצא באינטרנט.

כרטיס מאגר טוב מכיל שם, מקור וקישור, מספר דוגמאות, סוג התוויות, ממדי הקלט, תנאי איסוף, רישיון, תנאי שימוש, ערכים חסרים, חוסר איזון, סיכונים ופעולות עיבוד. בעת שימוש בנתונים אישיים מתבקשים לצמצם מידע מזהה, לבדוק רשות שימוש ולהימנע מהצגת תמונות, קולות או טקסטים פרטיים בדוח הפומבי.

דוגמה מלווה: **digits** מתוך **scikit-learn**

זהו מאגר לימודי מוכר ובו 1,797 תמונות של ספרות בכתב יד. כל תמונה בגודל 8×8 וכוללת ערכי עוצמה בטווח 0–16; לכן 64 מספרים מייצגים תמונה אחת. השימוש במאגר הזה מאפשר להתמקד בתהליך המחקר, האימון והיישום בלי להסתבך בהורדת קבצים. הוא אינו מייצג באופן מלא קלט חופשי שמגיע ממשתמשי מערכת אמיתית.

$$X \in R^{(1797\times64)} \qquad y \in \{0, 1, ..., 9\}^{(1797)}$$

16.6 חקר נתונים (EDA) אינו רק יצירת גרפים

חקר נתונים טוב מתחיל משאלה. לדוגמה: האם כל הספרות מיוצגות במידה דומה? האם חלק מהתמונות כהות במיוחד? האם יש תוויות שגויות? האם החלק המרכזי של התמונה נושא את רוב המידע? גרף שכיחויות וגלריית תמונות אינם קישוט לדוח אלא ראיות בעזרתן מחליטים כיצד לעבד נתונים ומהן מגבלותיהם.

יש לנסח לכל גרף ממצא, אפשרות לפרשנות ומגבלה. "בספרה 1 מופיעים פיקסלים בהירים באזור המרכז" הוא ממצא אפשרי; "לכן הרשת תזהה כל 1 בעולם" היא טענה שאין לה בסיס. בנוסף, כאשר מציגים ממוצעים או מתאמים, אסור להסיק סיבתיות מן הקשר בלבד.

16.7 חלוקת נתונים — שמירת מבחן אמיתי



Example split for illustration; adapt to dataset and dependence structure

חלוקה לדוגמה בלבד. חלוקה אקראית מתאימה רק אם אין תלות בין הדוגמאות שגורמת להדלפת מידע.

חלוקה בין Train, Validation ו־Test שומרת על שלוש מטרות שונות. נתוני האימון משמשים לעדכון משקולות. נתוני הולידציה משמשים להשוואת ארכיטקטורות, קצבי למידה וספי החלטה. נתוני המבחן נשמרים להערכת התוצר לאחר שהחלטות התכנון הוכרעו. אם בודקים שוב ושוב את Test ומשנים את הרשת בהתאם, הוא מפסיק להיות מבחן בלתי תלוי.

בנתוני סדרות זמן יש לשמור על הסדר הכרונולוגי; בנתונים של אותו אדם או אותו מוצר יש לשקול פיצול לפי קבוצות, כדי שדוגמאות קשורות לא יופיעו גם באימון וגם בבדיקה. קנה מידה כגון ממוצע וסטיית תקן חייב להילמד מקבוצת האימון בלבד, ולהיות מיושם באותו אופן בולידציה, במבחן ובאפליקציה.

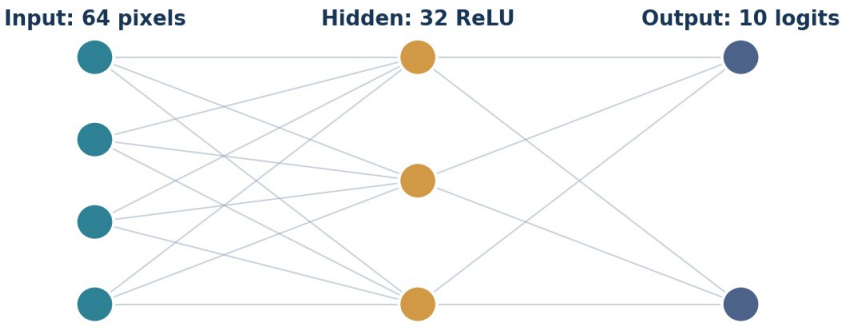
16.8 מהו Baseline ומדוע הוא חובה מקצועית?

מודל בסיס הוא פתרון פשוט שנבנה קודם למודל העמוק. בסיווג אפשר לבדוק ניבוי של המחלקה השכיחה ביותר, KNN שכבר למדנו, או כלל חישובי. בחיזוי סדרה אפשר לנבא שהערך הבא שווה לערך האחרון. אם רשת עמוקה אינה משפרת את הבסיס לפי המדד שנבחר, אין זה כישלון פדגוגי: זו ראיה שצריך לבחון את הנתונים, תהליך האימון או הצדקת מורכבות המודל.

נקודה למחשבה — יש להשוות מודלים על אותה חלוקה, באותו מדד ובתנאי קדם־עיבוד בני־השוואה. השוואה לפי *Accuracy* בלבד אינה מספיקה כאשר לטעויות מסוגים שונים יש מחיר שונה.

16.9 כיצד מתכננים ארכיטקטורת למידה עמוקה?

הארכיטקטורה מתרגמת את סוג הקלט לסדרה של פעולות. עבור 64 ערכי פיקסל ניתן להתחיל ברשת פשוטה בעלת שכבה חבויה של 32 יחידות ReLU ושכבת פלט של 10 ערכים. השכבה האחרונה מחזירה logits, כלומר ציונים גולמיים, ולא בהכרח הסתברויות. בעת אימון עם CrossEntropyLoss ב־PyTorch מוסרים את ה־logits ישירות; אין להחיל Softmax לפני פונקציית האובדן הזו.



ארכיטקטורה אפשרית לדוגמת *digits*. מספר העיגולים באיור הוא סכמטי; הממדים כתובים מעליו.

$$64 \times 32 + 32 + 32 \times 10 + 10 = 2410 \text{ parameters}$$

מספר הפרמטרים המחושבים הוא 2,410: 2,048 משקולות ו־32 הטיות בשכבה הראשונה, ו־320 משקולות ו־10 הטיות בשכבת הפלט. זהו תרגיל מתמטי חשוב משום שהוא מאפשר לתלמיד להעריך את מורכבות המודל ולבדוק את מספר הפרמטרים שמדפיסה הספרייה.

16.10 דוגמה מתמטית — מה פירוש Cross-Entropy?

נניח שהרשת מפיקה שלושה ציונים גולמיים עבור שלוש מחלקות: 2, 1 ו־0. פונקציית Softmax ממירה ציונים להסתברויות יחסיות שסכומן 1. אם המחלקה האמיתית היא הראשונה, השגיאה היא מינוס הלוגריתם של ההסתברות שהרשת נתנה למחלקה זו.

$$p_i = \exp(z_i) / \Sigma_j \exp(z_j)$$

$$p_{true} = \exp(2) / [\exp(2)+\exp(1)+\exp(0)] \approx 0.6652$$

$$L = -\ln(p_{true}) \approx 0.4076$$

המשמעות: כאשר הרשת מקצה הסתברות גבוהה למחלקה הנכונה, ערך האובדן קטן. עם זאת, ערך Loss של דוגמה בודדת אינו Accuracy ואינו מדד מספיק להצלחת המערכת. באימון יש לבחון גם התפלגות השגיאות, עקומות Train/Validation ומדדי הביצועים על קבוצות שלא שימשו לעדכון הפרמטרים.

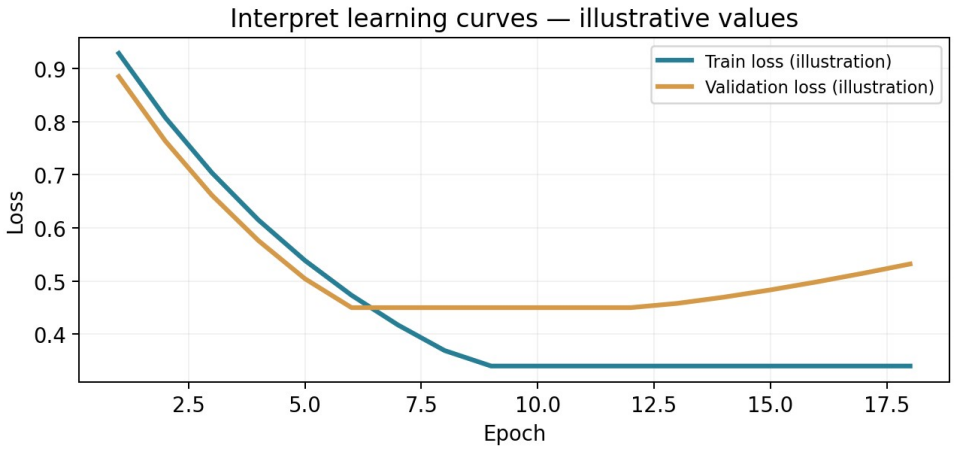
16.11 תכנון ניסוי: מה משתנה ומה נשאר קבוע?

ניסוי טוב משנה החלטת תכנון אחת בכל פעם כאשר הדבר מעשי. אפשר להתחיל ברשת 64–32–10, לשמור את החלוקה, את הזרע האקראי ואת מספר ה־Epochs, ואז לבדוק השפעת שינוי מספר היחידות החבויות. בניסוי אחר נשנה את קצב הלמידה. כך נוצרת יכולת לייחס הבדל שנצפה לבחירה מוגדרת, בלי להתבלבל בין שינויים רבים שבוצעו יחד.

מספר ניסוי	השערה	מה שינינו	מה מודדים
E01	רשת קטנה תספק בסיס הוגן	ארכיטקטורת התחלה	בולידציה Loss ו־Accuracy
E02	הגדלת השכבה עשויה לשפר יכולת ייצוג	64 → 32 יחידות	אותם מדדים ותנאי אימון
E03	קצב למידה נמוך ישפיע על התכנסות	η=0.01 → η=0.001	עקומת שגיאה לאורך Epochs

ביומן הניסויים נדרשים גם תאריך, גרסת קוד, נתוני אימון, חומרת הרצה אם רלוונטית, תוצאה, פירוש ומגבלה. מסקנה מקצועית אינה רק "מודל E02 יותר טוב", אלא "בחלוקה ובמספר Epochs שנבדקו, E02 השיג שיפור של כך וכך; יש לבדוק אם השיפור יציב".

16.12 עקומות אימון — מתי עוצרים?



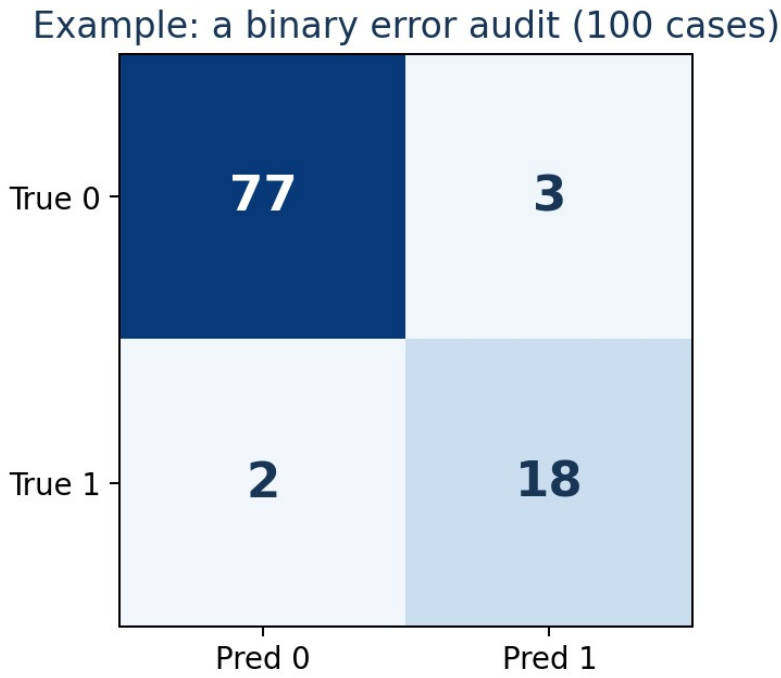
עקומות מומצאות להמחשת הרעיון; אין לפרש אותן כתוצאות ניסוי שבוצע.

עקומת Train Loss מלמדת אם המודל מצליח להתאים את עצמו לדוגמאות האימון. עקומת Validation Loss מלמדת אם השינוי משתפר גם על דוגמאות שלא שימשו לעדכון המשקולות. כאשר Train Loss ממשיך לרדת אבל Validation Loss מתחיל לעלות לאורך זמן, ייתכן Overfitting. זה אינו פסק דין אוטומטי: יש לבחון תנודתיות, גודל המדגם ואופן המדידה.

Early Stopping היא החלטה לעצור לפי תוצאות הולידציה ולהחזיר את מצב המשקולות הטוב שנשמר. אין להחליט על מועד העצירה לאחר הצצה לתוצאות Test. כמו כן חשוב להבדיל בין שימוש במדד האובדן לתהליך האימון לבין מדד העסקי או המחקרי שבו מקבלים החלטה על איכות הפרויקט.

16.13 הערכה סופית וניתוח טעויות

כאשר הוחלט על הארכיטקטורה ועל מדיניות האימון, מריצים הערכה סופית על Test. עבור סיווג רב־מחלקתי משתמשים במטריצת בלבול, במדדי Precision/Recall/F1 לפי צורך ובדוגמאות שגויות מייצגות. הדיוק הכללי חשוב, אבל אינו מסביר אילו מחלקות המודל מבלבל ומדוע.



דוגמה מספרית נפרדת של בעיית סיווג בינארית; היא אינה מטריצת הבלבול של מאגר הספרות.

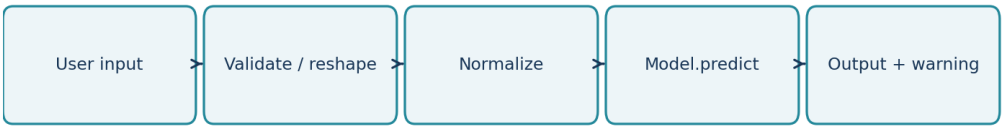
במטריצה זו TP=18, FN=2, FP=3, TN=77. לכן Accuracy הוא 95%, Precision הוא 18 חלקי 21 (כ־85.7%), ו־Recall הוא 18 חלקי 20 (90%). כאשר החלטה שגויה עשויה להזיק למשתמש, אין להסתפק ב־Accuracy. צריך גם לתעד דוגמאות שהמודל מחמיץ, ולבדוק האם יש תת־קבוצות שקשה לו במיוחד לזהות.

$$\text{Accuracy} = (TP + TN) / N = 95 / 100 = 0.95$$

$$\text{Precision} = 18/21 \approx 0.8571 \qquad \text{Recall} = 18/20 = 0.90$$

16.14 מדוע יישום תוכנה הוא רכיב נפרד מהמודל?

רשת שפועלת במחברת אינה בהכרח מערכת שמישהו מסוגל להשתמש בה. היישום נדרש לקבל נתון חדש, לבדוק שהוא חוקי, להתאים אותו לאותו פורמט ששימש באימון, להפעיל את המודל שהוכשר ולהציג פלט ברור. תלמיד שמגיע עם Notebook בלבד ועדיין לא שילב את המודל ביישום, לא הראה את מלוא מחזור החיים המתואר בהנחיות החלופה.



שרשרת היסק: אין לבצע באפליקציה עיבוד אחר מזה ששימש באימון.

לדוגמה: אם הרשת אומנה על תמונות 8×8 בעוצמות 0–16 המנורמלות לחלוקה ב־16, אין להזין ישירות צילום בגודל 224×224 ובטווח 0–255. צריך להגדיר תהליך המרה מפורש, לבדוק אותו ולתעד את

מגבלותיו. בלי התאמה זו המודל יכול להחזיר מספר, אבל המספר אינו תחזית אמינה באותה משמעות שבה אומן.

16.15 דוגמת קוד מונחית — שמירת מודל ובדיקת קלט

הדוגמה שלהלן מדגישה את מבנה האחריות של קוד היישום. היא אינה מהווה פרויקט מוגמר, ואינה מחליפה את תהליך האימון הנדרש לתלמיד. הקפדה על צורת הקלט מונעת שגיאות שניתן לאתר עוד לפני הרצת הרשת.

```
import numpy as np

def prepare_digit(image):
    arr = np.asarray(image, dtype=np.float32)
    if arr.shape != (8, 8):
        raise ValueError("Expected an 8x8 image")
    if not np.isfinite(arr).all():
        raise ValueError("Image contains invalid values")
    if np.any(arr < 0) or np.any(arr > 16):
        raise ValueError("Pixel values must be in [0, 16]")
    return (arr.reshape(1, 64) / 16.0)
```

יש להסביר מדוע פעולת reshape משנה את צורת הנתונים אך לא את ערכיהם, מדוע מחלקים ב־16 ולא ב־255 בדוגמה הזו, ומדוע חוסמים ערכים שאינם בתחום שהמודל ראה באימון. במערכת אמיתית יש להתאים את מדיניות החריגות לשימוש, ולא רק לזרוק שגיאה לא מוסברת למשתמש.

16.16 תכנון ארכיטקטורת מערכת

מומלץ להפריד בין שלושה רכיבים תוכנתיים: שירות קדם־עיבוד, שכבת חיזוי וממשק משתמש. ההפרדה מאפשרת לבדוק כל חלק בנפרד. אפליקציה שולחנית, Web או Android עשויות להתאים, בתנאי שהמודל אכן משולב ביישום המתאים ושאפשר להדגים את פעולתו באופן יציב בבחינה.

רכיב	אחריות	בדיקה חשובה
DataPreprocessor	מתאים קלט למבנה, לערכים ולסוג הנתונים	תמונה לא נכונה נדחית עם הודעה
ModelService	טוען מודל שמור ומבצע תחזית	אותו קלט מחזיר תחזית תקינה ועקבית
UserInterface	מקבל קלט ומציג תוצאה ומגבלות	המשתמש מבין איך להפעיל ולתקן
Logger (ללא נתונים רגישים)	מתעד תקלה בלי לשמור מידע אישי שלא לצורך	רישום מועיל ללא חשיפת פרטיות

16.17 בדיקות תוכנה שאינן קשורות למדד Accuracy

בדיקות המודל ובדיקות התוכנה משלימות זו את זו. Accuracy גבוה אינו מגן מפני קריסה של מסך, העלאת קובץ פגום, כשל בטעינת משקולות או הצגה שגויה של תווית. לכן נחלק את הבדיקות לבדיקות יחידה של פונקציות קדם־עיבוד, בדיקות אינטגרציה בין רכיבי המערכת ובדיקות שימוש עם קלט תקין ושגוי.

מקרה	קלט	התנהגות צפויה
קלט חוקי	מטריצה 8×8 עם ערכים 0–16	תוצג ספרה חזויה ומידע על מגבלות
גודל לא תקין	מערך 10×10	הודעה ידידותית; אין לבצע תחזית
ערך חסר	NaN בתוך תמונה	הודעת שגיאה והנחיה לתיקון
טווח לא תואם	פיקסל בעוצמה 255	דחייה או המרה מאומתת לפי האפיון
מודל חסר	קובץ משקולות לא קיים	הודעה ברורה ולא סגירה שקטה

16.18 בטיחות, פרטיות, ייחוס וכלי AI

פרויקט טוב אינו אוסף יותר מידע מכפי שנדרש. לפני שימוש בתמונות, הקלטות או נתונים אישיים, יש לברר הרשאת שימוש, רישיון ומגבלות שיתוף. אין להכניס סיסמאות או אסימוני גישה למחברות ציבוריות או ל־GitHub. בנוסף, אין לטעון באופן אוטומטי קובצי מודל ממקור לא מוכר בפורמט שמאפשר הרצת קוד לא מהימן.

מותר לשלב כלי AI לצורך למידה, ניסוח, חקר שגיאות ובדיקת הסברים, לפי מדיניות בית הספר. מבחינה פדגוגית חשוב לתעד איזו עזרה התקבלה, מה נבדק עצמאית ומה נכתב על ידי התלמיד. תיעוד כזה מגן על אמינות העבודה ומסייע לתלמיד להסביר באמת את הקוד שהוא מגיש. אין להציג פלט של כלי AI כהוכחה לתקינות ללא הרצה ובדיקה.

16.19 מבנה תיק הפרויקט

בהנחיות משרד החינוך מופיעים שער, תוכן עניינים, מבוא, ארכיטקטורה, מדריך למפתח, מדריך למשתמש, רפלקציה, מקורות ונספחים. להלן הצעת פירוק מעשית, השומרת על המבנה ובתוכה מסדרת את ראיות העבודה שלנו:

פרק בתיק	חומר שיש להכין
שער ותוכן עניינים	פרטי העבודה לפי הנחיות בית הספר, כותרות ומספור
מבוא	צורך, משתמשים, מטרות, מחקר מקדים ופתרונות חלופיים
ארכיטקטורה — נתונים	מקור, מבנה, EDA, חלוקה והכנה לאימון
ארכיטקטורה — מודל	תרשים רשת, אובדן, אימון, היפר־פרמטרים וניסויים
ארכיטקטורה — יישום	תרשים מערכת, מעבר נתונים, קוד קלט וחיזוי
מדריך למפתח	קבצים, מחלקות, פונקציות, התקנה וגרסאות
מדריך למשתמש	מסכים, הוראות, מגבלות והודעות
רפלקציה	מה למדתי, מה נכשל, ומה הייתי עושה אחרת

חומר שיש להכין	פרק בתיק
ראיות ניסוי, קוד, נתונים, בדיקות ומקורות	**נספחים**

16.20 כתיבת רפלקציה מקצועית

רפלקציה אינה המקום לכתוב רק "נהניתי מהפרויקט". היא מתארת את ההתפתחות של התלמיד: רעיון שהתברר כלא מתאים, ניסוי שהראה תוצאה מפתיעה, באג שנמצא בבדיקת קלט, שיחה עם משתמש ששינתה את המסך, או החלטה מקצועית שהתלמיד היה מקבל אחרת אילו התחיל מחדש.

דוגמה לניסוח מחקרי: "בתחילה בחרתי CNN מתוך הנחה שמודל עמוק ישיג תוצאה טובה יותר. לאחר השוואת קו בסיס גיליתי שהיתרון אינו מובטח במאגר הקטן. שיניתי את סדר הניסויים והוספתי בדיקות לנתוני קלט חדשים. מן התהליך למדתי שמורכבות אינה תחליף למדידה". אין להעתיק את הפסקה הזו כהצהרת תלמיד; היא מדגימה את סוג החשיבה המצופה.

16.21 הכנה להגנה בעל־פה — להראות הבנה ולא רק מצגת

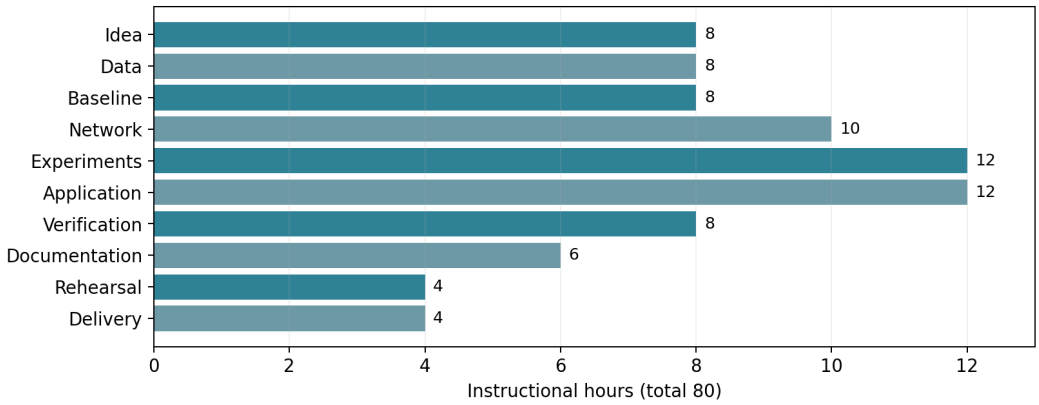
בוחן עשוי לבקש מן התלמיד להפעיל את התוכנה, להסביר את גבולות היכולת של המודל, לפתוח קובץ קוד ולתאר כיצד ישנה רכיב. תלמיד שמכיר רק את המצגת לא יוכל להגן על עבודתו. לכן כל תלמיד יכין הדגמה קצרה, יתאמן במענה לשאלות וידע להסביר נתיב מלא: מקלט חדש, דרך עיבוד וטעינת מודל, ועד הצגת תחזית.

שאלות לדוגמה לתרגול בעל־פה:

- מדוע בחרת בבעיה הזו, ולמה נדרש מודל למידה עמוקה?
- האם הדוגמאות במאגר דומות לקלט שמגיע מהמשתמש?
- כיצד מנעת Data Leakage? איך בחרת היפר־פרמטרים?
- מהו מספר הפרמטרים ברשת ומה תפקיד פונקציית ההפעלה?
- מה ההבדל בין Loss לבין Accuracy ולמה שניהם נמדדו?
- כיצד היישום ממיר קלט חדש למבנה מתאים למודל?
- איזה מקרה קצה עלול להכשיל את המערכת? איך תטפל בו?
- אם אבקש להוסיף מחלקה חדשה, אילו חלקים במערכת תצטרך לשנות?

16.22 תכנון עבודה ל־80 שעות

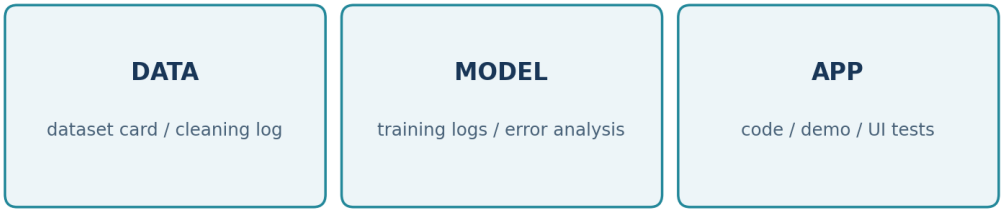
החלוקה להלן היא תכנית הוראה מוצעת של הספר, לא חלוקת שעות רשמית פנימית. זמני הפרויקט האמיתיים משתנים לפי רמת הקבוצה, סוג הנתונים ויכולת התלמידים. בכל שער־שלב נדרש תוצר מוחשי שישמר בתיק הפרויקט.



חלוקה מוצעת ל־80 שעות, עם בקרה ותוצר מסכם בסיום כל שלב.

שלב	שעות	תוצר מחייב בכיתה
בחירת בעיה ואפיון	8	מסמך אפיון ותבחיני הצלחה
נתונים ואישורים	8	כרטיס מאגר ואישור מקור
EDA וקו בסיס	8	ניתוח נתונים והשוואת Baseline
ארכיטקטורה	10	תרשים רשת וקוד אימון ראשון
ניסויים ושיפור	12	יומן ניסויים, גרפים והחלטת מודל
יישום תוכנה	12	ממשק פועל המשתמש במודל
בדיקות וניתוח כשלים	8	בדיקות תוכנה, הערכה ודיון במגבלות
כתיבת תיק	6	תיק פרויקט ומדריכי שימוש
הכנה להגנה	4	הדגמה ומענה מנומק לשאלות
אריזה ובקרה סופית	4	קבצים מסודרים, גרסאות ובדיקת הפעלה

16.23 מטריצת ראיות — כיצד נוכיח שהעבודה בוצעה?



לכל רכיב יש תוצרים שאפשר לבדוק. צילום מסך יחיד של תוצאה אינו מחליף רצף ראיות.

מטריצת ראיות מסייעת להימנע מהמצב שבו תלמיד עושה עבודה איכותית אך אינו מתעד אותה. הדגש אינו על כמות התמונות בדוח אלא על התאמה בין טענה לבין בדיקה. למשל, אם כתוב "שיפרתי את המודל", יש לצרף טבלת ניסויים שמראה את מצב ההתחלה ואת מצב הסיום באותן שיטות מדידה.

16.24 מחוון הוראה מוצע — לא מחוון משרד החינוך

לצורך משוב שוטף בכיתה מוצע להשתמש במפת 100 נקודות הבאה. היא אינה קובעת את ציון הבגרות ואינה מחליפה את משקלי המחוון התקף של הפיקוח; מטרתה לאפשר לתלמיד לזהות מוקדם חולשות מקצועיות ולתקנן.

שאלה מנחה	משקל מוצע	תחום
האם הבעיה ברורה והנתונים מתאימים ומותרים?	20	**בעיה, נתונים ואתיקה**
האם הרשת והאימון מוסברים ונבדקו היטב?	25	**מודל ואימון**
האם יש Baseline, Test וניתוח שגיאות?	20	**הערכה וניסוי**
האם מערכת פועלת עם קלט חדש ומקרי קצה?	20	**יישום תוכנה**
האם אפשר לשחזר ולהסביר עצמאית?	15	**תיעוד והגנה**

16.25 פרויקט מודרך: זיהוי ספרות — תרגיל העברה

כעת נחבר את החלקים לתכנון ראשוני של פרויקט אחד. בחרו את מאגר digits. נסחו מטרת מוצר ברורה, בנו כרטיס נתונים, השוו קו בסיס ורשת MLP, בחרו מדדי הערכה, שמרו את המודל, ובנו ממשק שמקבל תמונה בפורמט שהוגדר. אם תשתמשו בקלט של ציור חופשי, תצטרכו לתעד ולהוכיח את שלב המרת הקלט — לא די לשנות את גודל המערך ללא בדיקה.

המשימות במחברת Colab מלוות את תהליך החשיבה. אינן מספקות פתרון מלא לעבודת הגמר, משום שהפרויקט דורש גם בחירת נושא עצמאית, פיתוח יישום, תיעוד ועמידה בהנחיות העדכניות. ניתן להשתמש בדוגמה המודרכת כדי ללמוד כיצד לנהל ראיות ולהסביר החלטות, אבל אין להגיש אותה כשלעצמה כפרויקט מקורי ללא פיתוח עצמאי מתאים.

16.26 משימות הערכה מדורגות

רמה א׳ — שליטה ביסודות

משימה 1: הגדירו מהי אוכלוסיית השימוש, מהו Dataset ואילו תנאים דרושים לכך שביצועי Test יתנו הערכה הגיונית. משימה 2: חשבו כמה משקולות והטיות קיימות ב־MLP בגודל 64–32–10 והסבירו את שלבי החישוב. משימה 3: הסבירו מדוע בעת שימוש ב־CrossEntropyLoss אין להפעיל Softmax פעמיים.

רמה ב׳ — שימוש בידע

משימה 4: תוכננו שתי ארכיטקטורות. אחת טובה יותר באימון אך פחות טובה בולידציה. נתחו שלוש סיבות אפשריות והציעו ניסוי מבוקר לכל אחת. משימה 5: חשבו Accuracy, Precision ו־Recall מטבלה שבה

TP=18, FN=2, FP=3, TN=77, ובחרו מדד מתאים לבעיה שבה החמצה יקרה יותר מאזעקת שווא. משימה 6: תארו שלושה מקרי בדיקה ליישום שאינו קשור כלל לדיוק המודל.

משימה 7: תכננו מערך נתונים שבו חלוקה אקראית תיצור זליגת מידע, והציעו חלופה מתאימה. משימה 8: בחרו סוג העמקה אחד — נתונים, מודל או יישום — והגדירו כיצד תמדדו את תרומתו. משימה 9: כתבו מכתב קצר למשתמש המסביר מתי אסור להסתמך על המערכת. משימה 10: תכננו חקר של שגיאות בפרויקט שלכם. משימה 11: הציגו תרחיש שבו Baseline עדיף על הרשת. משימה 12: הכינו סרטוט ותסריט הדגמה להגנה בעל־פה שכולל תקלה מתוכננת אחת וכיצד מאתרים אותה.

16.27 כרטיסי עבודה מוכנים למילוי

א. כרטיס אפיון פרויקט

שם הפרויקט: _____

צורך של המשתמש: _____

קלט, פלט ומדד הצלחה: _____

מגבלה או סיכון ידוע: _____

ב. יומן ניסויים

מספר וגרסת ניסוי: _____

השערה והפרמטר ששונה: _____

תוצאת Validation: _____

מסקנה והניסוי הבא: _____

ג. כרטיס מודל

ארכיטקטורה וגרסה: _____

Dataset, רישוי וחלוקה: _____

מדדים ומגבלות שימוש: _____

קישור לראיות בדיקה: _____

ד. רשימת הגשה

האם קיים קובץ שמפעיל את היישום? _____

האם כל הנתיבים והגרסאות מתועדים? ____

האם ניתן להציג קלט חדש? _____

האם התיק כולל רפלקציה ומקורות? _____

16.28 הקשר בין שלוש תכניות הלימוד

תכנית	מה היא תורמת	המשך בספר המאוחד
יחידה 3 — כיתה י׳	קריאת נתונים, Python, EDA, מודלים בסיסיים	חשיבה מחקרית וקו בסיס
יחידה 5 — נושאי ליבה	וקטורים, סטטיסטיקה, מרחקים, KNN, הערכה	הצדקה מתמטית ומדידת איכות
התמחות DL — י״א-י״ב	רשתות עמוקות, אימון, CNN/RNN ופיתוח	מערכת AI, תיעוד ועבודת גמר

מפת ההקשרים היא בחירה פדגוגית שלנו. אין פירושה שכל מיומנות המוזכרת בפרק זה היא יעד חובה ביחידה 5 העיונית. הדרישה לעבודה הכוללת מודל למידה עמוקה ויישום תוכנה משויכת לחלופת ההתמחות בלבד, לפי ההנחיות שנסקרו.

16.29 מקורות והערת גרסה

- משרד החינוך — הנחיות לכתיבת תיק פרויקט בחלופה למידת מכונה, סמל 883589: https://meyda.education.gov.il/files/CSIT/MachinLearning-Project.pdf
- משרד החינוך — מחוון חלופת למידת מכונה 5 יחידות, תשפ״ג (לצורך עיון היסטורי; יש לבדוק עדכון): https://meyda.education.gov.il/files/CSIT/machinLearningProject_Finalt-5.pdf
- משרד החינוך — המרחב הפדגוגי של הנדסת תוכנה, בינה מלאכותית וסייבר: https://pop.education.gov.il/tchumey_daat/computer-science-software-engineering-cyber/high-school/pedagogy-of/high-school-software-cyber-engineering/
- Aurélien Géron — Hands-On Machine Learning, כבסיס לשיטה הנדסית ולניסויים.
- Zhang, Lipton, Li & Smola — Dive into Deep Learning, כבסיס להסבר מודלים ואימון.
- Simon J. D. Prince — Understanding Deep Learning, כבסיס לחשיבה על רשתות ואובדן.
- Google — Rules of Machine Learning: https://developers.google.com/machine-learning/guides/rules-of-ml
- Mitchell et al. — Model Cards for Model Reporting (2019): https://research.google/pubs/model-cards-for-model-reporting/
- מאגר ההוראה שלנו: https://github.com/samiabo75/Data-Science-ML-Course/tree/main/03-Deep-Learning-5-Units/16-Final-Project

נקודה למחשבה — מהדורת ההוראה הזו נכתבה כפרק עצמאי בספר המאוחד. לפני הגשה מעשית יש לוודא שהנחיות הפיקוח, מחוון ההערכה, מועדי ההגשה ודרישות הטכנולוגיה נכונים לשנת ההגשה הספציפית.